

# A Finitely Presented Non-Sofic Group

## A Guided Digest of the Lean Formalization

Digest of `NonSoficGroup.lean` from the repository [openai/ten-proofs](#) (commit `94bc0feb6a9f`)

Digest prepared automatically by Claude (Anthropic) for the LeanPIMS26 course materials

August 4, 2026

### Abstract

This document is a guided tour of `NonSoficGroup.lean`, a 34,440-line Lean 4 formalization published by OpenAI in the repository `ten-proofs` (August 2026). The file proves, without `sorry` and without extra axioms, that *there exists a finitely presented group that is not sofic*, answering the well-known question of Gromov and Weiss of whether every group admits finite approximate permutation models. Every one of the file's 1,421 declarations is accounted for here: the load-bearing definitions and theorems are displayed with their Lean statements and explained in detail, and the routine supporting lemmas are summarized in tables, one line each, in the order they appear in the file. The Lean source accompanies this document as `sophic.lean`.

## Contents

|                                                                                |     |
|--------------------------------------------------------------------------------|-----|
| About this document                                                            | 2   |
| 1 The main theorem and the shape of the proof                                  | 2   |
| 2 Foundations: soficity, LEF, property (T), and the multiplication-table trick | 4   |
| 3 The nine-prefix source group and its compressions                            | 17  |
| 4 Prefix transposition groups, midrank statistics, and projections             | 31  |
| 5 Property (T) for the elementary group and coarea repair                      | 63  |
| 6 Rooted models, Kazhdan contraction, and expander decompositions              | 80  |
| 7 Transporting and completing sofic approximations                             | 107 |
| 8 Poincaré inequalities, finite models, and the Thompson obstruction           | 137 |
| 9 Final assembly: selected components and the main theorem                     | 168 |
| 10 Provenance and verification notes                                           | 196 |

# About this document

The digest walks through the file in its original order, one namespace at a time. For each namespace you will find: a short summary of what it develops and why the argument needs it; *deep dives* into its most important declarations, showing the Lean statement (theorems are shown up to `:=` by, with an italic note recording the length of the omitted proof script) followed by a mathematical explanation; and a table listing every remaining declaration with a one-line description. Lean identifiers are set in typewriter font, e.g. `normalizedHamming`. Section headings carry the enclosing namespace, and the marginal tags `DEF`, `THM`, `STRUCT`, `CLASS`, `INST`, `ABBR`, `IND` record each declaration's kind.

|                                              |                       |
|----------------------------------------------|-----------------------|
| lines of Lean                                | 34,440                |
| declarations                                 | 1,421                 |
| theorems                                     | 1,045                 |
| definitions (def, noncomputable def, abbrev) | 334                   |
| structures, classes, inductives              | 22                    |
| instances                                    | 20                    |
| namespaces documented                        | 126 sections          |
| occurrences of sorry or axiom                | 0                     |
| toolchain                                    | Lean 4.32.0 + Mathlib |

## 1 The main theorem and the shape of the proof

A countable group is *sofic* if its multiplication can be modeled, with arbitrarily small error, by honest permutations of finite sets. The file makes this precise with a normalized Hamming distance on permutations, a bare-bones notion of a finite permutation model of a group, and a predicate saying such a model is good on a finite window:

```
def normalizedHamming {Y : Type*} [Fintype Y] [DecidableEq Y]
  (p q : Equiv.Perm Y) : ℝ :=
  (hammingDist (fun y => p y) (fun y => q y) : ℝ) / Fintype.card Y

structure PermutationModel (G : Type*) [Group G] where
  size : ℕ
  size_pos : 0 < size
  action : G → Equiv.Perm (Fin size)
  map_one : action 1 = 1

structure GoodOn {G : Type*} [Group G]
  (M : PermutationModel G) (F : Finset G) (ε : ℝ) : Prop where
  multiplicative : ∀ g ∈ F, ∀ h ∈ F,
    normalizedHamming (M.action (g * h)) (M.action g * M.action h) < ε
  separated : ∀ g ∈ F, g ≠ 1 →
    1 - ε < normalizedHamming (M.action g) 1

class Sofic (G : Type*) [Group G] : Prop where
  approximation : ∀ (F : Finset G) (ε : ℝ), 0 < ε → ε < 1 →
    ∃ M : PermutationModel G, GoodOn M F ε
```

So `Sofic G` demands, for every finite set  $F$  of group elements and every tolerance  $\varepsilon$ , a finite permutation assignment that is  $\varepsilon$ -multiplicative on  $F$  and moves at least a  $1 - \varepsilon$  fraction of points for each nonidentity element of  $F$ . Free groups, amenable groups, residually finite groups — all known groups — are sofic; whether *every* group is sofic was posed by Gromov (1999) and Weiss (2000) and has been open since. The file's final answer is:

```
theorem exists_finitelyPresented_nonsofic_group :
  ∃ (G : Type) (G : Group G),
    Group.IsFinitelyPresented G ∧ ¬ SoficGroups.Sofic G := by
  -- proof: 3 lines, omitted
```

## 1.1 The counterexample and the contradiction engine

The concrete non-sofic group is `binaryLeavittElementaryGroup 9`, a group of elementary  $2 \times 2$  matrices over a Leavitt-style binary ring, shown isomorphic (via `ninePrefixElementaryGroupEquiv`) to an elementary group `prefixElementaryGroup ninePrefixCode` built from a nine-word prefix code. The refutation of soficity runs through a second approximation notion, *local embeddability into finite groups*:

```
structure LocalMultiplicativeOn {G : Type u} {H : Type v}
  [Group G] [Group H] (s : Finset G) (f : G → H) : Prop where
  map_one : f 1 = 1
  map_mul : ∀ x ∈ s, ∀ y ∈ s, f (x * y) = f x * f y

class LEF (G : Type u) [Group G] : Prop where
  approximate : ∀ s : Finset G, ∃ (n : ℕ) (f : G → Equiv.Perm (Fin n)),
    Set.InjOn f (s : Set G) ∧ LocalMultiplicativeOn s f
```

The engine of the proof is the implication “sofic  $\Rightarrow$  LEF” for a carefully chosen witness group, played against a hand-checkable proof that the witness is *not* LEF:

```
theorem sourceLocalPrefixTranspositionGroup_lef_of_sofic :
  SoficGroups.Sofic
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode) →
  SoficGroups.LEF
    (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
     SoficGroups.ThompsonPrefixLocal.sourceLocalWord) := by
  -- proof: 6 lines, omitted

theorem ninePrefixElementaryGroup_not_sofic :
  ¬ SoficGroups.Sofic
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode) := by
  -- proof: 4 lines, omitted
```

The first theorem is where nearly all of the file’s 34,000 lines are spent. Assuming a sofic approximation of the nine-prefix elementary group, the machinery of Parts 2–9 upgrades its permutation models — using property (T) of the elementary group, spectral gaps, and an expander-decomposition transport argument — into *expanding centralizer finite models* that restrict to honest local embeddings of the Thompson-style local prefix transposition group. The second pillar is a short, self-contained obstruction: Thompson-like two-relator identities force any finite quotient to collapse (`ThompsonFFiniteQuotient`, `ThompsonFTwoRelatorLEF`), so that transposition group admits no local embeddings at all (`sourceLocalPrefixTranspositionGroup_notLEF`). The two pillars together refute soficity of the nine-prefix group, and soficity descends along injective homomorphisms (`sofic_of_injective`), which kills `binaryLeavittElementaryGroup 9`:

```
theorem binaryLeavittElementaryGroup_not_sofic :
  ¬ SoficGroups.Sofic (SoficGroups.binaryLeavittElementaryGroup 9) := by
  -- proof: 8 lines, omitted
```

## 1.2 From non-sofic to finitely presented and non-sofic

The counterexample above is not obviously finitely presented, and the final step is a pleasant general trick. Given any non-sofic group, a compactness argument (`exists_finite_obstruction`) produces a single finite window  $F$  and tolerance  $\varepsilon$  with no good model. The *multiplication-table group* `tableGroup F h` is then the finitely presented group whose generators are the elements of the closure of  $F$  under multiplication and whose relators are exactly the entries of its multiplication table; it maps onto enough of  $G$  to inherit the obstruction, and it is finitely presented by construction:

```
theorem exists_finitelyPresented_not_sofic_of_not_sofic
  {G : Type u} [Group G] (hG : ¬ Sofic G) :
  ∃ (H : Type u) (h_ : Group H),
    Group.IsFinitelyPresented H ∧ ¬ Sofic H := by
  -- proof: 3 lines, omitted
```

Applying this to `binaryLeavittElementaryGroup 9` yields the main theorem. The remainder of this document follows the file from top to bottom.

## 2 Foundations: soficity, LEF, property (T), and the multiplication-table trick

The opening stretch of the file lays down every actor of the drama: the normalized Hamming distance and permutation models defining `Sofic`, sofic approximations and their amplification, the compactness argument extracting a single finite obstruction from non-soficity, the finitely presented multiplication-table group that inherits the obstruction, local embeddability into finite groups (`LEF`), Kazhdan pairs and property (T), and the binary Leavitt ring whose elementary matrix group will be the counterexample.

### 2.1 Soficity, property (T), and table groups

Sets up the two frameworks the file mediates between. First, property (T): `UnitaryRepresentation`, `KazhdanPair` and the class `HasPropertyT`, transported along isomorphisms. Second, soficity: the normalized Hamming metric on finite symmetric groups with its bi-invariance lemmas, `PermutationModel`, the goodness predicate `GoodOn` ( $\varepsilon$ -multiplicative on a finite set, separating nonidentity elements), and the class `Sofic`. Amplification of a model by a trivial factor preserves all relevant distances, giving sofic approximations whose sizes tend to infinity, and soficity pulls back along injective homomorphisms. Then the multiplication-table trick: a non-sofic group has a finite obstruction  $(F, \varepsilon)$ , and the finitely presented group `tableGroup`, built from relators recording the multiplication table on  $F$ , inherits non-soficity; this reduces the main theorem to exhibiting any non-sofic group. The segment ends by defining `LocalMultiplicativeOn` and the class `LEF`.

**KazhdanPair** `struct`

```
structure KazhdanPair (G : Type u) [Group G] where
  generators : Finset G
  kazhdanConstant : ℝ
  positive : 0 < kazhdanConstant
  invariant :
    ∀ (H : Type v) [NormedAddCommGroup H] [InnerProductSpace ℂ H]
      [CompleteSpace H] (π : UnitaryRepresentation G H) (ξ : H),
      ‖ξ‖ = 1 →
      (∀ g ∈ generators, ‖π g ξ - ξ‖ < kazhdanConstant) →
      ∃ η : H, η ≠ 0 ∧ ∀ g : G, π g η = η
```

`KazhdanPair` packages a Kazhdan pair for a group  $G$ : a finite set of generators, a strictly positive real `kazhdanConstant`  $\kappa$ , and the defining property that for every complete complex inner product space  $H$  and every unitary representation  $\pi$  of  $G$  on  $H$  (a homomorphism into linear isometric equivalences, per `UnitaryRepresentation`), any unit vector  $\xi$  with  $\|\pi(g)\xi - \xi\| < \kappa$  for all generators  $g$  forces the existence of a nonzero  $G$ -invariant vector  $\eta$ . This is the standard quantitative formulation of property (T): almost-invariant vectors imply invariant vectors. The class `HasPropertyT` merely asserts that a Kazhdan pair exists, with a universe parameter for the representation spaces quantified over. The transfer lemma `hasPropertyT_of_mulEquiv` shows the property is an isomorphism invariant: given an isomorphism  $e$  from  $G$  to  $G'$ , push the generator set forward by  $e$  and keep the same constant; a representation of  $G'$  precomposed with  $e$  is a representation of  $G$ , and invariance of a vector under all of  $G$  transfers using surjectivity of  $e$ . Property (T) matters in this file because the spectral-gap and expander estimates that ultimately contradict soficity are driven by Kazhdan-type constants of auxiliary groups appearing much later, so this early section fixes the exact form of hypothesis all of that machinery consumes.

**Sofic** `class`

```
class Sofic (G : Type*) [Group G] : Prop where
  approximation : ∀ (F : Finset G) (ε : ℝ), 0 < ε → ε < 1 →
    ∃ M : PermutationModel G, GoodOn M F ε
```

`Sofic` is the file's official definition of soficity, phrased finitarily. A `PermutationModel` of  $G$  is a positive size  $n$  with a map action from  $G$  to permutations of  $\text{Fin } n$  sending the identity to the identity; no homomorphism property is required. The predicate `GoodOn` demands two things measured in the normalized Hamming metric

normalizedHamming (the fraction of points where two permutations differ): for all  $g, h \in F$  the permutation assigned to  $gh$  is within  $\varepsilon$  of the product of those assigned to  $g$  and  $h$ ; and every nonidentity  $g \in F$  has distance more than  $1 - \varepsilon$  from the identity, i.e. its permutation moves almost every point. Sofic then asserts that for every finite  $F$  and every  $0 < \varepsilon < 1$  some good model exists. The metric lemmas proved beforehand (symmetry, triangle inequality, boundedness by one, and invariance under left and right multiplication) make this notion workable, and `GoodOn.mono` lets one shrink the test set. Everything the file does afterwards is organized around refuting this definition for one specific finitely presented group, so all the analytic and combinatorial machinery is ultimately measured against these two quantitative clauses.

**nonempty\_soficApproximation\_of\_sofic** thm

```
theorem nonempty_soficApproximation_of_sofic
  (G : Type*) [Group G] [Countable G] [Sofic G] :
  Nonempty (SoficApproximation G) := by
-- proof: 49 lines, omitted
```

Converts the finitary definition of soficity into the sequential object `SoficApproximation`: a sequence of permutation models where, for each fixed pair  $g, h$ , the normalized Hamming distance between the model of  $gh$  and the product of the models tends to zero, and for each fixed  $g \neq 1$  the distance from the model of  $g$  to the identity tends to one. For a countable sofic group the proof takes a finite exhaustion of the universe (via `Set.FiniteExhaustion`), tolerances  $\varepsilon_n = 1/(n + 2)$ , and chooses for each  $n$  a model good on the  $n$ -th finite piece; since every element eventually belongs to the exhaustion, squeeze arguments yield both limits. Companions refine this: `amplifyApproximation` replaces the  $n$ -th model by its amplification by factor  $n + 1$ , which the earlier lemmas `amplifyModel_multiplicative_distance` and `amplifyModel_separation_distance` show changes no distances, so `exists_soficApproximation_size_tendsto` produces an approximation whose model sizes diverge. This is the form in which soficity is actually consumed by the long combinatorial argument later in the file: a hypothetical sofic approximation with growing size is fed into `expander` and `transport` estimates to manufacture a contradiction, and `pullbackSoficApproximation` moves such data across injective homomorphisms.

**exists\_nonsofic\_finite\_table** thm

```
theorem exists_nonsofic_finite_table {G : Type u} [Group G]
  (hG : ¬ Sofic G) :
  ∃ (F : Finset G) (h1 : 1 ∈ F),
  Group.IsFinitelyPresented (tableGroup F h1) ∧
  ¬ Sofic (tableGroup F h1) := by
-- proof: 4 lines, omitted
```

This theorem is the reduction that makes the main result finitely presented. Starting from any non-sofic group  $G$ , `exists_finite_obstruction` extracts a finite set  $F$  containing the identity and an  $\varepsilon \in (0, 1)$  such that no permutation model of  $G$  is good on  $(F, \varepsilon)$  — otherwise the goodness witnesses would assemble directly into an instance of `Sofic`. The construction then almost forgets  $G$ : `multiplicationTable` is  $F$  together with all pairwise products, and `tableGroup` is the `PresentedGroup` on these generators with relators (`tableRelators`) saying the generator at 1 is trivial and the generator at  $gh$  is the product of the generators at  $g$  and  $h$ ; this is visibly a finite presentation (`tableGroup_finitelyPresented`). The evaluation homomorphism `tableEvaluation` back to  $G$  shows generators of nonidentity elements are nontrivial. Crucially, `goodOn_pullbackTableModel` converts any model of the table group that is good on the generator set `tableTestSet` into a model of  $G$  good on  $F$ , by acting through generators on table elements and by the identity elsewhere. Hence a sofic table group would contradict the obstruction (`tableGroup_not_sofic_of_obstruction`), so the table group is a finitely presented non-sofic group, as packaged in `exists_finitelyPresented_not_sofic_of_not_sofic`.

*Remaining declarations in this section:*

**UnitaryRepresentation**

**ABBR** Abbreviates a unitary representation: a homomorphism from  $G$  into linear isometric equivalences of a complex inner product space.

|                                        |        |                                                                                                                                                 |
|----------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| HasPropertyT                           | CLASS  | Class asserting a Kazhdan pair exists, i.e. $G$ has property (T).                                                                               |
| hasPropertyT_of_mulEquiv               | THM    | Transfers property (T) across a group isomorphism by imaging the generators and precomposing representations.                                   |
| normalizedHamming                      | DEF    | Normalized Hamming distance: the fraction of points where two permutations of a finite type differ.                                             |
| normalizedHamming_self                 | THM    | The normalized Hamming distance of a permutation to itself is zero.                                                                             |
| normalizedHamming_comm                 | THM    | Symmetry of <code>normalizedHamming</code> .                                                                                                    |
| normalizedHamming_nonneg               | THM    | Nonnegativity of <code>normalizedHamming</code> .                                                                                               |
| normalizedHamming_triangle             | THM    | Triangle inequality for <code>normalizedHamming</code> , inherited from <code>hammingDist</code> .                                              |
| normalizedHamming_le_one               | THM    | <code>normalizedHamming</code> is at most one, since Hamming distance is bounded by the cardinality.                                            |
| normalizedHamming_mul_left             | THM    | Left multiplication by a fixed permutation preserves <code>normalizedHamming</code> .                                                           |
| normalizedHamming_mul_right            | THM    | Right multiplication by a fixed permutation preserves <code>normalizedHamming</code> , via a bijection of disagreement sets.                    |
| PermutationModel                       | STRUCT | Structure: a positive size $n$ with a map from $G$ to permutations of $\text{Fin } n$ fixing the identity.                                      |
| GoodOn                                 | STRUCT | Predicate: the model is $\varepsilon$ -multiplicative on $F$ and nonidentity elements of $F$ move all but an $\varepsilon$ -fraction of points. |
| GoodOn.mono                            | THM    | Goodness on a set restricts to any subset.                                                                                                      |
| normalizedHamming_permCongr            | THM    | Conjugating both permutations by an equivalence of finite types preserves <code>normalizedHamming</code> .                                      |
| hammingDist_prodCongr_refl_left        | THM    | Hamming distance of <code>Equiv.prodCongr</code> with an identity factor scales by the cardinality of that factor.                              |
| normalizedHamming_prodCongr_refl_left  | THM    | Normalized Hamming distance is unchanged by adding an identity product factor.                                                                  |
| amplifyModel                           | DEF    | Amplifies a model by factor $k$ , acting on $\text{Fin } k \times \text{Fin } n$ with identity on the first coordinate.                         |
| amplifyModel_normalizedHamming         | THM    | Amplification preserves normalized Hamming distances between model permutations.                                                                |
| amplifyModel_multiplicative_distance   | THM    | Amplification preserves the multiplicativity defect distance.                                                                                   |
| amplifyModel_separation_distance       | THM    | Amplification preserves distance from the identity permutation.                                                                                 |
| SoficApproximation                     | STRUCT | Structure: a sequence of models whose multiplicativity defect tends to zero and whose separation tends to one, pointwise in $G$ .               |
| pullbackPermutationModel               | DEF    | Precomposes a permutation model with a group homomorphism.                                                                                      |
| pullbackSoficApproximation             | DEF    | Pulls back a sofic approximation along an injective homomorphism.                                                                               |
| amplifyApproximation                   | DEF    | Amplifies the $n$ -th model of an approximation by factor $n + 1$ .                                                                             |
| amplifyApproximation_size_tendsto      | THM    | Sizes of the amplified approximation tend to infinity.                                                                                          |
| exists_soficApproximation_size_tendsto | THM    | A countable sofic group has a sofic approximation with sizes tending to infinity.                                                               |
| sofic_of_injective                     | THM    | Soficity pulls back along injective homomorphisms, by testing on image sets.                                                                    |

|                                                              |        |                                                                                                                      |
|--------------------------------------------------------------|--------|----------------------------------------------------------------------------------------------------------------------|
| <code>exists_finite_obstruction</code>                       | THM    | A non-sofic group has a finite obstruction: some $F \ni 1$ and $\varepsilon \in (0, 1)$ admit no good model.         |
| <code>multiplicationTable</code>                             | DEF    | The finite set $F$ together with all pairwise products.                                                              |
| <code>mem_multiplicationTable_of_mem</code>                  | THM    | $F$ is contained in its multiplication table.                                                                        |
| <code>mul_mem_multiplicationTable</code>                     | THM    | Products of two elements of $F$ lie in the multiplication table.                                                     |
| <code>tableRelators</code>                                   | DEF    | Relators: the generator at 1 is trivial and generators multiply according to the table.                              |
| <code>tableGroup</code>                                      | ABBR   | The presented group on multiplication-table generators modulo <code>tableRelators</code> .                           |
| <code>tableGenerator</code>                                  | DEF    | Canonical generator of the table group attached to a table element.                                                  |
| <code>tableGenerator_one</code>                              | THM    | The generator at the identity element equals one.                                                                    |
| <code>tableGenerator_mul</code>                              | THM    | Generators of $g$ and $h$ multiply to the generator of $gh$ .                                                        |
| <code>tableEvaluation</code>                                 | DEF    | Evaluation homomorphism from the table group to $G$ , sending each generator to its underlying element.              |
| <code>tableEvaluation_generator</code>                       | THM    | Evaluation computes on generators (simp lemma).                                                                      |
| <code>tableGenerator_ne_one</code>                           | THM    | Generators of nonidentity elements of $F$ are nontrivial, via evaluation.                                            |
| <code>tableGroup_finitelyPresented</code>                    | THM    | The table group is finitely presented (instance search).                                                             |
| <code>tableTestSet</code>                                    | DEF    | The finite subset of the table group consisting of the generators indexed by $F$ .                                   |
| <code>tableGenerator_mem_tableTestSet</code>                 | THM    | Generators coming from $F$ belong to the test set.                                                                   |
| <code>pullbackTableModel</code>                              | DEF    | Turns a model of the table group into a model of $G$ : act through generators on table elements, identity elsewhere. |
| <code>pullbackTableModel_action_of_mem</code>                | THM    | Computes the pulled-back action on multiplication-table elements.                                                    |
| <code>goodOn_pullbackTableModel</code>                       | THM    | A model good on the test set pulls back to a model of $G$ good on $F$ .                                              |
| <code>tableGroup_not_sofic_of_obstruction</code>             | THM    | If $(F, \varepsilon)$ admits no good model of $G$ , the table group is not sofic.                                    |
| <code>exists_finitelyPresented_not_sofic_of_not_sofic</code> | THM    | Packages the previous result: non-soficity implies existence of a finitely presented non-sofic group.                |
| <code>LocalMultiplicativeOn</code>                           | STRUCT | Predicate: $f$ sends 1 to 1 and is multiplicative on pairs from the finite set $s$ .                                 |
| LEF                                                          | CLASS  | Class: every finite subset admits an injective locally multiplicative map into a finite symmetric group.             |

## 2.2 Local multiplicativity lemmas (LocalMultiplicativeOn)

A short namespace of closure properties for `LocalMultiplicativeOn`, the predicate defined just above asserting that a map  $f$  between groups sends 1 to 1 and is multiplicative on products of elements of a fixed finite set  $s$ . `LocalMultiplicativeOn.mono` restricts the predicate along inclusions of test sets, and `LocalMultiplicativeOn.map_inv_of_mem` derives compatibility with inverses when both an element and its inverse lie in  $s$ . These small facts are the working interface for the LEF notion (local embeddability into finite groups): the later arguments that test LEF on finite fragments of a group repeatedly restrict approximate homomorphisms to smaller test sets and must handle inverses without any global homomorphism property being available.

`map_inv_of_mem` *thm*

```
theorem map_inv_of_mem {s : Finset G} {f : G → H}
  (h : LocalMultiplicativeOn s f) {x : G} (hx : x ∈ s) (hi : x⁻¹ ∈ s) :
  f x⁻¹ = (f x)⁻¹ := by
```

-- proof: 4 lines, omitted

`LocalMultiplicativeOn.map_inv_of_mem` shows that a locally multiplicative map automatically respects inverses inside its test set: if  $x \in s$  and  $x^{-1} \in s$ , then  $f(x^{-1}) = f(x)^{-1}$ . The proof is one line of algebra: applying local multiplicativity to the pair  $x, x^{-1}$  gives  $f(xx^{-1}) = f(x)f(x^{-1})$ , the left side is  $f(1) = 1$  by the `map_one` field, and a right inverse in a group is the inverse. The point of recording this is that LEF witnesses are bare functions into finite symmetric groups with no homomorphism structure, so every group-theoretic identity one wants to use about them must be re-derived from the two axioms of `LocalMultiplicativeOn` on a sufficiently large finite set. Together with `LocalMultiplicativeOn.mono` and the free-group evaluation lemma `exists_local_word_control` that follows immediately after the namespace, this gives a small calculus for pushing fixed words and inverses through approximate homomorphisms — exactly what is needed when the LEF property is tested on the finite fragments of the local prefix transposition group in the endgame of the file.

*Remaining declarations in this section:*

`mono` `thm` Local multiplicity on a set restricts to subsets.

## 2.3 Expansion, matched partitions, Leavitt algebra

Quantitative tools, followed by the algebraic protagonist. First, `exists_local_word_control` controls evaluation of a fixed free-group word under locally multiplicative maps. Then a boundary and expansion theory for the multigraphs given by finite families of permutations: commutation defects bound the boundary of agreement sets, yielding the rigidity dichotomy `hamming_dichotomy_of_expansion`, and a maximal bad cut argument (`exists_pruned_expander`) prunes an expander-with-additive-error down to a small deleted set outside of which induced expansion holds. Next, machinery for comparing two finite partitions matched through a dominant map  $D$ : the crossing set of a permutation for one partition is bounded by crossings for the other plus twice the unmatched mass, with density versions along a sequence. Auxiliary lemmas cover permutation increments, extension of partial permutations, a diagonal extraction (`diagonalRadius`), and vanishing bad-density lemmas. The final stretch constructs the binary Leavitt algebra `BinaryLeavitt` over  $\mathbb{Z}/2$ , its representation on binary sequences, and the word calculus `leavittWordS`, `leavittWordT`, `leavittCylinder` with the cylinder splitting identity.

**hamming\_dichotomy\_of\_expansion** `thm`

```
theorem hamming_dichotomy_of_expansion {V ι : Type*}
  [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (h : ℝ)
  (hexp : ∀ A : Finset V,
    h * min (A.card : ℝ)
      ((Fintype.card V : ℝ) - A.card) ≤
    (boundary σ A : ℝ))
  (c c' : Equiv.Perm V) :
  h * (hammingDist (fun x => c x) (fun x => c' x) : ℝ) ≤
    (permutationCommutationDefect σ c +
     permutationCommutationDefect σ c' : ℕ) v
  h * ((Fintype.card V : ℝ) -
    hammingDist (fun x => c x) (fun x => c' x)) ≤
    (permutationCommutationDefect σ c +
     permutationCommutationDefect σ c' : ℕ) := by
-- proof: 32 lines, omitted
```

`hamming_dichotomy_of_expansion` is the mechanism converting spectral-gap information into rigidity of permutations. Fix a family  $\sigma_i$  of permutations of a finite vertex set  $V$  satisfying an expansion hypothesis: every subset  $A$  has boundary at least  $h \cdot \min(|A|, |V| - |A|)$ , where `boundary` counts, for each  $i$ , the points of  $A$  mapped out of  $A$ . Given two further permutations  $c$  and  $c'$ , consider their `agreementSet`. Its boundary is bounded by the sum of the two `permutationCommutationDefects` — the counts of points where  $c$  (respectively  $c'$ ) fails to commute with each  $\sigma_i$  — because a point where  $c$  and  $c'$  agree while their values at  $\sigma_i x$  disagree forces a commutation failure for one of them (`boundary_agreementSet_le_commutationDefect`). Since the agreement set's cardinality is complementary to the Hamming distance between  $c$  and  $c'$  (`agreementSet_card_add_hammingDist`),



the expansion hypothesis yields a dichotomy: either  $h$  times the Hamming distance, or  $h$  times its complement  $|V| - \text{dist}$ , is bounded by the combined defects. In words: two permutations that both almost commute with an expanding family are either almost equal or almost completely disjoint. This is the basic rigidity used later to constrain the possible shapes of hypothetical sofic approximations.

**exists\_pruned\_expander** thm

```
theorem exists_pruned_expander {V ι : Type*}
  [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (γ ell a : ℝ)
  (hgap : ell < γ)
  (hadd : ∀ A : Finset V,
    γ * min (A.card : ℝ) ((Fintype.card V : ℝ) - (A.card : ℝ)) -
    a * (Fintype.card V : ℝ) ≤ (boundary σ A : ℝ))
  (hsmall :
    2 * a * (2 * (γ - ell) + (Fintype.card ι : ℝ)) ≤ (γ - ell) ^ 2) :
  ∃ B : Finset V,
    2 * B.card ≤ Fintype.card V ∧
    (B.card : ℝ) ≤ a * (Fintype.card V : ℝ) / (γ - ell) ∧
    ∀ E : Finset V, Disjoint B E →
      2 * E.card ≤ Fintype.card V - B.card →
      ell * (E.card : ℝ) ≤ (inducedBoundary σ B E : ℝ) := by
-- proof: 15 lines, omitted
```

**exists\_pruned\_expander** upgrades an additive-error expander to a genuine one after deleting a small set. The hypothesis gives, for every subset  $A$ ,  $\gamma \min(|A|, N - |A|) - aN \leq \partial A$  with  $N = |V|$ : expansion at rate  $\gamma$  up to an additive defect  $aN$ , the form in which spectral estimates for approximate representations naturally arrive. Choosing a target rate  $\ell < \gamma$  and assuming the smallness condition  $2a(2(\gamma - \ell) + |\iota|) \leq (\gamma - \ell)^2$ , the theorem produces a set  $B$  occupying at most half the vertices with  $|B| \leq aN/(\gamma - \ell)$ , such that every  $E$  disjoint from  $B$  with  $2|E| \leq N - |B|$  satisfies  $\ell|E| \leq \text{inducedBoundary}$ , the boundary of  $E$  counted avoiding  $B$ . The proof takes  $B$  to be a maximal-cardinality small set with cheap boundary (**exists\_maximal\_bad\_cut**); if some disjoint  $E$  violated induced expansion, then either  $B \cup E$  would be a larger bad cut, contradicting maximality (**maximal\_bad\_cut\_induced\_lower**), or  $E$  is so large that the purely algebraic estimate **large\_cut\_lower** applies, using **boundary\_le\_induced\_add\_deleted** to compare the two boundary notions. Later chunks apply this to the permutation multigraphs coming from sofic models, obtaining honest expanders after discarding a vanishing fraction of points.

**partitionWordCrossing\_card\_le\_target\_add\_unmatched** thm

```
theorem partitionWordCrossing_card_le_target_add_unmatched
  {V : Type*} [DecidableEq V] {U : Finset V}
  (P Q : Finpartition U) (R : Finset (Finset V))
  (hR : R ⊆ P.parts) (D : Finset V → Finset V)
  (hD : ∀ C ∈ R, D C ∈ Q.parts)
  (hinj : Set.InjOn D (R : Set (Finset V)))
  (w : Equiv.Perm V) :
  (partitionWordCrossing P w).card ≤
    (partitionWordCrossing Q w).card +
    2 * (U \ matchedCore R D).card := by
-- proof: 11 lines, omitted
```

**partitionWordCrossing\_card\_le\_target\_add\_unmatched** is the transport estimate for partitions along a matching. Setup: a finite set  $U$  carries two **Finpartitions**  $P$  and  $Q$ ; a subfamily  $R$  of  $P$ -parts is retained, and each retained part  $C$  is matched to a set  $D(C)$ , assumed to be a  $Q$ -part whose strict majority lies inside  $C$ ; this makes  $D$  injective on  $R$  (**finpartition\_dominant\_matching\_injOn**, resting on the pigeonhole fact **disjoint\_dominant\_intersections\_false**). The **matchedCore** is the union of the overlaps  $C \cap D(C)$ . For any permutation  $w$ , the crossing set **partitionWordCrossing** — the points moved out of their own part — for  $P$  is covered by the  $Q$ -crossings, the complement of the matched core, and the  $w$ -preimage of that complement (**partitionWordCrossing\_subset\_target\_or\_unmatched**): if both  $x$  and  $w x$  lie in the core and  $w$  preserves  $Q$ -parts at  $x$ , the injectivity of the matching forces it to preserve  $P$ -parts too (**matched\_partition\_part\_eq\_of\_target\_part\_eq**).

Cardinality bookkeeping then gives that the number of  $P$ -crossings is at most the number of  $Q$ -crossings plus  $2|U \setminus \text{core}|$ , and `matchedCore_missing_density_tendsto_zero` makes the correction term vanish asymptotically when the discarded support and the symmetric differences  $C \Delta D(C)$  have vanishing density. This lets later arguments transfer approximate partition-preservation between combinatorially matched partitions.

## BinaryLeavitt `abbr`

`abbrev BinaryLeavitt : Type := RingQuot LeavittRelation`

`BinaryLeavitt` is the coefficient ring over which the eventual non-sofic group is built: the Leavitt algebra of the binary shift over  $\mathbb{Z}/2$ . It is presented concretely as `RingQuot` of the free  $\mathbb{Z}/2$ -algebra on generators  $s_0, s_1, t_0, t_1$  (`LeavittGenerator`) by the relations (`LeavittRelation`)  $t_i s_j = \delta_{ij}$  and  $s_0 t_0 + s_1 t_1 = 1$ : the  $s_i$  behave like isometries with orthogonal ranges partitioning the identity. Countability and finite generation are recorded as instances. To see the quotient is not degenerate, the file represents the algebra on the space `leavitt` Sequence of  $\mathbb{Z}/2$ -valued sequences: `prefixOperator` interleaves a sequence into the positions congruent to  $i$  modulo 2, and `deletionOperator` reads the subsequence  $x(2n + i)$ ; these satisfy the defining relations (`deletion_mul_prefix`, `prefix_deletion_partition`), so `RingQuot.liftAlgHom` yields `leavittRepresentation`. Evaluating this representation shows  $s_0 t_0 \neq 1$  (`leavitt5_mul_T_ne_one`), giving nontriviality and, since in a finite ring one-sided inverses are two-sided, infiniteness of the algebra. The subsequent word calculus — `leavittWord5`, `leavittWord`  $\top$ , the orthogonality of incomparable words, and the cylinder idempotents with their splitting identity `leavittCylinder_split` — is the interface every later prefix-code construction uses.

*Remaining declarations in this section:*

|                                                         |     |                                                                                                                               |
|---------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------|
| <code>exists_local_word_control</code>                  | THM | For each free-group word there is a finite set on which local multiplicativity forces $f$ to commute with word evaluation.    |
| <code>boundary</code>                                   | DEF | Boundary of $A$ : total count, over the permutation family, of points of $A$ mapped outside $A$ .                             |
| <code>agreementSet</code>                               | DEF | The set of points where two permutations agree.                                                                               |
| <code>permutationCommutationDefect</code>               | DEF | Total count of points where $c$ fails to commute with each family permutation.                                                |
| <code>agreementSet_card_add_hammingDist</code>          | THM | Agreement-set cardinality plus Hamming distance equals the size of $V$ .                                                      |
| <code>boundary_agreementSet_le_commutationDefect</code> | THM | The boundary of the agreement set is at most the sum of the two commutation defects.                                          |
| <code>inducedBoundary</code>                            | DEF | Boundary of $E$ counting only exits avoiding both $B$ and $E$ .                                                               |
| <code>boundary_union_le</code>                          | THM | Boundary of a union is at most the boundary of $B$ plus the induced boundary of $E$ .                                         |
| <code>boundary_le_induced_add_deleted</code>            | THM | Boundary of $E$ is at most its induced boundary plus $ \iota  \cdot  B $ .                                                    |
| <code>maximal_bad_cut_induced_lower</code>              | THM | A maximal bad cut $B$ forces induced expansion of any disjoint $E$ with small union.                                          |
| <code>bad_cut_card_bound</code>                         | THM | Arithmetic bound $b \leq aN/(\gamma - \ell)$ for a bad cut, from the additive expansion estimate.                             |
| <code>large_cut_lower</code>                            | THM | Arithmetic inequality handling the case where $E$ exceeds half the remaining vertices.                                        |
| <code>prune_permutation_multigraph</code>               | THM | Main pruning estimate: after removing the maximal bad cut, every small disjoint set has induced boundary at least $\ell E $ . |
| <code>exists_maximal_bad_cut</code>                     | THM | Chooses a maximal-cardinality set among small sets with boundary below $\ell$ times cardinality.                              |
| <code>disjoint_dominant_intersections_false</code>      | THM | A finite set cannot have majority overlap with each of two disjoint sets.                                                     |
| <code>matchedRetainedSupport</code>                     | DEF | Union of the retained parts $R$ .                                                                                             |
| <code>matchedCore</code>                                | DEF | Union over retained parts of the overlaps $C \cap D(C)$ .                                                                     |

|                                                       |      |                                                                                                             |
|-------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------|
| partitionWordCrossing                                 | DEF  | Points of $U$ that a permutation moves out of their own partition part.                                     |
| matchedWordPreimageBad                                | DEF  | Points of $U$ whose image lands in $U$ minus the matched core.                                              |
| finpartition_dominant_matching_injOn                  | THM  | A dominant matching is injective on retained parts (majority overlap plus disjointness).                    |
| matchedRetainedSupport_subset                         | THM  | The retained support is contained in $U$ .                                                                  |
| matchedCore_subset_retainedSupport                    | THM  | The matched core is contained in the retained support.                                                      |
| matchedCore_subset                                    | THM  | The matched core is contained in $U$ .                                                                      |
| matchedRetainedSupport_card                           | THM  | Cardinality of the retained support is the sum of retained part sizes.                                      |
| matchedCore_card                                      | THM  | Cardinality of the matched core is the sum of overlap sizes.                                                |
| matchedCore_missing_card                              | THM  | Exact count: $U$ minus the core equals discarded support plus the per-part differences $C \setminus D(C)$ . |
| matchedCore_missing_card_le_symmDiff                  | THM  | Bounds the missing core by discarded support plus the symmetric differences $C \Delta D(C)$ .               |
| matched_partition_part_eq_of_target_part_eq           | THM  | On the matched core, equal target parts imply equal source parts, by injectivity of the matching.           |
| matchedWordPreimageBad_card_le                        | THM  | The bad-preimage set is no larger than $U$ minus the matched core.                                          |
| partitionWordCrossing_subset_target_or_unmatched      | THM  | Source crossings are covered by target crossings, the missing core, and its permutation preimage.           |
| matchedCore_missing_density_tendsto_zero              | THM  | If discarded-support and symmetric-difference densities vanish, the missing-core density vanishes.          |
| permutation_increment_sum_eq_zero                     | THM  | The increments $f(px) - fx$ of a permutation sum to zero.                                                   |
| permutation_squared_increment_le_twice_decreasing     | THM  | For $f$ valued in $[0, 1]$ , squared increments sum to at most twice the count of decreasing points.        |
| exists_completion_of_internal_permutation             | THM  | Any permutation extends its $Z$ -internal behavior to a permutation of the subtype $Z$ .                    |
| diagonalRadius                                        | DEF  | Diagonal radius: the greatest $k \leq n$ with error $e(n, k) < 1/(k + 1)$ .                                 |
| diagonalRadius_tendsto_atTop                          | THM  | The diagonal radius tends to infinity when each fixed- $k$ error vanishes.                                  |
| diagonalRadius_eventually_error_lt                    | THM  | Eventually the diagonal error is below $1/(r + 1)$ at the chosen radius.                                    |
| diagonalRadius_error_tendsto_zero                     | THM  | The diagonal error tends to zero along the diagonal radius.                                                 |
| exists_diverging_radius_with_vanishing_diagonal_error | THM  | Diagonal extraction: a radius sequence diverging to infinity with vanishing diagonal error.                 |
| finite_union_bad_density_tendsto_zero                 | THM  | A finite union of asymptotically negligible bad sets remains asymptotically negligible in density.          |
| sum_card_inter_partition                              | THM  | Summing $ C \cap B $ over the parts of a partition gives $ U \cap B $ .                                     |
| retained_bad_density_tendsto_zero                     | THM  | Vanishing bad density on $V$ transfers to a subset $U$ of relative density tending to one.                  |
| LeavittGenerator                                      | IND  | Inductive type of Leavitt generators: $s_i$ and $t_i$ for $i$ in $\text{Fin } 2$ .                          |
| LeavittFree                                           | ABBR | The free $\mathbb{Z}/2$ -algebra on the Leavitt generators.                                                 |
| leavittFreeCountable                                  | INST | Countability of the free algebra, via monoid-algebra coefficients.                                          |
| LeavittRelation                                       | IND  | The relations $t_i s_j = \delta_{ij}$ and $s_0 t_0 + s_1 t_1 = 1$ .                                         |
| binaryLeavittCountable                                | INST | Countability of <code>BinaryLeavitt</code> , from surjectivity of the quotient map.                         |

|                                                     |      |                                                                                                                      |
|-----------------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------|
| <code>leavittQuotient</code>                        | DEF  | The quotient algebra homomorphism from the free algebra onto <code>BinaryLeavitt</code> .                            |
| <code>binaryLeavittFiniteType</code>                | INST | <code>BinaryLeavitt</code> is a finitely generated algebra.                                                          |
| <code>leavittS</code>                               | DEF  | Image of the generator $s_i$ in <code>BinaryLeavitt</code> .                                                         |
| <code>leavittT</code>                               | DEF  | Image of the generator $t_i$ in <code>BinaryLeavitt</code> .                                                         |
| <code>leavittT_mul_S</code>                         | THM  | The relation $t_i s_j = \delta_{ij}$ holds in the quotient.                                                          |
| <code>leavitt_partition</code>                      | THM  | The partition relation $s_0 t_0 + s_1 t_1 = 1$ holds in the quotient.                                                |
| <code>leavittSequence</code>                        | ABBR | The module of $\mathbb{Z}/2$ -valued sequences indexed by the naturals.                                              |
| <code>prefixOperator</code>                         | DEF  | Linear operator interleaving a sequence into the positions congruent to $i$ modulo 2.                                |
| <code>deletionOperator</code>                       | DEF  | Linear operator reading positions $2n + i$ , deleting a leading letter.                                              |
| <code>deletion_mul_prefix</code>                    | THM  | Deletion after prefix is $\delta_{ij}$ on sequence operators.                                                        |
| <code>prefix_deletion_partition</code>              | THM  | The two prefix-deletion compositions sum to the identity operator.                                                   |
| <code>leavittGeneratorAction</code>                 | DEF  | Assigns the prefix and deletion operators to the corresponding Leavitt generators.                                   |
| <code>leavittFreeRepresentation</code>              | DEF  | Lifts the generator action to the free algebra.                                                                      |
| <code>leavittFreeRepresentation_respects</code>     | THM  | The free representation respects both Leavitt relations.                                                             |
| <code>leavittRepresentation</code>                  | DEF  | The induced representation of <code>BinaryLeavitt</code> on sequences, via <code>RingQuot.liftAlgHom</code> .        |
| <code>binaryLeavittNontrivial</code>                | INST | Nontriviality of <code>BinaryLeavitt</code> , together with simp lemmas evaluating the representation on generators. |
| <code>prefix_deletion_ne_one</code>                 | THM  | $s_0 t_0 \neq 1$ as sequence operators, by evaluating at a point.                                                    |
| <code>leavittS_mul_T_ne_one</code>                  | THM  | Transports the previous inequality to <code>BinaryLeavitt</code> : $s_0 t_0 \neq 1$ there.                           |
| <code>binaryLeavittInfinite</code>                  | INST | <code>BinaryLeavitt</code> is infinite: in a finite ring one-sided inverses would be two-sided.                      |
| <code>leavittWordS</code>                           | DEF  | Word monomial $S_a$ : the ordered product of $s_i$ along a binary word.                                              |
| <code>leavittWordT</code>                           | DEF  | Word monomial $T_a$ : the reversed product of $t_i$ along a binary word.                                             |
| <code>leavittWordT_mul_wordS_self</code>            | THM  | $T_a S_a = 1$ for every word $a$ .                                                                                   |
| <code>leavittWordT_mul_wordS_of_incomparable</code> | THM  | $T_a S_b = 0$ when neither word is a prefix of the other.                                                            |
| <code>leavittWordS_append</code>                    | THM  | $S$ -monomials are multiplicative under word concatenation.                                                          |
| <code>leavittWordT_append</code>                    | THM  | $T$ -monomials are anti-multiplicative under word concatenation.                                                     |
| <code>leavittCylinder</code>                        | DEF  | The cylinder idempotent $S_a T_a$ of a binary word.                                                                  |
| <code>leavittCylinder_split</code>                  | THM  | A cylinder splits as the sum of its two one-letter extensions.                                                       |

## 2.4 Matrix algebras as idempotent corners (MatrixCorner)

The `MatrixCorner` namespace proves a purely ring-theoretic fact used to move between matrix algebras and corners. Given families  $s, t : \iota \rightarrow A$  in a ring satisfying  $t_i s_j = \delta_{ij}$ , the element `codeIdempotent`  $e = \sum_i s_i t_i$  is an idempotent, and the maps `encode` (sending a matrix  $M$  to  $\sum_{i,j} s_i M_{ij} t_j$ ) and `decode` (sending  $x$  to the matrix with entries  $t_i x s_j$ ) are mutually inverse between the matrix algebra over  $A$  and the corner  $e A e$ . After verifying `decode_encode`, `encode_decode`, `encode_one` and multiplicativity `encode_mul`, the packaged `ringEquiv` exhibits the matrix algebra as the corner subring. Applied to the word families of a binary prefix code in the next segment, this is how nine-by-nine matrix pictures of the Leavitt algebra arise inside the algebra itself.

**ringEquiv** def

```
def ringEquiv [DecidableEq ι] (s t : ι → A)
  (h : ∀ i j, t i * s j = if i = j then 1 else 0) :
  Matrix ι ι A ≈+* (codeIdempotent_isIdempotent s t h).Corner where
toFun M :=
  (encode s t M, (encode s t M, by
    change
      codeIdempotent s t * encode s t M * codeIdempotent s t =
        encode s t M
    rw [← encode_decode, decode_encode s t h]))
invFun x := decode s t x.1
left_inv M := decode_encode s t h M
right_inv x := Subtype.ext <| by
  change encode s t (decode s t x.1) = x.1
  rw [encode_decode]
have hx :=
  (Subsemigroup.mem_corner_iff
    (codeIdempotent_isIdempotent s t h)).mp x.property
  rw [hx.1, hx.2]
map_mul' M N := Subtype.ext (encode_mul s t h M N)
map_add' M N := Subtype.ext <| by
  change encode s t (M + N) = encode s t M + encode s t N
  simp [encode, Matrix.add_apply, mul_add, add_mul, Finset.sum_add_distrib]
```

`MatrixCorner.ringEquiv` packages the computations of this namespace into a ring isomorphism between  $\text{Matrix } \iota \iota A$  and the corner  $eAe$ , where  $e$  is the idempotent  $\sum_i s_i t_i$  attached to families satisfying  $t_i s_j = \delta_{ij}$ , and the corner is Mathlib’s `IsIdempotentElem.Corner`. The forward map is `encode`, whose image lands in the corner because `encode_decode` identifies  $e \cdot \text{encode}(M) \cdot e$  with the encoding of a decoded round-trip, which `decode_encode` collapses; the inverse is `decode` applied to the underlying element. Additivity is a distributivity computation, and multiplicativity is `encode_mul`: expanding the product of two encoded matrices produces cross factors  $t_j s_k$ , which the delta relation contracts to exactly the matrix-product sum. Conceptually this is the standard fact that a ring containing a family of matrix units built from orthogonal isometries with total range projection  $e$  has corner isomorphic to a matrix algebra over  $A$ . Its role here: for a complete binary prefix code the idempotent is 1, so `binaryPrefixCornerEquiv` realizes the full nine-by-nine matrix algebra over `BinaryLeavitt` inside the Leavitt algebra itself — the self-similarity that powers the compression units and the elementary-group constructions that follow.

*Remaining declarations in this section:*

|                                          |     |                                                                                    |
|------------------------------------------|-----|------------------------------------------------------------------------------------|
| <code>codeIdempotent</code>              | DEF | The idempotent $\sum_i s_i t_i$ attached to families with orthogonality relations. |
| <code>encode</code>                      | DEF | Encodes a matrix as $\sum_{i,j} s_i M_{ij} t_j$ .                                  |
| <code>decode</code>                      | DEF | Decodes a ring element into the matrix with entries $t_i x s_j$ .                  |
| <code>decode_encode</code>               | THM | Decoding after encoding recovers the matrix, given $t_i s_j = \delta_{ij}$ .       |
| <code>encode_decode</code>               | THM | Encoding after decoding is two-sided compression by the code idempotent.           |
| <code>codeIdempotent_isIdempotent</code> | THM | The code element $\sum_i s_i t_i$ is idempotent under the orthogonality relations. |
| <code>encode_one</code>                  | THM | The identity matrix encodes to the code idempotent.                                |
| <code>encode_mul</code>                  | THM | Encoding is multiplicative: cross terms contract via $t_j s_k = \delta_{jk}$ .     |

## 2.5 Prefix codes, compressions, elementary groups

Concrete prefix-code combinatorics inside `BinaryLeavitt`. A `BinaryPrefixCode` is a family of pairwise prefix-incomparable binary words, so the associated word elements satisfy  $T_i S_j = \delta_{ij}$  (`binaryPrefixCode_orthogonal`). The `prefixTable` of two codes,  $\sum_i S_{\text{target}(i)} T_{\text{source}(i)}$ , rewires one code onto the other and is a unit when both

codes are complete (`prefixTableUnit`). Explicit nine-element codes are then built from the word families `alphaWord`, `betaWord`, `nuWord`, `etaWord`: the code `ninePrefixCode` and two rearrangements `uPrefixCode` and `vPrefixCode`, all proved complete by cylinder-splitting computations, giving the units `compressionU` and `compressionV`. Conjugation by these units transports elements  $S_a x T_b$  between codes; specialized to the alpha rows it appends the letter 0 to the words. Finally `prefixElementaryUnit` defines the elementary units  $1 + S_i a T_j$ , `prefixElementaryGroup` the subgroup they generate, and the compressions are shown to conjugate the alpha elementary group onto the `alphaZero` one, which embeds back into the alpha group.

#### **BinaryPrefixCode** `struct`

```
structure BinaryPrefixCode (ι : Type*) where
  word : ι → List (Fin 2)
  prefix_free : ∀ {i j}, i ≠ j → ¬word i <+: word j
```

`BinaryPrefixCode` is a family of binary words  $w_i$  indexed by a type  $\iota$  such that no word is a prefix of a distinct one. Prefix-freeness is exactly what makes the associated Leavitt-algebra elements behave like matrix units: `binaryPrefixCode_orthogonal` shows  $T_{w_i} S_{w_j} = \delta_{ij}$ , combining `leavittWordT_mul_wordS_self` (a word cancels itself) with `leavittWordT_mul_wordS_of_incomparable` (incomparable words annihilate: peel common letters until the first disagreement, where  $t_i s_j = 0$  kills the product). Consequently the families  $S_{w_i}$  and  $T_{w_i}$  satisfy the hypotheses of the `MatrixCorner` machinery, and `binaryPrefixCornerEquiv` identifies matrices over `BinaryLeavitt` indexed by  $\iota$  with the corner cut out by the code idempotent  $\sum_i S_{w_i} T_{w_i}$ , which is the sum of the cylinders of the code words. When the code is complete — its cylinders partition the binary Cantor set, algebraically the idempotent equals 1 — this identifies the matrix algebra with the whole Leavitt algebra. All the specific structures in the rest of the file (the nine-element code, its rearrangements, and their elementary subgroups) instantiate this notion, so this definition fixes the combinatorial data on which the eventual non-sofic group depends.

#### **prefixTableUnit** `def`

```
def prefixTableUnit {ι : Type*} [Fintype ι] [DecidableEq ι]
  (source target : BinaryPrefixCode ι)
  (hsource : MatrixCorner.codeIdempotent
    (fun i => leavittWordS (source.word i))
    (fun i => leavittWordT (source.word i)) = 1)
  (htarget : MatrixCorner.codeIdempotent
    (fun i => leavittWordS (target.word i))
    (fun i => leavittWordT (target.word i)) = 1) : BinaryLeavitt* where
  val := prefixTable source target
  inv := prefixTable target source
  val_inv := (prefixTable_mul_reverse source target).trans htarget
  inv_val := (prefixTable_mul_reverse target source).trans hsource
```

`prefixTableUnit` shows how two complete prefix codes on the same index type yield a unit of the Leavitt algebra. The candidate is `prefixTable`, the sum  $\sum_i S_{\text{target}(i)} T_{\text{source}(i)}$ , which reads off a source-code cylinder and rewrites it as the corresponding target-code cylinder. The computation `prefixTable_mul_reverse` multiplies the table by its reverse: expanding the double sum, the middle factors  $T_{\text{source}(i)} S_{\text{source}(j)}$  contract by orthogonality to  $\delta_{ij}$ , leaving exactly the target code idempotent  $\sum_i S_{\text{target}(i)} T_{\text{target}(i)}$ . When both codes are complete — their idempotents equal 1, as proved for the concrete codes by the cylinder-splitting lemmas `ninePrefixCode_complete`, `uPrefixCode_complete` and `vPrefixCode_complete` — both products are 1, so the table and its reverse form a unit and its inverse. This is the algebraic incarnation of a piecewise transformation of the binary Cantor set carrying the partition into source cylinders onto the partition into target cylinders. Applied to `ninePrefixCode` against `uPrefixCode` and `vPrefixCode`, it yields the units `compressionU` and `compressionV`, whose conjugation action implements the self-similarity (appending a letter to the alpha words) exploited by the elementary-group results closing this chunk.

#### **prefixElementaryUnit** `def`

```

def prefixElementaryUnit {ι : Type*} [DecidableEq ι]
  (E : BinaryPrefixCode ι) (i j : ι) (hij : i ≠ j)
  (a : BinaryLeavitt) : BinaryLeavittx := by
  let x := leavittWordS (E.word i) * a * leavittWordT (E.word j)
  have hzero : leavittWordT (E.word j) * leavittWordS (E.word i) = 0 := by
    simp [hij.symm] using binaryPrefixCode_orthogonal E j i
  have hx : x * x = 0 := by
    dsimp [x]
    calc
      (leavittWordS (E.word i) * a * leavittWordT (E.word j)) *
        (leavittWordS (E.word i) * a * leavittWordT (E.word j)) =
        leavittWordS (E.word i) * a *
          (leavittWordT (E.word j) * leavittWordS (E.word i)) *
            a * leavittWordT (E.word j) := by
              simp [mul_assoc]
      _ = 0 := by rw [hzero]; simp
  exact
    { val := 1 + x
      inv := 1 - x
      val_inv := by noncomm_ring [hx]
      inv_val := by noncomm_ring [hx] }

```

`prefixElementaryUnit` constructs, for a prefix code  $E$ , distinct indices  $i \neq j$ , and an arbitrary coefficient  $a$  in `BinaryLeavitt`, the unit  $1 + x$  where  $x = S_{E(i)} a T_{E(j)}$ . The key computation is  $x^2 = 0$ : squaring produces the inner factor  $T_{E(j)} S_{E(i)}$ , which vanishes by `binaryPrefixCode_orthogonal` since  $i \neq j$ . Hence  $1 + x$  and  $1 - x$  multiply to  $1 - x^2 = 1$  in both orders (verified by `noncomm_ring`), so  $1 + x$  is a unit with explicit inverse — in characteristic two, its own inverse. These are precisely elementary matrices: under the corner isomorphism `binaryPrefixCornerEquiv`,  $x$  corresponds to a matrix with the single off-diagonal entry  $a$  in position  $(i, j)$ , and  $1 + x$  to an elementary transvection. The subgroup of units generated by all of them is `prefixElementaryGroup`, the elementary group of the code — for the nine-element code this is the group whose non-soficity the entire file establishes. Defining elementary units directly from code words rather than through the matrix picture keeps all later conjugation computations, by `compressionU` and `compressionV`, at the level of word algebra.

#### `prefixElementaryGroup` def

```

def prefixElementaryGroup {ι : Type*} [DecidableEq ι]
  (E : BinaryPrefixCode ι) : Subgroup BinaryLeavittx :=
  Subgroup.closure
    {z | ∃ (i j : ι) (h : i ≠ j) (a : BinaryLeavitt),
      prefixElementaryUnit E i j h a = z}

```

`prefixElementaryGroup` is the subgroup of units of `BinaryLeavitt` generated by the elementary units `prefixElementaryUnit` as the distinct indices  $i \neq j$  and the coefficient  $a$  range freely; under the corner equivalence it plays the role of the elementary subgroup of the matrix group over the Leavitt algebra. The closing results of this chunk establish its equivariance and monotonicity. `prefixElementaryGroup_map_conj` is a generic transport lemma: if conjugation by a unit  $g$  carries each elementary unit of code  $E$  to the corresponding elementary unit of code  $E'$ , then the image of the whole subgroup under `MulAut.conj` applied to  $g$  is the elementary group of  $E'$  — immediate from `MonoidHom.map_closure`. Instantiated with the compression units via `compressionU_conjugate_alpha_elementaryUnit` and its  $V$ -analogue (which reduce to the root computations appending the letter 0 to alpha words), conjugation carries the alpha elementary group onto that of `alphaZeroPrefixCode`. Finally `alphaZero_prefixElementaryUnit_eq` rewrites an `alphaZero` elementary unit as an alpha one with coefficient  $s_{0at_0}$ , giving `alphaZero_prefixElementaryGroup_le`: the appended-zero elementary group sits inside the alpha elementary group. Two distinct units compressing the same group into itself is the seed of the Thompson-like structure exploited in later chunks.

*Remaining declarations in this section:*

|                                          |     |                                                                                                             |
|------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------|
| <code>binaryPrefixCode_orthogonal</code> | THM | Code words give matrix-unit relations: $T_{w_i} S_{w_j} = \delta_{ij}$ .                                    |
| <code>binaryPrefixCornerEquiv</code>     | DEF | The matrix algebra over <code>BinaryLeavitt</code> is ring-isomorphic to the corner of the code idempotent. |

|                                   |      |                                                                                                     |
|-----------------------------------|------|-----------------------------------------------------------------------------------------------------|
| prefixTable                       | DEF  | The rewiring element $\sum_i S_{\text{target}(i)} T_{\text{source}(i)}$ between two codes.          |
| prefixTable_mul_reverse           | THM  | A prefix table times its reverse equals the target code idempotent.                                 |
| alphaWord                         | DEF  | Three explicit words $[0, 0, 0]$ , $[0, 0, 1]$ , $[0, 1]$ refining the cylinder of 0.               |
| betaWord                          | DEF  | Three explicit words refining the cylinder $[1, 0]$ .                                               |
| nuWord                            | DEF  | Three explicit words refining the cylinder $[1, 1]$ .                                               |
| etaWord                           | DEF  | Three explicit words refining the cylinder of 1.                                                    |
| nineWord                          | DEF  | Nine words combining the alpha, beta and nu families.                                               |
| uWord                             | DEF  | Nine words: the alpha words extended by 0 and 1, plus the eta words.                                |
| vWord                             | DEF  | The same nine words as uWord, listed in a different order.                                          |
| alphaPrefixCode                   | DEF  | The alpha words as a prefix code, prefix-freeness checked by <b>decide</b> .                        |
| ninePrefixCode                    | DEF  | The nine-word prefix code central to the whole construction.                                        |
| uPrefixCode                       | DEF  | The u-words as a prefix code.                                                                       |
| vPrefixCode                       | DEF  | The v-words as a prefix code.                                                                       |
| alpha_cylinders                   | THM  | The three alpha cylinders sum to the cylinder $[0]$ .                                               |
| beta_cylinders                    | THM  | The three beta cylinders sum to the cylinder $[1, 0]$ .                                             |
| nu_cylinders                      | THM  | The three nu cylinders sum to the cylinder $[1, 1]$ .                                               |
| eta_cylinders                     | THM  | The three eta cylinders sum to the cylinder $[1]$ .                                                 |
| ninePrefixCode_complete           | THM  | The nine code cylinders sum to one: <b>ninePrefixCode</b> is complete.                              |
| uPrefixCode_complete              | THM  | The u code is complete, via cylinder splitting and the eta computation.                             |
| vPrefixCode_complete              | THM  | The v code is complete, being a reordering of the u code.                                           |
| compressionU                      | DEF  | The unit rewiring <b>ninePrefixCode</b> onto <b>uPrefixCode</b> .                                   |
| compressionV                      | DEF  | The unit rewiring <b>ninePrefixCode</b> onto <b>vPrefixCode</b> .                                   |
| binaryLeavittCharP                | INST | <b>BinaryLeavitt</b> has characteristic two.                                                        |
| matrixUnitTransport_root          | THM  | General ring identity: the rewiring sums conjugate $s_i a t_j$ to the target families' matrix unit. |
| prefixTable_transport_root        | THM  | Conjugation by prefix tables transports $S_a x T_b$ from source code words to target code words.    |
| compressionU_conjugate_root       | THM  | Conjugation by <b>compressionU</b> carries nine-code matrix units to u-code matrix units.           |
| compressionV_conjugate_root       | THM  | Conjugation by <b>compressionV</b> carries nine-code matrix units to v-code matrix units.           |
| alphaNineIndex                    | DEF  | Embeds <b>Fin 3</b> into <b>Fin 9</b> by $i \mapsto 3i$ , selecting the alpha rows.                 |
| nineWord_alphaNineIndex           | THM  | The nine word at an alpha index is the alpha word.                                                  |
| uWord_alphaNineIndex              | THM  | The u word at an alpha index is the alpha word with 0 appended.                                     |
| vWord_alphaNineIndex              | THM  | The v word at an alpha index is also the alpha word with 0 appended.                                |
| compressionU_conjugate_alpha_root | THM  | Conjugating alpha matrix units by <b>compressionU</b> appends 0 to both words.                      |
| compressionV_conjugate_alpha_root | THM  | Conjugating alpha matrix units by <b>compressionV</b> appends 0 to both words.                      |
| alphaZeroPrefixCode               | DEF  | The prefix code of alpha words with 0 appended.                                                     |



|                                                          |     |                                                                                                                                |
|----------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------|
| <code>compressionU_conjugate_alpha_elementaryUnit</code> | THM | <code>MulAut.conj</code> by <code>compressionU</code> sends alpha elementary units to <code>alphaZero</code> elementary units. |
| <code>compressionV_conjugate_alpha_elementaryUnit</code> | THM | <code>MulAut.conj</code> by <code>compressionV</code> sends alpha elementary units to <code>alphaZero</code> elementary units. |
| <code>prefixElementaryGroup_map_conj</code>              | THM | If conjugation matches the elementary units of two codes, it maps one elementary group onto the other.                         |
| <code>compressionU_map_alphaPrefixElementaryGroup</code> | THM | Conjugation by <code>compressionU</code> carries the alpha elementary group onto the <code>alphaZero</code> elementary group.  |
| <code>compressionV_map_alphaPrefixElementaryGroup</code> | THM | Conjugation by <code>compressionV</code> carries the alpha elementary group onto the <code>alphaZero</code> elementary group.  |
| <code>alphaZero_prefixElementaryUnit_eq</code>           | THM | An <code>alphaZero</code> elementary unit equals the alpha elementary unit with coefficient <code>s0at0</code> .               |
| <code>alphaZero_prefixElementaryGroup_le</code>          | THM | The <code>alphaZero</code> elementary group is contained in the alpha elementary group.                                        |

### 3 The nine-prefix source group and its compressions

This part constructs the counterexample's working avatar: the elementary group over the Leavitt-style ring of a nine-word binary prefix code, together with the finite family of compression elements that generate it. Two special compressions U and V are realized explicitly as products of elementary swaps, which later pins down a concrete finite generating set with concrete relations.

#### 3.1 Elementary matrix units over noncommutative rings (NoncommutativeElementaryGroup)

Develops elementary matrix units over an arbitrary noncommutative ring  $R$ . For  $i \neq j$ , the transvection  $1 + aE_{ij}$  is a unit with explicit inverse (`elementaryUnit`); these units are additive and injective in the coefficient, and `elementaryGroup` is the subgroup they generate inside the units of the matrix ring, an analogue of  $EL_n(R)$ . Root homomorphisms embed the additive group of  $R$  at each off-diagonal position, giving infiniteness whenever  $R$  is infinite. Two identities close the section: the Steinberg-type commutator relation `elementaryUnit_commutator` with its two-step membership consequence, and the characteristic-two factorization `cylinder_transposition_factorization`, later the key to evaluating cylinder transpositions in the Leavitt algebra. This apparatus underlies every group construction in the chunk.

**elementaryUnit** `def`

```
def elementaryUnit (i j : ι) (h : i ≠ j) (a : R) : (Matrix ι ι R)× where
  val := 1 + Matrix.single i j a
  inv := 1 - Matrix.single i j a
  val_inv := by
    have hx := single_mul_self_eq_zero i j h a
    noncomm_ring [hx]
  inv_val := by
    have hx := single_mul_self_eq_zero i j h a
    noncomm_ring [hx]
```

`elementaryUnit i j h a` is the transvection  $1 + aE_{ij}$  in the unit group of the matrix ring over an arbitrary ring  $R$ : since  $i \neq j$ , the matrix unit squares to zero (`single_mul_self_eq_zero`), so  $1 - aE_{ij}$  is a two-sided inverse, with both unit equations discharged by `noncomm_ring` from the square-zero relation. This is the basic building block of the whole chunk. The units at a fixed position form a copy of the additive group of  $R$  (`elementaryUnit_mul`, packaged as `elementaryRootHom`), which yields infiniteness of `elementaryGroup` for infinite  $R$  (`elementaryGroup_infinite`), and the family over all positions generates `elementaryGroup`, the analogue of  $EL_n(R)$ . After transport along the prefix-code isomorphisms of later segments these units become the `prefixElementaryUnit` elements of the binary Leavitt algebra, whose group is the object eventually proved non-sofic.

### elementaryUnit\_commutator thm

```

theorem elementaryUnit_commutator (i j k : ι)
  (hij : i ≠ j) (hjk : j ≠ k) (hik : i ≠ k) (a b : R) :
  [elementaryUnit i j hij a, elementaryUnit j k hjk b] =
    elementaryUnit i k hik (a * b) := by
-- proof: 19 lines, omitted

```

The Steinberg-type relation: for pairwise distinct indices, the commutator of `elementaryUnit` at  $(i, j, a)$  and  $(j, k, b)$  equals the elementary unit at  $(i, k, ab)$ . The proof expands the commutator into a product of four matrices of the form  $1 \pm aE_{ij}$  and  $1 \pm bE_{jk}$ , records the handful of matrix-unit products that vanish (mismatched inner indices) or collapse ( $aE_{ij} \cdot bE_{jk} = abE_{ik}$ ), and lets `noncomm_ring` finish. This identity is the algebraic engine of the chunk. It immediately gives `elementaryUnit_mem_of_two_step`: a subgroup containing units at  $(i, j, a)$  and  $(j, k, 1)$  contains the unit at  $(i, k, a)$ . It powers the multiplicative closure of `elementaryCoefficientSubalgebra`, hence finite generation of the elementary group, and its prefix-code analogue `prefixElementaryUnit_commutator` drives the hub argument by which `sourceGeneratedGroup` absorbs the whole nine-code elementary group.

*Remaining declarations in this section:*

|                                                   |     |                                                                                                                                            |
|---------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>single_mul_self_eq_zero</code>              | THM | The off-diagonal matrix unit with entry $a$ squares to zero, since its column and row indices mismatch.                                    |
| <code>elementaryUnit_mul</code>                   | THM | The unit at coefficient zero is one, and units at $a$ and $b$ multiply to the unit at $a + b$ .                                            |
| <code>elementaryUnit_injective</code>             | THM | The map $a \mapsto \text{elementaryUnit } i j a$ is injective, read off from the $(i, j)$ matrix entry.                                    |
| <code>elementaryGroup</code>                      | DEF | Defines <code>elementaryGroup</code> as the subgroup of invertible matrices generated by all elementary units.                             |
| <code>elementaryUnit_mem</code>                   | THM | Every elementary unit belongs to <code>elementaryGroup</code> , being a generator of the closure.                                          |
| <code>elementaryRootHom</code>                    | DEF | Packages $a \mapsto \text{elementaryUnit } i j a$ as a homomorphism from <code>Multiplicative R</code> into <code>elementaryGroup</code> . |
| <code>elementaryGroup_infinite</code>             | THM | If $R$ is infinite, <code>elementaryGroup</code> is infinite, via injectivity of a single root embedding.                                  |
| <code>cylinder_transposition_factorization</code> | THM | In characteristic two, $(1 + P)(1 + Q)(1 + P) = 1 - PQ - QP + P + Q$ when $P^2 = 0$ and $PQP = P$ .                                        |
| <code>elementaryUnit_mem_of_two_step</code>       | THM | If a subgroup contains the units at $(i, j, a)$ and $(j, k, 1)$ , the commutator trick yields the unit at $(i, k, a)$ .                    |

## 3.2 Finite generation of elementary groups (FiniteGeneration)

Proves finite generation of the elementary group of  $n \times n$  matrices when the coefficient ring is a finitely generated algebra over  $\mathbb{Z}\text{Mod } 2$  and  $n > 2$ . The proposed generators are the finitely many elementary units with coefficients in an algebra generating set enlarged by one (`finiteElementaryGenerators`). The decisive construction is `elementaryCoefficientSubalgebra`: the coefficients whose elementary units all lie in a given subgroup form a subalgebra, using coefficient additivity for sums and the commutator relation through a third index for products, which is where  $n > 2$  is needed. The case  $n = 3$  is recorded separately for later use with the binary Leavitt algebra, whose elementary group inherits finite generation from this theorem.

### elementaryGroup\_finitelyGenerated thm

```

theorem elementaryGroup_finitelyGenerated
  [Algebra.FiniteType (ZMod 2) R]
  (n : ℕ) (hn : 2 < n) :
  Group.FG (elementaryGroup (Fin n) R) := by
-- proof: 41 lines, omitted

```

States that for a ring  $R$  that is a finitely generated algebra over  $\mathbb{Z}\text{Mod } 2$  and any  $n > 2$ , the group elementary Group over  $n \times n$  matrices is finitely generated. The candidate generating set is `finiteElementaryGenerators`: all elementary units whose coefficient lies in a chosen finite algebra generating set  $s$  enlarged by 1. Let  $H$  be its closure. The pivotal device is `elementaryCoefficientSubalgebra`: the set of coefficients  $a$  all of whose elementary units lie in  $H$  is closed under addition, because coefficients add under multiplication of units, and closed under products by routing through a third index  $k$  (available since  $n > 2$ ) with the commutator relation `elementaryUnit_commutator`. This set is therefore a subalgebra containing  $s$ , hence all of  $R$ , so  $H$  contains every elementary unit and equals the full elementary group. The corollary `elementaryGroup_three_finitelyGenerated` for  $n = 3$  is what the file applies to `BinaryLeavitt`, giving `binaryLeavittEL3_finitelyGenerated` and, through the isomorphisms of the next segment, finite generation of the alpha prefix elementary group.

*Remaining declarations in this section:*

|                                                      |     |                                                                                                                                                |
|------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>finiteElementaryGenerators</code>              | DEF | Finite generating set: all elementary units whose coefficient lies in a finite set $s$ together with one.                                      |
| <code>mem_finiteElementaryGenerators</code>          | THM | Membership characterization of <code>finiteElementaryGenerators</code> : exactly the elementary units with coefficient in $s$ or equal to one. |
| <code>elementaryCoefficientSubalgebra</code>         | DEF | Coefficients whose elementary units all lie in a subgroup $H$ form a $\mathbb{Z}\text{Mod } 2$ subalgebra when $n > 2$ .                       |
| <code>elementaryGroup_three_finitelyGenerated</code> | THM | Specializes the finite generation theorem to $n = 3$ .                                                                                         |

### 3.3 Matrix and prefix group isomorphisms

Bridges matrix elementary groups over `BinaryLeavitt` and prefix-code elementary groups inside its unit group. For a complete prefix code the entire matrix ring is ring-isomorphic to `BinaryLeavitt` via `MatrixCorner.encode`, producing `ninePrefixElementaryGroupEquiv`: `binaryLeavittElementaryGroup 9` is isomorphic to `prefixElementaryGroup ninePrefixCode`. For the incomplete three-word alpha code the matrix ring is only a corner of the algebra; `idempotentCornerUnitExtension` extends corner units by adding  $1 - e$ , and composing with the corner equivalence gives the injective homomorphism `prefixCornerUnitHom` and the isomorphism `alphaPrefixElementaryGroupEquiv` for `binaryLeavittElementaryGroup 3`. Both constructions are verified on generators, mapping elementary units to prefix elementary units, so group-theoretic properties transfer between the matrix picture and the Leavitt picture.

**completePrefixMatrixEquiv** `def`

```
def completePrefixMatrixEquiv {ι : Type*} [Fintype ι] [DecidableEq ι]
  (E : BinaryPrefixCode ι)
  (hcomplete : MatrixCorner.codeIdempotent
    (fun i => leavittWordS (E.word i))
    (fun i => leavittWordT (E.word i)) = 1) :
  Matrix ι ι BinaryLeavitt ==> BinaryLeavitt where
toFun := MatrixCorner.encode
  (fun i => leavittWordS (E.word i))
  (fun i => leavittWordT (E.word i))
invFun := MatrixCorner.decode
  (fun i => leavittWordS (E.word i))
  (fun i => leavittWordT (E.word i))
left_inv := MatrixCorner.decode_encode _ _ (binaryPrefixCode_orthogonal E)
right_inv x := by
  rw [MatrixCorner.encode_decode, hcomplete]
  simp
map_mul' := MatrixCorner.encode_mul _ _ (binaryPrefixCode_orthogonal E)
map_add' M N := by
  simp [MatrixCorner.encode, Matrix.add_apply, mul_add, add_mul,
    Finset.sum_add_distrib]
```

For a binary prefix code  $E$  on a finite index type whose code idempotent  $\sum_i S_i T_i$  equals 1 (a complete code), the full matrix ring over `BinaryLeavitt` is isomorphic, as a ring, to `BinaryLeavitt` itself. The forward map `MatrixCorner.encode` sends a matrix  $M$  to  $\sum_{i,j} S_i M_{ij} T_j$ , where  $S_i$  and  $T_i$  denote `leavittWordS` and `leavittWordT` of the code words; `MatrixCorner.decode` recovers entries by sandwiching between the dual generators. Orthogonality of distinct code words (`binaryPrefixCode_orthogonal`) makes decode after encode the identity, and completeness makes encode after decode the identity. This matrix-absorption property of the Leavitt algebra is what lets a nine-index elementary group live naturally inside `BinaryLeavitt`: passing to unit groups (`completePrefixUnitEquiv`) and tracking generators (`completePrefixUnitEquiv_elementaryUnit`, `completePrefixElementaryGroup_map`) produces `ninePrefixElementaryGroupEquiv`, the isomorphism between `binaryLeavittElementaryGroup 9` and `prefixElementaryGroup ninePrefixCode` through which all later structural results move between the two presentations.

#### `idempotentCornerUnitExtension` def

```
def idempotentCornerUnitExtension {A : Type*} [Ring A]
  {e : A} (he : IsIdempotentElem e) : he.Corner* →* A* where
toFun u :=
  { val := u.val.val + (1 - e)
    inv := u.inv.val + (1 - e)
    val_inv := by
      have hu := (Subsemigroup.mem_corner_iff he).mp u.val.property
      have hv := (Subsemigroup.mem_corner_iff he).mp u.inv.property
      have huv : u.val.val * u.inv.val = e := by
        have h := congrArg (fun z : he.Corner => z.val) u.val_inv
        exact h
      noncomm_ring [he.eq, hu.1, hu.2, hv.1, hv.2, huv]
    inv_val := by
      have hu := (Subsemigroup.mem_corner_iff he).mp u.val.property
      have hv := (Subsemigroup.mem_corner_iff he).mp u.inv.property
      have hvu : u.inv.val * u.val.val = e := by
        have h := congrArg (fun z : he.Corner => z.val) u.inv_val
        exact h
      noncomm_ring [he.eq, hu.1, hu.2, hv.1, hv.2, hvu] }
map_one' := by
  apply Units.ext
  change e + (1 - e) = 1
  noncomm_ring
map_mul' u v := by
  apply Units.ext
  have hu := (Subsemigroup.mem_corner_iff he).mp u.val.property
  have hv := (Subsemigroup.mem_corner_iff he).mp v.val.property
  change
    u.val.val * v.val.val + (1 - e) =
      (u.val.val + (1 - e)) * (v.val.val + (1 - e))
  noncomm_ring [he.eq, hu.1, hu.2, hv.1, hv.2]
```

Given an idempotent  $e$  in a ring  $A$ , every unit  $u$  of the corner ring  $eAe$  (whose identity is  $e$ ) extends to the unit  $u + (1 - e)$  of  $A$ ; the assignment is a monoid homomorphism, checked by `noncomm_ring` from the corner equations  $ex = x = xe$ , and it is injective because the summand  $1 - e$  cancels (`idempotentCornerUnitExtension_injective`). This handles the case the completeness argument cannot: the three-word `alphaPrefixCode` is not complete, its cylinders sum to an idempotent strictly below 1, so the  $3 \times 3$  matrix ring is only isomorphic to a corner of `BinaryLeavitt`. Composing the corner equivalence `binaryPrefixCornerEquiv` with this extension yields `prefixCornerUnitHom`, an injective homomorphism carrying elementary units to prefix elementary units (`prefixCornerUnitHom_elementaryUnit`) and inducing `alphaPrefixElementaryGroupEquiv`, through which finite generation of `binaryLeavittElementaryGroup 3` is transported to the alpha prefix elementary group in `alphaPrefixElementaryGroup_finitelyGenerated`.

*Remaining declarations in this section:*

|                                           |      |                                                                                                                                              |
|-------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>binaryLeavittElementaryGroup</code> | ABBR | Abbreviation: <code>binaryLeavittElementaryGroup n</code> is the elementary group of $n \times n$ matrices over <code>BinaryLeavitt</code> . |
|-------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------|

|                                                      |     |                                                                                                                                         |
|------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>completePrefixUnitEquiv</code>                 | DEF | Transfers <code>completePrefixMatrixEquiv</code> to a multiplicative equivalence of unit groups.                                        |
| <code>completePrefixUnitEquiv_elementaryUnit</code>  | THM | The unit equivalence carries each <code>elementaryUnit</code> to the corresponding <code>prefixElementaryUnit</code> of the code.       |
| <code>completePrefixElementaryGroup_map</code>       | THM | Under the unit equivalence, the image of <code>elementaryGroup</code> is exactly <code>prefixElementaryGroup</code> of the code.        |
| <code>ninePrefixElementaryGroupEquiv</code>          | DEF | Group isomorphism between <code>binaryLeavittElementaryGroup 9</code> and <code>prefixElementaryGroup ninePrefixCode</code> .           |
| <code>idempotentCornerUnitExtension_injective</code> | THM | The corner-unit extension is injective, by cancelling the common summand $1 - e$ .                                                      |
| <code>prefixCornerUnitHom</code>                     | DEF | Composes the prefix corner equivalence with the idempotent extension, mapping matrix units into <code>BinaryLeavitt</code> units.       |
| <code>prefixCornerUnitHom_injective</code>           | THM | <code>prefixCornerUnitHom</code> is injective, as a composition of injective maps.                                                      |
| <code>prefixCornerUnitHom_elementaryUnit</code>      | THM | <code>prefixCornerUnitHom</code> sends each <code>elementaryUnit</code> to the corresponding <code>prefixElementaryUnit</code> .        |
| <code>prefixCornerElementaryGroup_map</code>         | THM | The image of <code>elementaryGroup</code> under <code>prefixCornerUnitHom</code> equals <code>prefixElementaryGroup</code> of the code. |
| <code>alphaPrefixElementaryGroupEquiv</code>         | DEF | Group isomorphism between <code>binaryLeavittElementaryGroup 3</code> and <code>prefixElementaryGroup alphaPrefixCode</code> .          |

### 3.4 Generating the nine-code group from compressions (SourceGeneration)

Shows the nine-code prefix elementary group is generated by the alpha-code elementary group together with the two compression units. `sourceGeneratedGroup` is the closure of these generators. Word-splitting lemmas express how the compressions interact with alpha word generators, and the block sandwich identity collapses conjugates: conjugation by `compressionU` turns alpha-code units into nine-code units at alpha and beta indices, conjugation by `compressionV` reaches the nu indices. The prefix-code commutator relation, its two-step consequence, and the hub criterion `prefixElementaryGroup_le_of_hub` at index zero then absorb every nine-code elementary unit into `sourceGeneratedGroup`. Conversely, alpha units are nine-code units at the alpha indices, so once both compressions are shown to lie in the nine-code group (done in the following sections) the two groups are equal, the step that will transfer finite generation to the nine-code group.

**sourceGeneratedGroup** `def`

```
def sourceGeneratedGroup : Subgroup BinaryLeavitt* :=
  Subgroup.closure
    ((prefixElementaryGroup alphaPrefixCode : Set BinaryLeavitt*) u
     {compressionU, compressionV})
```

`sourceGeneratedGroup` is the subgroup of `BinaryLeavitt` units generated by the alpha-code prefix elementary group together with the two units `compressionU` and `compressionV`. It is the compact generating description around which this whole segment revolves: the alpha elementary group is known (by `alphaPrefixElementaryGroup_finitelyGenerated` in the next segment) to be finitely generated, so if the nine-code prefix elementary group coincides with `sourceGeneratedGroup`, it too is generated by finitely many alpha generators plus two compressions. The rest of the section proves the inclusion of the nine-code group into this closure unconditionally (`ninePrefixElementaryGroup_le_sourceGenerated`, via conjugation and the hub criterion) and the reverse inclusion conditionally on the two compressions lying in the nine-code group (`sourceGeneratedGroup_le_nine_of_compressions`); those two membership facts are exactly what the `PrefixCompressionU` and `PrefixCompressionV` sections later supply. The definition thus sets up the reduction of a structural generation statement to two concrete unit memberships.

**alphaBetaRoot\_mem\_sourceGenerated** `thm`

```

theorem alphaBetaRoot_mem_sourceGenerated
  (i j : Fin 3) (hij : i ≠ j) (p q : Fin 2) (a : BinaryLeavitt) :
  prefixElementaryUnit ninePrefixCode
    (alphaBetaNineIndex p i) (alphaBetaNineIndex q j)
    (alphaBetaNineIndex_ne hij p q) a ∈ sourceGeneratedGroup := by
  -- proof: 52 lines, omitted

```

For distinct blocks  $i \neq j$  in `Fin 3` and letters  $p, q$ , the nine-code elementary unit at indices `alphaBetaNineIndex p i` and `alphaBetaNineIndex q j` with coefficient  $a$  belongs to `sourceGeneratedGroup`. The construction conjugates the alpha-code unit with the modified coefficient  $S_p a T_q$  by `compressionU`. Two computational lemmas expand the conjugation: `compressionU_inv_mul_alphaWordS` rewrites the inverse compression acting on an alpha word generator as a sum of alpha and beta refinements, and `alphaWordT_mul_compressionU` does the dual; then `binaryBlockSandwich` collapses the resulting three-factor product to the single term  $S(w) a T(w')$ , where  $w$  and  $w'$  are the alpha or beta words selected by  $p$  and  $q$ . Thus conjugation by a compression unit transforms coarse alpha-code elementary units into elementary units over six of the nine code words; the parallel result `alphaNuRoot_mem_sourceGenerated`, conjugating by `compressionV`, reaches the remaining nu words. Together they feed the exhaustive hub computations `nineRoot_to_alphaHub_mem_sourceGenerated` and `nineRoot_from_alphaHub_mem_sourceGenerated`.

#### **sourceGeneratedGroup\_eq\_nine\_of\_compressions thm**

```

theorem sourceGeneratedGroup_eq_nine_of_compressions
  (hu : compressionU ∈ prefixElementaryGroup ninePrefixCode)
  (hv : compressionV ∈ prefixElementaryGroup ninePrefixCode) :
  sourceGeneratedGroup = prefixElementaryGroup ninePrefixCode :=
  le_antisymm (sourceGeneratedGroup_le_nine_of_compressions hu hv)
  ninePrefixElementaryGroup_le_sourceGenerated

```

Under the hypotheses that `compressionU` and `compressionV` lie in `prefixElementaryGroup ninePrefixCode`, this theorem concludes that `sourceGeneratedGroup` equals that group. One direction is `sourceGeneratedGroup_le_nine_of_compressions`: alpha elementary units are literally nine-code elementary units at the alpha indices (`alphaPrefixElementaryUnit_eq_nine`, giving `alphaPrefixElementaryGroup_le_nine`), and the compressions are members by hypothesis. The other direction, `ninePrefixElementaryGroup_le_sourceGenerated`, holds unconditionally: the hub criterion `prefixElementaryGroup_le_of_hub` at index 0 reduces the inclusion to elementary units into and out of the hub, and those are supplied by the exhaustive case analyses built from compression conjugation and two-step commutator moves. The hypotheses are subsequently discharged by `compressionU_mem_ninePrefixElementaryGroup` and `compressionV_mem_of_compressionU`, so the nine-code prefix elementary group is generated by the alpha group and two extra units; combined with `alphaPrefixElementaryGroup_finitelyGenerated` this exhibits the group at the heart of the non-soficity argument as finitely generated.

*Remaining declarations in this section:*

|                                               |     |                                                                                                                              |
|-----------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------|
| <code>alphaRoot_mem_sourceGenerated</code>    | THM | Every alpha-code elementary unit lies in <code>sourceGeneratedGroup</code> by construction.                                  |
| <code>compressionU_mem_sourceGenerated</code> | THM | <code>compressionU</code> is one of the declared generators of <code>sourceGeneratedGroup</code> .                           |
| <code>compressionV_mem_sourceGenerated</code> | THM | <code>compressionV</code> is one of the declared generators of <code>sourceGeneratedGroup</code> .                           |
| <code>prefixTable_mul_wordS</code>            | THM | The prefix table sends each source code word generator <code>leavittWordS</code> to the corresponding target word generator. |
| <code>wordT_mul_prefixTable</code>            | THM | Dually, target word <code>leavittWordT</code> elements multiplied by the prefix table give source word elements.             |
| <code>leavittWordS_split</code>               | THM | Splits <code>leavittWordS</code> of a word into its two one-letter extensions using the Leavitt partition relation.          |
| <code>leavittWordT_split</code>               | THM | Splits <code>leavittWordT</code> of a word into its two one-letter extensions, dual to the previous lemma.                   |

|                                                           |     |                                                                                                                                                                     |
|-----------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>betaNineIndex</code>                                | DEF | Index function placing the beta word of block $i$ at position $3i + 1$ in the nine code.                                                                            |
| <code>nuNineIndex</code>                                  | DEF | Index function placing the nu word of block $i$ at position $3i + 2$ in the nine code.                                                                              |
| <code>compressionU_inv_mul_alphaWordsS</code>             | THM | Computes the inverse of <code>compressionU</code> acting on an alpha word generator, producing alpha and beta refinements.                                          |
| <code>alphaWordT_mul_compressionU</code>                  | THM | Computes <code>leavittWordT</code> of an alpha word times <code>compressionU</code> , producing dual alpha and beta refinements.                                    |
| <code>alphaBetaNineIndex</code>                           | DEF | Index $3i + p$ selecting either the alpha ( $p = 0$ ) or beta ( $p = 1$ ) word of block $i$ .                                                                       |
| <code>alphaBetaNineIndex_ne</code>                        | THM | Distinct blocks give distinct <code>alphaBetaNineIndex</code> values, by arithmetic on the index formula.                                                           |
| <code>binaryBlockSandwich</code>                          | THM | Sandwich identity collapsing a split-generator product to the single term selected by the letters $p$ and $q$ .                                                     |
| <code>prefixElementaryEntry</code>                        | DEF | Defines <code>prefixElementaryEntry</code> : source word generator times coefficient times dual target word generator.                                              |
| <code>prefixElementaryEntry_mul</code>                    | THM | Products of prefix elementary entries compose like matrix units, with a Kronecker delta on the middle indices.                                                      |
| <code>prefixElementaryUnit_commutator</code>              | THM | Prefix-code analogue of the commutator identity: commutators of prefix elementary units multiply their coefficients.                                                |
| <code>prefixElementaryUnit_mem_of_two_step</code>         | THM | Two-step membership: from units at $(i, j, a)$ and $(j, k, 1)$ in $H$ , the unit at $(i, k, a)$ follows.                                                            |
| <code>prefixElementaryGroup_le_of_hub</code>              | THM | Hub criterion: if all elementary units into and out of one index lie in $H$ , the whole <code>prefixElementaryGroup</code> does.                                    |
| <code>compressionV_inv_mul_alphaWordsS</code>             | THM | Analogue of <code>compressionU_inv_mul_alphaWordsS</code> for <code>compressionV</code> , producing alpha and nu refinements.                                       |
| <code>alphaWordT_mul_compressionV</code>                  | THM | Analogue of <code>alphaWordT_mul_compressionU</code> for <code>compressionV</code> , with nu words replacing beta words.                                            |
| <code>alphaNuNineIndex</code>                             | DEF | Index $3i + 2p$ selecting either the alpha ( $p = 0$ ) or nu ( $p = 1$ ) word of block $i$ .                                                                        |
| <code>alphaNuNineIndex_ne</code>                          | THM | Distinct blocks give distinct <code>alphaNuNineIndex</code> values.                                                                                                 |
| <code>alphaNuRoot_mem_sourceGenerated</code>              | THM | Conjugating alpha elementary units by <code>compressionV</code> yields nine-code units at alpha and nu indices, inside <code>sourceGeneratedGroup</code> .          |
| <code>nineRoot_to_alphaHub_mem_sourceGenerated</code>     | THM | Case analysis over all nine indices: every elementary unit into hub index zero lies in <code>sourceGeneratedGroup</code> .                                          |
| <code>nineRoot_from_alphaHub_mem_sourceGenerated</code>   | THM | Symmetric case analysis: every elementary unit out of hub index zero lies in <code>sourceGeneratedGroup</code> .                                                    |
| <code>ninePrefixElementaryGroup_le_sourceGenerated</code> | THM | Combines the hub criterion with both hub lemmas: <code>prefixElementaryGroup</code> <code>ninePrefixCode</code> is contained in <code>sourceGeneratedGroup</code> . |
| <code>alphaNineIndex_ne</code>                            | THM | Distinct blocks give distinct <code>alphaNineIndex</code> values.                                                                                                   |
| <code>alphaPrefixElementaryUnit_eq_nine</code>            | THM | An alpha-code elementary unit equals the nine-code elementary unit at the corresponding alpha indices.                                                              |
| <code>alphaPrefixElementaryGroup_le_nine</code>           | THM | Hence <code>prefixElementaryGroup</code> <code>alphaPrefixCode</code> is contained in <code>prefixElementaryGroup</code> <code>ninePrefixCode</code> .              |
| <code>sourceGeneratedGroup_le_nine_of_compressions</code> | THM | If both compressions lie in the nine-code elementary group, then <code>sourceGeneratedGroup</code> is contained in it.                                              |



### 3.5 Countability facts and midrank variance toolkit

Two independent developments. First, bookkeeping: subgroups of matrix units over the countable ring `BinaryLeavitt` are countable; the three-by-three elementary group is finitely generated and infinite; these properties transfer along the previous isomorphisms to the alpha and nine prefix elementary groups. Second, a self-contained combinatorial toolkit. Midrank moments and variance of a list of masses are defined recursively and computed in closed form; when all masses are at most  $m$ , the variance is at least  $(1 - m)/12$ , with a weighted summed version. Rank masses and vertex midranks of an integer labeling on finite components are then introduced, culminating in `actual_weighted_midrank_dominant_mass_le`: the number of vertices missing their component's dominant label is at most twelve times the summed size-weighted midrank variances. Finally, `maximumOverlapPart` selects a partition part of maximal overlap with a given set. These tools serve the later expander and transport arguments.

**midrankVariance** `def`

```
noncomputable def midrankVariance (ps : List ℝ) : ℝ :=
  midrankSecondMoment 0 ps - (midrankFirstMoment 0 ps) ^ 2
```

`midrankVariance` is the variance of the midrank statistic of a list of masses: `midrankFirstMoment` walks the list with a running offset  $a$ , adding  $p(a + p/2)$  for each block, the center of an interval of length  $p$  laid down consecutively, and `midrankSecondMoment` adds  $p(a + p/2)^2$ . The closed forms `midrankFirstMoment_eq` and `midrankSecondMoment_eq` telescope these recursions by induction with a ring step, and for lists summing to 1 the variance simplifies to  $(1 - \sum p^3)/12$  (`midrankVariance_eq`). The point of the definition is the chain of inequalities that follows: since  $\sum p^3 \leq m$  whenever every mass is at most  $m$  (`cube_sum_le_dominant_mass`), one gets  $(1 - m)/12 \leq \text{midrankVariance}$  (`midrankVariance_controls_dominant_mass`), with a weighted summed form in `weighted_midrank_dominant_mass_le`. A distribution without a dominant atom must therefore have midrank variance bounded away from zero, which is the quantitative mechanism behind the counting bound of this segment.

**componentVertexMidrank** `def`

```
def componentVertexMidrank {V : Type*}
  (C : Finset V) (b : V → ℤ) (x : V) : ℝ :=
  (((C.filter fun z => b z < b x).card : ℝ) +
   ((C.filter fun z => b z = b x).card : ℝ) / 2) /
  (C.card : ℝ)
```

`componentVertexMidrank` assigns to a vertex  $x$  of a finite component  $C$ , labeled by an integer function  $b$ , its normalized midrank: the number of vertices with strictly smaller label plus half the number sharing its label, divided by the size of  $C$ . The identity `componentVertexMidrank_eq_sum_componentRankMass` rewrites it as the total rank mass strictly below the vertex's value plus half the mass at its own value, precisely the midrank of the vertex's label under the distribution `componentRankMass`. Supporting lemmas show the statistic lies in  $[0, 1]$  (`componentVertexMidrank_nonneg`, `componentVertexMidrank_le_one`, via a disjointness and cardinality comparison) and is monotone in the label (`componentVertexMidrank_mono`). This furnishes a bounded, monotone observable on vertices, canonically attached to any integer labeling of a component, whose distribution over the component is captured by the sorted mass list `componentRankMassList`; it is the observable whose variance the later transport argument controls with spectral-gap estimates.

**actual\_weighted\_midrank\_dominant\_mass\_le** `thm`

```
theorem actual_weighted_midrank_dominant_mass_le
  {V : Type*}
  (I : Finset V) (C : V → Finset V) (b : V → ℤ) (j : V → ℤ)
  (hC : ∀ i ∈ I, (C i).Nonempty)
  (hmax : ∀ i ∈ I, ∀ k ∈ (C i).image b,
    componentRankMass (C i) b k ≤ componentRankMass (C i) b (j i)) :
  (∑ i ∈ I,
    (((C i).card - ((C i).filter fun x => b x = j i).card : ℕ) : ℝ)) ≤
```



```

12 *  $\sum_{i \in I}$ ,
  ((C i).card :  $\mathbb{R}$ ) *
  midrankVariance (componentRankMassList (C i) b) := by
-- proof: 25 lines, omitted

```

The main counting output of the toolkit: given finitely many nonempty components  $C_i$  with an integer labeling  $b$  and, for each, a label  $j_i$  of maximal rank mass, the total number of vertices whose label differs from their component's dominant value satisfies  $\sum_i (|C_i| - |\{x \in C_i : b(x) = j_i\}|) \leq 12 \sum_i |C_i| \cdot V_i$ , where  $V_i$  is the `midrankVariance` of the component's sorted mass list. The proof rewrites each summand as  $|C_i|(1 - m_i)$  with  $m_i$  the dominant mass (`component_omitted_rank_card_eq`) and applies the weighted inequality `weighted_midrank_dominant_mass_le`; its hypotheses, that the mass lists are nonnegative, sum to one, and are entrywise bounded by the dominant mass, come from `componentRankMassList_sum`, `componentRankMassList_nonneg`, and `componentRankMassList_le_max`, with the maximal value supplied by `exists_maximal_componentRankMass`. It is a purely combinatorial bridge: any later upper bound on the summed midrank variances forces almost all vertices of the components to carry the dominant label.

*Remaining declarations in this section:*

|                                                              |      |                                                                                                                                         |
|--------------------------------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>binaryLeavittMatrixUnitSubgroup_countable</code>       | THM  | Any subgroup of the units of finite matrices over <code>BinaryLeavitt</code> is countable, since the matrix ring is countable.          |
| <code>binaryLeavittElementaryGroupCountable</code>           | INST | Instance: <code>binaryLeavittElementaryGroup n</code> is countable.                                                                     |
| <code>binaryLeavittEL3_finitelyGenerated</code>              | THM  | <code>binaryLeavittElementaryGroup 3</code> is finitely generated, from the general finite generation theorem.                          |
| <code>binaryLeavittEL3_infinite</code>                       | THM  | <code>binaryLeavittElementaryGroup 3</code> is infinite, via a single infinite root subgroup.                                           |
| <code>alphaPrefixElementaryGroup_finitelyGenerated</code>    | THM  | Transfers finite generation across <code>alphaPrefixElementaryGroupEquiv</code> to <code>prefixElementaryGroup alphaPrefixCode</code> . |
| <code>ninePrefixElementaryGroup_countable</code>             | THM  | <code>prefixElementaryGroup ninePrefixCode</code> is countable, transported through <code>ninePrefixElementaryGroupEquiv</code> .       |
| <code>midrankFirstMoment</code>                              | DEF  | Recursive first moment of the midrank distribution of a list of masses, starting at offset $a$ .                                        |
| <code>midrankSecondMoment</code>                             | DEF  | Recursive second moment of the same midrank distribution.                                                                               |
| <code>midrankFirstMoment_eq</code>                           | THM  | Closed form: the first moment telescopes to $((a + \sum p)^2 - a^2)/2$ .                                                                |
| <code>midrankSecondMoment_eq</code>                          | THM  | Closed form: the second moment equals $((a + \sum p)^3 - a^3)/3 - \sum p^3/12$ .                                                        |
| <code>midrankVariance_eq</code>                              | THM  | For mass lists summing to one, the variance equals $(1 - \sum p^3)/12$ .                                                                |
| <code>cubeSum_le_dominant_mul_sum</code>                     | THM  | Cube sum bound: $\sum p^3 \leq m \sum p$ when entries lie in $[0, 1]$ below $m$ .                                                       |
| <code>cube_sum_le_dominant_mass</code>                       | THM  | Specialization to lists summing to one: $\sum p^3 \leq m$ when every entry is at most $m$ .                                             |
| <code>midrankVariance_controls_dominant_mass</code>          | THM  | Key inequality: $(1 - m)/12 \leq \text{midrankVariance}$ when every mass is at most $m$ .                                               |
| <code>weighted_midrank_dominant_mass_le</code>               | THM  | Weighted version: total weighted dominant-mass defect is bounded by twelve times the total weighted midrank variance.                   |
| <code>componentRankMass</code>                               | DEF  | Fraction of a finite component $C$ whose label $b$ equals $j$ .                                                                         |
| <code>componentVertexMidrank_eq_sum_componentRankMass</code> | THM  | The vertex midrank equals the rank mass strictly below its value plus half the mass at its value.                                       |
| <code>componentRankMass_nonneg</code>                        | THM  | Rank masses are nonnegative.                                                                                                            |
| <code>sum_componentRankMass</code>                           | THM  | The rank masses over the image of $b$ sum to one for nonempty components.                                                               |
| <code>componentVertexMidrank_nonneg</code>                   | THM  | Vertex midranks are nonnegative.                                                                                                        |

|                                                 |     |                                                                                                      |
|-------------------------------------------------|-----|------------------------------------------------------------------------------------------------------|
| <code>component_lower_equal_disjoint</code>     | THM | The strictly-lower and equal-rank fibers of a vertex are disjoint subsets of the component.          |
| <code>componentVertexMidrank_le_one</code>      | THM | Vertex midranks are at most one, by a cardinality comparison.                                        |
| <code>componentVertexMidrank_mono</code>        | THM | Vertex midrank is monotone in the label value.                                                       |
| <code>componentRankMassList</code>              | DEF | The list of rank masses of a component, indexed by the sorted image of $b$ .                         |
| <code>sum_map_sort_eq</code>                    | THM | Summing a function over a sorted finset list agrees with the finset sum.                             |
| <code>componentRankMassList_sum</code>          | THM | The rank mass list of a nonempty component sums to one.                                              |
| <code>componentRankMassList_nonneg</code>       | THM | All entries of the rank mass list are nonnegative.                                                   |
| <code>componentRankMassList_le_max</code>       | THM | Every entry of the rank mass list is bounded by any maximal rank mass.                               |
| <code>exists_maximal_componentRankMass</code>   | THM | A nonempty component admits a label value of maximal rank mass.                                      |
| <code>component_omitted_rank_card_eq</code>     | THM | The number of vertices avoiding rank $j$ equals the component size times one minus its rank mass.    |
| <code>exists_maximum_overlap_component</code>   | THM | Every nonempty subset of a partitioned finset admits a part maximizing the intersection cardinality. |
| <code>sum_card_component_inter_partition</code> | THM | The intersection cardinalities of $C$ with all parts of the partition sum to the size of $C$ .       |
| <code>maximumOverlapPart</code>                 | DEF | Selects, noncomputably, a partition part of maximal overlap with a given set.                        |
| <code>maximumOverlapPart_mem</code>             | THM | The chosen maximum overlap part is indeed a part of the partition.                                   |
| <code>maximumOverlapPart_maximal</code>         | THM | The chosen part maximizes the intersection cardinality among all parts.                              |

### 3.6 Compression U realized by elementary swaps (PrefixCompressionU)

Proves that `compressionU` itself lies in `prefixElementaryGroup ninePrefixCode`. Cross roots, elementary units whose coefficient is a word generator times a dual word generator, combine into cross swaps: transpositions of two refined cylinders whose ring value is computed with the characteristic-two factorization. A nineteen-word complete refinement `refinedSourcePrefixCode` of the nine code is constructed so that `compressionU` sends each refined source generator to the parent's  $u$  word extended by the same suffix. A chronological list of twenty-one cross swaps is then defined; its product `elementaryCompressionU` is shown by exhaustive word-cancellation computation to act identically on all nineteen generators, and the extensionality principle `completePrefixWord_ext` over the complete refined code concludes `elementaryCompressionU_eq_compressionU`, hence the desired membership `compressionU_mem_ninePrefixElementaryGroup`.

**sourceCrossSwap\_val** *thm*

```
theorem sourceCrossSwap_val (i j : Fin 9) (h : i ≠ j)
  (a b : List (Fin 2)) :
  (↑(sourceCrossSwap i j h a b) : BinaryLeavitt) =
    1 - leavittWordS (nineWord i ++ a) *
      leavittWordT (nineWord i ++ a) -
      leavittWordS (nineWord j ++ b) *
        leavittWordT (nineWord j ++ b) +
      leavittWordS (nineWord i ++ a) *
        leavittWordT (nineWord j ++ b) +
      leavittWordS (nineWord j ++ b) *
        leavittWordT (nineWord i ++ a) := by
  -- proof: 74 lines, omitted
```

Computes the ring value of `sourceCrossSwap`: with  $A$  the nine-code word of index  $i$  extended by  $a$  and  $B$

the word of  $j$  extended by  $b$ , the swap equals  $1 - S_A T_A - S_B T_B + S_A T_B + S_B T_A$ . Setting  $P = S_A T_B$  and  $Q = S_B T_A$ , orthogonality of refinements of distinct nine-code words (`sourceRefinementOrthogonal`) gives  $P^2 = 0$ , while the isometry relation `leavittWordT_mul_words_self` gives  $PQ = S_A T_A$ ,  $QP = S_B T_B$ , and  $PQP = P$ ; the characteristic-two identity `cylinder_transposition_factorization` from the first segment then evaluates the triple product  $(1 + P)(1 + Q)(1 + P)$ . The resulting element acts as a transposition of the cylinders  $A$  and  $B$ , fixing everything orthogonal to both: an elementary-group realization of a prefix exchange. Its action formula on arbitrary word generators (`sourceCrossSwap_mul_leavittWords`), phrased through `binaryWordCancellation`, is what makes the later exhaustive verification against `compressionU` a finite, decidable computation.

**completePrefixWord\_ext** thm

```
theorem completePrefixWord_ext {ι : Type*} [Fintype ι]
  (E : BinaryPrefixCode ι)
  (hcomplete : MatrixCorner.codeIdempotent
    (fun i => leavittWordS (E.word i))
    (fun i => leavittWordT (E.word i)) = 1)
  (x y : BinaryLeavitt)
  (haction : ∀ i : ι,
    x * leavittWordS (E.word i) =
      y * leavittWordS (E.word i)) :
  x = y := by
-- proof: 27 lines, omitted
```

An extensionality principle for complete prefix codes: if the code idempotent equals one and two elements  $x, y$  of `BinaryLeavitt` satisfy  $xS_i = yS_i$  for every code word generator, then  $x = y$ . The proof multiplies  $x$  by  $1 = \sum_i S_i T_i$ , reassociates so each summand contains  $xS_i$ , replaces it by  $yS_i$ , and reassembles  $y$ . Its significance is strategic: equality of two units of the infinite-dimensional Leavitt algebra is reduced to finitely many equations, one for each word of a complete code. Applied with the nineteen-word `refinedSourcePrefixCode`, proved complete in `refinedSourcePrefixCode_complete` by merging cylinder splits step by step down to the nine-code cylinders, it turns the identification of the chronological swap product with `compressionU` into the exhaustive, decidable computation carried out in `elementaryCompressionU_mul_refinedSourceWord`. The same pattern reappears in the next section as the completeness argument identifying the transition table with a product of parent swaps.

**elementaryCompressionU\_eq\_compressionU** thm

```
theorem elementaryCompressionU_eq_compressionU :
  elementaryCompressionU = compressionU := by
-- proof: 12 lines, omitted
```

The culmination of the section: the product `elementaryCompressionU` of the twenty-one chronological cross swaps equals `compressionU`. By the extensionality principle `completePrefixWord_ext` over the complete nineteen-word refined code, it suffices to show both units act identically on each refined source word generator. For `compressionU` this is structural: each refined word is its nine-code parent followed by a suffix (`refinedSourceWord_eq_parent_append`, checked by `decide`), and the compression, being the prefix table from the nine code to the  $u$  code, replaces the parent by the corresponding  $u$  word while keeping the suffix (`compressionU_mul_refinedSourceWord` via `compressionU_mul_nine_refinement`). For the swap product it is the exhaustive computation `elementaryCompressionU_mul_refinedSourceWord`, which pushes each generator through all twenty-one swaps using the explicit action formula `sourceCrossSwap_mul_leavittWords` and word cancellation. Since every cross swap lies in `prefixElementaryGroup_ninePrefixCode`, the equality yields `compressionU_mem_ninePrefixElementaryGroup`, the first of the two membership hypotheses required by `sourceGeneratedGroup_eq_nine_of_compressions`.

*Remaining declarations in this section:*

|                                  |     |                                                                                                                   |
|----------------------------------|-----|-------------------------------------------------------------------------------------------------------------------|
| <code>sourceCrossRoot</code>     | DEF | Elementary unit of the nine code whose coefficient is a source word generator times a dual target word generator. |
| <code>sourceCrossRoot_mem</code> | THM | Cross roots belong to <code>prefixElementaryGroup_ninePrefixCode</code> as generators.                            |

|                                                           |     |                                                                                                                                         |
|-----------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>sourceCrossSwap</code>                              | DEF | Computes the cross root value and defines the triple product <code>sourceCrossSwap</code> designed to transpose two refined cylinders.  |
| <code>sourceCrossSwap_mem</code>                          | THM | Cross swaps lie in <code>prefixElementaryGroup ninePrefixCode</code> , being products of members.                                       |
| <code>sourceRefinementOrthogonal</code>                   | THM | Refinements of distinct nine-code words stay orthogonal: the dual generator times the other generator vanishes.                         |
| <code>refinedSourceWord</code>                            | DEF | Nineteen explicit binary words refining the nine code, adapted to the action of <code>compressionU</code> .                             |
| <code>refinedSourcePrefixCode</code>                      | DEF | Packages the nineteen words as a prefix code; prefix freeness is checked by <code>decide</code> .                                       |
| <code>refinedSourcePrefixCode_complete</code>             | THM | The refined code is complete: its cylinder idempotents sum to one, merged stepwise down to the nine code.                               |
| <code>prefixTable_mul_sourceWord</code>                   | THM | Restates that the prefix table maps source word generators to target word generators.                                                   |
| <code>compressionU_mul_nine_refinement</code>             | THM | <code>compressionU</code> maps a refined generator over nine word $i$ to the same refinement over the $u$ word.                         |
| <code>refinedSourceParent</code>                          | DEF | Assigns to each of the nineteen refined words its parent index in the nine code.                                                        |
| <code>refinedSourceSuffix</code>                          | DEF | The residual suffix of each refined word after removing its nine-code parent prefix.                                                    |
| <code>refinedTargetWord</code>                            | DEF | Target words: the parent's $u$ word followed by the same suffix.                                                                        |
| <code>refinedSourceWord_eq_parent_append</code>           | THM | Verifies by <code>decide</code> that each refined word is its parent's word followed by its suffix.                                     |
| <code>compressionU_mul_refinedSourceWord</code>           | THM | <code>compressionU</code> maps each refined source generator to the corresponding refined target generator.                             |
| <code>binaryWordCancellation</code>                       | DEF | Recursive normal form for a dual word generator times a word generator: cancel common prefixes, else zero.                              |
| <code>leavittWordT_mul_wordS_cancel</code>                | THM | Proves <code>leavittWordT</code> times <code>leavittWordS</code> equals the recursive cancellation normal form.                         |
| <code>leavittWordS_mul_wordS</code>                       | THM | Concatenation rule: the product of two word generators is the generator of the concatenation.                                           |
| <code>sourceCrossSwap_mul_leavittWordS</code>             | THM | Explicit action of a cross swap on any word generator, expressed through <code>binaryWordCancellation</code> .                          |
| <code>sourceUChronologicalSwaps</code>                    | DEF | A chronological list of twenty-one cross swaps whose composition will realize <code>compressionU</code> .                               |
| <code>elementaryCompressionU</code>                       | DEF | Defines <code>elementaryCompressionU</code> as the reversed product of the chronological swap list.                                     |
| <code>elementaryCompressionU_mem</code>                   | THM | The candidate product lies in <code>prefixElementaryGroup nine PrefixCode</code> , since each swap does.                                |
| <code>elementaryCompressionU_mul_refinedSourceWord</code> | THM | Verifies by exhaustive computation that the swap product acts on all nineteen refined generators exactly as <code>compressionU</code> . |
| <code>compressionU_mem_ninePrefixElementaryGroup</code>   | THM | Therefore <code>compressionU</code> belongs to <code>prefixElementaryGroup ninePrefixCode</code> .                                      |

### 3.7 Top-level compression U membership export

A single top-level restatement. The membership of `compressionU` in `prefixElementaryGroup ninePrefixCode`, established inside the `PrefixCompressionU` namespace by the twenty-one-swap computation, is re-exported unqualified so that subsequent sections can consume it directly. It is the first of the two compression membership facts demanded by `sourceGeneratedGroup_eq_nine_of_compressions`; the second, for `compressionV`, is

derived from this one in the immediately following `PrefixCompression` section via the transition factorization. No new mathematics is introduced here, but the export marks the point where the hard computational content of the previous section becomes available to the global source generation argument.

**compressionU\_mem\_ninePrefixElementaryGroup** thm

```
theorem compressionU_mem_ninePrefixElementaryGroup :
  compressionU ∈ prefixElementaryGroup ninePrefixCode :=
  PrefixCompressionU.compressionU_mem_ninePrefixElementaryGroup
```

A one-line re-export: the theorem restates `PrefixCompressionU.compressionU_mem_ninePrefixElementaryGroup` outside its namespace, so later sections can cite the membership of `compressionU` in `prefixElementaryGroup ninePrefixCode` without qualification. There is no new mathematical content, but the statement is one of the two load-bearing inputs of the source generation argument: `sourceGeneratedGroup_eq_nine_of_compressions` needs both compressions to lie in the nine-code elementary group, and this theorem supplies the first, while the following `PrefixCompression` section derives the second from it via `compressionV_mem_of_compressionU`. Together they identify the nine-code prefix elementary group with the group generated by the alpha elementary group and the two compression units, the step that ultimately makes the group at the center of the non-soficity argument finitely generated. Organizationally, the export separates the heavy word computations of the namespace from the clean membership fact the rest of the file consumes.

### 3.8 Compression V via transition swaps (PrefixCompression)

Derives the membership of `compressionV` from that of `compressionU`. Since prefix tables compose, `compressionV` factors as the transition table from the  $u$  code to the  $v$  code times `compressionU`. The transition is then exhibited inside the elementary group: six explicit cross swaps are listed, and the abstract refinement law `transpositionValue_refine`, specialized to words as `wordSwapValue_refine`, merges child swaps into parent swaps, collapsing the six-swap product to three transpositions of  $u$  words. Acting on every  $u$  word generator, these three swaps exchange the pairs one-two, four-five, and seven-eight, which is exactly the prefix table to the  $v$  code by completeness of the  $u$  code. Hence the transition, and with it `compressionV`, lies in `prefixElementaryGroup ninePrefixCode`, completing the inputs to the source generation theorem.

**compressionV\_eq\_transition\_mul** thm

```
theorem compressionV_eq_transition_mul :
  compressionV = compressionTransition * compressionU := by
  -- proof: 7 lines, omitted
```

Factors `compressionV` as `compressionTransition` times `compressionU`, where the transition is the invertible prefix table `prefixTableUnit` from the  $u$  code to the  $v$  code. The proof is one application of `prefixTable_compose`: the table from the nine code to the  $v$  code equals the table from the  $u$  code to the  $v$  code composed with the table from the nine code to the  $u$  code, each table acting on word generators as established earlier. The strategic effect is to decouple the hard membership question for `compressionV` into two easier ones: membership of `compressionU`, already settled in the previous section by the twenty-one-swap computation, and membership of the transition, which is more tractable because the  $u$  and  $v$  codes share the same cylinder decomposition and, as the parent swap computation `compressionTransitionParentSwaps_mul_word` shows, differ only by exchanging the word pairs at indices one-two, four-five, and seven-eight, a genuine permutation realized by three cross swaps.

**transpositionValue\_refine** thm

```
theorem transpositionValue_refine {A : Type*} [Ring A]
  (sa ta sb tb p₀ q₀ p₁ q₁ : A)
  (haa : ta * sa = 1) (hbb : tb * sb = 1)
  (hab : ta * sb = 0) (hba : tb * sa = 0)
  (h₀₀ : q₀ * p₀ = 1) (h₁₁ : q₁ * p₁ = 1)
  (h₀₁ : q₀ * p₁ = 0) (h₁₀ : q₁ * p₀ = 0)
```

```

(hpartition : p0 * q0 + p1 * q1 = 1) :
transpositionValue (sa * p0) (q0 * ta) (sb * p0) (q0 * tb) *
  transpositionValue (sa * p1) (q1 * ta) (sb * p1) (q1 * tb) =
  transpositionValue sa ta sb tb := by
-- proof: 35 lines, omitted

```

The abstract refinement law for transposition values: in any ring, suppose  $s_a, t_a, s_b, t_b$  satisfy the partial isometry relations  $t_a s_a = t_b s_b = 1$  with orthogonality  $t_a s_b = t_b s_a = 0$ , and  $p_0, q_0, p_1, q_1$  form a two-element partition,  $p_0 q_0 + p_1 q_1 = 1$ , with the analogous isometry and orthogonality relations. Then the product of the transposition values built on the two refined quadruples  $(s_a p_k, q_k t_a, s_b p_k, q_k t_b)$  equals the transposition value of the coarse quadruple. The proof normalizes both sides with `noncomm_ring` using the sixteen associativity-adjusted rewriting facts and then collapses the sums with the partition identity. Specialized to Leavitt word generators via `wordSwapValue_refine`, it says a swap of two parent cylinders equals the product of the swaps of their two children, and this is the mechanism by which the six explicit `nineCrossSwap` elements, each visibly in the elementary group, merge in `compressionTransitionCrossSwaps_prod_val` into the three parent swaps forming `compressionTransition`.

#### **compressionV\_mem\_of\_compressionU thm**

```

theorem compressionV_mem_of_compressionU
  (hU : compressionU ∈ prefixElementaryGroup ninePrefixCode) :
  compressionV ∈ prefixElementaryGroup ninePrefixCode := by
-- proof: 4 lines, omitted

```

The closing theorem of the chunk: if `compressionU` lies in `prefixElementaryGroup ninePrefixCode`, then so does `compressionV`. The proof composes the section's results: `compressionV_eq_transition_mul` factors `compressionV` as the transition times `compressionU`, and `compressionTransition_mem` places the transition in the elementary group by identifying it (`compressionTransitionCrossSwaps_prod_eq`) with the product of six cross swaps. That identification runs through two value computations: `compressionTransitionCrossSwaps_prod_val` collapses the six swaps to three parent word swaps by the refinement law, and `compressionTransitionParentSwaps_eq_table` shows the three parent swaps act on every  $u$  word generator exactly as the prefix table to the  $v$  code, hence equal it by completeness of the  $u$  code (the same extensionality pattern as in the previous section). Combined with the top-level `compressionU_mem_ninePrefixElementaryGroup`, both hypotheses of `sourceGeneratedGroup_eq_nine_of_compressions` are now available, so the nine-code prefix elementary group is finitely generated from the alpha group and the two compressions.

*Remaining declarations in this section:*

|                                                       |     |                                                                                                                      |
|-------------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------|
| <code>prefixTable_compose</code>                      | THM | Prefix tables compose: the table from middle to target times the table from source to middle gives source to target. |
| <code>compressionTransition</code>                    | DEF | Defines <code>compressionTransition</code> as the invertible prefix table unit from the $u$ code to the $v$ code.    |
| <code>nineCrossRoot</code>                            | DEF | Cross root over the nine code, identical in shape to <code>sourceCrossRoot</code> .                                  |
| <code>nineCrossRoot_mem</code>                        | THM | Cross roots belong to the nine-code elementary group.                                                                |
| <code>nineCrossSwap</code>                            | DEF | Triple product of cross roots forming a transposition-shaped unit.                                                   |
| <code>nineCrossSwap_mem</code>                        | THM | Cross swaps belong to the nine-code elementary group.                                                                |
| <code>compressionTransitionCrossSwaps</code>          | DEF | Six explicit cross swaps whose product will equal <code>compressionTransition</code> .                               |
| <code>compressionTransitionCrossSwaps_prod_mem</code> | THM | The product of the six swaps lies in the nine-code elementary group.                                                 |
| <code>transpositionValue</code>                       | DEF | Abstract transposition value $1 - s_a t_a - s_b t_b + s_a t_b + s_b t_a$ in any ring.                                |
| <code>nineRefinement_orthogonal</code>                | THM | Refinements of distinct nine-code words remain orthogonal.                                                           |

|                                           |     |                                                                                                                  |
|-------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------|
| nineCrossSwap_val                         | THM | Each nineCrossSwap evaluates to the transpositionValue of its two refined cylinder words.                        |
| wordSwapValue                             | DEF | Specializes transpositionValue to a pair of binary words.                                                        |
| wordSwapValue_refine                      | THM | Word-level refinement: swap values on the two extensions multiply to the swap value of the parent words.         |
| compressionTransitionCrossSwaps_prod_val  | THM | The six-swap product collapses, via refinement identities, to three parent word swaps between $u$ code words.    |
| transpositionValue_mul_codeWord           | THM | A transposition value acts on code word generators by exchanging the two designated words and fixing the others. |
| uSwap12_mul                               | THM | Specializes the action to the swap of $u$ words one and two.                                                     |
| uSwap45_mul                               | THM | Specializes the action to the swap of $u$ words four and five.                                                   |
| uSwap78_mul                               | THM | Specializes the action to the swap of $u$ words seven and eight.                                                 |
| compressionTransitionParentSwaps_mul_word | THM | The three parent swaps map each $u$ word generator to the matching $v$ word generator.                           |
| compressionTransitionParentSwaps_eq_table | THM | The three parent swaps multiply to the prefix table from the $u$ code to the $v$ code.                           |
| compressionTransitionCrossSwaps_prod_eq   | THM | Hence the six-swap product equals compressionTransition.                                                         |
| compressionTransition_mem                 | THM | compressionTransition lies in the nine-code elementary group.                                                    |

## 4 Prefix transposition groups, midrank statistics, and projections

Here the file builds the Thompson-style local prefix transposition groups that will carry the LEF obstruction, embeds them into products, and develops two bodies of technique used throughout the second half: exact midrank statistics of permutations on components (a discrete variance calculus), and sums-of-squares identities for triples of orthogonal projections in a Hilbert space, the analytic germ of the spectral criterion for property (T).

### 4.1 Floor rank drops and common offsets

Two loose ends of the source-generation argument are closed: `compressionV` lies in the nine-prefix-code elementary group, so `sourceGeneratedGroup_eq_nine` identifies the source generated group with `prefixElementaryGroup` of `ninePrefixCode`. The bulk of the segment is a measure-theoretic averaging device for floor ranks. For reals  $u, v$  and a scale  $H$ , the set of offsets  $r \in [0, H]$  where  $\lfloor (v+r)/H \rfloor < \lfloor (u+r)/H \rfloor$  is measurable and, when  $v \leq u$ , has volume at most  $u - v$ . Averaging the finite count `rankDropCount` over  $r$  therefore produces one common offset at which the number of coordinates whose floor rank drops is bounded by  $(1/H) \sum_i \delta_i$ ; a logarithmic variant turns multiplicative  $(1 - \eta)$ -comparability into such additive bounds, with error  $|\log(1 - \eta)|/H \rightarrow 0$ . These offset lemmas later convert nearly monotone component sizes into integer ranks that rarely decrease.

**shifted\_floor\_drop\_volume\_bound\_sharp** thm

```
theorem shifted_floor_drop_volume_bound_sharp
  (u v H : ℝ) (hH : 0 < H) (hvu : v ≤ u) :
  MeasureTheory.volume
    {r : ℝ | r ∈ Set.Ico 0 H ∧
      ⌊(v + r) / H⌋ < ⌊(u + r) / H⌋} ≤
    ENNReal.ofReal (u - v) := by
  -- proof: 80 lines, omitted
```

For  $v \leq u$  and  $H > 0$  this bounds the Lebesgue volume of the offset set  $\{r \in [0, H) : \lfloor (v+r)/H \rfloor < \lfloor (u+r)/H \rfloor\}$  by exactly  $u - v$ . Setting  $k = \lfloor u/H \rfloor$ , the earlier lemma `shifted_floor_drop_subset_two_intervals` confines the drop set to two half-open intervals with endpoints  $kH - u, kH - v$  and  $(k+1)H - u, (k+1)H - v$ . With  $c = (k+1)H - u$  and  $d = u - v$ , intersecting with  $[0, H)$  shows the set is covered by  $[0, \max(0, c - H + d))$



and  $[c, \min(H, c + d))$ ; a case split on whether  $d \leq H - c$  shows the two lengths always add to exactly  $d$ , so subadditivity of volume gives the bound. The point of sharpness is that the bound carries no factor depending on  $H$  or on how many integer thresholds separate  $v$  from  $u$ : summed over coordinates in `rankDrop_measure` `Real_bound_sharp` and `exists_common_rank_offset_bound_sharp`, it makes the expected number of floor-rank drops at a random offset at most  $(1/H) \sum_i (u_i - v_i)$ , which is what lets the argument tolerate only a logarithmically small total decrement later.

**exists\_common\_log\_rank\_offset** *thm*

```
theorem exists_common_log_rank_offset {ι : Type*} [Fintype ι]
  (x y : ι → ℝ) (eta H : ℝ)
  (hx : ∀ i : ι, 0 < x i)
  (heta0 : 0 ≤ eta) (heta1 : eta < 1) (hH : 0 < H)
  (hcomparison : ∀ i : ι, (1 - eta) * x i ≤ y i) :
  ∃ r ∈ Set.Ico 0 H,
    rankDropCount (fun i => Real.log (x i))
      (fun i => Real.log (y i)) H r ≤
      (Fintype.card ι : ℝ) * |Real.log (1 - eta)| / H := by
-- proof: 14 lines, omitted
```

Given positive data  $x_i$  and comparisons  $(1 - \eta)x_i \leq y_i$  with  $0 \leq \eta < 1$ , the theorem produces a single offset  $r \in [0, H)$  such that the count of indices whose floor rank  $\lfloor (\log y_i + r)/H \rfloor$  falls below  $\lfloor (\log x_i + r)/H \rfloor$  is at most  $\text{card } \iota \cdot |\log(1 - \eta)|/H$ . The proof composes two prior steps: `log_one_sided_of_multiplicative` converts the multiplicative hypothesis into the additive one-sided bound  $\log x_i - |\log(1 - \eta)| \leq \log y_i$ , and `exists_common_rank_offset_one_sided_sharp` then applies the averaged sharp volume estimate with the constant slack  $\delta_i = |\log(1 - \eta)|$ . This is the precise device later applied to component sizes of a transported partition: there, sizes are only comparable up to the factor  $1 - \eta$  coming from imperfect overlaps, and the resulting integer labels  $\lfloor (\log s + r)/H \rfloor$  of a size  $s$  serve as the rank function fed into the midrank machinery. The companion asymptotics `abs_log_one_sub_le_two_mul` and `abs_log_one_sub_div_tendsto_zero` guarantee that with  $\eta_n \rightarrow 0$  and  $\eta_n/H_n \rightarrow 0$  the density of rank-decreasing indices vanishes, which is exactly the rank-drop hypothesis of the final energy estimate.

*Remaining declarations in this section:*

|                                                                          |     |                                                                                                                                                                   |
|--------------------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>compressionV_mem_ninePrefix</code><br><code>ElementaryGroup</code> | THM | <code>compressionV</code> lies in the nine-prefix-code elementary group, deduced from the corresponding membership of <code>compression u</code> .                |
| <code>sourceGeneratedGroup_eq_nine</code>                                | THM | Identifies <code>sourceGeneratedGroup</code> with <code>prefixElementaryGroup</code> of <code>ninePrefixCode</code> , given membership of both compression units. |
| <code>shifted_floor_drop_subset_two_intervals</code>                     | THM | Offsets $r \in [0, H)$ where the shifted floor of $v$ drops below that of $u$ lie in two explicit half-open intervals.                                            |
| <code>measurableSet_shifted_floor_drop</code>                            | THM | The set of offsets where $\lfloor (v + r)/H \rfloor < \lfloor (u + r)/H \rfloor$ is measurable.                                                                   |
| <code>exists_common_offset_below_average</code>                          | THM | On $[0, H)$ , any integrable function attains a value at most its average at some point $r$ .                                                                     |
| <code>rankDropCount</code>                                               | DEF | Real-valued count of indices $i$ whose shifted floor rank drops from $u i$ to $v i$ at offset $r$ .                                                               |
| <code>measurable_rankDropCount</code>                                    | THM | <code>rankDropCount</code> is measurable in the offset $r$ , as a finite sum of indicator functions.                                                              |
| <code>rankDropCount_nonneg_and_le</code>                                 | THM | <code>rankDropCount</code> is nonnegative and at most the cardinality of the index type.                                                                          |
| <code>integrableOn_rankDropCount</code>                                  | THM | <code>rankDropCount</code> is integrable on $[0, H)$ : measurable and bounded on a finite-volume set.                                                             |
| <code>integrable_rankDropIndicator</code>                                | THM | Each individual floor-drop indicator is integrable with respect to volume restricted to $[0, H)$ .                                                                |



|                                                        |     |                                                                                                                                                       |
|--------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>exists_common_rank_offset</code>                 | THM | Some offset $r \in [0, H)$ realizes <code>rankDropCount</code> at most its average over the interval.                                                 |
| <code>rankDropCount_integral_eq</code>                 | THM | The integral of <code>rankDropCount</code> equals the sum over indices of the measures of the individual floor-drop sets.                             |
| <code>rankDropCount_average_eq</code>                  | THM | The average of <code>rankDropCount</code> over $[0, H)$ is $H^{-1}$ times the summed floor-drop measures.                                             |
| <code>rankDropCount_clamp_eq</code>                    | THM | Clamping each $v_i$ to $\min(v_i, u_i)$ leaves <code>rankDropCount</code> unchanged: floor ranks only drop when $v_i < u_i$ .                         |
| <code>rankDrop_measureReal_bound_sharp</code>          | THM | Real-valued form: the restricted measure of the floor-drop offset set is at most $u - v$ .                                                            |
| <code>exists_common_rank_offset_bound_sharp</code>     | THM | With $v_i \leq u_i$ pointwise, some common offset makes <code>rankDropCount</code> at most $(1/H) \sum_i (u_i - v_i)$ .                               |
| <code>exists_common_rank_offset_one_sided_sharp</code> | THM | One-sided variant: from $u_i - \delta_i \leq v_i$ with $\delta_i \geq 0$ , some offset bounds <code>rankDropCount</code> by $(1/H) \sum_i \delta_i$ . |
| <code>log_one_sided_of_multiplicative</code>           | THM | Multiplicative comparison $(1 - \eta)x \leq y$ yields the additive bound $\log x -  \log(1 - \eta)  \leq \log y$ .                                    |
| <code>abs_log_one_sub_le_two_mul</code>                | THM | For $0 \leq \eta \leq 1/2$ , the bound $ \log(1 - \eta)  \leq 2\eta$ holds.                                                                           |
| <code>abs_log_one_sub_div_tendsto_zero</code>          | THM | If $\eta_n \rightarrow 0$ and $\eta_n/H_n \rightarrow 0$ , then $ \log(1 - \eta_n) /H_n \rightarrow 0$ , by squeezing.                                |

## 4.2 Finite means, medians, and variances (CheegerPoincare)

Elementary statistics for real functions on a finite vertex type, packaged for the Cheeger and Poincare estimates used against the hypothetical sofic approximation. `finiteMean` and `finiteVariance` are defined with the standard facts: deviations from the mean sum to zero, the mean minimizes the sum of squared deviations, and the variance is nonnegative. Strict sublevel and superlevel sets `lowerLevel` and `upperLevel` are introduced, and `exists_finite_real_median` produces a median  $m$ : both strict level sets occupy at most half the vertices. Finally, `positiveSupport_max_sub` and its reverse identify the positive supports of the clamped functions  $\max(f - m, 0)$  and  $\max(m - f, 0)$  with the level sets at  $m$ . This is the classical reduction allowing a Poincare (variance versus energy) inequality to be deduced from a Cheeger-type inequality applied to functions supported on at most half of the graph.

**`exists_finite_real_median`** thm

```
@[simp] theorem mem_lowerLevel {V : Type*} [Fintype V]
  (f : V → ℝ) (a : ℝ) (x : V) :
  x ∈ lowerLevel f a ↔ f x < a := by
  -- proof: 87 lines, omitted
```

For any real function  $f$  on a finite nonempty vertex type the theorem produces a median value  $m$ : both the strict sublevel set  $\{f < m\}$  and the strict superlevel set  $\{f > m\}$  contain at most half of the vertices. The construction is order-theoretic: among the finitely many attained values, let  $m$  be the largest value whose strict sublevel set has at most half the vertices (the minimum attained value qualifies, since its sublevel set is empty). If the superlevel set at  $m$  exceeded half, its smallest attained value  $b > m$  would have a strict sublevel set disjoint from  $\{f > m\}$ , hence of size at most half, making  $b$  a larger candidate and contradicting maximality. The median matters because of `positiveSupport_max_sub` and `positiveSupport_max_sub_reverse`: subtracting  $m$  and clamping at zero splits  $f$  into two nonnegative functions whose positive supports are exactly the two level sets, each of size at most half the graph. This is the standard reduction from a Poincare inequality (variance controlled by Dirichlet energy) to a Cheeger-type inequality that only applies to functions supported on at most half the vertex set, and it is against exactly this decomposition that the small-energy midrank observable is eventually tested.

*Remaining declarations in this section:*

|                              |     |                                                        |
|------------------------------|-----|--------------------------------------------------------|
| <code>positiveSupport</code> | DEF | The finset of vertices where $f$ is strictly positive. |
|------------------------------|-----|--------------------------------------------------------|

|                                              |     |                                                                                                                                             |
|----------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>finiteMean</code>                      | DEF | The mean of $f$ : sum over all vertices divided by the cardinality; the span also proves the <code>positiveSupport</code> membership lemma. |
| <code>sum_sub_finiteMean_eq_zero</code>      | THM | Deviations of $f$ from <code>finiteMean</code> sum to zero.                                                                                 |
| <code>sum_sq_sub_finiteMean_le</code>        | THM | The mean minimizes squared deviations: centering at any constant $c$ only increases the sum.                                                |
| <code>lowerLevel</code>                      | DEF | The strict sublevel finset of vertices with $f x < a$ .                                                                                     |
| <code>upperLevel</code>                      | DEF | The strict superlevel finset of vertices with $a < f x$ .                                                                                   |
| <code>positive_max_iff</code>                | THM | $0 < \max(a, 0)$ holds exactly when $0 < a$ .                                                                                               |
| <code>positiveSupport_max_sub</code>         | THM | The positive support of $x \mapsto \max(f x - m, 0)$ equals the upper level set at $m$ .                                                    |
| <code>positiveSupport_max_sub_reverse</code> | THM | Dually, the positive support of $x \mapsto \max(m - f x, 0)$ equals the lower level set at $m$ .                                            |
| <code>finiteVariance</code>                  | DEF | The variance of $f$ : average squared deviation from <code>finiteMean</code> .                                                              |
| <code>finiteVariance_nonneg</code>           | THM | <code>finiteVariance</code> is nonnegative.                                                                                                 |
| <code>card_mul_finiteVariance</code>         | THM | Cardinality times <code>finiteVariance</code> equals the sum of squared deviations from the mean.                                           |

### 4.3 Orthogonal idempotent corners in Leavitt algebra (ThompsonPrefixLocal)

Develops the corner-group calculus that will produce two commuting subgroups with trivial intersection inside the unit group of `BinaryLeavitt`. For idempotents  $e, f$  with  $ef = fe = 0$ , corner units extend to global units  $u + (1 - e)$ ; such extensions commute, and if extensions over the two idempotents coincide they are trivial, so the corner subgroups `idempotentCornerGroup` centralize each other and meet in the trivial subgroup. Specializing, each cylinder  $S_a T_a$  in the Leavitt algebra is idempotent, cylinders of prefix-incomparable words multiply to zero, and a prefix code yields a code idempotent whose corner group contains `prefixElementaryGroup`. For the concrete pair `alphaZeroPrefixCode` versus `sourceLocalWord`  $[0, 0, 0, 1]$ , the incomparability checks are decidable, giving: the local cylinder corner group centralizes the alpha-zero elementary group and intersects it trivially, the two hypotheses of the product embedding built later.

#### `idempotentCornerUnitExtensions_commute` thm

```

theorem idempotentCornerUnitExtensions_commute
  {A : Type*} [Ring A] {e f : A}
  (he : IsIdempotentElem e) (hf : IsIdempotentElem f)
  (hef : e * f = 0) (hfe : f * e = 0)
  (u : he.Corner*) (v : hf.Corner*) :
  Commute
    (idempotentCornerUnitExtension he u)
    (idempotentCornerUnitExtension hf v) := by
  -- proof: 41 lines, omitted

```

In any ring  $A$ , an idempotent  $e$  determines the corner  $eAe$ , and a unit  $u$  of the corner extends to the global unit  $u + (1 - e)$ , the map `idempotentCornerUnitExtension`. This theorem shows that for idempotents  $e, f$  with  $ef = fe = 0$ , the extensions of any corner units  $u, v$  commute in the unit group. The proof extracts from corner membership the identities  $u = eu = ue$  and  $v = fv = vf$ , then derives the annihilation relations  $uv = vu = 0$ ,  $ev = ve = 0$ ,  $uf = fu = 0$ , after which `noncomm_ring` verifies that  $(u + (1 - e))(v + (1 - f))$  equals the reversed product. The companion `idempotentCornerUnitExtensions_eq_one` shows an element realized as an extension over both idempotents is trivial, yielding `idempotentCornerGroup_le_centralizer` and `idempotentCornerGroup_inf_eq_bot`. This is the entire algebraic mechanism for producing an internal direct product: two subgroups of the Leavitt unit group supported on orthogonal idempotents automatically commute elementwise and intersect trivially. It is applied with  $e$  the code idempotent of `alphaZeroPrefixCode` and  $f$  the cylinder of `sourceLocalWord`.

## alphaZero\_inf\_localCylinderCorner\_eq\_bot thm

```
theorem alphaZero_inf_localCylinderCorner_eq_bot :
  prefixElementaryGroup alphaZeroPrefixCode n
    cylinderCornerGroup sourceLocalWord = 1 := by
  -- proof: 16 lines, omitted
```

States that `prefixElementaryGroup` of `alphaZeroPrefixCode` intersects the corner group of the cylinder of `sourceLocalWord`  $[0, 0, 0, 1]$  trivially. Two ingredients feed the abstract lemma `idempotentCornerGroup_inf_eq_bot`. First, `prefixElementaryGroup_le_corner` embeds the elementary group of any prefix code into the corner group of its code idempotent, via the matrix-corner description of the elementary generators. Second, the code idempotent of the alpha-zero code and the local cylinder annihilate one another: each of the three code words is prefix-incomparable with  $[0, 0, 0, 1]$  (verified by `decide` in `alphaZeroWord_incomparable_local` and `localWord_incomparable_alphaZero`), so the corresponding cylinders multiply to zero by `leavittCylinder_mul_eq_zero_of_incomparable`, and summing over the code gives `alphaZeroCode_mul_localCylinder` and its reverse. Together with the parallel centralizer statement `localCylinderCorner_le_centralizer_alphaZero`, this supplies exactly the two hypotheses, elementwise commutation and trivial intersection, that make the multiplication map from the product of the alpha-zero elementary group and the local prefix transposition group injective in Thompson PrefixInsertion, the embedding on which the reduction from soficity of the nine-code group to LEF of the transposition group rests.

*Remaining declarations in this section:*

|                                                           |     |                                                                                                                                               |
|-----------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>idempotentCornerUnitExtensions_eq_one</code>        | THM | If corner-unit extensions over orthogonal idempotents coincide, both equal one; multiplying by $e$ forces $u = e$ .                           |
| <code>idempotentCornerGroup</code>                        | DEF | The subgroup of units arising as corner-unit extensions from the idempotent $e$ .                                                             |
| <code>idempotentCornerGroup_le_centralizer</code>         | THM | For orthogonal idempotents, one corner group lies in the centralizer of the other.                                                            |
| <code>idempotentCornerGroup_inf_eq_bot</code>             | THM | Corner groups of orthogonal idempotents have trivial intersection.                                                                            |
| <code>leavittCylinder_isIdempotent</code>                 | THM | Each cylinder $S_a T_a$ in the Leavitt algebra is an idempotent.                                                                              |
| <code>leavittCylinder_mul_eq_zero_of_incomparable</code>  | THM | Cylinders of prefix-incomparable words multiply to zero, via <code>leavittWordT_mul_wordS_of_incomparable</code> .                            |
| <code>cylinderCornerGroup</code>                          | DEF | The corner group attached to the cylinder idempotent of a word $a$ .                                                                          |
| <code>prefixCodeIdempotent</code>                         | DEF | The sum-of-cylinders idempotent of a binary prefix code, via <code>MatrixCorner.codeIdempotent</code> .                                       |
| <code>prefixCodeCornerGroup</code>                        | DEF | The corner group of the code idempotent of a prefix code.                                                                                     |
| <code>prefixElementaryGroup_le_corner</code>              | THM | The prefix elementary group of a code is contained in its code-idempotent corner group.                                                       |
| <code>sourceLocalWord</code>                              | DEF | The localizing word $[0, 0, 0, 1]$ whose cylinder will host the transposition group.                                                          |
| <code>alphaZeroWord_incomparable_local</code>             | THM | No word of <code>alphaZeroPrefixCode</code> is a prefix of <code>sourceLocalWord</code> ; verified by case analysis and <code>decide</code> . |
| <code>localWord_incomparable_alphaZero</code>             | THM | <code>sourceLocalWord</code> is a prefix of no word of <code>alphaZeroPrefixCode</code> .                                                     |
| <code>alphaZeroCode_mul_localCylinder</code>              | THM | The alpha-zero code idempotent times the local cylinder is zero, summing incomparability term by term.                                        |
| <code>localCylinder_mul_alphaZeroCode</code>              | THM | The reverse product, local cylinder times alpha-zero code idempotent, is also zero.                                                           |
| <code>localCylinderCorner_le_centralizer_alphaZero</code> | THM | The corner group of the local cylinder centralizes the alpha-zero prefix elementary group.                                                    |

## 4.4 Matched partitions and diagonal error extraction

Combinatorial transfer machinery comparing a partition  $P$  with a target partition  $Q$  through a family  $R$  of retained parts matched by a map  $D$  with majority overlap. `target_majority_of_small_symmDiff` shows a small symmetric difference forces  $|D| < 2|C \cap D|$ , which is what makes the matching injective. The word-crossing counts of  $P$  are bounded by those of  $Q$  plus twice the mass missed by the matched core; if the discarded and symmetric-difference densities vanish, the retained support covers almost everything. The centrepiece, `exists_matched_slow_diagonal_word_errors`, is a diagonal extraction: from per-radius vanishing of  $Q$ -crossings and bad-set densities it produces one radius sequence  $r(n) \rightarrow \infty$  along which the  $P$ -crossing counts plus bad mass still have vanishing density. Averaging lemmas then locate good components and, eventually, vertices avoiding the bad set.

**target\_majority\_of\_small\_symmDiff** *thm*

```
theorem target_majority_of_small_symmDiff
  {V : Type*} [DecidableEq V] (C D : Finset V)
  (hsmall : 2 * (C Δ D).card < C.card) :
  D.card < 2 * (C ∩ D).card := by
  -- proof: 9 lines, omitted
```

A purely counting lemma: if finsets  $C, D$  satisfy  $2|C \Delta D| < |C|$ , then  $|D| < 2|C \cap D|$ , that is, the intersection is a strict majority of  $D$ . The proof combines `Finset.card_sdiff_add_card_inter` for both orders with the fact that each one-sided difference embeds in the symmetric difference, and closes with `omega`. Its role is structural: the matching  $D$  from retained  $P$ -parts to  $Q$ -parts is only known to have small symmetric difference with its source in aggregate, but the injectivity argument (`finpartition_dominant_matching_injOn`, invoked inside `exists_matched_slow_diagonal_word_errors`) needs each matched  $Q$ -part to be majority-covered by its source component, since two disjoint parts cannot both contain a strict majority of the same target. This lemma is the hinge converting the analytic smallness hypothesis on symmetric differences into that combinatorial majority condition.

**exists\_matched\_slow\_diagonal\_word\_errors** *thm*

```
theorem exists_matched_slow_diagonal_word_errors
  {V : ℕ → Type*} [∀ n, DecidableEq (V n)] {ι : Type*}
  (U : ∀ n, Finset (V n))
  (P Q : ∀ n, Finpartition (U n))
  (R : ∀ n, Finset (Finset (V n)))
  (hR : ∀ n, R n ⊆ (P n).parts)
  (D : ∀ n, Finset (V n) → Finset (V n))
  (hD : ∀ n C, C ∈ R n → D n C ∈ (Q n).parts)
  (hmajor : ∀ n C, C ∈ R n →
    (D n C).card < 2 * (C ∩ D n C).card)
  (hdiscard : Tendsto
    (fun n => (((U n \ matchedRetainedSupport (R n)).card : ℝ) /
      (U n).card)) atTop (ℳ 0))
  (hsymm : Tendsto
    (fun n =>
      ((∑ C ∈ R n, (C Δ D n C).card : ℕ) : ℝ) / (U n).card)
    atTop (ℳ 0))
  (I : ℕ → Finset ι)
  (w : ∀ n, ι → Equiv.Perm (V n))
  (hword : ∀ k, Tendsto
    (fun n =>
      ((∑ i ∈ I k,
        (partitionWordCrossing (Q n) (w n i)).card : ℕ) : ℝ) /
      (U n).card)
    atTop (ℳ 0))
  (B : ∀ n, ℕ → Finset (V n))
  (hbad : ∀ k, Tendsto
    (fun n => (((U n ∩ B n k).card : ℝ) / (U n).card))
    atTop (ℳ 0)) :
  ∃ r : ℕ → ℕ,
```

```

Tendsto r atTop atTop λ
  Tendsto
    (fun n =>
      ((∑ i ∈ I (r n),
        (partitionWordCrossing (P n) (w n i)).card : ℕ) : ℝ) +
      (U n n B n (r n)).card) / (U n).card)
    atTop (N 0) := by
-- proof: 88 lines, omitted

```

The diagonal extraction at the heart of the transport argument. The data: a sequence of vertex sets  $U_n$  with partitions  $P_n$  (from the sofic approximation) and  $Q_n$  (the model), retained parts  $R_n$  matched by  $D_n$  into  $Q_n$ -parts with majority overlap, with the discarded density and the aggregate symmetric-difference density both tending to zero; for each fixed radius  $k$ , the summed  $Q_n$ -crossing counts of the group words indexed by  $I_k$  and the bad sets  $B_n(k)$  both have vanishing density. The conclusion produces one radius function  $r(n) \rightarrow \infty$  such that along the diagonal the  $P_n$ -crossing counts of the words in  $I_{r(n)}$  plus the bad mass still have vanishing density. The proof first transfers crossings from  $Q$  to  $P$ : `partitionWordCrossing_indexed_sum_card_le_target_add_unmatched` bounds the  $P$ -crossing total by the  $Q$ -crossing total plus  $2|I_k|$  times the mass missing from the matched core, and `matchedCore_missing_density_tendsto_zero` kills the correction. Each fixed- $k$  error then tends to zero, and the generic lemma `exists_diverging_radius_with_vanishing_diagonal_error` selects the slowly diverging radius. This converts per-radius control, which is all a sofic approximation provides, into a single sequence of statements with growing radius, the form needed to run the spectral-gap contradiction.

*Remaining declarations in this section:*

|                                                                             |     |                                                                                                                           |
|-----------------------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------|
| <code>matchedRetainedFinpartition</code>                                    | DEF | Restricts a finpartition to a retained subfamily $R$ , partitioning <code>matchedRetainedSupport</code> .                 |
| <code>matchedRetainedSupport_nonempty</code>                                | THM | A nonempty retained family has nonempty support, since parts of a finpartition are nonempty.                              |
| <code>partitionWordCrossing_indexed_sum_card_le_target_add_unmatched</code> | THM | Summed $P$ -crossing counts are bounded by $Q$ -crossing counts plus twice the index count times the unmatched-core mass. |
| <code>matchedRetainedSupport_cover_density_tendsto_one</code>               | THM | If discarded density vanishes, the retained support covers a fraction of $U_n$ tending to one.                            |
| <code>matchedRadiusBad</code>                                               | DEF | The bad set at one radius: $B$ united with all word-crossing vertices for the index set.                                  |
| <code>matchedRadiusBad_card_le</code>                                       | THM | Its cardinality inside $U$ is at most the bad mass plus the total crossing counts.                                        |
| <code>matched_exists_le_weighted_average</code>                             | THM | In a positively weighted family, some index has bad-to-weight ratio at most the aggregate ratio.                          |
| <code>matched_sum_card_inter_partition</code>                               | THM | Summing $ C \cap B $ over the parts of a finpartition of $U$ yields $ U \cap B $ .                                        |
| <code>matchedRetained_bad_density_tendsto_zero</code>                       | THM | If bad density in $U_n$ vanishes and retained support covers half, bad density inside the retained support vanishes.      |
| <code>matched_eventually_exists_good_vertex</code>                          | THM | Eventually every component contains a vertex outside the bad set, once the relative bad density falls below one.          |

#### 4.5 Exact midrank statistics on components (ComponentMidrankVariance)

Computes the exact statistics of the midrank observable on one component. Within a component  $C$  carrying an integer rank  $b$ , `componentVertexMidrank` places each vertex at the midpoint of its rank block, normalized into  $[0, 1]$ . Writing `lowerRankWeight` for the cumulative mass strictly below a rank, an induction over the maximal element gives the closed form  $\sum_j w_j(L_j + w_j/2)^2 = (\sum w)^3/3 - (\sum w^3)/12$ . Consequently the midranks over  $C$  sum to  $|C|/2$ , their squares sum to  $|C|(1/3 - \sum_j m_j^3/12)$  in terms of the rank masses  $m_j$ , and the mean and variance of the midrank function are exactly  $1/2$  and `midrankVariance` of `componentRankMassList`. This pins the variance appearing in the Poincare inequality to a concrete cube-sum of rank masses, which stays

macroscopic unless a single rank dominates.

**sum\_weighted\_ordered\_midpoints\_sq thm**

```
theorem sum_weighted_ordered_midpoints_sq (s : Finset ℤ) (w : ℤ → ℝ) :
  (∑ j ∈ s, w j * (lowerRankWeight s w j + w j / 2) ^ 2) =
    (∑ j ∈ s, w j) ^ 3 / 3 -
    (∑ j ∈ s, w j ^ 3) / 12 := by
  -- proof: 45 lines, omitted
```

The exact second-moment identity behind the midrank variance. For a finset  $s$  of integer ranks with weights  $w$ , let  $L_j$  be `lowerRankWeight`, the total weight strictly below  $j$ . The theorem evaluates  $\sum_{j \in s} w_j (L_j + w_j/2)^2 = (\sum_j w_j)^3/3 - (\sum_j w_j^3)/12$ . The proof is an induction using `Finset.induction_on_max`: inserting a new maximal element makes its lower weight the entire previous total while leaving the other cumulative weights unchanged, and the resulting algebraic identity closes by `ring`. When the weights are the rank masses of a component (summing to one), the left side is the sum of squared midranks, and the identity gives second moment  $1/3 - (\sum m_j^3)/12$ ; combined with the first moment  $1/2$  obtained from the pair-counting lemma `twice_lower_rank_pairs_add_equal_rank_pairs`, the variance of the midrank function is exactly  $1/12 - (\sum m_j^3)/12$ , which is `midrankVariance` of the rank-mass list. The significance is a clean dichotomy: unless a single rank value carries almost all the mass of a component, the cube sum is bounded away from one and the variance stays macroscopic, providing the lower bound the Poincare inequality will play against the vanishing energy.

**componentVertexMidrank\_finiteVariance\_eq thm**

```
theorem componentVertexMidrank_finiteVariance_eq
  {V : Type*}
  (C : Finset V) (b : V → ℤ) (hC : C.Nonempty) :
  SoficGroups.CheegerPoincare.finiteVariance
    (fun x : {x // x ∈ C} =>
      SoficGroups.componentVertexMidrank C b x) =
    SoficGroups.midrankVariance (SoficGroups.componentRankMassList C b) := by
  -- proof: 12 lines, omitted
```

Bridges the concrete midrank computation with the abstract statistics of the `CheegerPoincare` section. Viewing the midrank as a function on the subtype of vertices of the component  $C$ , its `finiteMean` is exactly  $1/2$  (from `sum_componentVertexMidrank`, which gives total  $|C|/2$ ), and its `finiteVariance` equals `midrankVariance` applied to `componentRankMassList`, the sorted list of rank masses. The proof transports the sums over the subtype back to sums over  $C$  with `Finset.sum_subtype`, then invokes `componentVertexMidrank_variance_eq`, itself assembled from the centering identity and the closed-form second moment of `sum_componentVertexMidrank_sq_eq`. This is the form in which the estimate enters the endgame: the Poincare or Cheeger inequality on the approximating graph bounds `finiteVariance` of any observable by its Dirichlet energy up to the spectral-gap constant, and this theorem certifies that for the midrank observable that variance is a completely explicit combinatorial quantity, bounded below whenever the component's rank distribution is spread out. The contradiction will be between this lower bound and the vanishing energy proved in `MidrankPermutationEnergy`.

*Remaining declarations in this section:*

|                                                          |     |                                                                                                 |
|----------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------|
| <code>lowerRankWeight</code>                             | DEF | The total weight of ranks in $s$ strictly below $j$ : a cumulative distribution function.       |
| <code>twice_lower_rank_pairs_add_equal_rank_pairs</code> | THM | Double counting: twice the strictly-lower-rank pairs plus the equal-rank pairs equals $ C ^2$ . |
| <code>sum_componentVertexMidrank</code>                  | THM | Midranks over a component sum to $ C /2$ .                                                      |
| <code>componentVertexMidrank_eq_ordered_midpoint</code>  | THM | A vertex's midrank equals the cumulative mass below its rank plus half its own rank mass.       |
| <code>sum_componentVertexMidrank_sq_eq</code>            | THM | Squared midranks sum to $ C (1/3 - \sum_j m_j^3/12)$ in the rank masses $m_j$ .                 |

|                                                        |     |                                                                                                                  |
|--------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------|
| <code>componentRankMassList_cube_sum</code>            | THM | Cubing the sorted rank-mass list and summing matches the cube sum over the image of $b$ .                        |
| <code>componentVertexMidrank_variance_eq</code>        | THM | Squared deviations of midranks from $1/2$ sum to $ C $ times <code>midrankVariance</code> of the rank-mass list. |
| <code>componentVertexMidrank_finiteMean_eq_half</code> | THM | The <code>finiteMean</code> of the midrank function on a component is exactly $1/2$ .                            |

## 4.6 Overlap loss and transported component sizes

Measures how a partition survives transport by a permutation. `insufficientOverlapComponents` collects the parts of  $P$  whose intersection with their best-overlapping part of  $Q$  falls below the fraction  $1 - \eta$ ; a Markov-type inequality bounds  $\eta$  times their total mass by the global overlap deficit. A permutation  $T$  induces an order isomorphism on finsets, hence a transported finpartition whose parts are the images of the original parts, with a variant for partitions of the whole vertex set. `partitionComponentSize` records the size of the part containing a vertex; the key estimates show that for a retained (well-overlapping) transported component, following  $T$  can shrink the component size by at most the factor  $1 - \eta$ . Combined with the logarithmic offset lemmas of the earlier segment, component size becomes an integer-valued rank that rarely decreases along the approximating permutations.

### `insufficientOverlapComponents_mass_le_loss` thm

```
theorem insufficientOverlapComponents_mass_le_loss
  {V : Type u} [DecidableEq V] {U : Finset V}
  (P Q : Finpartition U) (eta : ℝ) :
  eta * (∑ C ∈ insufficientOverlapComponents P Q eta,
    (C.card : ℝ)) ≤
    ∑ C ∈ P.parts,
      ((C.card : ℝ) - ((C.n maximumOverlapPart Q C).card : ℝ)) := by
-- proof: 27 lines, omitted
```

A Markov-type mass bound for badly transported components. `insufficientOverlapComponents` filters the parts  $C$  of  $P$  for which the intersection with `maximumOverlapPart` of  $Q$  at  $C$  has fewer than  $(1 - \eta)|C|$  elements. The theorem bounds  $\eta$  times the total cardinality of these bad parts by the global overlap deficit  $\sum_C (|C| - |C \cap \text{best}(C)|)$  taken over all parts of  $P$ . The proof is two inequalities: on each bad part the deficit strictly exceeds  $\eta|C|$  by the defining filter condition, and the deficit is nonnegative on every part, so the sum may be extended from the bad family to all of  $P$ . Consequently, when the transport estimates show the aggregate deficit has vanishing density relative to the vertex set, the bad components occupy only a fraction of order deficit over  $\eta$  of the mass; choosing  $\eta = \eta_n \rightarrow 0$  slowly keeps this vanishing. This is what justifies discarding the insufficiently overlapping components, feeding the discarded-density hypothesis of the matched-support lemmas, while retaining components on which component size is nearly preserved by the permutation.

### `partitionComponentSize_transport_lower_of_retained` thm

```
theorem partitionComponentSize_transport_lower_of_retained
  {V : Type u} [Fintype V] [DecidableEq V]
  (Q : Finpartition (Finset.univ : Finset V))
  (T : Equiv.Perm V) (C : Finset V) (hC : C ∈ Q.parts)
  (eta : ℝ)
  (hretained : C.map T.toEmbedding ∈
    insufficientOverlapComponents
      (transportedUnivFinpartition Q T) Q eta)
  (x : V) (hx : x ∈ C)
  (hTx : T x ∈ maximumOverlapPart Q (C.map T.toEmbedding)) :
  (1 - eta) * (partitionComponentSize Q x : ℝ) ≤
    (partitionComponentSize Q (T x) : ℝ) := by
-- proof: 21 lines, omitted
```



The pointwise conclusion of the overlap analysis. Fix a partition  $Q$  of the full vertex set, a permutation  $T$ , and a part  $C$  of  $Q$  whose image under  $T$  is not an insufficient-overlap component of the transported partition relative to  $Q$ . Then for any  $x \in C$  whose image  $Tx$  lands in the maximum-overlap part of the image of  $C$ , the component sizes satisfy  $(1 - \eta) \text{size}(x) \leq \text{size}(Tx)$ , where `partitionComponentSize` is the cardinality of the part containing a vertex. The proof chains three facts: retention means the image of  $C$  meets its best  $Q$ -part in at least  $(1 - \eta)|C|$  elements (`maximumOverlapPart_overlap_of_not_mem_insufficient`); that intersection is contained in the target part, so the target has at least  $(1 - \eta)|C|$  elements (`partitionComponentSize_transport_lower_of_overlap`); and both sizes equal the cardinalities of the respective parts (`partitionComponentSize_eq_card_of_mem`). The upshot is that, away from a small exceptional set of components and vertices, the observable sending a vertex to the logarithm of its component size is  $(1 - \eta)$ -multiplicatively almost nondecreasing along  $T$ . This is exactly the comparison the common log-rank offset lemma consumes to manufacture an integer rank function that rarely decreases under the approximating permutations.

*Remaining declarations in this section:*

|                                                                 |     |                                                                                                              |
|-----------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------|
| <code>insufficientOverlapComponents</code>                      | DEF | Parts of $P$ overlapping their best $Q$ -part in less than the fraction $1 - \eta$ .                         |
| <code>insufficientOverlapComponents_subset</code>               | THM | These insufficiently overlapping components form a subset of the parts of $P$ .                              |
| <code>finsetPermutationOrderIso</code>                          | DEF | A permutation of $V$ induces an order isomorphism of finsets via <code>Finset.map</code> .                   |
| <code>transportedFinpartition</code>                            | DEF | Pushes a finpartition of $U$ forward along a permutation to partition the image of $U$ .                     |
| <code>transportedFinpartition_parts</code>                      | THM | The transported partition's parts are the images of the original parts.                                      |
| <code>transportedUnivFinpartition</code>                        | DEF | Transporting a partition of the full vertex set again partitions <code>Finset.univ</code> .                  |
| <code>transportedUnivFinpartition_parts</code>                  | THM | Parts of the transported universe partition are the images of the original parts.                            |
| <code>partitionComponentSize</code>                             | DEF | The size of the part of the partition containing a given vertex.                                             |
| <code>partitionComponentSize_eq_card_of_mem</code>              | THM | If $x$ belongs to a part $C$ , its component size equals $ C $ .                                             |
| <code>partitionComponentSize_transport_lower_of_overlap</code>  | THM | Heavy overlap of the transported part with a $Q$ -part bounds size loss along $T$ by the factor $1 - \eta$ . |
| <code>maximumOverlapPart_overlap_of_not_mem_insufficient</code> | THM | Retained components achieve overlap at least $(1 - \eta) C $ with their maximum-overlap $Q$ -part.           |

## 4.7 Prefix transposition groups and product embedding (ThompsonPrefixInsertion)

Constructs the local prefix transposition group and embeds its product with the alpha-zero group into the nine-code elementary group. The singleton prefix code of a word  $a$  identifies its code idempotent with the cylinder of  $a$ , so the associated corner homomorphism becomes `prefixInsertionHom`, an injective endomorphism of the unit group with explicit value  $S_a u T_a + (1 - S_a T_a)$ . The involutions `cylinderSwap` exchange incomparable cylinders, and prefix insertion carries the swap of  $a, b$  to the swap of the prefixed words. A braid identity through elementary cross roots shows swaps of two extensions of one code word lie in `prefixElementaryGroup`, hence the whole group `binaryPrefixTranspositionGroup` inserted at `sourceLocalWord` lands in the alpha (so nine) elementary group, centralizes the alpha-zero group, and meets it trivially. Multiplication therefore embeds the direct product injectively into `prefixElementaryGroup` of `ninePrefixCode`.

**`prefixInsertionHom_val`** thm

```
theorem prefixInsertionHom_val (a : List (Fin 2)) (u : BinaryLeavitt*) :
  (↑(prefixInsertionHom a u) : BinaryLeavitt) =
    leavittWordS a * (u : BinaryLeavitt) * leavittWordT a +
```



```

    (1 - leavittCylinder a) := by
-- proof: 12 lines, omitted

```

Gives the closed formula for the prefix-insertion endomorphism: for a unit  $u$  of `BinaryLeavitt`, the value of `prefixInsertionHom` at  $a$  is  $S_a u T_a + (1 - S_a T_a)$ , where  $S_a, T_a$  are the word isometries and  $S_a T_a$  is the cylinder idempotent of  $a$ . The definition routes  $u$  through the ring equivalence with  $1 \times 1$  matrices and then through `prefixCornerUnitHom` for `singletonPrefixCode`, whose code idempotent was identified with the cylinder in `singletonPrefixCode_codeIdempotent`; unfolding `MatrixCorner.encode` and simplifying yields the displayed value. Conceptually this compresses the whole unit group into the corner of the cylinder of  $a$ , acting as  $u$  inside the cylinder and as the identity outside, which is why the map is injective (`prefixInsertionHom_injective`) and lands in `cylinderCornerGroup` (`prefixInsertionHom_mem_cylinderCorner`). All the later structure rests on this formula: `prefixInsertionHom_cylinderSwap` computes that swaps go to swaps of prefixed words by a direct `noncomm_ring` calculation from it, and the corner localization is what makes the inserted transposition group commute with, and intersect trivially, the alpha-zero elementary group supported on incomparable cylinders.

**samePrefixCylinderSwap\_mem** thm

```

theorem samePrefixCylinderSwap_mem {ι : Type*} [DecidableEq ι]
  (E : BinaryPrefixCode ι)
  (i j : ι) (hij : i ≠ j)
  (a b c : List (Fin 2))
  (hab : ~ a <+ b) (hba : ~ b <+ a) :
  cylinderSwap (E.word i ++ a) (E.word i ++ b)
    (prepend_not_prefix_prepend (E.word i) a b hab)
    (prepend_not_prefix_prepend (E.word i) b a hba) ∈
  prefixElementaryGroup E := by
-- proof: 63 lines, omitted

```

The membership theorem that pulls the transposition group inside the elementary group. For a binary prefix code  $E$ , distinct indices  $i \neq j$ , and incomparable suffixes  $a, b$  (plus an auxiliary word  $c$ ), the involution `cylinderSwap` exchanging the cylinders of the concatenations of the word at  $i$  with  $a$  and with  $b$  belongs to `prefixElementaryGroup` of  $E$ . The difficulty is that this swap moves mass within a single code cylinder, while the elementary generators `prefixElementaryUnit` connect two different code indices. The proof routes through the second index: `prefixCrossSwap` builds, from three elementary cross roots of the form  $1 + S_i(S_a T_b)T_j$ , a transposition between the extended words at  $i$  and  $j$  (computed in `prefixCrossSwap_val` via `cylinder_transposition_factorization`), and the braid identity `transpositionValue_braid` shows that conjugating the swap through  $b, c$  by the swap through  $a, c$  produces exactly the desired same-prefix swap; its nine hypotheses are isometry relations and orthogonality of extended words, supplied by `append_not_prefix_append` and `prefix-freeness` of  $E$ . Specialized to `alphaPrefixCode` in `sourceLocal_cylinderSwap_mem_alpha`, it proves every generator of the inserted transposition group lies in the alpha elementary group, giving the chain of inclusions into `prefixElementaryGroup` of `ninePrefixCode`.

**sourceCompressedLocalProductHom** def

```

def sourceCompressedLocalProductHom :
  (prefixElementaryGroup alphaZeroPrefixCode ×
   localPrefixTranspositionGroup ThompsonPrefixLocal.sourceLocalWord) →*
  BinaryLeavitt* where
toFun x := (x.1 : BinaryLeavitt*) * (x.2 : BinaryLeavitt*)
map_one' := by simp
map_mul' p q := by
  have hcomm :
    (q.1 : BinaryLeavitt*) * (p.2 : BinaryLeavitt*) =
    (p.2 : BinaryLeavitt*) * (q.1 : BinaryLeavitt*) :=
  Subgroup.mem_centralizer_iff.mp
  (sourceLocalPrefixTranspositionGroup_le_centralizer_alphaZero
   p.2.property)
  (q.1 : BinaryLeavitt*) q.1.property
change
  ((p.1 : BinaryLeavitt*) * (q.1 : BinaryLeavitt*)) *
  ((p.2 : BinaryLeavitt*) * (q.2 : BinaryLeavitt*)) =

```

```

      ((p.1 : BinaryLeavitt*) * (p.2 : BinaryLeavitt*)) *
      ((q.1 : BinaryLeavitt*) * (q.2 : BinaryLeavitt*))
calc
  ((p.1 : BinaryLeavitt*) * (q.1 : BinaryLeavitt*)) *
  ((p.2 : BinaryLeavitt*) * (q.2 : BinaryLeavitt*)) =
  (p.1 : BinaryLeavitt*) *
  (((q.1 : BinaryLeavitt*) * (p.2 : BinaryLeavitt*)) *
   (q.2 : BinaryLeavitt*)) := by simp [mul_assoc]
_ = (p.1 : BinaryLeavitt*) *
  (((p.2 : BinaryLeavitt*) * (q.1 : BinaryLeavitt*)) *
   (q.2 : BinaryLeavitt*)) := by rw [hcomm]
_ = ((p.1 : BinaryLeavitt*) * (p.2 : BinaryLeavitt*)) *
  ((q.1 : BinaryLeavitt*) * (q.2 : BinaryLeavitt*)) := by
  simp [mul_assoc]

```

Defines the multiplication map from the direct product of the subgroup `prefixElementaryGroup` of `alphaZero` `PrefixCode` and `localPrefixTranspositionGroup` at `sourceLocalWord` into the unit group of `BinaryLeavitt`, sending a pair to the product of its coordinates. Multiplicativity is not free for such a map; it holds because the second factor centralizes the first (`sourceLocalPrefixTranspositionGroup_le_centralizer_alphaZero`, descending from the orthogonal-idempotent corner calculus), which lets the middle terms be exchanged in the four-fold product. Injectivity (`sourceCompressedLocalProductHom_injective`) follows from the kernel criterion: if the product of a pair is one, the first coordinate equals the inverse of the second and so lies in both subgroups, and the trivial-intersection theorem `alphaZero_inf_sourceLocalPrefixTranspositionGroup_eq_bot` forces both coordinates to be one. Finally the range lies in the nine-code elementary group, since both factors do, so corestriction yields the injective homomorphism `sourceCompressedLocalProductEmbedding` into `prefixElementaryGroup` of nine `PrefixCode`. This realizes an internal direct product inside the finitely presented target group; it is the structural input to the reduction that soficity of the nine-code elementary group would make the local prefix transposition group LEF, which the endgame refutes.

*Remaining declarations in this section:*

|                                                    |     |                                                                                                                                        |
|----------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>singletonPrefixCode</code>                   | DEF | The one-word prefix code containing only $a$ ; prefix-freeness holds vacuously.                                                        |
| <code>singletonPrefixCode_codeIdempotent</code>    | THM | Its code idempotent equals the cylinder <code>leavittCylinder</code> of $a$ .                                                          |
| <code>prefixInsertionHom</code>                    | DEF | Endomorphism of the Leavitt unit group compressing into the cylinder corner of $a$ , via the one-by-one matrix equivalence.            |
| <code>prefixInsertionHom_injective</code>          | THM | <code>prefixInsertionHom</code> is injective, as a composite of injective homomorphisms.                                               |
| <code>leavittCylinder_isIdempotent</code>          | THM | Local restatement that cylinders are idempotent elements.                                                                              |
| <code>cylinderCornerGroup</code>                   | DEF | The range of the corner-unit extension for the cylinder idempotent of $a$ .                                                            |
| <code>cylinderCornerGroup_eq_source</code>         | THM | This corner group coincides definitionally with the Thompson <code>PrefixLocal</code> version.                                         |
| <code>prefixInsertionHom_mem_cylinderCorner</code> | THM | Every value of <code>prefixInsertionHom</code> lies in the cylinder corner group of $a$ .                                              |
| <code>transpositionValue_mul_self</code>           | THM | Under isometry and orthogonality relations, <code>PrefixCompression.transpositionValue</code> squares to one.                          |
| <code>cylinderSwap</code>                          | DEF | The involutive unit swapping the cylinders of two prefix-incomparable words, built from <code>PrefixCompression.wordSwapValue</code> . |
| <code>append_not_prefix_append</code>              | THM | Prefix-incomparability of $a$ and $b$ persists after appending arbitrary suffixes.                                                     |
| <code>prepend_not_prefix_prepend</code>            | THM | Prefix-incomparability persists after prepending a common word $l$ .                                                                   |
| <code>prefixInsertionHom_cylinderSwap</code>       | THM | Prefix insertion at $l$ carries the swap of $a, b$ to the swap of the words prefixed by $l$ .                                          |

|                                                                           |     |                                                                                                                                             |
|---------------------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>transpositionValue_braid</code>                                     | THM | Braid identity: conjugating the $b, c$ transposition by the $a, c$ transposition yields the $a, b$ transposition.                           |
| <code>prefixCrossRoot</code>                                              | DEF | The elementary unit $1 + S_i(S_a T_b) T_j$ crossing between code indices $i$ and $j$ .                                                      |
| <code>prefixCrossRoot_mem</code>                                          | THM | Cross roots are elementary generators, hence lie in the prefix elementary group.                                                            |
| <code>prefixCrossSwap</code>                                              | DEF | Product of three cross roots realizing a transposition between two extended code words.                                                     |
| <code>prefixCrossSwap_mem</code>                                          | THM | The cross swap lies in the prefix elementary group, being a product of members.                                                             |
| <code>prefixCrossSwap_val</code>                                          | THM | Evaluates the cross swap as <code>wordSwapValue</code> of the concatenated words, using <code>cylinder_transposition_factorization</code> . |
| <code>sourceLocal_cylinderSwap_mem_alpha</code>                           | THM | Inserting $[0, 0, 0, 1]$ sends every cylinder swap into the alpha prefix elementary group.                                                  |
| <code>binaryPrefixTranspositionGroup</code>                               | DEF | The subgroup of Leavitt units generated by all cylinder swaps of incomparable word pairs.                                                   |
| <code>localPrefixTranspositionGroup</code>                                | DEF | The image of <code>binaryPrefixTranspositionGroup</code> under prefix insertion at a localizing word $l$ .                                  |
| <code>localPrefixTranspositionGroup_le_cylinderCorner</code>              | THM | The local transposition group is contained in the cylinder corner group of $l$ .                                                            |
| <code>localPrefixTranspositionGroup_le_sourceCylinderCorner</code>        | THM | Transfers the inclusion to the <code>ThompsonPrefixLocal</code> corner group via their definitional equality.                               |
| <code>sourceLocalPrefixTranspositionGroup_le_alpha</code>                 | THM | For $l = [0, 0, 0, 1]$ , the local transposition group lies in the alpha prefix elementary group.                                           |
| <code>sourceLocalPrefixTranspositionGroup_le_alpha_sourceWord</code>      | THM | Restates that inclusion with the localizing word written as <code>sourceLocalWord</code> .                                                  |
| <code>sourceLocalPrefixTranspositionGroup_le_nine</code>                  | THM | Composing with the alpha-to-nine inclusion places the local transposition group inside the nine-code elementary group.                      |
| <code>sourceLocalPrefixTranspositionGroup_le_centralizer_alphaZero</code> | THM | The local transposition group centralizes the alpha-zero elementary group, through the cylinder corner group.                               |
| <code>alphaZero_inf_sourceLocalPrefixTranspositionGroup_eq_bot</code>     | THM | The alpha-zero elementary group and the local transposition group intersect trivially.                                                      |
| <code>sourceCompressedLocalProductHom_injective</code>                    | THM | Injectivity: a kernel pair forces both coordinates into the trivial intersection, hence both equal one.                                     |
| <code>sourceCompressedLocalProductHom_range_le_nine</code>                | THM | The range of the product homomorphism lies in the nine-prefix-code elementary group.                                                        |
| <code>sourceCompressedLocalProductEmbedding</code>                        | DEF | Corestriction: an injective homomorphism from the product group into <code>prefixElementaryGroup</code> of <code>ninePrefixCode</code> .    |
| <code>sourceCompressedLocalProductEmbedding_injective</code>              | THM | The corestricted embedding is injective, inherited from <code>sourceCompressedLocalProductHom_injective</code> .                            |

## 4.8 Vanishing midrank permutation energy (MidrankPermutationEnergy)

Assembles the vanishing-energy estimate for the midrank observable. `partitionVertexMidrank` evaluates the midrank of a vertex inside its own part; `offsetFloorRank` is the integer rank  $\lfloor (ux + r)/H \rfloor$ , and `rankDropCount_eq_sum_rankDecreasingVertices` identifies the earlier analytic drop count with counts of rank-decreasing vertices per permutation. The structural lemma shows a midrank can strictly decrease under a permutation only at a part-crossing vertex or a rank-decreasing vertex. Since midranks lie in  $[0, 1]$ , a permutation inequality bounds each squared-increment energy sum by twice the crossing count plus twice the rank-drop count, and summing over the finitely many permutations preserves the bound. Hence if both totals vanish after normalization, the full Dirichlet energy of the midrank function tends to zero: the almost-invariant test function contradicting

the spectral gap in the endgame.

#### partitionVertexMidrank\_permutation\_energy\_tendsto\_zero thm

```

theorem partitionVertexMidrank_permutation_energy_tendsto_zero
  (V 1 : ℕ → Type*)
  [V n, Fintype (V n)] [V n, DecidableEq (V n)]
  [V n, Fintype (1 n)]
  (P : (n : ℕ) → Finpartition (Finset.univ : Finset (V n)))
  (b : (n : ℕ) → V n → ℤ)
  (p : (n : ℕ) → 1 n → Equiv.Perm (V n))
  (N : ℕ → ℝ)
  (hN : ∀ n, 0 < N n)
  (hcross : Tendsto
    (fun n =>
      (∑ i : 1 n, ((partitionWordCrossing (P n) (p n i)).card : ℝ)) /
      N n)
    atTop (nhds 0))
  (hrank : Tendsto
    (fun n =>
      (∑ i : 1 n,
        ((rankDecreasingVertices Finset.univ (b n) (p n i)).card : ℝ)) /
        N n)
    atTop (nhds 0)) :
  Tendsto
    (fun n =>
      (∑ i : 1 n, ∑ x : V n,
        (partitionVertexMidrank (P n) (b n) (p n i x) -
          partitionVertexMidrank (P n) (b n) x) ^ 2) / N n)
    atTop (nhds 0)) := by
  -- proof: 52 lines, omitted

```

The culminating estimate of the segment: given normalizations  $N_n > 0$ , if both the total part-crossing count of the permutations  $p_{n,i}$  with respect to  $P_n$  and the total rank-decreasing count for the integer ranks  $b_n$  vanish after division by  $N_n$ , then the full Dirichlet energy of the midrank observable,  $\sum_i \sum_x (\text{mid}(p_{n,i} x) - \text{mid}(x))^2 / N_n$ , tends to zero. The mechanism has two steps. Structurally, `midpointDecreasing_subset_crossing_union_rankDecreasing` shows a vertex's midrank can strictly decrease under a permutation only if it crosses parts or its integer rank drops, using monotonicity of `componentVertexMidrank` in the rank within a fixed part. Quantitatively, since midranks take values in  $[0, 1]$ , the general inequality `permutation_squared_increment_le_twice_decreasing` bounds each permutation's squared-increment sum by twice the cardinality of its decreasing set, hence by twice crossings plus twice rank drops (`partitionVertexMidrank_permutation_energy_le`); summing over the index set and squeezing finishes. The two hypotheses are exactly what the transport apparatus delivers: crossings vanish by the diagonal extraction, and rank drops vanish by the common log-rank offset applied to component sizes. The resulting low-energy, mean-one-half, macroscopic-variance observable is the witness that violates the Poincare and Cheeger spectral-gap inequalities of the expander construction, driving the final contradiction with soficity.

*Remaining declarations in this section:*

|                                                          |     |                                                                                                                   |
|----------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------|
| <code>partitionVertexMidrank</code>                      | DEF | The midrank of $x$ computed inside its own part of the partition $P$ .                                            |
| <code>rankDecreasingVertices</code>                      | DEF | Vertices of $U$ whose integer rank strictly decreases under the permutation $p$ .                                 |
| <code>offsetFloorRank</code>                             | DEF | The integer rank $\lfloor (ux + r)/H \rfloor$ determined by an offset and a scale.                                |
| <code>rankDropCount_eq_sum_rankDecreasingVertices</code> | THM | Identifies <code>rankDropCount</code> over index-vertex pairs with the summed counts of rank-decreasing vertices. |
| <code>midpointDecreasingVertices</code>                  | DEF | Vertices whose partition midrank strictly decreases under the permutation $p$ .                                   |

|                                     |     |                                                                                                           |
|-------------------------------------|-----|-----------------------------------------------------------------------------------------------------------|
| midpointDecreasing_subset_crossing_ | THM | A midrank decrease forces either crossing parts or an integer rank drop, by midrank monotonicity.         |
| union_rankDecreasing                |     |                                                                                                           |
| partitionVertexMidrank_permutation_ | THM | Per permutation, the summed squared midrank increments are at most twice crossings plus twice rank drops. |
| energy_le                           |     |                                                                                                           |
| sum_partitionVertexMidrank_         | THM | Sums the energy bound over a finite index set of permutations.                                            |
| permutation_energy_le               |     |                                                                                                           |

## 4.9 Charging rank-changing arcs (RankArcCharging)

This segment sets up a charging argument that bounds how many points of a finite set  $U$  change an integer rank label  $b$  when moved by a permutation  $w$ . Given a finite partition  $P$  of  $U$  and a chosen value  $j(C)$  for each part  $C$ , `selectedRankSupport` collects the points whose label agrees with the choice on their part. Any point whose label changes under  $w$  must cross parts, be unselected, or map to an unselected point, giving the cardinality bound `rankChangingArc_card_le_crossing_add_unselected` and, in the limit, `rankChangingArc_density_tendsto_zero`. In the global argument this converts vanishing partition-crossing densities of hypothetical sofic models into vanishing densities of rank-label changes, so rank labels behave like conserved quantities of the model permutations.

**rankChangingArc\_density\_tendsto\_zero** thm

```

theorem rankChangingArc_density_tendsto_zero
  (V : ℕ → Type*) [V n, DecidableEq (V n)]
  (U : ∀ n, Finset (V n))
  (P : ∀ n, Finpartition (U n))
  (b : ∀ n, V n → ℤ)
  (j : ∀ n, Finset (V n) → ℤ)
  (w : ∀ n, Equiv.Perm (V n))
  (N : ℕ → ℝ) (hN : ∀ n, 0 < N n)
  (hcross : Filter.Tendsto
    (fun n => ((partitionWordCrossing (P n) (w n)).card : ℝ) / N n)
    Filter.atTop (nhds 0))
  (homit : Filter.Tendsto
    (fun n =>
      (((U n \ selectedRankSupport (P n) (b n) (j n)).card : ℝ) /
       N n)
    Filter.atTop (nhds 0)) :
  Filter.Tendsto
    (fun n => ((rankChangingArc (U n) (b n) (w n)).card : ℝ) / N n)
    Filter.atTop (nhds 0) := by
-- proof: 39 lines, omitted

```

For a sequence of finite vertex sets  $U_n$  with partitions  $P_n$ , integer labels  $b_n$ , per-part selected values  $j_n$ , permutations  $w_n$ , and positive normalizations  $N_n$ , the theorem states that if both the density of partition-crossing points `partitionWordCrossing` and the density of points outside the selected rank support tend to zero, then the density of points whose label changes under  $w_n$  tends to zero as well. The proof is a direct squeeze: the finite counting bound `rankChangingArc_card_le_crossing_add_unselected` says each rank-changing count is at most the crossing count plus twice the unselected count; dividing by  $N_n$  and applying `squeeze_zero'` with the hypothesis limits finishes. Its role in the global argument is to certify that a rank-style potential function built from a hypothetical sofic approximation is asymptotically invariant under the model permutations: once a common quantization offset makes the unselected density vanish (the `ExceptionalRankOffset` segment provides this) and equipartition estimates control the crossing density, rank labels behave like conserved quantities. This near-invariance is one of the constraints that the later Kun-style transport machinery plays against the spectral-gap inequalities to derive a contradiction with soficity.

*Remaining declarations in this section:*

|                                  |     |                                                                                                             |
|----------------------------------|-----|-------------------------------------------------------------------------------------------------------------|
| <code>selectedRankSupport</code> | DEF | Points of each part $C$ whose rank label $b$ equals the selected value $j(C)$ , unioned over the partition. |
|----------------------------------|-----|-------------------------------------------------------------------------------------------------------------|

|                                                              |     |                                                                                                         |
|--------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------|
| <code>rankChangingArc</code>                                 | DEF | Points of $U$ whose rank label $b$ changes when moved by the permutation $w$ .                          |
| <code>rankSupportPreimageBad</code>                          | DEF | Points of $U$ that $w$ sends into $U \setminus S$ , the unselected region.                              |
| <code>selectedRankSupport_subset</code>                      | THM | The selected rank support is contained in $U$ .                                                         |
| <code>selectedRankSupport_card</code>                        | THM | Its cardinality is the sum over parts of the number of label-matching points, since parts are disjoint. |
| <code>card_sdiff_selectedRankSupport</code>                  | THM | Cardinality of the complement equals the sum over parts of the count of label-mismatching points.       |
| <code>selectedRankSupport_rank_eq</code>                     | THM | On the selected support the label $b(x)$ equals $j$ of the part containing $x$ .                        |
| <code>rankSupportPreimageBad_card_le</code>                  | THM | The bad-preimage set injects into $U \setminus S$ via $w$ , bounding its cardinality.                   |
| <code>rankChangingArc_subset_crossing_or_unselected</code>   | THM | A rank-changing point crosses its part, is unselected, or is a preimage of an unselected point.         |
| <code>rankChangingArc_card_le_crossing_add_unselected</code> | THM | Counts rank-changing points by crossings plus twice the unselected count.                               |

## 4.10 Almost centralizer cluster groups

Working with permutations of a finite vertex set, `permutationDistance` is the Hamming metric, shown bi-invariant and compatible with products and inverses. The commutation defect against a generator family  $\sigma$  vanishes at the identity, is inversion invariant, and subadditive. An `AlmostCentralizerElement` is a permutation whose defect is at most a tolerance; expansion of the generators forces the distance dichotomy `AlmostCentralizerGap`, so closeness is an equivalence and the quotient `AlmostCentralizerClusters` carries a group law once an `AlmostCentralizerRepair` corrects products back into tolerance. `ExpandingCentralizerFiniteModel` packages generators, expansion, repair and an approximately multiplicative, separated map from a group  $G$ ; it yields a finite group receiving a finite subset  $F$  injectively and multiplicatively, hence `lef_of_expanding_centralizer_models`: such models for every finite  $F$  make  $G$  LEF. Two closing lemmas supply slowly vanishing threshold scales used by later quantitative sections.

**`almostCentralizerGap_of_expansion`** thm

```

theorem almostCentralizerGap_of_expansion {V ι : Type*}
  [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (tolerance : ℕ) (h : ℝ)
  (hpositive : 0 < h)
  (hexp : ∀ A : Finset V,
    h * min (A.card : ℝ)
      ((Fintype.card V : ℝ) - A.card) ≤ (boundary σ A : ℝ))
  (hsmall : (10 : ℝ) * tolerance < h * Fintype.card V) :
  AlmostCentralizerGap σ tolerance := by
-- proof: 46 lines, omitted

```

The statement: if the generator family  $\sigma$  satisfies the vertex-expansion inequality  $h \min(|A|, n - |A|) \leq |\partial A|$  for every subset  $A$ , with  $h > 0$  and  $10t < hn$  for the tolerance  $t$  and vertex count  $n$ , then `AlmostCentralizerGap` holds: any two permutations with commutation defect at most  $t$  are either within distance  $n/5$  of each other or farther apart than  $4n/5$ . The proof invokes the earlier `hamming_dichotomy_of_expansion`, which says that for any two permutations either  $h$  times their distance or  $h$  times the complementary distance is bounded by the sum of their commutation defects, here at most  $2t$ . In each branch, assuming the negation of the desired inequality gives  $n < 5d$  or  $5d \leq 4n$ , and combining with  $10t < hn$  produces a numerical contradiction via `nlinarith`. Conceptually this is the discreteness phenomenon at the heart of the construction: property (T) expansion forbids permutations that almost centralize the generators from sitting at intermediate Hamming distances, so almost-centralizing permutations organize into widely separated clusters. That separation is exactly what makes the quotient by closeness a well-defined finite group in the following declarations.

### **almostCentralizerClusterGroup** `def`

```
@[instance_reducible]
def almostCentralizerClusterGroup {V ι : Type*}
  [Fintype V] [Fintype ι] [DecidableEq V]
  {σ : ι → Equiv.Perm V} {tolerance : ℕ}
  (hgap : AlmostCentralizerGap σ tolerance)
  (R : AlmostCentralizerRepair σ tolerance) :
  Group (AlmostCentralizerClusters σ tolerance hgap) := by
letI : One (AlmostCentralizerClusters σ tolerance hgap) :=
  (almostCentralizerCluster hgap (almostCentralizerOne σ tolerance))
letI : Mul (AlmostCentralizerClusters σ tolerance hgap) :=
  (almostCentralizerClusterMul hgap R)
letI : Inv (AlmostCentralizerClusters σ tolerance hgap) :=
  (almostCentralizerClusterInv hgap)
apply Group.ofLeftAxioms
· intro x y z
  induction x, y, z using Quotient.inductionOn₃ with
  | _ p q r =>
    apply Quotient.sound
    exact correctedAlmostCentralizerProduct_assoc hgap R p q r
· intro x
  induction x using Quotient.inductionOn with
  | _ p =>
    apply Quotient.sound
    change 5 * permutationDistance
      (R.correct (1 * p.permutation)) p.permutation ≤ Fintype.card V
    simp [almostCentralizerOne] using
      R.corrected_distance (almostCentralizerOne σ tolerance) p
· intro x
  induction x using Quotient.inductionOn with
  | _ p =>
    apply Quotient.sound
    change 5 * permutationDistance
      (R.correct (p.permutation⁻¹ * p.permutation)) 1 ≤ Fintype.card V
    simp [almostCentralizerInverse] using
      R.corrected_distance (almostCentralizerInverse p) p
```

`almostCentralizerClusterGroup` endows the quotient `AlmostCentralizerClusters` with a group structure, given the gap `AlmostCentralizerGap` and a repair operator `AlmostCentralizerRepair`. The unit is the class of the identity permutation, multiplication is `almostCentralizerClusterMul` (the repaired product `correctedAlmostCentralizerProduct` descended to classes, well defined by `correctedAlmostCentralizerProduct_congr`), and inversion is `almostCentralizerClusterInv`. The group axioms are verified through `Group.ofLeftAxioms`: associativity holds up to the closeness relation by `correctedAlmostCentralizerProduct_assoc`, which chains triangle inequalities through the true products and lets the gap dichotomy rule out the far alternative; the left unit and left inverse laws follow because the repair moves a corrected value at most  $n/5$  away from the honest product  $1 \cdot p$  or  $p^{-1}p$ . The construction is notable because no single permutation in a cluster need be exactly central or multiplicative: the group law lives only on the quotient, where the fifth-of- $n$  error margins collapse. Since `almostCentralizerElementFinite` makes the element type finite, this yields the finite groups into which fragments of the abstract group are embedded, the key ingredient of the LEF conclusion in `lef_of_expanding_centralizer_models`.

### **ExpandingCentralizerFiniteModel** `struct`

```
structure ExpandingCentralizerFiniteModel (G : Type*) [Group G]
  (F : Finset G) where
  vertices : Type
  [verticesFintype : Fintype vertices]
  [verticesDecidableEq : DecidableEq vertices]
  generatorIndex : Type
  [generatorFintype : Fintype generatorIndex]
  generators : generatorIndex → Equiv.Perm vertices
  tolerance : ℕ
  cheeger : ℝ
  cheeger_positive : 0 < cheeger
```

```

expansion : ∀ A : Finset vertices,
  cheeger * min (A.card : ℝ)
    ((Fintype.card vertices : ℝ) - A.card) ≤
    (boundary generators A : ℝ)
small : (10 : ℝ) * tolerance < cheeger * Fintype.card vertices
repair : AlmostCentralizerRepair generators tolerance
approximation : G → AlmostCentralizerElement generators tolerance
map_one : (approximation 1).permutation = 1
multiplicative : ∀ x ∈ F, ∀ y ∈ F,
  5 * permutationDistance (approximation (x * y)).permutation
    ((approximation x).permutation * (approximation y).permutation) ≤
    Fintype.card vertices
separated : ∀ x ∈ F, ∀ y ∈ F, x ≠ y →
  Fintype.card vertices <
    5 * permutationDistance
      (approximation x).permutation (approximation y).permutation

```

`ExpandingCentralizerFiniteModel` is the data structure that the whole soficity contradiction funnels through. For a group  $G$  and a finite subset  $F$ , it records: a finite vertex type with decidable equality; a finite family of generator permutations; a tolerance; a Cheeger constant  $h > 0$  with the expansion inequality for the generator boundary; the smallness condition  $10t < hn$ ; a repair operator `AlmostCentralizerRepair`; and an approximation map assigning to each element of  $G$  an almost centralizer element, such that the identity maps to the identity, the map is multiplicative on  $F \times F$  up to distance  $n/5$ , and distinct elements of  $F$  are separated by more than  $n/5$ . These are precisely the hypotheses under which the cluster-group construction turns the approximation into an honest homomorphism-like map into a finite group. The constructor `expandingCentralizerFiniteModel OfImprovement` shows the repair can be replaced by the weaker `HasAlmostCentralizerImprovement`. In the global argument, a hypothetical sofic approximation of the nine-prefix elementary group, combined with the property (T) expansion of commutation-defect boundaries, produces such a model of the local prefix transposition group for every finite  $F$ , which by `lef_of_expanding_centralizer_models` would force that group to be LEF.

**lef\_of\_expanding\_centralizer\_models** *thm*

```

theorem lef_of_expanding_centralizer_models {G : Type*} [Group G]
  (hmodels : ∀ F : Finset G,
    Nonempty (ExpandingCentralizerFiniteModel G F)) : LEF G := by
-- proof: 7 lines, omitted

```

`lef_of_expanding_centralizer_models` states that if every finite subset  $F$  of  $G$  admits an `ExpandingCentralizerFiniteModel`, then  $G$  is LEF, locally embeddable into finite groups. The proof composes the two preceding theorems. First, `exists_finite_group_embedding_of_expanding_centralizer` builds the gap from expansion via `almostCentralizerGap_of_expansion`, forms the finite cluster group with `almostCentralizerClusterGroup`, and maps  $G$  into it by taking clusters of approximations; injectivity on  $F$  comes from the separation axiom against the cluster-equality criterion, the identity is preserved by `map_one`, and multiplicativity on  $F$  follows because the cluster of the approximation of  $xy$  and the corrected product of the approximations of  $x$  and  $y$  are within the closeness margin, using the model's multiplicativity plus the repair's distance bound and one more application of the dichotomy. Second, `exists_permutation_embedding_of_expanding_centralizer` replaces the abstract finite group with a symmetric group via the Cayley action `MulAction.toPermHom` composed with an equivalence to `Fin`. This theorem is the hinge of step 2 in the global chain: soficity of the nine-prefix elementary group would supply the models, hence LEF of the source local prefix transposition group, contradicting `sourceLocalPrefixTranspositionGroup_notLEF`.

*Remaining declarations in this section:*

|                                           |     |                                                                               |
|-------------------------------------------|-----|-------------------------------------------------------------------------------|
| <code>permutationDistance</code>          | DEF | Hamming distance between two permutations, counting points where they differ. |
| <code>permutationDistance_self</code>     | THM | The distance from a permutation to itself is zero.                            |
| <code>permutationDistance_comm</code>     | THM | Permutation distance is symmetric.                                            |
| <code>permutationDistance_triangle</code> | THM | Triangle inequality for permutation distance.                                 |



|                                                    |        |                                                                                                                                 |
|----------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------|
| permutationDistance_mul_left                       | THM    | Left multiplication by a fixed permutation preserves the distance.                                                              |
| permutationDistance_mul_right                      | THM    | Right multiplication by a fixed permutation preserves the distance, via an explicit bijection of disagreement sets.             |
| permutationDistance_mul_le                         | THM    | Distance between products is at most the sum of the factors' distances.                                                         |
| permutationDistance_inv                            | THM    | Inversion preserves permutation distance.                                                                                       |
| permutationCommutationDefect_one                   | THM    | The identity permutation has zero commutation defect.                                                                           |
| permutationCommutationDefect_inv                   | THM    | Commutation defect is invariant under inverting the test permutation.                                                           |
| permutationCommutationDefect_mul_le                | THM    | Commutation defect is subadditive: the defect of a product is at most the sum of defects.                                       |
| AlmostCentralizerElement                           | STRUCT | Bundles a permutation with a proof that its commutation defect is at most the tolerance.                                        |
| AlmostCentralizerElement.ext                       | THM    | Two almost centralizer elements with equal underlying permutations are equal.                                                   |
| almostCentralizerElementFinite                     | INST   | There are finitely many almost centralizer elements, by injectivity of the underlying permutation.                              |
| AlmostCentralizerGap                               | STRUCT | Dichotomy: any two almost centralizer elements lie within $n/5$ or beyond $4n/5$ in permutation distance, $n$ the vertex count. |
| almostCentralizerSetoid                            | DEF    | Closeness $5d(p, q) \leq n$ is an equivalence relation, transitivity using the gap dichotomy.                                   |
| AlmostCentralizerClusters                          | ABBR   | The quotient of almost centralizer elements by the closeness relation.                                                          |
| almostCentralizerCluster                           | DEF    | Sends an almost centralizer element to its cluster.                                                                             |
| almostCentralizerCluster_eq_iff                    | THM    | Clusters coincide exactly when the underlying permutations satisfy $5d \leq n$ .                                                |
| AlmostCentralizerRepair                            | STRUCT | A correction operator making products defect-small again while moving them at most $n/5$ .                                      |
| HasAlmostCentralizerImprovement                    | STRUCT | Every permutation with defect at most twice the tolerance is close to one within tolerance.                                     |
| almostCentralizerRepairOfImprovement               | DEF    | Builds a repair operator from the improvement property by choosing improved permutations.                                       |
| correctedAlmostCentralizerProduct                  | DEF    | The repaired product of two almost centralizer elements, corrected back within tolerance.                                       |
| correctedAlmostCentralizerProduct_congr            | THM    | The corrected product respects the closeness relation in both arguments.                                                        |
| almostCentralizerClusterMul                        | DEF    | Multiplication on clusters induced by the corrected product.                                                                    |
| almostCentralizerOne                               | DEF    | The identity permutation as an almost centralizer element.                                                                      |
| almostCentralizerInverse                           | DEF    | The inverse of an almost centralizer element, using defect inversion invariance.                                                |
| almostCentralizerClusterInv                        | DEF    | Inversion descends to clusters since permutation distance is inversion invariant.                                               |
| correctedAlmostCentralizerProduct_assoc            | THM    | Corrected products are associative up to closeness, by triangle inequalities and the gap dichotomy.                             |
| expandingCentralizerFiniteModelVerticesFintype     | INST   | The model's vertex type is a fintype.                                                                                           |
| expandingCentralizerFiniteModelVerticesDecidableEq | INST   | The model's vertex type has decidable equality.                                                                                 |

|                                                            |      |                                                                                                                                           |
|------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------|
| expandingCentralizerFiniteModel<br>GeneratorFintype        | INST | The model's generator index type is a fintype.                                                                                            |
| expandingCentralizerFiniteModelOf<br>Improvement           | DEF  | Constructs a model from expansion, smallness, an improvement property, and a separated approximately multiplicative approximation map.    |
| exists_finite_group_embedding_of_<br>expanding_centralizer | THM  | Extracts from a model a finite group with a map injective and multiplicative on $F$ .                                                     |
| exists_permutation_embedding_of_<br>expanding_centralizer  | THM  | Upgrades the finite group to a symmetric group via the Cayley embedding.                                                                  |
| exists_positive_antitone_slow_<br>vanishing_threshold      | THM  | Builds positive antitone $\eta \rightarrow 0$ vanishing slowly enough that $e_n/\eta_n \rightarrow 0$ , via square roots of tail suprema. |
| exists_positive_antitone_slow_<br>overlap_scales           | THM  | Produces two nested slow scales $\eta$ and $H$ with $e/\eta \rightarrow 0$ and $\eta/H \rightarrow 0$ .                                   |

#### 4.11 Rank offsets with exceptional indices (ExceptionalRankOffset)

Extends the earlier common logarithmic rank offset selection to tolerate an exceptional set of indices. `rankDropCount_goodSubtype_eq` rewrites the drop count restricted to a good set  $G$  as a sum of indicators, and `rankDropCount_le_goodSubtype_add_exceptions` bounds the full drop count by the good-subtype count plus the number of exceptional indices. Choosing the offset via the earlier existence theorem on the subtype yields `exists_common_log_rank_offset_except`, and the sequential version `exists_common_log_rank_offsets_tendsto_zero_except` shows the normalized drop count tends to zero when  $\eta_n/H_n \rightarrow 0$  and the exception density vanishes. This robust offset selection lets the transport argument quantize logarithmic ranks so that comparisons  $(1 - \eta)x \leq y$ , valid only on the good indices, almost never drop the quantized rank.

**exists\_common\_log\_rank\_offsets\_tendsto\_zero\_except** thm

```

theorem exists_common_log_rank_offsets_tendsto_zero_except
  (ι : ℕ → Type*)
  [∀ n, Fintype (ι n)] [∀ n, DecidableEq (ι n)]
  (G : (n : ℕ) → Finset (ι n))
  (x y : (n : ℕ) → ι n → ℝ)
  (eta H N : ℕ → ℝ) (C : ℝ)
  (hx : ∀ n i, i ∈ G n → 0 < x n i)
  (heta0 : ∀ n, 0 ≤ eta n)
  (heta1 : ∀ n, eta n < 1)
  (hH : ∀ n, 0 < H n)
  (hN : ∀ n, 0 < N n)
  (hcard : ∀ n, ((G n).card : ℝ) ≤ C * N n)
  (hcomparison : ∀ n i, i ∈ G n →
    (1 - eta n) * x n i ≤ y n i)
  (heta : Tendsto eta atTop (nhds 0))
  (hratio : Tendsto (fun n => eta n / H n) atTop (nhds 0))
  (hexceptions : Tendsto
    (fun n =>
      (((Finset.univ : Finset (ι n)) \ G n).card : ℝ) / N n)
    atTop (nhds 0)) :
  ∃ r : ℕ → ℝ,
    (∀ n, r n ∈ Set.Ico 0 (H n)) ∧
    Tendsto
      (fun n =>
        SoficGroups.rankDropCount
          (fun i => Real.log (x n i))
          (fun i => Real.log (y n i)) (H n) (r n) / N n)
      atTop (nhds 0) := by
-- proof: 64 lines, omitted

```

This is the sequential form of the exceptional-offset selection. Given index types  $\iota_n$ , good sets  $G_n$  whose cardinality is at most  $CN_n$ , positive quantities  $x_n$  on  $G_n$ , comparisons  $(1 - \eta_n)x_n(i) \leq y_n(i)$  on  $G_n$ , and scales with  $\eta_n \rightarrow 0$ ,  $\eta_n/H_n \rightarrow 0$ , and exceptional density tending to zero, it produces offsets  $r_n \in [0, H_n)$  such that

the normalized count of indices where the quantized logarithm  $\lfloor (\log x + r)/H \rfloor$  strictly drops when passing from  $x$  to  $y$  tends to zero. The proof selects each  $r_n$  by `exists_common_log_rank_offset_except`, which averages over offsets on the good subtype (via the earlier `exists_common_log_rank_offset`) and adds the exceptional count through `rankDropCount_le_goodSubtype_add_exceptions`; the resulting bound  $C|\log(1 - \eta_n)|/H_n$  plus exception density is squeezed to zero using `abs_log_one_sub_div_tendsto_zero`. The point of the quantization is to convert multiplicative near-monotonicity of ranks (a  $(1 - \eta)$  loss) into an integer-valued label that almost never decreases, which the `RankArcCharging` segment then treats as an almost-conserved quantity of the permutation model; this pairing is a recurring device in the transport argument against soficity.

*Remaining declarations in this section:*

|                                                          |     |                                                                                                           |
|----------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------|
| <code>rankDropCount_goodSubtype_eq</code>                | THM | Rewrites the rank drop count over the subtype of $G$ as a sum of drop indicators over $G$ .               |
| <code>rankDropCount_le_goodSubtype_add_exceptions</code> | THM | Bounds the full rank drop count by the good-subtype count plus the number of exceptional indices.         |
| <code>exists_common_log_rank_offset_except</code>        | THM | Finds an offset $r \in [0, H)$ bounding log-rank drops by $ G  \cdot  \log(1 - \eta) /H$ plus exceptions. |

## 4.12 Cylinder swap word identity (ThompsonPrefixInsertion)

A single computational lemma inside the `ThompsonPrefixInsertion` namespace, placed here ahead of the main development of prefix word actions. For incomparable binary words  $a$  and  $b$ , the unit `cylinderSwap` of the binary Leavitt algebra, defined through `PrefixCompression.transpositionValue`, satisfies: the swap times `leavittWordS` of  $b$  equals `leavittWordS` of  $a$ . The proof expands the transposition value and uses the relations that  $T$ -words annihilate incomparable  $S$ -words and invert their own. It is the right-hand companion of the left action law proved later and supplies the `prefixing` field of `cylinderSwap_prefixWordAction_right`, feeding the `PrefixWordAction` interface developed below.

**cylinderSwap\_mul\_wordS\_right** thm

```
theorem cylinderSwap_mul_wordS_right
  (a b : List (Fin 2))
  (hab : ¬ a <+ b) (hba : ¬ b <+ a) :
  (↑(cylinderSwap a b hab hba) : BinaryLeavitt) *
    leavittWordS b = leavittWordS a := by
-- proof: 9 lines, omitted
```

The lemma asserts that for binary words  $a$  and  $b$  with neither a prefix of the other, the transposition unit `cylinderSwap` satisfies, as an element of `BinaryLeavitt`, the identity: swap times  $S_b$  equals  $S_a$ , where  $S_w$  denotes `leavittWordS` of  $w$ . The proof unfolds the definition through `PrefixCompression.transpositionValue`, which is  $S_a T_b + S_b T_a + (1 - S_a T_a - S_b T_b)$ , multiplies on the right by  $S_b$ , and simplifies with the two structural relations of the Leavitt algebra: `leavittWordT_mul_wordS_self` gives  $T_w S_w = 1$ , and `leavittWordT_mul_wordS_of_incomparable` gives  $T_u S_w = 0$  for incomparable words. Only the summand  $S_a T_b S_b = S_a$  survives. Conceptually the unit implements the tree transformation exchanging the cylinder of  $a$  with the cylinder of  $b$  while fixing everything else, and this computation is its action on the generating isometry indexed by  $b$ . It is stated in this early slice because it is exactly the `prefixing` field of `cylinderSwap_prefixWordAction_right` proved later, and hence feeds the `PrefixWordAction` calculus used to conjugate prefix insertion homomorphisms.

## 4.13 Elementary groups under ring maps (ElementaryRingQuotient)

Functoriality of elementary matrix groups in the coefficient ring. A ring homomorphism  $f : R \rightarrow S$  induces `elementaryMatrixUnitMap`, a monoid homomorphism on units of matrix rings; it sends the elementary unit with entry  $a$  to the elementary unit with entry  $f(a)$  (`elementaryMatrixUnitMap_elementaryUnit`). Consequently the image of `elementaryGroup` lands in the elementary group over  $S$ , with equality when  $f$  is surjective (`elementaryGroup_map_eq_of_surjective`). These transfer results underpin the Morita-style equivalences of elementary

groups established in the `LeavittElementaryMorita` section, used to identify prefix elementary groups with one another and to move property (T) between them along ring equivalences of coefficients.

#### `elementaryGroup_map_eq_of_surjective` thm

```
theorem elementaryGroup_map_eq_of_surjective
  (f : R →+ S) (hf : Function.Surjective f) :
  (elementaryGroup ι R).map (elementaryMatrixUnitMap f) =
    elementaryGroup ι S := by
-- proof: 8 lines, omitted
```

`elementaryGroup_map_eq_of_surjective` states that when the coefficient ring homomorphism  $f : R \rightarrow S$  is surjective, the image of the elementary group over  $R$  under the induced map on matrix unit groups is exactly the elementary group over  $S$ . One inclusion is `elementaryGroup_map_le`, functoriality: since `elementaryMatrixUnitMap_elementaryUnit` shows the induced map carries an elementary unit with entry  $a$  to the elementary unit with entry  $f(a)$ , the image of the closure of elementary units lands in the closure over  $S$ . For the converse, every generating elementary unit over  $S$  has entry  $b = f(a)$  for some  $a$  by surjectivity, so it is the image of the corresponding elementary unit over  $R$ ; the closure characterization `Subgroup.closure_le` completes the argument. Though short, this lemma is the workhorse behind `elementaryCoefficientGroupEquiv` in the `LeavittElementaryMorita` section: a ring equivalence is in particular surjective, so it induces an isomorphism of elementary groups. That is one of the three moves (coefficient change, block flattening, reindexing) composed to identify the elementary groups of sizes three and nine over the binary Leavitt algebra, along which property (T) is transported.

*Remaining declarations in this section:*

|                                                     |     |                                                                                              |
|-----------------------------------------------------|-----|----------------------------------------------------------------------------------------------|
| <code>elementaryMatrixUnitMap</code>                | DEF | Monoid homomorphism on matrix unit groups induced by a coefficient ring homomorphism.        |
| <code>elementaryMatrixUnitMap_elementaryUnit</code> | THM | Maps the elementary unit with entry $a$ to the elementary unit with entry $f(a)$ .           |
| <code>elementaryGroup_map_le</code>                 | THM | The image of the elementary group under the induced map lies in the target elementary group. |

## 4.14 Prefix elementary group equivalence

A single definition, `binaryPrefixElementaryGroupEquiv`, identifying the elementary group of matrices over the binary Leavitt algebra indexed by a finite type  $\iota$  with the prefix elementary group of any binary prefix code  $E$  on  $\iota$ . It is the composite of the isomorphism onto the image of the injective corner unit homomorphism `prefixCornerUnitHom` with the identification `prefixCornerElementaryGroup_map` of that image with `prefixElementaryGroup`. This equivalence is the change-of-model tool that lets properties proved uniformly for matrix elementary groups, notably property (T) after the Morita reductions, be transported to the prefix elementary groups appearing in the non-soficity chain, for example to the code `alphaZeroPrefixCode` later in this slice.

#### `binaryPrefixElementaryGroupEquiv` def

```
def binaryPrefixElementaryGroupEquiv
  {ι : Type u} [Fintype ι] [DecidableEq ι]
  (E : BinaryPrefixCode ι) :
  elementaryGroup ι BinaryLeavitt ≃ prefixElementaryGroup E :=
  ((elementaryGroup ι BinaryLeavitt).equivMapOfInjective
    (prefixCornerUnitHom E) (prefixCornerUnitHom_injective E)).trans
    (MulEquiv.subgroupCongr (prefixCornerElementaryGroup_map E))
```

`binaryPrefixElementaryGroupEquiv` is a multiplicative equivalence between `elementaryGroup` of  $\iota$ -indexed matrices over `BinaryLeavitt` and `prefixElementaryGroup` of an arbitrary binary prefix code  $E$  on the finite index type  $\iota$ . It is assembled from two general pieces: `Subgroup.equivMapOfInjective` identifies the elementary group with its image under the corner unit homomorphism `prefixCornerUnitHom`, which is injective by `prefixCornerUnitHom_`

injective; then `MulEquiv.subgroupCongr` transports along the equality `prefixCornerElementaryGroup_map`, which says this image is precisely the prefix elementary group of the code. The corner homomorphism realizes an  $\iota$ -indexed matrix algebra inside the Leavitt algebra using the partial isometries  $S$  and  $T$  attached to the code words, so the equivalence is a concrete instance of corner Morita equivalence. Its importance is organizational: statements proved uniformly for matrix elementary groups transfer immediately to the prefix elementary groups that actually appear in the non-soficity chain, in particular to the prefix elementary group of `alphaZeroPrefixCode` in `alphaZeroPrefixElementaryGroup_hasPropertyT_of_nine`, and ultimately toward the nine-prefix code group whose soficity is refuted at the end of the file.

#### 4.15 Sofic separation and tracked products (CompressionCriterion)

Preliminary facts for the compression criterion. `normalizedHamming_distinct_tendsto` shows that in any sofic approximation the actions of two distinct group elements become fully separated: the normalized Hamming distance between their images tends to one, deduced from separation of  $gh^{-1}$  from the identity and asymptotic multiplicativity, via bi-invariance and triangle properties of the normalized Hamming distance. The second half defines `productTrackedTable`, the multiplication table of  $F$  with the identity adjoined, and records that it contains the identity, the elements of  $F$ , and all products of pairs from  $F$ . These closure properties let later constructions demand approximate multiplicativity on a finite set that is stable under exactly the products the argument uses.

**normalizedHamming\_distinct\_tendsto** *thm*

```
theorem normalizedHamming_distinct_tendsto
  {G : Type*} [Group G]
  (A : SoficGroups.SoficApproximation G)
  {g h : G} (hne : g ≠ h) :
  Tendsto
    (fun n => SoficGroups.normalizedHamming
      ((A.model n).action g) ((A.model n).action h))
    atTop (N 1) := by
  -- proof: 43 lines, omitted
```

For a sofic approximation  $A$  of  $G$  and distinct elements  $g \neq h$ , the normalized Hamming distance between the model actions of  $g$  and  $h$  tends to one. The proof reduces the two-element statement to the defining axioms about single elements:  $gh^{-1} \neq 1$ , so the separation axiom makes the distance from the action of  $gh^{-1}$  to the identity tend to one, while the multiplicativity axiom makes the distance between the action of  $(gh^{-1})h$  and the product of the actions tend to zero. Right invariance of the normalized Hamming metric (`normalizedHamming_mul_right`) converts the first quantity into the distance between the product of actions and the action of  $h$ , and a triangle inequality then bounds the distance between the actions of  $g$  and  $h$  from below by a sequence tending to one; the constant upper bound `normalizedHamming_le_one` closes the squeeze. The lemma upgrades soficity from the statement that each nonidentity element moves almost all points to the statement that any two distinct elements act almost nowhere equally, the form needed when a sofic approximation must distinguish all elements of a finite subset simultaneously, as in the compression and tracked-table constructions that follow.

*Remaining declarations in this section:*

|                                          |     |                                                              |
|------------------------------------------|-----|--------------------------------------------------------------|
| <code>productTrackedTable</code>         | DEF | The multiplication table of $F$ with the identity adjoined.  |
| <code>one_mem_productTrackedTable</code> | THM | The identity belongs to the tracked table.                   |
| <code>mem_productTrackedTable</code>     | THM | Every element of $F$ belongs to the tracked table.           |
| <code>mul_mem_productTrackedTable</code> | THM | Products of two elements of $F$ belong to the tracked table. |

#### 4.16 Permutation graphs and Kazhdan displacement

Two bridges between combinatorics and representation theory. `permutationGraph` realizes a permutation  $p$  as the finset of pairs  $(x, p(x))$ ; the boundary of this subset under the diagonal action of the generator family, in

the sense of the earlier boundary count, is exactly `permutationCommutationDefect` of  $p$  (`boundary_permutationGraph_eq_commutationDefect`). Hence generator expansion on  $V \times V$  controls how far from central a permutation can be. Separately, `kazhdan_generator_displacement` extracts from a Kazhdan pair and a unitary representation with no nonzero invariant vectors a generator moving each nonzero vector by at least the Kazhdan constant times its norm. Together these supply the spectral inputs behind the expansion hypotheses of the expanding centralizer models.

#### **boundary\_permutationGraph\_eq\_commutationDefect** thm

```
theorem boundary_permutationGraph_eq_commutationDefect
  {V 1 : Type*} [Fintype V] [Fintype 1] [DecidableEq V]
  (σ : 1 → Equiv.Perm V) (p : Equiv.Perm V) :
  boundary (fun i => (σ i).prodCongr (σ i)) (permutationGraph p) =
    permutationCommutationDefect σ p := by
-- proof: 32 lines, omitted
```

This theorem identifies two counts: the boundary, in the sense of the earlier `boundary` function, of the graph of a permutation  $p$  inside  $V \times V$  under the diagonal generator family  $i \mapsto \sigma_i \times \sigma_i$ , and the commutation defect `permutationCommutationDefect` of  $p$  against  $\sigma$ . The proof works generator by generator and exhibits an explicit bijection: a pair  $z$  in the graph leaves the graph under  $\sigma_i \times \sigma_i$  precisely when  $p(\sigma_i(z_1)) \neq \sigma_i(p(z_1))$ , so projecting to the first coordinate is a bijection onto the set of points where  $p$  fails to commute with  $\sigma_i$ ; injectivity uses that the second coordinate of a graph element is determined as  $p$  of the first. The significance is that it converts a representation-theoretic quantity (failure to centralize the generators) into a combinatorial isoperimetric one (boundary of a subset of the product vertex set). Consequently, an expansion inequality for the product action, ultimately derived from property (T) through spectral estimates, directly bounds how well a permutation can almost commute with all generators, feeding the hypotheses of `almostCentralizerGap_of_expansion` and thereby the entire almost-centralizer cluster construction.

#### **kazhdan\_generator\_displacement** thm

```
theorem kazhdan_generator_displacement
  {G : Type u} {H : Type v} [Group G]
  [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H]
  (P : KazhdanPair.{u, v} G)
  (π : UnitaryRepresentation G H)
  (hfixed : ∀ η : H, (∀ g : G, π g η = η) → η = 0)
  (ξ : H) (hξ : ξ ≠ 0) :
  ∃ g ∈ P.generators,
    P.kazhdanConstant * ||ξ|| ≤ ||π g ξ - ξ|| := by
-- proof: 29 lines, omitted
```

Given a Kazhdan pair  $P$  for  $G$  with generating set and constant  $\kappa$ , and a unitary representation  $\pi$  on a complete complex inner product space whose only invariant vector is zero, the theorem produces for every nonzero  $\xi$  a generator  $g$  with  $\kappa\|\xi\| \leq \|\pi(g)\xi - \xi\|$ . The proof is by contradiction: if every generator moved  $\xi$  by less than  $\kappa\|\xi\|$ , then the normalized vector  $\zeta = \xi/\|\xi\|$  would be a unit vector displaced by less than  $\kappa$  by all generators, and the defining property of `KazhdanPair` (the field invariant) would yield a nonzero invariant vector  $\eta$ , contradicting the hypothesis `hfixed`. Unitarity enters only through linearity of  $\pi(g)$  under the scalar normalization. This quantitative displacement statement is the standard way property (T) is consumed downstream: applied to representations built from hypothetical sofic approximations, for instance on spaces of functions on the model vertices with the invariant part removed, it shows some generator moves every vector a definite amount, which translates into the Cheeger-type expansion inequalities appearing in `ExpandingCentralizerFiniteModel` and in the spectral-gap estimates of the Kun-style transport sections.

*Remaining declarations in this section:*

|                                   |     |                                                                   |
|-----------------------------------|-----|-------------------------------------------------------------------|
| <code>permutationGraph</code>     | DEF | The graph of a permutation as the finset of pairs $(x, p(x))$ .   |
| <code>mem_permutationGraph</code> | THM | Membership criterion: $(x, y)$ lies in the graph iff $y = p(x)$ . |

## 4.17 Prefix word actions and insertion conjugation (ThompsonPrefixInsertion)

Develops the `PrefixWordAction` interface: a unit  $g$  of the binary Leavitt algebra acts on prefix words  $a, b$  when  $gS_a = S_b$  and  $T_a g^{-1} = T_b$ . The relation is stable under appending a common suffix (`prefixWordAction_append`) and composes multiplicatively (`prefixWordAction_mul`). Cylinder swaps realize it concretely: the swap of incomparable  $a$  and  $b$  sends  $a$  to  $b$ , sends  $b$  to  $a$ , and fixes every word incomparable with both. The payoff is `prefixInsertionHom_conjugate_of_prefixWordAction`: conjugating the prefix insertion homomorphism at  $a$  by such a  $g$  gives the insertion at  $b$ ; moreover insertion preserves the binary prefix transposition group. This equivariance is what makes the Thompson-like prefix transposition groups act coherently on inserted copies of unit groups, a mechanism used in the finite generation and locality analysis.

**PrefixWordAction** `struct`

```
structure PrefixWordAction (g : BinaryLeavitt*)
  (a b : List (Fin 2)) : Prop where
  prefixing : (↑g : BinaryLeavitt) * leavittWordS a = leavittWordS b
  deletion :
    leavittWordT a * (↑(g⁻¹) : BinaryLeavitt) = leavittWordT b
```

`PrefixWordAction` of  $g, a, b$  is a two-field proposition about a unit  $g$  of the binary Leavitt algebra: `prefixing` says  $g \cdot S_a = S_b$ , and `deletion` says  $T_a \cdot g^{-1} = T_b$ , where  $S_w, T_w$  are the composite isometries `leavittWordS` and `leavittWordT` attached to a binary word  $w$ . Together they express that  $g$  intertwines the partial isometries of the cylinder of  $a$  with those of the cylinder of  $b$ : thinking of  $S_w$  as the inclusion of the whole space onto cylinder  $w$  and  $T_w$  as its left inverse, the relation says  $g$  restricted to cylinder  $a$  is exactly the prefix replacement sending  $ax$  to  $bx$ . The interface is closed under appending a common suffix (`prefixWordAction_append`) and under composition (`prefixWordAction_mul`), giving a small calculus over binary words, and the cylinder swaps furnish concrete instances. The abstraction matters because the key consequence, conjugation of prefix insertion homomorphisms (`prefixInsertionHom_conjugate_of_prefixWordAction`), only needs these two equations; any group element realizing a prefix substitution therefore transports inserted subgroups identically, which is used to compare local copies of transposition groups inside the Thompson-style group.

**prefixInsertionHom\_conjugate\_of\_prefixWordAction** `thm`

```
theorem prefixInsertionHom_conjugate_of_prefixWordAction
  (g : BinaryLeavitt*) (a b : List (Fin 2))
  (h : PrefixWordAction g a b) (u : BinaryLeavitt*) :
  g * prefixInsertionHom a u * g⁻¹ = prefixInsertionHom b u := by
-- proof: 34 lines, omitted
```

The theorem states that if  $g$  has a prefix word action from  $a$  to  $b$ , then for every unit  $u$ , conjugation by  $g$  carries the value of `prefixInsertionHom` at  $a$  on  $u$  to its value at  $b$  on  $u$ . The insertion homomorphism sends  $u$  to  $S_a u T_a + (1 - S_a T_a)$ , acting by  $u$  inside the cylinder of  $a$  and trivially outside. The proof passes to underlying algebra elements, expands the conjugation with `noncomm_ring` (using the characteristic-two identification of subtraction with addition via `CharTwo.sub_eq_add`), and rewrites the three grouped factors with the `prefixing` equation, the `deletion` equation, and  $gg^{-1} = 1$ , which turns every occurrence of the  $a$ -data into  $b$ -data. The companion result `prefixInsertionHom_mem_binaryPrefixTranspositionGroup` shows insertions keep the prefix transposition group inside itself, since inserted cylinder swaps are again cylinder swaps at prepended words (`prefixInsertionHom_cylinderSwap` with `prepend_not_prefix_prepend`). Together they mean the family of inserted subgroups indexed by binary words is permuted by any elements realizing prefix substitutions, in particular by cylinder swaps; this equivariance underlies the comparison between local and global prefix transposition groups and the finite generation arguments in the Thompson-style analysis.

*Remaining declarations in this section:*

|                                      |     |                                                                                    |
|--------------------------------------|-----|------------------------------------------------------------------------------------|
| <code>prefixWordAction_append</code> | THM | A prefix word action persists after appending the same suffix to both words.       |
| <code>prefixWordAction_mul</code>    | THM | Prefix word actions compose: $gh$ acts along the concatenation of the two actions. |



|                                                  |     |                                                                    |
|--------------------------------------------------|-----|--------------------------------------------------------------------|
| <code>cylinderSwap_prefixWordAction_left</code>  | THM | The cylinder swap of incomparable $a, b$ acts sending $a$ to $b$ . |
| <code>cylinderSwap_prefixWordAction_right</code> | THM | The same swap also sends $b$ back to $a$ .                         |
| <code>cylinderSwap_prefixWordAction_fixed</code> | THM | The swap fixes any word incomparable with both $a$ and $b$ .       |
| <code>prefixInsertionHom_mem_binaryPrefix</code> | THM | Prefix insertion maps the binary prefix transposition group        |
| <code>TranspositionGroup</code>                  |     | into itself, sending swaps to swaps at prepended words.            |

## 4.18 Morita equivalences of elementary groups (LeavittElementaryMorita)

A chain of multiplicative equivalences between elementary groups. Block flattening: matrices over matrices are matrices over the product index (`elementaryBlockMatrixEquiv`), and this identifies the corresponding elementary groups (`elementaryBlockGroupEquiv`), with the diagonal-block case handled by a two-step commutator trick. Reindexing along a bijection and changing coefficients along a ring equivalence give `elementaryReindexGroupEquiv` and `elementaryCoefficientGroupEquiv`. The complete ternary prefix code with words  $[0, 0]$ ,  $[0, 1]$ ,  $[1]$  makes the binary Leavitt algebra isomorphic to three-by-three matrices over itself (`binaryLeavittMatrixThreeEquiv`), and composing everything yields `binaryLeavittElementaryThreeEquivNine`: the elementary groups of sizes three and nine over `BinaryLeavitt` are isomorphic. Property (T), assumed for size nine, therefore transfers to size three and onward to the alpha prefix elementary groups. This is how the property (T) input reaches the groups of the main non-soficity chain.

**`elementaryBlockGroupEquiv`** def

```
def elementaryBlockGroupEquiv [Nontrivial ι] :
  elementaryGroup ι (Matrix κ κ R) ≈*
    elementaryGroup (ι × κ) R :=
  ((elementaryBlockUnitEquiv
    (ι := ι) (κ := κ) (R := R)).subgroupMap
    (elementaryGroup ι (Matrix κ κ R))).trans
    (MulEquiv.subgroupCongr
      (elementaryBlockGroup_map (ι := ι) (κ := κ) (R := R)))
```

`elementaryBlockGroupEquiv` is the multiplicative equivalence between the elementary group of  $\iota$ -indexed matrices whose entries are themselves  $\kappa$ -indexed matrices over  $R$ , and the elementary group over the flattened index  $\iota \times \kappa$ . It is built by restricting the unit-group equivalence `elementaryBlockUnitEquiv`, induced by `Matrix.compRingEquiv`, to the elementary subgroup and transporting along `elementaryBlockGroup_map`, the statement that the image is exactly the flat elementary group. The interesting inclusion is surjectivity onto flat generators: an elementary unit at  $((i, k), (j, l))$  with  $i \neq j$  is directly the image of a block elementary unit with a single-entry block (`elementaryBlockUnitEquiv_elementaryUnit_single`); but when  $i = j$  the flat generator sits inside a diagonal block, and the proof routes through `elementaryUnit_mem_of_two_step`, a commutator-style factorization through an auxiliary index available since  $\iota$  is nontrivial. The other inclusion decomposes an arbitrary block entry by `Matrix.induction_on'` into single entries, using additivity of elementary units via `elementaryUnit_mul`. This Morita move is one of the three composed in `binaryLeavittElementaryThreeEquivNine`, letting size-three elementary groups over a matrix ring be recognized as size-nine elementary groups over the base ring.

**`binaryLeavittElementaryThreeEquivNine`** def

```
def binaryLeavittElementaryThreeEquivNine :
  binaryLeavittElementaryGroup 3 ≈*
    binaryLeavittElementaryGroup 9 :=
  (elementaryCoefficientGroupEquiv
    (ι := Fin 3) binaryLeavittMatrixThreeEquiv).trans
    ((elementaryBlockGroupEquiv
      (ι := Fin 3) (κ := Fin 3) (R := BinaryLeavitt)).trans
      (elementaryReindexGroupEquiv
        (R := BinaryLeavitt) (finProdFinEquiv :
          Fin 3 × Fin 3 ≈ Fin 9)))
```



`binaryLeavittElementaryThreeEquivNine` exhibits a multiplicative equivalence between `binaryLeavittElementaryGroup` at size three and at size nine. It composes three moves. First, `elementaryCoefficientGroupEquiv` along `binaryLeavittMatrixThreeEquiv`, the ring identification of the binary Leavitt algebra with three-by-three matrices over itself; that identification comes from `completePrefixMatrixEquiv` applied to the ternary prefix code with words  $[0, 0]$ ,  $[0, 1]$ ,  $[1]$ , whose completeness `ternaryLeavittPrefixCode_complete` is verified by cylinder splitting identities: the two length-two cylinders sum to the cylinder of  $[0]$ , and that plus the cylinder of  $[1]$  is one. Second, `elementaryBlockGroupEquiv` flattens elementary groups of matrices over matrices to the product index. Third, `elementaryReindexGroupEquiv` along `finProdFinEquiv` renames the product of two three-element index types as a nine-element one. This chain is the formal counterpart of the self-similarity of the binary Leavitt algebra, which is isomorphic to three-by-three matrices over itself, and it is exactly what allows property (T), which the development establishes for the size-nine elementary group, to be pulled back to size three by `binaryLeavittElementaryThree_hasPropertyT_of_nine` and from there to the prefix elementary groups in the main chain.

**`alphaPrefixElementaryGroup_hasPropertyT_of_nine` thm**

```
theorem alphaPrefixElementaryGroup_hasPropertyT_of_nine
  [HasPropertyT.{0, v} (binaryLeavittElementaryGroup 9)] :
  HasPropertyT.{0, v} (prefixElementaryGroup alphaPrefixCode) := by
-- proof: 4 lines, omitted
```

`alphaPrefixElementaryGroup_hasPropertyT_of_nine` transfers property (T) to the prefix elementary group of `alphaPrefixCode`, assuming it for `binaryLeavittElementaryGroup` at size nine. The proof is two instance moves: first `binaryLeavittElementaryThree_hasPropertyT_of_nine` converts the size-nine hypothesis into property (T) for size three through the Morita equivalence `binaryLeavittElementaryThreeEquivNine` and the general transfer principle `hasPropertyT_of_mulEquiv`, which moves property (T) across any multiplicative equivalence; then a second application of the transfer principle along `alphaPrefixElementaryGroupEquiv` lands on the alpha prefix elementary group. The companion `alphaZeroPrefixElementaryGroup_hasPropertyT_of_nine` plays the same game for `alphaZeroPrefixCode` using the generic `binaryPrefixElementaryGroupEquiv` from earlier in this slice. These theorems are the terminal links of the property (T) supply chain here: the hard analytic content lives in the size-nine group, established elsewhere in the file through Kazhdan pairs and spectral estimates, and everything downstream, including expansion of commutation-defect boundaries, the almost centralizer gap, and the spectral-gap inequalities that a sofic approximation would violate, consumes property (T) for precisely these prefix elementary groups.

*Remaining declarations in this section:*

|                                                             |     |                                                                                                          |
|-------------------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------|
| <code>elementaryBlockMatrixEquiv</code>                     | DEF | Ring equivalence flattening matrices of matrices into matrices over the product index type.              |
| <code>elementaryBlockUnitEquiv</code>                       | DEF | The induced multiplicative equivalence on unit groups of these matrix rings.                             |
| <code>elementaryBlockUnitEquiv_elementaryUnit_single</code> | THM | Flattening sends an elementary unit with single-entry block to an elementary unit at the paired indices. |
| <code>elementaryBlockUnitEquiv_elementaryUnit_mem</code>    | THM | The image of any elementary unit lies in the flat elementary group, by matrix induction on the block.    |
| <code>elementaryBlockGroup_map</code>                       | THM | The flattened image of the block elementary group equals the elementary group over the product index.    |
| <code>elementaryReindexUnitEquiv</code>                     | DEF | Unit group equivalence induced by reindexing matrices along a bijection.                                 |
| <code>elementaryReindexUnitEquiv_elementaryUnit</code>      | THM | Reindexing sends the elementary unit at $(i, j)$ to the elementary unit at $(e(i), e(j))$ .              |
| <code>elementaryReindexGroup_map</code>                     | THM | The reindexed image of the elementary group is the elementary group over the new index.                  |
| <code>elementaryReindexGroupEquiv</code>                    | DEF | Multiplicative equivalence of elementary groups along an index bijection.                                |

|                                                                  |     |                                                                                                                      |
|------------------------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------|
| <code>elementaryCoefficientGroupEquiv</code>                     | DEF | A coefficient ring equivalence induces an equivalence of elementary groups.                                          |
| <code>ternaryLeavittWord</code>                                  | DEF | The three binary words $[0, 0]$ , $[0, 1]$ , $[1]$ .                                                                 |
| <code>ternaryLeavittPrefixCode</code>                            | DEF | The prefix code on three indices given by <code>ternaryLeavittWord</code> , prefix-freeness checked by decidability. |
| <code>ternaryLeavittPrefixCode_complete</code>                   | THM | The code idempotent of the ternary code equals one, by splitting cylinders at the root and at zero.                  |
| <code>binaryLeavittMatrixThreeEquiv</code>                       | DEF | Ring equivalence of <code>BinaryLeavitt</code> with three-by-three matrices over itself, from the complete code.     |
| <code>binaryLeavittElementaryThree_hasPropertyT_of_nine</code>   | THM | Transfers property (T) from the size-nine to the size-three elementary group.                                        |
| <code>alphaZeroPrefixElementaryGroup_hasPropertyT_of_nine</code> | THM | Property (T) for the alpha-zero prefix elementary group, given size nine.                                            |

#### 4.19 Swap action case analysis (ThompsonFiniteGeneration)

A packaging lemma in the `ThompsonFiniteGeneration` namespace. Given incomparable words  $a$  and  $b$  and a case hypothesis describing how a word  $w$  relates to them, namely  $a$  is a prefix of  $w$  and the output  $v$  replaces it by  $b$ ; or  $b$  is a prefix and  $v$  replaces it by  $a$ ; or  $w$  is incomparable with both and  $v = w$ , it concludes `PrefixWordAction` of the cylinder swap from  $w$  to  $v$ . The proof simply dispatches to the `append`, `left`, `right`, and `fixed` lemmas of the preceding `ThompsonPrefixInsertion` development. It gives the exact combinatorial description of how prefix transpositions act on arbitrary words, in the form needed when generating the Thompson-style groups from finitely many swaps.

**cylinderSwap\_prefixWordAction\_of\_cases** *thm*

```
theorem cylinderSwap_prefixWordAction_of_cases
  {a b w v : List (Fin 2)}
  (hab : ¬ a <+ b) (hba : ¬ b <+ a)
  (hcase :
    (a <+ w ∧ v = b ++ w.drop a.length) v
    (b <+ w ∧ v = a ++ w.drop b.length) v
    (v = w ∧ ¬ a <+ w ∧ ¬ w <+ a ∧
     ¬ b <+ w ∧ ¬ w <+ b)) :
  PrefixWordAction (cylinderSwap a b hab hba) w v := by
-- proof: 16 lines, omitted
```

The lemma condenses the action of a cylinder swap on an arbitrary binary word into a single case split. Its hypothesis offers three scenarios relating  $w$  to the incomparable pair  $a, b$ : either  $a$  is a prefix of  $w$  and the output  $v$  is  $b$  followed by the remainder of  $w$  after  $a$ ; or symmetrically  $b$  is a prefix and  $v$  replaces it by  $a$ ; or  $w$  is incomparable with both and  $v = w$ . In each case the conclusion is `PrefixWordAction` of `cylinderSwap` from  $w$  to  $v$ . The proof rewrites  $w$  as its prefix followed by the drop via `List.prefix_append_drop` and applies `prefixWordAction_append` to the left or right swap actions `cylinderSwap_prefixWordAction_left` and `cylinderSwap_prefixWordAction_right`, or the fixed-word lemma `cylinderSwap_prefixWordAction_fixed` in the third case. The value of the lemma is that it matches exactly the combinatorial description of a prefix transposition acting on the boundary of the binary tree: swap the two subtrees below  $a$  and  $b$ , fix everything else. In the `ThompsonFiniteGeneration` development this normal form is what allows the words moved by generators to be tracked explicitly when expressing arbitrary transpositions in terms of finitely many generating swaps.

#### 4.20 Three-vector sum-of-squares identities (ThreeProjectionSumOfSquares)

Elementary Hilbert space inequalities preparing the three-projection spectral gap. `three_pair_sum_norm_sq` is a polarization identity expressing the sum of the three pairwise sums' squared norms as the squared norm of the total sum plus the individual squared norms. `norm_add_three_sq_le_three_mul` bounds the squared norm of a triple sum by three times the sum of squares, via the triangle inequality and a quadratic mean estimate. `three_pairwise_residual_spectral_gap` combines pairwise lower bounds of the form  $(1 - \varepsilon)(\|a\|^2 + \|b\|^2) \leq \|a + b\|^2$

into the triple bound with constant  $1 - 2\varepsilon$ . Applied later with the residual projections of three subspaces, this reduces the three-subspace spectral gap of the residual map to two-subspace angle estimates, one for each of the three pairs.

#### three\_pairwise\_residual\_spectral\_gap thm

```
theorem three_pairwise_residual_spectral_gap
  (a b c : H) (ε : ℝ)
  (hab : (1 - ε) * (||a|| ^ 2 + ||b|| ^ 2) ≤ ||a + b|| ^ 2)
  (hbc : (1 - ε) * (||b|| ^ 2 + ||c|| ^ 2) ≤ ||b + c|| ^ 2)
  (hca : (1 - ε) * (||c|| ^ 2 + ||a|| ^ 2) ≤ ||c + a|| ^ 2) :
  (1 - 2 * ε) * (||a|| ^ 2 + ||b|| ^ 2 + ||c|| ^ 2) ≤
    ||a + b + c|| ^ 2 := by
  -- proof: 2 lines, omitted
```

Given three vectors and a parameter  $\varepsilon$ , suppose each pairwise sum satisfies the lower bound that its squared norm is at least  $(1 - \varepsilon)$  times the sum of the two squared norms. The theorem concludes the triple bound: the squared norm of  $a + b + c$  is at least  $(1 - 2\varepsilon)$  times the total sum of squares. The proof is one `nlinarith` call from the exact identity `three_pair_sum_norm_sq`, which states that the three pairwise squared norms sum to the triple squared norm plus the individual squares; substituting the three hypotheses converts the pairwise information directly into the triple estimate, with the factor two in  $2\varepsilon$  coming from each vector appearing in two of the three pairs. Though completely elementary, the lemma carries the structural weight of the three-subspace argument later: pairwise angle estimates between fixed subspaces of pairs of subgroups, obtained from the sector analysis of `TwoOrthogonalSectors`, each give a  $(1 - c)$  bound for the corresponding pair of residual projections, and this lemma assembles them into the spectral gap of the three-residual map in `ejThreeResidualMap_spectral_gap`, from which the projection-average iteration converges geometrically to a common invariant vector.

*Remaining declarations in this section:*

|                                             |     |                                                                                                                  |
|---------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------|
| <code>three_pair_sum_norm_sq</code>         | THM | Identity: the three pairwise sums' squared norms equal the total sum's squared norm plus the individual squares. |
| <code>norm_add_three_sq_le_three_mul</code> | THM | Triangle bound $\ a + b + c\ ^2 \leq 3(\ a\ ^2 + \ b\ ^2 + \ c\ ^2)$ .                                           |

## 4.21 Angle bounds across orthogonal sectors (TwoOrthogonalSectors)

Shows the quantitative angle bound  $8\|\langle a, b \rangle\|^2 \leq \|a\|^2\|b\|^2$  is stable under orthogonal decompositions. `inner_sq_le_of_orthogonal_two_sector` combines the bound on two mutually orthogonal sectors, using a scalar inequality of arithmetic-geometric type together with vanishing cross inner products. `inner_sq_le_of_finite_orthogonal_sector_splitting` iterates it: given a finite family of subspaces with orthogonal projections preserving  $U$ ,  $V$ , and each other, if the bound holds on each orthocomplement sector and inner products vanish on the common intersection, then it holds for all vectors of  $U$  and  $V$ , by induction on the finite index set, splitting along one projection at a time. In the property (T) argument this propagates local angle estimates between subgroup fixed spaces to global ones.

#### inner\_sq\_le\_of\_finite\_orthogonal\_sector\_splitting thm

```
theorem inner_sq_le_of_finite_orthogonal_sector_splitting
  {ι : Type*} [Fintype ι]
  (F : ι → Submodule ℂ H)
  [∀ i, (F i).HasOrthogonalProjection]
  (U V : Submodule ℂ H)
  (hU : ∀ (i : ι) ⟨x : H⟩, x ∈ U →
    (F i).starProjection x ∈ U)
  (hV : ∀ (i : ι) ⟨x : H⟩, x ∈ V →
    (F i).starProjection x ∈ V)
  (hF : ∀ (i j : ι) ⟨x : H⟩, x ∈ F j →
    (F i).starProjection x ∈ F j)
```

```

(hsector : ∀ (i : ι) (a b : H),
  a ∈ U → b ∈ V → a ∈ (F i)⊥ → b ∈ (F i)⊥ →
  8 * ||inner C a b||2 ≤ ||a||2 * ||b||2)
(hterminal : ∀ (a b : H), a ∈ U → b ∈ V →
  (∀ i, a ∈ F i) → (∀ i, b ∈ F i) → inner C a b = 0)
(a b : H) (ha : a ∈ U) (hb : b ∈ V) :
  8 * ||inner C a b||2 ≤ ||a||2 * ||b||2 := by
-- proof: 82 lines, omitted

```

This is an induction principle for propagating the quantitative angle bound  $8\|\langle a, b \rangle\|^2 \leq \|a\|^2\|b\|^2$ . The setting: a finite family of subspaces  $F_i$  with orthogonal projections, and two subspaces  $U, V$  such that each projection preserves  $U, V$ , and every  $F_j$ ; the bound is assumed on each sector where both vectors are orthogonal to some  $F_i$  (with  $a \in U, b \in V$ ), and inner products vanish outright when both vectors lie in every  $F_i$ . The conclusion holds for all  $a \in U, b \in V$ . The proof strengthens the statement to allow vectors already inside  $F_j$  for all  $j$  outside a finite set  $s$  and inducts on  $s$ : removing an index  $i$ , it splits  $x = x_0 + x_1$  and  $y = y_0 + y_1$  along the projection onto  $F_i$  and its orthocomplement, checks all the membership invariances, applies the inductive hypothesis to the  $F_i$ -components and the sector hypothesis to the orthogonal components, and merges the two bounds with `inner_sq_le_of_orthogonal_two_sector`, whose cross terms vanish by orthogonality. In the property (T) argument this transports angle estimates, proved sectorwise on orthocomplements of auxiliary fixed spaces, to the full fixed subspaces of pairs of subgroups.

*Remaining declarations in this section:*

`inner_sq_le_of_orthogonal_two_sector` *THM*    The bound  $8\|\langle a, b \rangle\|^2 \leq \|a\|^2\|b\|^2$  persists under summing orthogonal sector pairs.

## 4.22 Averaged projections and spectral gaps (ThreeOrthogonalProjections)

Builds the alternating-projection machinery for three subspaces  $V_0, V_1, V_2$  of a complex inner product space. `ejResidualProjection` is projection onto the orthocomplement; `ejThreeResidualMap` sums the three residuals; `ejThreeResidualEnergy` is the total squared residual; `ejThreeProjectionAverage` is the average of the three projections, written as identity minus one third of the residual map. Exact identities compute the energy of an averaged vector, and `ejThreeResidualMap_spectral_gap` converts pairwise lower bounds into a gap for the residual map. Two-subspace angle bounds give `ejTwoResidual_spectral_gap_of_angle`, hence `ejThreeResidual_spectral_gap_of_pair_angles`; iterating the average then contracts the energy geometrically and successive iterates form a geometric Cauchy sequence. Fixed points of the average are exactly common members of the three subspaces. This furnishes the quantitative route from pairwise angle estimates between fixed subspaces to common invariant vectors in the property (T) proof.

**ejThreeProjectionAverage** *def*

```

def ejThreeProjectionAverage : H →L[C] H :=
  ContinuousLinearMap.id C H -
  (3 : C)-1 • ejThreeResidualMap V

```

`ejThreeProjectionAverage` is the continuous linear map  $A = \text{id} - \frac{1}{3}D$ , where  $D$  is `ejThreeResidualMap`, the sum of the residual projections onto the orthocomplements of the three subspaces  $V_0, V_1, V_2$ . Since each residual is identity minus the projection,  $A$  is precisely the average  $\frac{1}{3}(P_0 + P_1 + P_2)$  of the three orthogonal projections, a simultaneous version of the von Neumann alternating projection operator. The accompanying quantity `ejThreeResidualEnergy` measures the total squared distance of a vector to the three subspaces, and the exact identity `ejThreeResidualEnergy_average` computes how one application of  $A$  changes it: the energy drops by two thirds of  $\|Dx\|^2$  and regains one ninth of the energy of  $Dx$ . When a spectral gap  $\delta E(x) \leq \|Dx\|^2$  holds, this yields the contraction factor  $1 - \delta/3$  of `ejThreeResidualEnergy_average_le`, so iterating  $A$  drives a vector geometrically toward the common intersection while, by `ejThreeProjectionAverage_dist_sq_le`, moving it a controlled total distance. This operator is the engine that converts pairwise angle information into common almost-fixed vectors for three subgroups, the analytic mechanism behind the property (T) criterion instantiated in the following unitary sections.

**ejTwoResidual\_spectral\_gap\_of\_angle** thm

```

theorem ejTwoResidual_spectral_gap_of_angle
  (U V : Submodule C H)
  [U.HasOrthogonalProjection] [V.HasOrthogonalProjection]
  [(U ∩ V).HasOrthogonalProjection]
  (c : ℝ) (hc : 0 ≤ c) (hc' : c ≤ 1)
  (hangle : ∀ (u v : H),
    u ∈ U → u ∈ (U ∩ V)⊥ →
    v ∈ V → v ∈ (U ∩ V)⊥ →
    ‖inner C u v‖ ≤ c * ‖u‖ * ‖v‖)
  (x : H) :
  (1 - c) *
    (‖ejResidualProjection U x‖2 +
     ‖ejResidualProjection V x‖2) ≤
    ‖ejResidualProjection U x +
     ejResidualProjection V x‖2 := by
  -- proof: 57 lines, omitted

```

**ejTwoResidual\_spectral\_gap\_of\_angle** is the two-subspace spectral gap. Assume every pair of vectors  $u \in U$ ,  $v \in V$ , both orthogonal to  $U \cap V$ , satisfies the angle bound  $\|\langle u, v \rangle\| \leq c\|u\|\|v\|$  with  $0 \leq c \leq 1$ . Then for every  $x$ ,  $(1 - c)$  times the sum of the two squared residual norms is at most the squared norm of the sum of the residuals. The proof first splits  $x = q + w$  with  $q$  the projection onto  $U \cap V$ , on which both residuals vanish, reducing to  $w$  orthogonal to the intersection. There, **ejTwoProjection\_norm\_sq\_sum\_le\_of\_angle** bounds the projections' energies by  $(1 + c)\|w\|^2$ , an almost-Pythagoras estimate using the angle bound between the two projected components, whence the coercivity **ejTwoResidualEnergy\_ge\_of\_angle**:  $(1 - c)\|w\|^2$  lower-bounds the residual energy  $e$ . Finally the duality step:  $e$  equals the real inner product of the residual sum  $z$  against  $w$  by **ejTwoResidual\_sum\_re\_inner**, so  $e \leq \|z\|\|w\|$ , and squaring against the coercivity gives  $(1 - c)e \leq \|z\|^2$ . This inequality is the precise sense in which a bounded angle between two subspaces forces their residual projections to reinforce rather than cancel; three such bounds feed **ejThreeResidual\_spectral\_gap\_of\_pair\_angles**.

**ejThreeProjectionAverage\_iterate\_dist\_le\_geometric** thm

```

theorem ejThreeProjectionAverage_iterate_dist_le_geometric
  (δ : ℝ) (hδ : δ ≤ 3)
  (hgap : ∀ x : H,
    δ * ejThreeResidualEnergy V x ≤ ‖ejThreeResidualMap V x‖2)
  (x : H) (n : ℕ) :
  dist ((ejThreeProjectionAverage V : H → H)[n] x)
    (((ejThreeProjectionAverage V : H → H)[n + 1] x) ≤
    Real.sqrt (ejThreeResidualEnergy V x / 3) *
    (1 - δ / 6)n := by
  -- proof: 55 lines, omitted

```

Under a spectral gap hypothesis  $\delta E(x) \leq \|Dx\|^2$  valid for all vectors, with  $\delta \leq 3$ , the theorem bounds the distance between consecutive iterates of the projection average: the distance from  $A^n x$  to  $A^{n+1} x$  is at most  $\sqrt{E(x)/3}$  times  $(1 - \delta/6)^n$ . The proof combines two earlier facts. The energy along the orbit decays geometrically, with  $E$  of  $A^n x$  at most  $(1 - \delta/3)^n E(x)$ , by **ejThreeResidualEnergy\_iterate\_le**; and a single averaging step moves a point by at most the square root of a third of its current energy, by **ejThreeProjectionAverage\_dist\_sq\_le**. Writing  $q = 1 - \delta/3$  and  $r = 1 - \delta/6$ , the inequality  $q \leq r^2$  lets the square root of the decayed energy be dominated by  $r^n$  times the initial scale. Consequently the orbit is a Cauchy sequence with geometrically summable increments; in a complete space it converges to a fixed point of  $A$ , which **ejThreeProjectionAverage\_fixed\_mem** identifies as a common element of the three subspaces, and the total displacement from the starting vector is explicitly controlled. This effective convergence, with all constants explicit, is what the unitary applications need to produce genuinely invariant vectors near almost-invariant ones.

**ejThreeProjectionAverage\_fixed\_mem** thm

```

theorem ejThreeProjectionAverage_fixed_mem
  (x : H) (hx : ejThreeProjectionAverage V x = x)
  (i : Fin 3) : x ∈ V i := by

```

-- proof: 10 lines, omitted

The final theorem of the section shows that any fixed point  $x$  of the projection average lies in each of the three subspaces. The chain has three links. By `ejThreeProjectionAverage_fixed_iff_residualMap_eq_zero`, fixedness means the three-residual map vanishes at  $x$ : the average differs from the identity by one third of the residual map, and a scalar multiple of a vector vanishes only when the vector does. By `ejThreeResidualEnergy_eq_re_inner`, the energy of  $x$  equals the real inner product of the residual map against  $x$ , hence is zero. By `ejThreeResidualEnergy_eq_zero_iff`, zero energy forces each individual residual projection of  $x$  to vanish, since the energy is a sum of three squares; and a vanishing residual means  $x$  equals its own projection onto  $V_i$ , that is  $x \in V_i$ , via `Submodule.starProjection_eq_self_iff`. The subtlety is that vanishing of the sum of the residuals does not trivially imply termwise vanishing; positivity enters through the inner-product identity. Together with the geometric convergence of the iteration, this pins down the limit of the averaging process as a common vector of the three fixed spaces, completing the abstract three-projection criterion that the subsequent `ThreeFiniteUnitaryFixedSpaces` section applies to unitary representations of three subgroups.

*Remaining declarations in this section:*

|                                                     |     |                                                                                                                            |
|-----------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------------|
| <code>ejStarProjection_re_inner</code>              | THM | Real part of $\langle P_W x, x \rangle$ equals $\ P_W x\ ^2$ for an orthogonal projection.                                 |
| <code>ejResidualProjection</code>                   | DEF | The residual projection: orthogonal projection onto $V^\perp$ as a continuous linear map.                                  |
| <code>ejResidualProjection_apply</code>             | THM | The residual of $x$ is $x$ minus its projection onto $V$ .                                                                 |
| <code>ejResidualProjection_idempotent</code>        | THM | The residual projection is idempotent.                                                                                     |
| <code>ejResidualProjection_inner</code>             | THM | The residual projection is self-adjoint for the inner product.                                                             |
| <code>ejResidualProjection_re_inner</code>          | THM | Real part of $\langle Rx, x \rangle$ equals $\ Rx\ ^2$ .                                                                   |
| <code>ejThreeResidualMap</code>                     | DEF | Sum of the residual projections of the three subspaces.                                                                    |
| <code>ejThreeResidualEnergy</code>                  | DEF | Total energy: the sum of the three squared residual norms.                                                                 |
| <code>ejThreeResidualMap_apply</code>               | THM | Evaluation formula for the three-residual map.                                                                             |
| <code>ejThreeProjectionAverage_apply</code>         | THM | Evaluation formula for the projection average.                                                                             |
| <code>ejThreeResidualEnergy_nonneg</code>           | THM | The residual energy is nonnegative.                                                                                        |
| <code>ejThreeResidualEnergy_eq_re_inner</code>      | THM | The energy equals the real inner product of the residual map against the vector.                                           |
| <code>ejThreeResidualMap_norm_sq_le</code>          | THM | The residual map's squared norm is at most three times the energy.                                                         |
| <code>ejResidualProjection_norm_le</code>           | THM | Each residual projection is norm nonincreasing.                                                                            |
| <code>ejThreeResidualEnergy_le_three_norm_sq</code> | THM | The energy is at most three times the squared norm.                                                                        |
| <code>ejResidualProjection_cross_inner</code>       | THM | Cross identity: $\langle Rx, Rz \rangle = \langle Rx, z \rangle$ by idempotence and self-adjointness.                      |
| <code>ejThreeResidual_cross_re_inner</code>         | THM | The three cross real inner products sum to the residual map's squared norm.                                                |
| <code>norm_sub_one_third_smul_sq</code>             | THM | Expansion of $\ a - \frac{1}{3}b\ ^2$ into norms and a real inner product.                                                 |
| <code>ejResidualProjection_average_norm_sq</code>   | THM | Norm-squared expansion of a residual projection applied to an averaged vector.                                             |
| <code>ejThreeResidualEnergy_average</code>          | THM | Exact identity for the energy after one application of the projection average.                                             |
| <code>ejThreeResidualMap_spectral_gap</code>        | THM | Pairwise $(1 - \varepsilon)$ bounds on residual pairs give a $(1 - 2\varepsilon)$ energy lower bound for the residual map. |
| <code>ejThreeResidualEnergy_average_le</code>       | THM | A spectral gap $\delta$ makes the average contract the energy by factor $1 - \delta/3$ .                                   |
| <code>ejStarProjection_mem_inf_orthogonal</code>    | THM | Star projection onto $U$ preserves the orthocomplement of $U \cap V$ .                                                     |

|                                                                      |     |                                                                                                   |
|----------------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------|
| <code>ejTwoProjection_norm_sq_sum_le_of_angle</code>                 | THM | For $w \perp U \cap V$ , the two projections' squared norms sum to at most $(1 + c)\ w\ ^2$ .     |
| <code>ejTwoResidualEnergy_ge_of_angle</code>                         | THM | Coercivity: $(1 - c)\ w\ ^2$ lower-bounds the two-residual energy on the orthocomplement.         |
| <code>ejTwoResidual_sum_re_inner</code>                              | THM | The real inner product of the sum of two residuals with $w$ equals their energy.                  |
| <code>ejThreeProjectionAverage_dist_sq</code>                        | THM | The squared distance moved by one averaging step is one ninth of the residual map's squared norm. |
| <code>ejThreeProjectionAverage_dist_sq_le</code>                     | THM | That squared step distance is at most a third of the energy.                                      |
| <code>ejThreeResidualEnergy_iterate_le</code>                        | THM | Under a gap $\delta$ , iterated averaging decays the energy like $(1 - \delta/3)^n$ .             |
| <code>ejThreeResidual_spectral_gap_of_pair_angles</code>             | THM | Pairwise angle bounds give the three-residual spectral gap with constant $1 - 2c$ .               |
| <code>ejThreeProjectionAverage_iterate_dist_le_of_pair_angles</code> | THM | Angle bounds with $c < 1/2$ give geometric contraction of iterate step distances.                 |
| <code>ejThreeResidualEnergy_sqrt_div_three_le</code>                 | THM | If every residual norm is at most $T$ , then $\sqrt{E/3} \leq T$ .                                |
| <code>ejThreeProjectionAverage_fixed_iff_residualMap_eq_zero</code>  | THM | A vector is fixed by the average iff its three-residual map vanishes.                             |
| <code>ejThreeResidualEnergy_eq_zero_iff</code>                       | THM | The energy vanishes iff each of the three residual projections vanishes.                          |

## 5 Property (T) for the elementary group and coarea repair

The Ershov–Jaikin-style spectral criterion is assembled and applied: root subgroups over block matrices, averaging over finite subgroups, and an eightfold pair estimate over the field with eight elements combine into an unconditional Kazhdan pair for the elementary group. The part closes with the Kun-Thomas coarea machinery that repairs almost-permutations into exact ones with controlled loss.

### 5.1 Closed fixed subspaces for projections (ThreeFiniteUnitaryFixedSpaces)

This short section supplies the Hilbert-space infrastructure behind the alternating-projection argument for property (T). For a unitary representation  $\pi$  of a group  $G$  on a complex inner product space and a subgroup  $K$ , it defines the submodule of vectors fixed by every element of  $K$ , characterizes membership, and proves that this submodule, as well as the intersection of two such submodules, is closed. In a complete ambient space this yields `CompleteSpace` instances for the fixed subspaces, so orthogonal projections via `Submodule.starProjection` exist. These are exactly the subspaces that the three-projection averaging operator `ejThreeProjectionAverage` (defined earlier in the file) projects onto in the spectral criterion of the next section.

**ejUnitaryFixedSubmodule** `def`

```
def ejUnitaryFixedSubmodule
  {G : Type u2} {H : Type v2} [Group G]
  [NormedAddCommGroup H] [InnerProductSpace ℂ H]
  (π : UnitaryRepresentation G H) (K : Subgroup G) :
  Submodule ℂ H where
  carrier := {x : H | ∀ k : K, π (k : G) x = x}
  zero_mem' k := by simp
  add_mem' hx hy k := by
    simp [map_add, hx k, hy k]
  smul_mem' c x hx k := by
    simp [map_smul, hx k]
```

`ejUnitaryFixedSubmodule` takes a unitary representation  $\pi$  of  $G$  on a complex inner product space  $H$  and a subgroup  $K$ , and forms the  $\mathbb{C}$ -submodule  $\{x \mid \pi(k)x = x \text{ for all } k \in K\}$ . Closure under addition and scalar



multiplication is immediate since each  $\pi(k)$  is linear. The companion results show the carrier is an intersection over  $k \in K$  of the closed sets where the continuous map  $\pi(k)$  agrees with the identity, hence closed (`ejUnitaryFixedSubmodule_isClosed`), and likewise for a binary intersection (`ejUnitaryFixedSubmodule_inf_isClosed`). When  $H$  is complete these closedness facts give `CompleteSpace` instances, which is precisely the hypothesis `Submodule.HasOrthogonalProjection` needs so that `Submodule.starProjection` onto these subspaces is defined. This matters because the property (T) proof works with three finite subgroups of the elementary group and iterates an average of the three orthogonal projections onto their fixed subspaces; every later use of projections onto fixed spaces in this file routes through these instances.

*Remaining declarations in this section:*

|                                                                         |      |                                                                                             |
|-------------------------------------------------------------------------|------|---------------------------------------------------------------------------------------------|
| <code>mem_ejUnitaryFixedSubmodule</code>                                | THM  | Membership unfolding: $x$ lies in the fixed submodule iff $\pi(k)x = x$ for all $k \in K$ . |
| <code>ejUnitaryFixedSubmodule_isClosed</code>                           | THM  | The fixed submodule is closed, being an intersection of equalizers of continuous maps.      |
| <code>ejUnitaryFixedSubmodule_inf_isClosed</code>                       | THM  | The intersection of two fixed submodules is closed.                                         |
| <code>ejUnitaryFixedSubmodule_complete</code><br><code>Space</code>     | INST | Instance: a fixed submodule of a complete space is a complete space.                        |
| <code>ejUnitaryFixedSubmodule_inf_complete</code><br><code>Space</code> | INST | Instance: the intersection of two fixed submodules of a complete space is complete.         |

## 5.2 Spectral criterion for property (T) (`ErshovJaikinFiniteSpectral`)

This section builds the abstract spectral machine that converts geometry of three finite subgroups into Kazhdan's property (T). It introduces the normalized averaging operator over a finite subgroup, shows the average is fixed by the subgroup, and bounds the distance from a vector to its projection onto a fixed subspace by the displacement of its average. The central result `hasPropertyT_of_geometric_finite_averaging` extracts property (T) from any operator whose iterates contract geometrically toward representation-invariant vectors. This is then specialized to a generating triple of finite subgroups, and finally to the Friedrichs-angle form `hasPropertyT_of_three_finite_subgroups_friedrichs_angle`: a uniform bound  $c < 1/2$  on inner products between pairwise fixed subspaces (modulo their intersection) suffices. Auxiliary lemmas prepare the specific constant  $(\sqrt{8})^{-1}$  used later.

**`hasPropertyT_of_geometric_finite_averaging` thm**

```
theorem hasPropertyT_of_geometric_finite_averaging
  {G : Type u} [Group G] (S : Finset G) (q : ℝ)
  (hq1 : q < 1)
  (haverage :
    ∀ (H : Type v) [NormedAddCommGroup H] [InnerProductSpace ℂ H]
    [CompleteSpace H] (n : SoficGroups.UnitaryRepresentation G H),
    ∃ A : H → L[ℂ] H,
      (∀ (x : H) (n : ℕ),
        dist ((A : H → H)^[n] x)
          ((A : H → H)^[n+1] x) ≤
          (∑ g ∈ S, ‖π g x - x‖) * q ^ n) ∧
      (∀ x : H, A x = x → ∀ g : G, π g x = x)) :
    SoficGroups.HasPropertyT.{u, v} G := by
  -- proof: 66 lines, omitted
```

`hasPropertyT_of_geometric_finite_averaging` assumes a finset  $S$  of group elements, a ratio  $q < 1$ , and, for every complete unitary representation  $\pi$ , a continuous linear operator  $A$  satisfying two conditions: the iterate steps obey  $\text{dist}(A^n x, A^{n+1} x) \leq (\sum_{g \in S} \|\pi(g)x - x\|) q^n$ , and every fixed point of  $A$  is invariant under all of  $G$ . It concludes `SoficGroups.HasPropertyT` for  $G$ , exhibiting the Kazhdan pair with generators  $S$  and constant  $\delta = (1 - q)/(2(|S| + 1))$ . The proof: given a unit vector  $\xi$  whose displacements under  $S$  are all below  $\delta$ , the iterates  $A^n \xi$  form a Cauchy sequence by `cauchySeq_of_le_geometric`, converging to some  $\eta$  which is a fixed point of  $A$  by continuity. The geometric tail estimate gives  $\text{dist}(\xi, \eta) \leq C/(1 - q)$  with  $C < (1 - q)/2$ , so  $\eta$  is

within distance  $1/2$  of the unit vector  $\xi$  and hence nonzero. Fixedness under  $A$  then upgrades to  $G$ -invariance, producing the required nonzero invariant vector. All later property (T) results in the file are instances of this criterion.

**hasPropertyT\_of\_three\_finite\_subgroups\_friedrichs\_angle** thm

```

theorem hasPropertyT_of_three_finite_subgroups_friedrichs_angle
  {G : Type u} [Group G]
  (K : Fin 3 → Subgroup G) [V i, Fintype (K i)]
  (hgen : (⋂ i : Fin 3, K i) = T)
  (c : ℝ) (hc : 0 ≤ c) (hchalf : c < (1 / 2 : ℝ))
  (hangle :
    ∀ (H : Type v) [NormedAddCommGroup H]
      [InnerProductSpace ℂ H] [CompleteSpace H]
      (π : SoficGroups.UnitaryRepresentation G H)
      (i j : Fin 3), i ≠ j →
      ∀ (a b : H),
        a ∈ SoficGroups.ejUnitaryFixedSubmodule π (K i) →
        a ∈ (SoficGroups.ejUnitaryFixedSubmodule π (K i) ⊓
          SoficGroups.ejUnitaryFixedSubmodule π (K j))⊥ →
        b ∈ SoficGroups.ejUnitaryFixedSubmodule π (K j) →
        b ∈ (SoficGroups.ejUnitaryFixedSubmodule π (K i) ⊓
          SoficGroups.ejUnitaryFixedSubmodule π (K j))⊥ →
        ||inner ℂ a b|| ≤ c * ||a|| * ||b||) :
    SoficGroups.HasPropertyT.{u, v} G := by
-- proof: 94 lines, omitted

```

**hasPropertyT\_of\_three\_finite\_subgroups\_friedrichs\_angle** is the quantitative angle criterion. Hypotheses: three finite subgroups  $K_0, K_1, K_2$  whose join is all of  $G$ , and a constant  $0 \leq c < 1/2$  such that in every complete unitary representation, for  $i \neq j$ , any  $a$  fixed by  $K_i$  and  $b$  fixed by  $K_j$ , both orthogonal to the joint fixed space of  $K_i$  and  $K_j$ , satisfy  $|\langle a, b \rangle| \leq c \|a\| \|b\|$  — a Friedrichs-angle bound between the fixed subspaces. The proof sets  $q = 1 - (1 - 2c)/6$  and feeds the three-subgroup geometric criterion with the operator **ejThreeProjectionAverage** on the fixed subspaces  $V_i$ . The iterate distance bound is exactly the earlier lemma **ejThreeProjectionAverage\_iterate\_dist\_le\_of\_pair\_angles**, whose residual-energy input is controlled by the subgroup averages: the projection defect  $\|x - P_{V_i}x\|$  is at most the displacement of the  $K_i$ -average of  $x$  (idx 629), which is bounded by the total generator displacement  $T$ . Fixed points of the projection average lie in every  $V_i$ , giving invariance under the generating subgroups. This is the engine that idx 719 instantiates with  $c = (\sqrt{8})^{-1}$ .

*Remaining declarations in this section:*

|                                                                |     |                                                                                                                                    |
|----------------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------|
| <b>finiteUnitarySubgroupAverage</b>                            | DEF | Averages $\pi(k)x$ over a finite subgroup $K$ , normalized by the cardinality of $K$ .                                             |
| <b>finiteUnitarySubgroupAverage_fixed</b>                      | THM | The subgroup average is fixed by every element of $K$ , by reindexing the sum.                                                     |
| <b>finiteUnitarySubgroupAverage_sub</b>                        | THM | Rewrites the average minus $x$ as the normalized sum of the displacements $\pi(k)x - x$ .                                          |
| <b>norm_finiteUnitarySubgroupAverage_sub_le</b>                | THM | Bounds the norm of the average's displacement by the mean displacement norm over $K$ .                                             |
| <b>norm_sub_starProjection_le_finiteUnitarySubgroupAverage</b> | THM | The distance from $x$ to its projection onto $U$ is at most the average's displacement, when the average lies in $U$ .             |
| <b>threeFiniteSubgroupGenerators</b>                           | DEF | Finset collecting all elements of three finite subgroups, used as the Kazhdan generating set.                                      |
| <b>mem_threeFiniteSubgroupGenerators</b>                       | THM | Membership: $g$ is a generator iff it lies in some $K_i$ .                                                                         |
| <b>unitaryVectorStabilizer</b>                                 | DEF | The subgroup of elements of $G$ that fix a given vector $x$ under $\pi$ .                                                          |
| <b>hasPropertyT_of_three_finite_subgroups_geometric</b>        | THM | Property (T) from the geometric criterion when three finite subgroups generate $G$ and fixed points of $A$ are subgroup-invariant. |

|                                                 |     |                                                                                                                   |
|-------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------|
| <code>inv_sqrt_eight_pos</code>                 | THM | $(\sqrt{8})^{-1}$ is positive.                                                                                    |
| <code>inv_sqrt_eight_lt_one_half</code>         | THM | $(\sqrt{8})^{-1} < 1/2$ , since $\sqrt{8} > 2$ .                                                                  |
| <code>norm_inner_le_inv_sqrt_eight_of_sq</code> | THM | Converts $8 \langle a, b \rangle ^2 \leq \ a\ ^2 \ b\ ^2$ into the linear bound with constant $(\sqrt{8})^{-1}$ . |

### 5.3 The eight-element Galois field

A brief interlude constructing the finite field  $\mathbb{F}_8$  and its realization by matrices. `finiteBlockGaloisField` abbreviates `GaloisField 2 3`; a  $\mathbb{Z}/2$ -basis indexed by `Fin 3` gives the left-multiplication algebra homomorphism `finiteBlockGaloisMatrix` into  $3 \times 3$  matrices over  $\mathbb{Z}/2$ , which `finiteBlockGaloisMatrixOver` pushes forward to  $3 \times 3$  matrices over an arbitrary  $\mathbb{Z}/2$ -algebra  $R$ . The cardinality computation `finiteBlockGaloisField_natCard` records that the field has exactly eight elements. These eight commuting scalar matrices, with all differences of distinct elements invertible, are the source of the eight pairwise-commuting conjugated root subgroups and hence of the constant 8 in the spectral estimate.

**`finiteBlockGaloisMatrix`** `def`

```
def finiteBlockGaloisMatrix :
  finiteBlockGaloisField →*[ZMod 2]
  Matrix (Fin 3) (Fin 3) (ZMod 2) :=
  Algebra.leftMulMatrix finiteBlockGaloisBasis
```

`finiteBlockGaloisMatrix` is the left-regular representation of  $\mathbb{F}_8$ : using the basis `finiteBlockGaloisBasis` obtained from the `finrank` computation `GaloisField.finrank`, the `mathlib` construction `Algebra.leftMulMatrix` turns each field element  $t$  into the  $3 \times 3$  matrix over  $\mathbb{Z}/2$  of multiplication by  $t$ . Composing with the entrywise `algebraMap` into an arbitrary  $\mathbb{Z}/2$ -algebra  $R$  gives `finiteBlockGaloisMatrixOver`, a ring homomorphism from  $\mathbb{F}_8$  to  $3 \times 3$  matrices over  $R$ . Two features are exploited downstream. First, since the source is a field, for  $s \neq t$  the element  $(t - s)^{-1}$  exists, giving the telescoping identity of `idx 675` which writes any coefficient matrix as a difference of two Galois multiples; this is the algebraic heart of `idx 676`, placing product-central roots inside the join of two Galois conjugates. Second, the image matrices commute with each other and act as scalars on binary matrices, so multiplying a matrix with entries in a  $\mathbb{Z}/2$ -submodule  $W$  by them stays entrywise in  $W$  (`idx 646, 647`). The count  $|\mathbb{F}_8| = 8$  (`idx 642`) is exactly the cardinality of the commuting family in `idx 693`, producing the Friedrichs constant  $(\sqrt{8})^{-1} < 1/2$ .

*Remaining declarations in this section:*

|                                             |      |                                                                                                                |
|---------------------------------------------|------|----------------------------------------------------------------------------------------------------------------|
| <code>finiteBlockGaloisField</code>         | ABBR | Abbreviation for the field $\mathbb{F}_8$ , realized as <code>GaloisField 2 3</code> .                         |
| <code>finiteBlockGaloisBasis</code>         | DEF  | A $\mathbb{Z}/2$ -basis of $\mathbb{F}_8$ indexed by <code>Fin 3</code> , from the rank computation.           |
| <code>finiteBlockGaloisField_natCard</code> | THM  | $\mathbb{F}_8$ has exactly eight elements.                                                                     |
| <code>finiteBlockGaloisMatrixOver</code>    | DEF  | Extends the matrix embedding of $\mathbb{F}_8$ to $3 \times 3$ matrices over any $\mathbb{Z}/2$ -algebra $R$ . |

### 5.4 Root subgroups over block matrices (ActualBlockRoots)

Working in the elementary group of  $3 \times 3$  matrices whose entries are themselves  $3 \times 3$  matrices over a  $\mathbb{Z}/2$ -algebra  $R$ , this section builds finite root subgroups from a  $\mathbb{Z}/2$ -submodule  $W \leq R$ . Closure lemmas show  $W$ -entried matrices absorb multiplication by binary and Galois-scalar matrices. Root homomorphisms send additive  $W$ -matrices to elementary units  $e_{ij}(A)$ ; their ranges are finite when  $W$  is finite. The Chevalley commutator formula  $[e_{ij}(A), e_{jk}(B)] = e_{ik}(AB)$  and non-adjacency commutation rules drive everything. Product-central subgroups collect  $(i, k)$ -roots with coefficients  $AB$  for fixed  $B$ ; they are centralized by both root subgroups. Finally, Galois conjugation by  $e_{jk}(tB)$  produces, for each  $t \in \mathbb{F}_8$ , a conjugated copy of the  $(i, j)$ -root subgroup, distinct conjugates commute elementwise, and every product-central subgroup lies in the join of any two distinct conjugates — the Heisenberg-type configuration used in the spectral estimate.

### finiteOuterBlockRootSubgroup def

```
def finiteOuterBlockRootSubgroup
  (W : Submodule (ZMod 2) R) (i j : Fin 3) (hij : i ≠ j) :
  Subgroup (elementaryGroup (Fin 3) (Matrix (Fin 3) (Fin 3) R)) :=
  (finiteOuterBlockRootHom W i j hij).range
```

`finiteOuterBlockRootSubgroup` is the range of `finiteOuterBlockRootHom`, which composes three maps: the entrywise inclusion of  $W$ -matrices into  $R$ -matrices (`finiteBlockCoefficientMatrixHom`, idx 648), the passage to `Multiplicative`, and the elementary root homomorphism `elementaryRootHom` sending an additive coefficient  $A$  to the elementary unit  $1 + \text{single}(i, j, A)$  inside `elementaryGroup` of  $3 \times 3$  matrices over the ring of  $3 \times 3$  matrices over  $R$ . The result is an abelian subgroup — the root subgroup at block position  $(i, j)$  with coefficients confined to  $W$  — and `finite_finiteOuterBlockRootSubgroup` (idx 651) shows it is finite whenever  $W$  is, by surjectivity from the finite coefficient space. These subgroups are the fundamental objects of the whole Ershov–Jaikin-style argument: the three cyclic ones  $W_{01}, W_{12}, W_{20}$  (idx 700) serve as the generating triple of finite subgroups in the property (T) criterion, their commutator relations (idx 652–656) create the product-central subgroups, and their fixed subspaces in unitary representations are the objects whose pairwise angles the rest of the chunk estimates.

### finiteOuterBlockProductCentralRoot\_le\_galoisConjugates thm

```
theorem finiteOuterBlockProductCentralRoot_le_galoisConjugates
  (W : Submodule (ZMod 2) R)
  (i j k : Fin 3) (hij : i ≠ j) (hjk : j ≠ k) (hik : i ≠ k)
  (B : Matrix (Fin 3) (Fin 3) W)
  (s t : finiteBlockGaloisField) (hst : s ≠ t) :
  finiteOuterBlockProductCentralRootSubgroup W i k hik B ≤
    finiteOuterBlockGaloisConjugateRootWithCoefficient
      W i j k hij hjk B s ∪
    finiteOuterBlockGaloisConjugateRootWithCoefficient
      W i j k hij hjk B t := by
  -- proof: 88 lines, omitted
```

`finiteOuterBlockProductCentralRoot_le_galoisConjugates` states that for distinct Galois parameters  $s \neq t$  in  $\mathbb{F}_8$ , the product-central subgroup at  $(i, k)$  with coefficient  $B$  is contained in the join of the two conjugated root subgroups with parameters  $s$  and  $t$ . Given a generator  $e_{ik}(C_0 B_0)$  (with  $C_0, B_0$  the embeddings of  $A, B$ ), the proof forms the corrected coefficient  $D = A(t - s)^{-1}$  (idx 673) and the root element  $x = e_{ij}(D_0)$ . Conjugating  $x$  by the two Galois conjugators  $y_s = e_{jk}(s B_0)$  and  $y_t = e_{jk}(t B_0)$  and applying the conjugation formula (idx 653) shows the product  $(y_s x y_s^{-1})(y_t x y_t^{-1})^{-1}$ , which manifestly lies in the join, group-theoretically collapses to  $e_{ik}(D_0 t B_0 - D_0 s B_0)$ ; the telescoping identity (idx 675), using invertibility of  $t - s$  in the field, evaluates this coefficient to exactly  $C_0 B_0$ . This is the pivotal structural fact: it verifies the center-inside-pairwise-joins hypothesis of the abstract Heisenberg family estimate (idx 692), turning the eight Galois conjugates into a configuration that forces the inner-product bound with constant 8.

*Remaining declarations in this section:*

|                                                          |     |                                                                                                |
|----------------------------------------------------------|-----|------------------------------------------------------------------------------------------------|
| <code>finiteBlockCoefficientMatrix_mul_binary_mem</code> | THM | Entries of $AB$ stay in $W$ when $A$ has entries in $W$ and $B$ is binary.                     |
| <code>finiteBlockCoefficientMatrix_binary_mul_mem</code> | THM | The same closure holds for left multiplication by a binary matrix.                             |
| <code>finiteBlockCoefficientMatrix_mul_galois_mem</code> | THM | Entries of $A$ times a Galois-scalar matrix stay in $W$ .                                      |
| <code>finiteBlockCoefficientMatrix_galois_mul_mem</code> | THM | Entries of a Galois-scalar matrix times $A$ stay in $W$ .                                      |
| <code>finiteBlockCoefficientMatrixHom</code>             | DEF | Additive homomorphism embedding $W$ -entried $3 \times 3$ matrices into $R$ -entried matrices. |
| <code>finiteOuterBlockRootHom</code>                     | DEF | Monoid homomorphism from additive $W$ -matrices to elementary roots at position $(i, j)$ .     |

|                                                             |     |                                                                                                            |
|-------------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------|
| finite_finiteOuterBlockRootSubgroup                         | THM | Root subgroups are finite when the coefficient module $W$ is finite.                                       |
| finiteOuterElementaryRoot_commutator                        | THM | Chevalley commutator formula: $[e_{ij}(A), e_{jk}(B)] = e_{ik}(AB)$ for elementary roots.                  |
| finiteOuterElementaryRoot_conjugate_by_successor            | THM | Conjugating $e_{ij}(A)$ by $e_{jk}(B)$ equals $e_{ik}(AB)^{-1}e_{ij}(A)$ .                                 |
| finiteOuterElementaryUnit_commute_of_nonadjacent            | THM | Elementary units at non-adjacent positions commute, since the single-entry matrices multiply to zero.      |
| finiteOuterElementaryRoot_commute_of_nonadjacent            | THM | Elementary roots at non-adjacent positions commute.                                                        |
| finiteOuterElementaryRoot_commute_same                      | THM | Two elementary roots at the same position commute, via additivity of the root homomorphism.                |
| finiteBlockCoefficientMatrixRightMul                        | DEF | Additive map sending $A$ to $AB$ inside $R$ -matrices, for a fixed $W$ -matrix $B$ .                       |
| finiteOuterBlockProductCentralRootHom                       | DEF | Root homomorphism at $(i, k)$ whose coefficients are right-multiplied by a fixed matrix $B$ .              |
| finiteOuterBlockProductCentralRootSubgroup                  | DEF | The product-central subgroup of $(i, k)$ -roots with coefficients of the form $AB$ .                       |
| finiteOuterBlockRoot_commutator_mem_productCentral          | THM | Commutators of $(i, j)$ - and $(j, k)$ -roots land in the product-central $(i, k)$ -subgroup.              |
| finiteOuterBlockRoot_commutator_exists_productCentral       | THM | For elements of the two root subgroups, some $B$ makes their commutator product-central.                   |
| finiteOuterBlockProductCentralRoot_commute_left             | THM | Product-central roots commute with the $(i, j)$ -root subgroup.                                            |
| finiteOuterBlockProductCentralRoot_commute_right            | THM | Product-central roots commute with the $(j, k)$ -root subgroup.                                            |
| finiteOuterBlockProductCentralRoot_pair_le_centralizer      | THM | The join of the two root subgroups centralizes each product-central subgroup.                              |
| finiteOuterBlockProductCentralRoots_commute                 | THM | Any two product-central roots at $(i, k)$ commute with each other.                                         |
| finiteBlockGaloisLeftProductCoefficientMatrix               | DEF | $W$ -matrix representing the Galois left product: $t$ times the embedding of $B$ .                         |
| finiteBlockCoefficientMatrixHom_galoisLeftProduct           | THM | Embedding that coefficient matrix recovers the Galois matrix times the embedding of $B$ .                  |
| finiteOuterBlockGaloisConjugatorWithCoefficient             | DEF | The $(j, k)$ -root element with coefficient $tB$ , used as a conjugator.                                   |
| finiteOuterBlockGaloisConjugatorWithCoefficient_mem         | THM | That Galois conjugator lies in the $(j, k)$ -root subgroup.                                                |
| finiteOuterBlockGaloisConjugateRootWithCoefficient          | DEF | The $(i, j)$ -root subgroup conjugated by the Galois conjugator with parameter $t$ .                       |
| finiteOuterElementaryRootConjugates_commute                 | THM | Conjugates of $(i, j)$ -roots by two $(j, k)$ -roots commute, by the commutator calculus.                  |
| finiteOuterBlockGaloisConjugateRootsWithCoefficient_commute | THM | Elements of two Galois-conjugated root subgroups with distinct parameters commute.                         |
| finiteBlockGaloisRightInverseCoefficientMatrix              | DEF | $W$ -matrix representing $A$ times the Galois scalar $(t - s)^{-1}$ .                                      |
| finiteBlockCoefficientMatrixHom_galoisRightInverse          | THM | Its embedding equals the embedding of $A$ times the $(t - s)^{-1}$ Galois matrix.                          |
| finiteBlockGalois_right_inverse_difference                  | THM | Telescoping identity: multiplying by $(t - s)^{-1}$ , then by $t$ and $s$ , and subtracting recovers $C$ . |

## 5.5 Orthogonality via commuting stabilizers

This block develops general Hilbert-space lemmas about fixed subspaces of unitary representations and then proves the key Heisenberg-type inner-product estimate. After redefining fixed submodules via `Representation.invariants` and re-proving closedness and completeness, it shows: the fixed space of a join is the intersection of fixed spaces; centralizing elements map fixed spaces and their orthogonal complements to themselves; and orthogonal projections onto fixed spaces commute with centralizing group elements. A Pythagoras identity and a Cauchy–Schwarz counting argument give `unitaryImages_inner_sq_mul_card_le`: pairwise orthogonal images  $U_i a$  against a common fixed vector  $b$  force  $n|\langle a, b \rangle|^2 \leq \|a\|^2 \|b\|^2$ . The commuting-subgroup orthogonality criterion (idx 690) then combines with a family of conjugate subgroups (idx 692), and idx 693 instantiates everything with the eight Galois conjugators to obtain the constant 8 for one product-central sector.

**heisenbergFiniteFamily\_centerComplement\_inner\_sq\_le** *thm*

```
theorem heisenbergFiniteFamily_centerComplement_inner_sq_le
  {G : Type u₁} {H : Type v₁} {ι : Type*}
  [Group G] [Fintype ι]
  [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H]
  (π : UnitaryRepresentation G H)
  (X Z : Subgroup G) (y : ι → G)
  (hXZ : ∀ x : X, ∀ z : Z, Commute (x : G) (z : G))
  (hyZ : ∀ i : ι, ∀ z : Z, Commute (y i) (z : G))
  (hcomm : ∀ [i j : ι], i ≠ j →
    ∀ x : heisenbergConjugateSubgroup X (y i),
    ∀ x' : heisenbergConjugateSubgroup X (y j),
    Commute (x : G) (x' : G))
  (hcenter : ∀ [i j : ι], i ≠ j →
    Z ≤ heisenbergConjugateSubgroup X (y i) ⊔
    heisenbergConjugateSubgroup X (y j))
  {a b : H}
  (ha : a ∈ unitaryFixedSubmodule π X)
  (haZ : a ∈ (unitaryFixedSubmodule π Z)⊥)
  (hb : ∀ i : ι, π (y i) b = b) :
  (Fintype.card ι : ℝ) * ‖inner ℂ a b‖ ^ 2 ≤
    ‖a‖ ^ 2 * ‖b‖ ^ 2 := by
  -- proof: 27 lines, omitted
```

`heisenbergFiniteFamily_centerComplement_inner_sq_le` is the abstract Heisenberg-type estimate. The data: subgroups  $X$  and  $Z$  of  $G$ , a finite family of elements  $y_i$ , with  $X$  and every  $y_i$  commuting with  $Z$ ; for  $i \neq j$  the conjugate subgroups  $y_i X y_i^{-1}$  and  $y_j X y_j^{-1}$  commute elementwise; and  $Z$  is contained in the join of any two distinct conjugates. The conclusion: if  $a$  is  $X$ -fixed and orthogonal to the  $Z$ -fixed space, and  $b$  is fixed by every  $\pi(y_i)$ , then  $|\iota| \cdot |\langle a, b \rangle|^2 \leq \|a\|^2 \|b\|^2$ . The proof shows each image  $\pi(y_i)a$  is fixed by the  $i$ -th conjugate subgroup and, since  $y_i$  centralizes  $Z$ , remains orthogonal to the  $Z$ -fixed space (idx 689). For  $i \neq j$ , the two images are fixed by commuting subgroups whose join contains  $Z$ , so the orthogonality criterion idx 690 (project onto one fixed space, land in the joint fixed space, get killed by  $Z$ -orthogonality) makes them orthogonal. The counting lemma idx 687 then converts pairwise orthogonality of the  $n$  unit-preserving images into the factor- $n$  improvement over plain Cauchy–Schwarz. This is the analytic mechanism that turns the Galois-conjugate group structure into a spectral gap.

**finiteOuterBlockProductCentral\_centerComplement\_inner\_sq\_le** *thm*

```
theorem finiteOuterBlockProductCentral_centerComplement_inner_sq_le
  {R : Type*} [Ring R] [Algebra (ZMod 2) R]
  {H : Type v₁} [NormedAddCommGroup H]
  [InnerProductSpace ℂ H] [CompleteSpace H]
  (W : Submodule (ZMod 2) R)
  (i j k : Fin 3) (hij : i ≠ j) (hjk : j ≠ k) (hik : i ≠ k)
  (B : Matrix (Fin 3) (Fin 3) W)
  (π : UnitaryRepresentation
    (elementaryGroup (Fin 3) (Matrix (Fin 3) (Fin 3) R)) H)
  {a b : H}
  (ha : a ∈ unitaryFixedSubmodule π
```

```

(finiteOuterBlockRootSubgroup W i j hij))
(haZ : a ∈
  (unitaryFixedSubmodule π
    (finiteOuterBlockProductCentralRootSubgroup W i k hik B))⊥)
(hb : b ∈ unitaryFixedSubmodule π
  (finiteOuterBlockRootSubgroup W j k hjk)) :
8 * ‖inner C a b‖2 ≤ ‖a‖2 * ‖b‖2 := by
-- proof: 43 lines, omitted

```

`finiteOuterBlockProductCentral_centerComplement_inner_sq_le` instantiates the Heisenberg family estimate in the block elementary group. Take  $X$  to be the  $(i, j)$ -root subgroup,  $Z$  the product-central subgroup at  $(i, k)$  with coefficient  $B$ , and as family the eight Galois conjugators  $y_t = e_{jk}(tB_0)$  for  $t \in \mathbb{F}_8$  (idx 668). The hypotheses of idx 692 are exactly the section's structural lemmas:  $X$  commutes with  $Z$  by idx 662, each  $y_t$  commutes with  $Z$  by idx 663 (the conjugators live in the  $(j, k)$ -root subgroup, idx 669), distinct conjugated copies of  $X$  commute elementwise by idx 672, and  $Z$  lies in the join of any two distinct conjugates by the pivotal inclusion idx 676. A vector  $b$  fixed by the whole  $(j, k)$ -root subgroup is in particular fixed by every conjugator. Since  $|\mathbb{F}_8| = 8$ , the conclusion reads  $8|\langle a, b \rangle|^2 \leq \|a\|^2 \|b\|^2$  whenever  $a$  is  $X$ -fixed and orthogonal to the  $Z$ -fixed space. This is the single-sector estimate; idx 699 aggregates it over all coefficients  $B$  to remove the  $Z$ -orthogonality hypothesis in favor of orthogonality to the joint fixed space.

*Remaining declarations in this section:*

|                                                                               |      |                                                                                                                       |
|-------------------------------------------------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------|
| <code>UnitaryRepresentation.toRepresentation</code>                           | DEF  | Converts a unitary representation into a linear <code>Representation</code> over $\mathbb{C}$ .                       |
| <code>unitaryFixedSubmodule</code>                                            | DEF  | Fixed submodule of a subgroup $K$ , defined via <code>Representation.invariants</code> .                              |
| <code>unitaryVectorStabilizer</code>                                          | DEF  | The subgroup of elements fixing a given vector $x$ , restated at top level.                                           |
| <code>unitaryFixedSubmodule_sup</code>                                        | THM  | The fixed submodule of a join $K \sqcup J$ equals the intersection of the two fixed submodules.                       |
| <code>unitaryFixedSubmodule_isClosed</code>                                   | THM  | Fixed submodules are closed.                                                                                          |
| <code>unitaryFixedSubmodule_completeSpace</code>                              | INST | Instance: fixed submodules of a complete space are complete.                                                          |
| <code>unitaryFixedSubmodule_map_of_centralizes</code>                         | THM  | $\pi(g)$ maps the fixed submodule of $K$ onto itself when $g$ commutes with $K$ .                                     |
| <code>unitaryFixedSubmodule_starProjection_commute</code>                     | THM  | Orthogonal projection onto a fixed submodule commutes with $\pi(g)$ for centralizing $g$ .                            |
| <code>unitaryFixedSubmodule_starProjection_mem_of_commute</code>              | THM  | Projection onto the $K$ -fixed space preserves the $J$ -fixed space when $K$ and $J$ commute.                         |
| <code>norm_finset_sum_sq_eq_of_pairwise_inner_zero</code>                     | THM  | Pythagoras: the squared norm of a sum of pairwise orthogonal vectors is the sum of squared norms.                     |
| <code>unitaryImages_inner_sq_mul_card_le</code>                               | THM  | Pairwise orthogonal images $U_i a$ with each $U_i$ fixing $b$ give $n \langle a, b \rangle ^2 \leq \ a\ ^2 \ b\ ^2$ . |
| <code>unitaryFixedSubmodule_starProjection_mem_orthogonal_of_commute</code>   | THM  | Projection onto the $K$ -fixed space preserves the orthogonal complement of the $Z$ -fixed space.                     |
| <code>unitaryFixedSubmodule_orthogonal_map_of_centralizes</code>              | THM  | $\pi(g)$ preserves the orthogonal complement of the $Z$ -fixed space for centralizing $g$ .                           |
| <code>unitaryFixedSubmodules_inner_eq_zero_of_commute_centerComplement</code> | THM  | Vectors fixed by commuting subgroups are orthogonal when one is orthogonal to the fixed space of a central $Z$ .      |
| <code>heisenbergConjugateSubgroup</code>                                      | DEF  | The conjugate subgroup $gXg^{-1}$ , as the image of $X$ under conjugation.                                            |



## 5.6 Averaging reductions for orthogonality (FiniteSubgroupAveraging)

A small toolkit, restated inside the main namespace, around the normalized average  $\frac{1}{|K|} \sum_k \pi(k)b$  over a finite subgroup  $K$ . The average is  $K$ -fixed; taking inner products with a  $K$ -fixed vector leaves them unchanged; and if a second subgroup  $J$  fixes every orbit point  $\pi(k)b$ , then  $J$  also fixes the average. These combine into `inner_eq_zero_of_finite_subgroupAverage_pair_orthogonal`, which reduces the vanishing of  $\langle a, b \rangle$  to vanishing against vectors fixed simultaneously by  $K$  and  $J$ . This reduction supplies the last hypothesis of the sector-splitting lemma applied in idx 699, where genuine joint invariance is only available after averaging because the two root subgroups do not commute on the nose but only up to product-central commutators.

**inner\_eq\_zero\_of\_finite\_subgroupAverage\_pair\_orthogonal** *thm*

```
theorem inner_eq_zero_of_finite_subgroupAverage_pair_orthogonal
  (π : UnitaryRepresentation G H)
  (K J : Subgroup G) [Fintype K] (a b : H)
  (ha : ∀ k : K, π (k : G) a = a)
  (horbit : ∀ (k : K) (j : J),
    π (j : G) (π (k : G) b) = π (k : G) b)
  (horth : ∀ z : H,
    (∀ k : K, π (k : G) z = z) →
    (∀ j : J, π (j : G) z = z) →
    inner ℂ a z = 0) :
  inner ℂ a b = 0 := by
-- proof: 12 lines, omitted
```

`inner_eq_zero_of_finite_subgroupAverage_pair_orthogonal` assumes:  $a$  is fixed by the finite subgroup  $K$ ; every orbit point  $\pi(k)b$  of  $b$  under  $K$  is fixed by the subgroup  $J$ ; and  $a$  is orthogonal to every vector fixed by both  $K$  and  $J$ . It concludes  $\langle a, b \rangle = 0$ . The proof is three lines given the preceding lemmas: set  $z$  to be the  $K$ -average of  $b$ ; then  $z$  is  $K$ -fixed (idx 695) and  $J$ -fixed (idx 697, using the orbit hypothesis), so the joint-orthogonality hypothesis kills  $\langle a, z \rangle$ ; and since  $a$  is  $K$ -fixed, averaging does not change the inner product,  $\langle a, b \rangle = \langle a, z \rangle$  (idx 696). Its significance is in how it is used in idx 699: there  $b$  is fixed by the  $(j, k)$ -root subgroup  $Y$  and by all product-central subgroups, and a commutator computation ( $ts = sc^{-1}t$  with  $c$  the product-central commutator) shows the  $X$ -orbit of  $b$  is still  $Y$ -fixed even though  $X$  and  $Y$  do not commute. Averaging over  $X$  then produces an honest jointly fixed vector, letting orthogonality to the joint fixed space  $Q$  finish the argument.

*Remaining declarations in this section:*

|                                                                |     |                                                                                       |
|----------------------------------------------------------------|-----|---------------------------------------------------------------------------------------|
| <code>finiteUnitarySubgroupAverage</code>                      | DEF | Restates the normalized finite-subgroup averaging operator inside the main namespace. |
| <code>finiteUnitarySubgroupAverage_fixed</code>                | THM | The subgroup average is fixed by every element of $K$ , by reindexing (restated).     |
| <code>inner_finiteUnitarySubgroupAverage_of_fixed_left</code>  | THM | Averaging over $K$ does not change inner products against $K$ -fixed vectors.         |
| <code>finiteUnitarySubgroupAverage_fixed_of_orbit_fixed</code> | THM | If $J$ fixes each orbit point $\pi(k)b$ , then $J$ fixes the $K$ -average of $b$ .    |

## 5.7 The eightfold pair estimate

The culmination of the spectral half of this chunk. `finiteOuterBlockRootSubgroups_pair_inner_sq_le` proves the Friedrichs-angle inequality  $8|\langle a, b \rangle|^2 \leq \|a\|^2 \|b\|^2$  for vectors fixed by two adjacent root subgroups and orthogonal to their joint fixed space, by splitting into sectors indexed by the product-central subgroups and invoking the single-sector constant-8 bound together with the averaging reduction. `finiteOuterBlockCyclicRootFamily` packages the three root subgroups  $W_{i,i+1}$  cyclically over `Fin 3`, and idx 701 extends the estimate to every ordered pair of the family by exhaustive case analysis, symmetry of the inner product, and commutativity of the intersection, so it exactly matches the hypothesis shape of the angle criterion idx 638.

## finiteOuterBlockRootSubgroups\_pair\_inner\_sq\_le thm

```

theorem finiteOuterBlockRootSubgroups_pair_inner_sq_le
  {R : Type*} [Ring R] [Algebra (ZMod 2) R]
  {H : Type v₁} [NormedAddCommGroup H]
  [InnerProductSpace ℂ H] [CompleteSpace H]
  (W : Submodule (ZMod 2) R) [Finite W]
  (i j k : Fin 3)
  (hij : i ≠ j) (hjk : j ≠ k) (hik : i ≠ k)
  (π : UnitaryRepresentation
    (elementaryGroup (Fin 3) (Matrix (Fin 3) (Fin 3) R)) H)
  {a b : H}
  (ha : a ∈ unitaryFixedSubmodule π
    (finiteOuterBlockRootSubgroup W i j hij))
  (hb : b ∈ unitaryFixedSubmodule π
    (finiteOuterBlockRootSubgroup W j k hjk))
  (haQ : a ∈
    (unitaryFixedSubmodule π
      (finiteOuterBlockRootSubgroup W i j hij) ∩
      unitaryFixedSubmodule π
      (finiteOuterBlockRootSubgroup W j k hjk))⊥)
  (hbQ : b ∈
    (unitaryFixedSubmodule π
      (finiteOuterBlockRootSubgroup W i j hij) ∩
      unitaryFixedSubmodule π
      (finiteOuterBlockRootSubgroup W j k hjk))⊥) :
  8 * ‖inner ℂ a b‖ ^ 2 ≤ ‖a‖ ^ 2 * ‖b‖ ^ 2 := by
  -- proof: 142 lines, omitted

```

`finiteOuterBlockRootSubgroups_pair_inner_sq_le` is the estimate the whole block-root apparatus was built for: with  $X$  the  $(i, j)$ -root subgroup and  $Y$  the  $(j, k)$ -root subgroup, if  $a$  is  $X$ -fixed,  $b$  is  $Y$ -fixed, and both are orthogonal to  $Q$ , the intersection of the two fixed spaces, then  $8|\langle a, b \rangle|^2 \leq \|a\|^2 \|b\|^2$ . The proof applies the earlier abstract lemma `inner_sq_le_of_finite_orthogonal_sector_splitting` to the finite family of subspaces  $F(B)$ , the fixed spaces of the product-central subgroups  $Z(B)$  as  $B$  ranges over  $W$ -matrices. Its hypotheses are verified one by one: projections onto  $F(B)$  preserve the sectors  $U = \text{Fix}(X) \cap Q^\perp$  and  $V = \text{Fix}(Y) \cap Q^\perp$  because  $Z(B)$  centralizes  $X, Y$ , and their join (idx 662–664, via idx 685 and 688); projections onto different  $F(B), F(C)$  are compatible because product-central roots commute among themselves (idx 665); within one sector the constant-8 bound is idx 693; and for vectors orthogonal to all the  $F(B)$ , full orthogonality follows from the averaging reduction idx 698 combined with the commutator identity  $ts = sc^{-1}t$  showing the  $X$ -orbit of a  $Y$ -and-central-fixed vector stays  $Y$ -fixed. Through idx 701, 637, and 638 this becomes property (T).

*Remaining declarations in this section:*

|                                                                                 |     |                                                                                              |
|---------------------------------------------------------------------------------|-----|----------------------------------------------------------------------------------------------|
| <code>finiteOuterBlockCyclicRootFamily</code>                                   | DEF | The three cyclically indexed root subgroups $W_{i,i+1}$ for $i$ in <code>Fin 3</code> .      |
| <code>finiteOuterBlockCyclicRootFamily_</code><br><code>pair_inner_sq_le</code> | THM | Extends the pair estimate to every ordered pair of the cyclic family, by cases and symmetry. |

## 5.8 Finite binary coefficient spans (BinaryCoefficientSpan)

A four-declaration section supplying the finite coefficient modules that feed the root-subgroup machinery. Given a finset  $s$  of elements of a  $\mathbb{Z}/2$ -algebra  $R$ , `finiteBlockCoefficientSpan` is the  $\mathbb{Z}/2$ -submodule spanned by  $\{1\} \cup s$ . It contains 1 and every element of  $s$ , and it is finite because it is a finitely generated module over the two-element field. Finiteness is the crucial point: the property (T) criterion needs the three generating subgroups to be finite, while the algebra  $R$  (ultimately the binary Leavitt algebra) is infinite; a finite  $\mathbb{Z}/2$ -span of a generating set is small enough to be finite and large enough, by the results of the next section, to generate the full elementary group.

```
finiteBlockCoefficientSpan def
```

```
def finiteBlockCoefficientSpan (s : Finset R) : Submodule (ZMod 2) R :=
  Submodule.span (ZMod 2) ({1} ∪ {↑s : Set R})
```

`finiteBlockCoefficientSpan` takes a finset  $s$  in a  $\mathbb{Z}/2$ -algebra  $R$  and returns  $\text{span}_{\mathbb{Z}/2}(\{1\} \cup s)$ . The three companion facts are exactly what later constructions consume: 1 belongs to the span (idx 703, needed for the two-step commutator tricks in idx 716–717 which insert units with coefficient 1); every generator belongs (idx 704, so the coefficient subalgebra of idx 718 contains  $s$ ); and the span is finite (idx 705, via `Module.Finite.span_of_finite` over the finite field  $\mathbb{Z}/2$ ). The design point is that  $W$  is deliberately not a subring: the root-subgroup lemmas never multiply two  $W$ -elements together — they only multiply  $W$ -entried matrices by binary matrices or Galois-scalar matrices, operations under which  $W$  is closed entrywise (idx 644–647). This lets the argument thread the needle between two requirements: the three cyclic root subgroups built from  $W$  must be finite groups (so the Ershov–Jaikin finite-subgroup criterion applies), yet when  $s$  generates  $R$  as an algebra their join must be the entire elementary group (idx 718, 722). Both hold precisely for this span.

*Remaining declarations in this section:*

|                                                    |      |                                                                                          |
|----------------------------------------------------|------|------------------------------------------------------------------------------------------|
| <code>one_mem_finiteBlockCoefficientSpan</code>    | THM  | The identity element lies in the coefficient span.                                       |
| <code>mem_finiteBlockCoefficientSpan_of_mem</code> | THM  | Each element of $s$ lies in the coefficient span.                                        |
| <code>finiteBlockCoefficientSpan_finite</code>     | INST | Instance: the coefficient span is finite, being finitely generated over $\mathbb{Z}/2$ . |

## 5.9 Flattened roots generate elementary group (FiniteBlockRoots)

This section flattens the block picture:  $3 \times 3$  matrices of  $3 \times 3$  matrices over  $R$  are identified with  $9 \times 9$  matrices via `finiteBlockFlattenRingEquiv`, with `finiteBlockIndex` encoding pairs of `Fin 3` indices as `Fin 9` indices. Each block root homomorphism transports to a homomorphism into units of  $9 \times 9$  matrices; on single-entry coefficient matrices it produces literal elementary units (idx 711), so the flattened root subgroups sit inside `elementaryGroup` of  $9 \times 9$  matrices. The join of the three cyclic flattened root subgroups is analyzed: elementary units at block-distinct positions are reached directly or by two-step commutator factorizations, same-block positions by a detour through a third block, and finally, when  $s$  generates  $R$  as a  $\mathbb{Z}/2$ -algebra, the join equals the entire elementary group (idx 718).

**`cyclicFiniteBlockRootSubgroup_eq_elementaryGroup`** thm

```
theorem cyclicFiniteBlockRootSubgroup_eq_elementaryGroup
  (s : Finset R)
  (hs : Algebra.adjoin (ZMod 2) {↑s : Set R} = T) :
  cyclicFiniteBlockRootSubgroup (finiteBlockCoefficientSpan s) =
    elementaryGroup (Fin 9) R := by
  -- proof: 31 lines, omitted
```

`cyclicFiniteBlockRootSubgroup_eq_elementaryGroup` shows that if `Algebra.adjoin` of a finset  $s$  is all of  $R$ , then the join  $H$  of the three cyclic flattened root subgroups over  $W = \text{finiteBlockCoefficientSpan } s$  equals `elementaryGroup` of  $9 \times 9$  matrices over  $R$ . One inclusion is idx 715. For the other, the preparatory results establish that every elementary unit  $e_{xy}(a)$  with  $a \in W$  lies in  $H$ : units at cyclic block positions are direct images (idx 712); the remaining block-distinct positions are reached by the two-step factorization `elementaryUnit_mem_of_two_step`, writing  $e_{xz}(a)$  as a commutator of  $e_{xy}(a)$  and  $e_{yz}(1)$  — this is where  $1 \in W$  is essential (idx 716); and same-block positions detour through an auxiliary block (idx 717). The endgame uses `elementaryCoefficientSubalgebra`: the set of coefficients  $a$  such that every  $e_{xy}(a)$  lies in  $H$  forms a  $\mathbb{Z}/2$ -subalgebra of  $R$ ; it contains  $s$ , hence equals  $R$  by the generation hypothesis, and since the elementary group is generated as a group by elementary units,  $H$  is everything. This converts finite generation of the algebra  $R$  into generation of an infinite group by three finite subgroups.

*Remaining declarations in this section:*

|                                                                           |     |                                                                                                                          |
|---------------------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------|
| <code>finiteBlockIndex</code>                                             | DEF | Encodes a pair of <code>Fin 3</code> indices as a <code>Fin 9</code> index via <code>finProd FinEquiv</code> .           |
| <code>finiteBlockIndex_ne_of_ne</code>                                    | THM | Distinct block rows give distinct <code>Fin 9</code> indices.                                                            |
| <code>finiteBlockFlattenRingEquiv</code>                                  | DEF | Ring equivalence flattening $3 \times 3$ blocks of $3 \times 3$ matrices into $9 \times 9$ matrices.                     |
| <code>finiteBlockRootHom</code>                                           | DEF | Root homomorphism into units of $9 \times 9$ matrices, composing block roots with flattening.                            |
| <code>finiteBlockRootSubgroup</code>                                      | DEF | The flattened root subgroup: range of the flattened root homomorphism.                                                   |
| <code>finiteBlockRootHom_single</code>                                    | THM | On single-entry coefficient matrices, the flattened root homomorphism gives an elementary unit at block-encoded indices. |
| <code>elementaryUnit_mem_finiteBlockRootSubgroup</code>                   | THM | Elementary units with coefficient in $W$ at block-compatible positions lie in the flattened root subgroup.               |
| <code>finiteBlockRootSubgroup_le_elementaryGroup</code>                   | THM | The flattened root subgroup is contained in the elementary group of $9 \times 9$ matrices.                               |
| <code>cyclicFiniteBlockRootSubgroup</code>                                | DEF | Join of the three cyclic flattened root subgroups at block positions $(0, 1), (1, 2), (2, 0)$ .                          |
| <code>cyclicFiniteBlockRootSubgroup_le_elementaryGroup</code>             | THM | The cyclic join is contained in the elementary group.                                                                    |
| <code>elementaryUnit_mem_cyclicFiniteBlockRootSubgroup_of_block_ne</code> | THM | Elementary units with distinct block rows lie in the cyclic join, via two-step commutator factorizations.                |
| <code>elementaryUnit_mem_cyclicFiniteBlockRootSubgroup</code>             | THM | All elementary units with coefficient in $W$ lie in the cyclic join, handling same-block positions by a detour.          |

## 5.10 Unconditional property (T)

This block assembles the pieces into the property (T) results the non-soficity argument needs. First, generation by the three cyclic root families implies property (T) for the block elementary group, by instantiating the Friedrichs-angle criterion with  $c = (\sqrt{8})^{-1}$  and the pair estimate. Then `finiteOuterBlockFlattenHom` and idx 721–722 identify the block cyclic family with the flattened one, so the generation theorem idx 718 applies whenever the coefficient set generates  $R$ . Since `BinaryLeavitt` is a finitely generated  $\mathbb{Z}/2$ -algebra, this yields `binaryLeavittElementaryNine_hasPropertyT_unconditional`, property (T) for `binaryLeavittElementaryGroup 9`, with no outstanding hypotheses. The final three theorems transport property (T) along multiplicative equivalences to the alpha, alpha-zero, and nine prefix-code elementary groups; the last of these is the group whose hypothetical soficity the Kun-style argument will refute.

**hasPropertyT\_of\_finiteOuterBlockCyclicRootFamily\_generate** thm

```
theorem hasPropertyT_of_finiteOuterBlockCyclicRootFamily_generate
  {R : Type*} [Ring R] [Algebra (ZMod 2) R]
  (W : Submodule (ZMod 2) R) [Finite W]
  (hgen :
    (⋂ i : Fin 3, finiteOuterBlockCyclicRootFamily W i) = T) :
  HasPropertyT.{_, vSharp}
    (elementaryGroup (Fin 3) (Matrix (Fin 3) (Fin 3) R)) := by
  -- proof: 28 lines, omitted
```

`hasPropertyT_of_finiteOuterBlockCyclicRootFamily_generate` states: if  $W$  is a finite  $\mathbb{Z}/2$ -submodule of  $R$  and the supremum of the three cyclic root subgroups `finiteOuterBlockCyclicRootFamily` is the full block elementary group, then that group satisfies `HasPropertyT`. The proof first checks each family member is finite (idx 651 applied to the three cyclic positions), then invokes the Friedrichs-angle criterion `hasPropertyT_of_three_finite_subgroups_friedrichs_angle` (idx 638) with constant  $c = (\sqrt{8})^{-1}$ , whose admissibility  $0 \leq c < 1/2$  is idx 635 and 636. The angle hypothesis for an arbitrary complete unitary representation and an arbitrary ordered pair  $i \neq j$  of the family is discharged by the all-pairs estimate idx 701, converted from its squared form to the linear form by idx 637; a small `rfl`-style identification reconciles the two definitions of fixed submodule (unitary

FixedSubmodule versus ejUnitaryFixedSubmodule). This theorem is the precise junction where the group-theoretic construction (roots, Galois conjugates, product-central subgroups) hands off to the abstract spectral criterion: everything before it produced the number 8; everything after it is bookkeeping and transport.

**binaryLeavittElementaryNine\_hasPropertyT\_unconditional** thm

```
theorem binaryLeavittElementaryNine_hasPropertyT_unconditional :
  HasPropertyT.{0, vSharp} (binaryLeavittElementaryGroup 9) := by
-- proof: 16 lines, omitted
```

`binaryLeavittElementaryNine_hasPropertyT_unconditional` proves property (T) for `binaryLeavittElementaryGroup 9` with no side conditions — the word `unconditional` distinguishes it from earlier conditional variants in the file. The proof: `Algebra.FiniteType.out` extracts a finset  $s$  with `Algebra.adjoin ( $\mathbb{Z}/2$ )  $s = \mathbb{T}$`  for the binary Leavitt algebra `BinaryLeavitt`; take  $W = \text{finiteBlockCoefficientSpan } s$ , which is finite (idx 705). The generation theorem idx 722 — proved by mapping the block cyclic family through `finiteOuterBlockFlattenHom`, identifying its image with `cyclicFiniteBlockRootSubgroup` (idx 721), applying idx 718, and pulling back along the injective flattening — shows the cyclic family generates the block elementary group over `BinaryLeavitt`. Then idx 719 gives property (T) for that block group, and `hasPropertyT_of_mulEquiv` transports it along the Morita-style equivalence (`elementaryBlockGroupEquiv` composed with `elementaryReindexGroupEquiv` for `finProdFinEquiv`) onto the elementary group of  $9 \times 9$  matrices. The three successors (idx 724–726) push property (T) to the prefix-code elementary groups; in particular `ninePrefixElementaryGroup_hasPropertyT_unconditional` equips the group at the heart of the non-soficity proof with the spectral gap that its sofic approximations would have to respect.

*Remaining declarations in this section:*

|                                                                        |     |                                                                                                                   |
|------------------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------|
| <code>finiteOuterBlockFlattenHom</code>                                | DEF | Group homomorphism from the block elementary group into units of $9 \times 9$ matrices, via flattening.           |
| <code>finiteOuterBlockRootSubgroup_map_flatten</code>                  | THM | Flattening maps each block root subgroup onto the corresponding flattened root subgroup.                          |
| <code>finiteOuterBlockCyclicRootFamily_iSup_eq_top</code>              | THM | The cyclic root family generates the whole block elementary group when $s$ generates $R$ .                        |
| <code>alphaPrefixElementaryGroup_hasPropertyT_unconditional</code>     | THM | Transfers property (T) to the alpha prefix-code elementary group.                                                 |
| <code>alphaZeroPrefixElementaryGroup_hasPropertyT_unconditional</code> | THM | Transfers property (T) to the alpha-zero prefix-code elementary group.                                            |
| <code>ninePrefixElementaryGroup_hasPropertyT_unconditional</code>      | THM | Transfers property (T) to the nine-prefix-code elementary group via <code>ninePrefixElementaryGroupEquiv</code> . |

## 5.11 Coarea repair of almost permutations (KunThomFiberCoarea)

A long, purely combinatorial namespace: given a finite relation  $U \subseteq V \times V$  that is approximately invariant under the diagonal action of an expanding family of permutations  $\sigma_i$ , it repairs  $U$  into an honest permutation nearly commuting with the family. Fibers and multiplicities of  $U$  over each coordinate are introduced, their total variation under the diagonal action is bounded by the diagonal boundary of  $U$ , and a layer-cake coarea inequality (idx 757) converts the Cheeger-type expansion hypothesis into mass bounds: few vertices with zero or excess multiplicity. The matched edges — those whose endpoints both have multiplicity one — form a partial bijection extendable to a permutation whose commutation defect and distance to a reference permutation are controlled by the boundary (idx 794, 797). The results are packaged as `HasAlmostCentralizerImprovement` (idx 798), finally in a form rooted at a reference permutation graph (idx 806), the interface consumed by the later Kun-style iteration against sofic approximations.

**permutation\_small\_support\_coarea** thm

```
theorem permutation_small_support_coarea
```

```

{V ι : Type*} [Fintype V] [Fintype ι] [DecidableEq V]
(σ : ι → Equiv.Perm V) (h : ℝ)
(hexp : ∀ A : Finset V,
  h * min (A.card : ℝ)
    ((Fintype.card V : ℝ) - A.card) ≤
    (SoficGroups.boundary σ A : ℝ))
(f : V → ℕ)
(hhalf : 2 * (positiveNatSupport f).card ≤ Fintype.card V) :
2 * h * (∑ x : V, f x) ≤
  (permutationVariation σ f : ℝ) := by
-- proof: 115 lines, omitted

```

`permutation_small_support_coarea` is the discrete coarea inequality driving the whole namespace. Hypotheses: a family  $\sigma_i$  of permutations of a finite vertex set  $V$  satisfying the Cheeger-type expansion bound  $h \cdot \min(|A|, |V| - |A|) \leq \partial_{\sigma} A$  for every vertex set  $A$  (where `SoficGroups.boundary` counts points leaving  $A$  under each  $\sigma_i$ ), and a function  $f : V \rightarrow \mathbb{N}$  whose positive support occupies at most half of  $V$ . Conclusion:  $2h \sum_x f(x) \leq \text{permutationVariation } \sigma f$ . The proof is a strong induction on the support size, peeling the lowest layer of the layer-cake: with  $A$  the positive support and  $m$  the minimum positive value, the exact decomposition `permutationVariation_subtract_layer` (idx 756) splits the variation of  $f$  into that of the reduced function  $g = (f - m)\mathbf{1}_A$  plus  $2m \cdot \partial_{\sigma} A$ ; the expansion hypothesis (with the minimum resolved to  $|A|$  by the half-support assumption) converts the boundary term into at least  $2hm|A|$ , while  $\sum f = \sum g + m|A|$ , closing the induction since  $g$  has strictly smaller support. Through the contraction lemmas (idx 760, 761) this yields all the excess-mass and zero-mass bounds (idx 766–770): expansion forces the multiplicity profile of an almost-invariant relation to be almost exactly one everywhere.

#### **exists\_boundary\_controlled\_permutation\_repair** *thm*

```

theorem exists_boundary_controlled_permutation_repair
{V ι : Type*} [Fintype V] [Fintype ι] [DecidableEq V]
(σ : ι → Equiv.Perm V) (U : Finset (V × V))
(h : ℝ) (hpositive : 0 < h)
(hexp : ∀ A : Finset V,
  h * min (A.card : ℝ)
    ((Fintype.card V : ℝ) - A.card) ≤
    (SoficGroups.boundary σ A : ℝ))
(hfirst : Fintype.card V ≤
  2 * (singletonSupport (firstMultiplicity U)).card)
(hsecond : Fintype.card V ≤
  2 * (singletonSupport (secondMultiplicity U)).card) :
∃ q : Equiv.Perm V,
  matchedFiberEdges U ⊆ SoficGroups.permutationGraph q ∧
  h * (SoficGroups.permutationCommutationDefect σ q : ℝ) ≤
    (h + 8 * (Fintype.card ι : ℝ)) *
    (SoficGroups.boundary
      (fun i => (σ i).prodCongr (σ i)) U : ℝ) := by
-- proof: 57 lines, omitted

```

`exists_boundary_controlled_permutation_repair` is the repair theorem. Given an expanding family  $\sigma$  with constant  $h > 0$  and a relation  $U \subseteq V \times V$  whose first and second multiplicities are exactly one on at least half the vertices, it produces a permutation  $q$  such that the matched edges of  $U$  are contained in the graph of  $q$ , and  $h$  times the commutation defect `SoficGroups.permutationCommutationDefect`  $\sigma q$  is at most  $(h + 8|ι|)$  times the diagonal boundary of  $U$ . The construction: the matched edges, having unit multiplicity at both endpoints, define injections in both coordinates, so `Equiv.Perm.exists_extending_pair` extends them to a genuine permutation  $q$  (idx 789, 790). The commutation defect of  $q$  equals the diagonal boundary of its graph (`boundary_permutationGraph_eq_commutationDefect`), which idx 793 compares to the boundary of  $U$  at a cost proportional to the symmetric difference of the graph and  $U$ ; that difference is bounded by the unmatched vertex and edge counts (idx 792), each of which the coarea machinery caps at four boundaries over  $h$  (idx 787, 788). The message: a hypothetical almost-equivariant correspondence with a small diagonal boundary can always be straightened into an actual permutation almost centralizing the expander family.

### hasAlmostCentralizerImprovement\_of\_expanding\_diagonal\_cuts thm

```

theorem hasAlmostCentralizerImprovement_of_expanding_diagonal_cuts
  {V ι : Type*} [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (tolerance : ℕ)
  (h : ℝ) (hpositive : 0 < h)
  (hexp : ∀ A : Finset V,
    h * min (A.card : ℝ)
      ((Fintype.card V : ℝ) - A.card) ≤
      (SoficGroups.boundary σ A : ℝ))
  (hcut : ∀ p : Equiv.Perm V,
    SoficGroups.permutationCommutationDefect σ p ≤ 2 * tolerance →
    ∃ U : Finset (V × V),
      Fintype.card V ≤
        2 * (singletonSupport (firstMultiplicity U)).card ∧
      Fintype.card V ≤
        2 * (singletonSupport (secondMultiplicity U)).card ∧
      (h + 8 * (Fintype.card ι : ℝ)) *
        (SoficGroups.boundary
          (fun i => (σ i).prodCongr (σ i)) U : ℝ) ≤
        h * (tolerance : ℝ) ∧
      (5 : ℝ) *
        (4 * (SoficGroups.boundary
          (fun i => (σ i).prodCongr (σ i)) U : ℝ) +
          h * ((U \ SoficGroups.permutationGraph p).card : ℝ)) ≤
        h * (Fintype.card V : ℝ)) :
    SoficGroups.HasAlmostCentralizerImprovement σ tolerance := by
  -- proof: 24 lines, omitted

```

`hasAlmostCentralizerImprovement_of_expanding_diagonal_cuts` packages the repair pipeline into the interface predicate `SoficGroups.HasAlmostCentralizerImprovement σ tolerance`. The hypothesis `hcut` says: for every permutation  $p$  whose commutation defect with the family is at most twice the tolerance, there exists a cut relation  $U$  that is half-singleton in both multiplicities, whose weighted diagonal boundary  $(h + 8|\iota|) \partial U$  is at most  $h \cdot$  tolerance, and which is close to the graph of  $p$  in the sense that  $5(4\partial U + h|U \setminus \Gamma_p|) \leq h|V|$ . The conclusion supplies, for every such  $p$ , a permutation  $q$  with defect at most `tolerance` and  $5 \cdot$  permutation distance  $(q, p) \leq |V|$ . The proof simply feeds  $U$  into `idx 797` (repair close to the reference  $p$ ) and translates the real-number inequalities back to naturals. This improvement property — any almost-centralizing permutation can be halved in defect while moving at most a fifth of the points — is the iterative step that the subsequent Kun-style argument applies repeatedly: a sofic approximation of the property (T) group would generate such expander families with cuts, and iterating the improvement leads to the contradiction with the spectral estimates.

### hasAlmostCentralizerImprovement\_of\_rooted\_reference\_cuts thm

```

theorem hasAlmostCentralizerImprovement_of_rooted_reference_cuts
  {V ι : Type*} [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (tolerance : ℕ)
  (h : ℝ) (hpositive : 0 < h)
  (hexp : ∀ A : Finset V,
    h * min (A.card : ℝ)
      ((Fintype.card V : ℝ) - A.card) ≤
      (SoficGroups.boundary σ A : ℝ))
  (hcut : ∀ p : Equiv.Perm V,
    SoficGroups.permutationCommutationDefect σ p ≤ 2 * tolerance →
    ∃ U : Finset (V × V),
      2 * ((U \ SoficGroups.permutationGraph p).card +
        (SoficGroups.permutationGraph p \ U).card) ≤
        Fintype.card V ∧
      (h + 8 * (Fintype.card ι : ℝ)) *
        (SoficGroups.boundary
          (fun i => (σ i).prodCongr (σ i)) U : ℝ) ≤
        h * (tolerance : ℝ) ∧
      (5 : ℝ) *
        (4 * (SoficGroups.boundary

```



```

      (fun i => (σ i).prodCongr (σ i)) U : ℝ) +
      h * ((U \ SoficGroups.permutationGraph p).card : ℝ)) ≤
      h * (Fintype.card V : ℝ)) :
    SoficGroups.HasAlmostCentralizerImprovement σ tolerance := by
-- proof: 9 lines, omitted

```

`hasAlmostCentralizerImprovement_of_rooted_reference_cuts` is the user-facing variant of idx 798 in which the awkward half-singleton hypotheses on the cut  $U$  are replaced by a single rootedness condition:  $2(|U \setminus \Gamma_p| + |\Gamma_p \setminus U|) \leq |V|$ , that is,  $U$  differs from the reference permutation graph on at most half the vertices. The reduction rests on the observation that a permutation graph has all multiplicities exactly one (idx 799, 800), so the number of vertices where  $U$  has non-singleton first or second multiplicity is bounded by the total multiplicity variation between  $U$  and  $\Gamma_p$ , which idx 736 and 737 cap by the symmetric difference (idx 802, 803). A counting argument (idx 804, 805) then shows closeness to  $\Gamma_p$  forces the singleton supports of both multiplicities to occupy at least half of  $V$ , recovering exactly the hypotheses of idx 798. This matters for the downstream architecture: in the eventual application the cut  $U$  is manufactured from spectral or measure-theoretic data localized around the given almost-centralizing permutation  $p$ , so a condition stated relative to  $\Gamma_p$  is what can actually be verified, while the singleton conditions are internal bookkeeping.

*Remaining declarations in this section:*

|                                                                  |     |                                                                                                         |
|------------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------|
| <code>firstFiber</code>                                          | DEF | Edges of $U$ with first coordinate $x$ .                                                                |
| <code>secondFiber</code>                                         | DEF | Edges of $U$ with second coordinate $x$ .                                                               |
| <code>firstMultiplicity</code>                                   | DEF | Out-multiplicity of $x$ : the cardinality of its first fiber.                                           |
| <code>secondMultiplicity</code>                                  | DEF | In-multiplicity of $x$ : the cardinality of its second fiber.                                           |
| <code>sum_firstFiber_card</code>                                 | THM | First-fiber cardinalities sum to the cardinality of $U$ .                                               |
| <code>sum_secondFiber_card</code>                                | THM | Second-fiber cardinalities sum to the cardinality of $U$ .                                              |
| <code>natDist_card_le_sdiff</code>                               | THM | The distance of two finset cardinalities is at most the size of their symmetric difference.             |
| <code>firstFiber_sdiff</code>                                    | THM | First fibers commute with set difference of relations.                                                  |
| <code>secondFiber_sdiff</code>                                   | THM | Second fibers commute with set difference of relations.                                                 |
| <code>firstMultiplicity_variation_le_relation_difference</code>  | THM | Total variation of first multiplicities between $U$ and $W$ is at most their symmetric difference size. |
| <code>secondMultiplicity_variation_le_relation_difference</code> | THM | The same bound for second multiplicities.                                                               |
| <code>diagonalImage</code>                                       | DEF | The image of $U$ under the diagonal action $(p, p)$ of a permutation.                                   |
| <code>diagonalExit</code>                                        | DEF | The number of edges of $U$ whose diagonal image leaves $U$ .                                            |
| <code>diagonalImage_card</code>                                  | THM | The diagonal image has the same cardinality as $U$ .                                                    |
| <code>firstFiber_diagonalImage</code>                            | THM | The first fiber of the diagonal image at $p(x)$ is the image of the fiber at $x$ .                      |
| <code>secondFiber_diagonalImage</code>                           | THM | The analogous identity for second fibers.                                                               |
| <code>firstMultiplicity_diagonalImage</code>                     | THM | First multiplicities are preserved by the diagonal action, up to relabeling by $p$ .                    |
| <code>secondMultiplicity_diagonalImage</code>                    | THM | Second multiplicities are preserved by the diagonal action, up to relabeling by $p$ .                   |
| <code>diagonalImage_sdiff_eq_image_exit</code>                   | THM | The difference of the diagonal image over $U$ is the image of the exiting edges.                        |
| <code>diagonalImage_sdiff_cards</code>                           | THM | Both set differences between $U$ and its diagonal image have cardinality <code>diagonalExit</code> .    |
| <code>firstMultiplicity_diagonal_variation_le_twice_exit</code>  | THM | Variation of first multiplicities along $p$ is at most twice the diagonal exit count.                   |
| <code>secondMultiplicity_diagonal_variation_le_twice_exit</code> | THM | The same bound for second multiplicities.                                                               |

|                                                             |     |                                                                                                                           |
|-------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------|
| firstMultiplicity_totalVariation_le_twice_diagonalBoundary  | THM | Summed over the family, first-multiplicity variation is at most twice the diagonal boundary of $U$ .                      |
| secondMultiplicity_totalVariation_le_twice_diagonalBoundary | THM | The same bound for second multiplicities.                                                                                 |
| permutationVariation                                        | DEF | Total variation $\sum_i \sum_x \text{dist}(f(\sigma_i x), f(x))$ of a natural-valued function under a permutation family. |
| positiveNatSupport                                          | DEF | The finset of vertices where $f$ is positive.                                                                             |
| mem_positiveNatSupport                                      | THM | Membership characterization for the positive support.                                                                     |
| card_entering_eq_card_exiting                               | THM | For a permutation, as many points enter a set $A$ as leave it.                                                            |
| sum_indicator_nat_eq_mul_card                               | THM | An indicator sum with weight $m$ equals $m$ times a filtered cardinality.                                                 |
| permutationVariation_subtract_layer                         | THM | Peeling a height- $m$ layer over $A$ splits the variation into the reduced variation plus $2m$ times the boundary.        |
| natDist_sub_one_le                                          | THM | Truncated subtraction of one contracts $\mathbf{Nat.dist}$ .                                                              |
| natDist_zero_indicator_le                                   | THM | The zero-indicator transform contracts $\mathbf{Nat.dist}$ .                                                              |
| permutationVariation_sub_one_le                             | THM | The variation of $f - 1$ is at most that of $f$ .                                                                         |
| permutationVariation_zero_indicator_le                      | THM | The variation of the zero-set indicator is at most that of $f$ .                                                          |
| singletonSupport                                            | DEF | The finset of vertices where $f$ equals one.                                                                              |
| mem_singletonSupport                                        | THM | Membership characterization for the singleton support.                                                                    |
| excess_support_card_le_half_of_singletons                   | THM | If singletons cover half the vertices, the support of $f - 1$ covers at most half.                                        |
| zero_support_card_le_half_of_singletons                     | THM | The same half bound for the support of the zero-set indicator.                                                            |
| excess_mass_le_of_diagonal_variation                        | THM | Excess mass $\sum_x (f(x) - 1)$ is at most $B/h$ when the variation is at most $2B$ .                                     |
| zero_mass_le_of_diagonal_variation                          | THM | The number of zero vertices is at most $B/h$ under the same hypotheses.                                                   |
| first_excess_mass_le_diagonalBoundary                       | THM | Specializes the excess bound to first multiplicities, with $B$ the diagonal boundary.                                     |
| second_excess_mass_le_diagonalBoundary                      | THM | The same specialization for second multiplicities.                                                                        |
| first_zero_mass_le_diagonalBoundary                         | THM | The zero-vertex count for first multiplicities is bounded by the diagonal boundary over $h$ .                             |
| nonSingletonFirstEdges                                      | DEF | Edges whose first coordinate has multiplicity different from one.                                                         |
| nonSingletonSecondEdges                                     | DEF | Edges whose second coordinate has multiplicity different from one.                                                        |
| matchedFiberEdges                                           | DEF | Matched edges: both endpoints have multiplicity exactly one.                                                              |
| firstFiber_nonSingletonFirstEdges                           | THM | The first fiber of the non-singleton edges is the full fiber or empty.                                                    |
| secondFiber_nonSingletonSecondEdges                         | THM | The analogous dichotomy for second fibers.                                                                                |
| card_nonSingletonFirstEdges_le_twice_excess                 | THM | Non-singleton first edges number at most twice the first excess mass.                                                     |
| card_nonSingletonSecondEdges_le_twice_excess                | THM | Non-singleton second edges number at most twice the second excess mass.                                                   |
| nonmatchedFiberEdges_subset                                 | THM | Non-matched edges lie in the union of the two non-singleton edge sets.                                                    |
| card_nonmatchedFiberEdges_le_twice_excess                   | THM | Non-matched edges number at most twice the two excess masses combined.                                                    |
| singletonFirstEdges                                         | DEF | Edges whose first coordinate has multiplicity exactly one.                                                                |

|                                                                  |     |                                                                                                                          |
|------------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------|
| firstFiber_singletonFirstEdges                                   | THM | The first fiber of singleton edges is the full fiber or empty.                                                           |
| card_singletonFirstEdges                                         | THM | Singleton first edges biject with the singleton support of the first multiplicity.                                       |
| singletonFirstEdges_subset_matching_union                        | THM | Singleton first edges are matched or have a non-singleton second coordinate.                                             |
| card_singletonSupport_le_matching_add_column_excess              | THM | The singleton-support size is at most the matched edges plus twice the second excess.                                    |
| card_le_zero_singleton_excess                                    | THM | Counts all vertices by zero, singleton, and excess contributions of $f$ .                                                |
| unmatched_vertices_le_zero_and_excess                            | THM | The unmatched vertex count is bounded by the zero count plus both excess masses.                                         |
| matchedFiberEdges_relation_loss_bound                            | THM | Under expansion and half-singleton hypotheses, $h$ times the non-matched edge count is at most four boundaries.          |
| matchedFiberEdges_vertex_loss_bound                              | THM | Similarly, $h$ times the unmatched vertex count is at most four boundaries.                                              |
| exists_permutation_extending_matchedFiberEdges                   | THM | The matched edges form a partial injection, so some permutation extends them.                                            |
| exists_permutationGraph_containing_matchedFiberEdges             | THM | Some permutation graph contains all matched edges.                                                                       |
| permutationGraph_card                                            | THM | A permutation graph has exactly $ V $ edges.                                                                             |
| permutationGraph_sdiff_bounds_of_matchedFiberEdges               | THM | Bounds both differences between $U$ and an extending permutation graph by matching losses.                               |
| boundary_le_boundary_add_sdiff                                   | THM | The boundary of $U$ exceeds that of $W$ by at most the family size times their symmetric difference.                     |
| permutationDistance_eq_card_permutationGraph_sdiff               | THM | The Hamming distance between permutations equals the cardinality of their graph difference.                              |
| permutationDistance_le_matching_loss_add_reference               | THM | The repaired permutation is close to a reference $p$ : distance bounded by matching loss plus $ U \setminus \Gamma_p $ . |
| exists_boundary_controlled_permutation_repair_close_to_reference | THM | Combines the repair with the distance bound relative to a reference permutation.                                         |
| firstMultiplicity_permutationGraph                               | THM | Every vertex has first multiplicity one in a permutation graph.                                                          |
| secondMultiplicity_permutationGraph                              | THM | Every vertex has second multiplicity one in a permutation graph.                                                         |
| card_nonSingletonMultiplicity_le_total_distance                  | THM | Vertices with multiplicity different from one are at most the total distance of $f$ from one.                            |
| card_nonSingletonFirstMultiplicity_le_reference_difference       | THM | Non-singleton first vertices are bounded by the symmetric difference with a reference permutation graph.                 |
| card_nonSingletonSecondMultiplicity_le_reference_difference      | THM | The same bound for non-singleton second vertices.                                                                        |
| first_singleton_half_of_reference_difference                     | THM | If $U$ is within half of a permutation graph, first singletons cover at least half the vertices.                         |
| second_singleton_half_of_reference_difference                    | THM | The same conclusion for second singletons.                                                                               |

## 6 Rooted models, Kazhdan contraction, and expander decompositions

Assuming a sofic approximation exists, this part converts it into rooted Markov models on finite graphs. Property (T) acts as a geometric contraction on the orthogonal complement of invariant vectors, word powers

acquire a uniform rootedness radius, and residual graphs are decomposed into honest expander components with vanishing bad-set density.

## 6.1 Indicator crossing and limit kernel realization (KunRootedIndicatorCrossing)

This segment converts a hypothetical sequence of finite permutation models carrying a crossing 0/1 indicator into an equivariant Hilbert-space object. A same-sign trick shows that each generator's displacement of an indicator vector is controlled by the Markov-average defect, so normalized displacements are bounded by word length. Pair displacements  $z(g, h)$  satisfy Chasles additivity and, on rooted models (multiplicative only on the indicator's support), diagonal translation invariance of inner products. Ultrafilter compactness extracts a positive semidefinite, diagonally invariant, additive limit kernel from the bounded Gram data, and an inline RKHS/GNS construction realizes any such kernel as vectors in a complete Hilbert space with a unitary representation of  $G$ . The capstone `exists_equivariant_hilbert_normalized_rooted_pair_realization` produces an equivariant cocycle whose generator average has norm exactly one, feeding the Kazhdan spectral-gap contraction of the next segment.

**permutation\_indicator\_displacement\_norm\_le\_card\_mul\_markov\_defect** thm

```
theorem permutation_indicator_displacement_norm_le_card_mul_markov_defect
  {ι V : Type*} [Fintype ι] [Nonempty ι] [Fintype V]
  (p : ι → Equiv.Perm V) (f : V → ℝ)
  (hf : ∀ x, f x = 0 ∨ f x = 1) (i : ι) :
  ||permutationUnitary (p i) (indicatorVector f) - indicatorVector f|| ≤
    (Fintype.card ι : ℝ) *
    ||permutationMarkov p (indicatorVector f) - indicatorVector f|| := by
  -- proof: 23 lines, omitted
```

This is the crossing inequality that powers the whole segment. For a zero-one valued  $f$  on a finite vertex set and permutations  $p_i$ , it bounds each individual displacement norm  $\|U_{p_i} \mathbf{1} - \mathbf{1}\|$  of the indicator vector by  $|\iota|$  times the Markov defect norm  $\|M\mathbf{1} - \mathbf{1}\|$ , where  $M$  is the average `permutationMarkov` of the generator unitaries. The proof exploits that  $f$  takes only the values 0 and 1: at each fixed vertex  $x$  the displacements  $f(p_i^{-1}x) - f(x)$  all share a common sign (`indicator_displacements_same_sign`), so the sum of their squares is dominated by the square of their sum (`sum_sq_le_sq_sum_of_same_sign`). Summing over vertices and rewriting the inner sum as  $|\iota|$  times the Markov average gives `sum_sum_indicator_displacement_sq_le_card_sq_mul_markov_defect`; extracting a single generator term via `Finset.single_le_sum` finishes. Its significance: for general vectors an average being nearly fixed says nothing about individual generators, but for indicator vectors it does. This lets `normalizedIndicatorDisplacement` of every generator be uniformly bounded, hence every word displacement grows only linearly in word length (`norm_normalizedIndicatorDisplacement_list_prod_le`), which is the pointwise bound needed for ultrafilter compactness later.

**exists\_hyperfilter\_diagonal\_positive\_pair\_kernel\_of\_rooted\_indicators** thm

```
theorem exists_hyperfilter_diagonal_positive_pair_kernel_of_rooted_indicators
  {G ι : Type*} [Group G] [Fintype ι] [Nonempty ι]
  {V : ℕ → Type*} [∀ n, Fintype (V n)]
  (p : ∀ n, ι → Equiv.Perm (V n))
  (f : ∀ n, V n → ℝ)
  (σ : ∀ n, G → Equiv.Perm (V n))
  (w : G → List ι)
  (hf : ∀ n x, f n x = 0 ∨ f n x = 1)
  (hdefect : ∀ n, permutationMarkov (p n) (indicatorVector (f n)) ≠
    indicatorVector (f n))
  (hw : ∀ n g, σ n g = (w g).map (p n)).prod
  (hroot : ∀ a g : G, ∀f n in Filter.atTop,
    ∀ x : V n, indicatorVector (f n) x ≠ 0 →
    σ n (a * g) x = (σ n a * σ n g) x) :
  ∃ K : Matrix (G × G) (G × G) ℂ,
    K.PosSemidef ∧
    (∀ a g h j k : G,
      K (a * g, a * h) (a * j, a * k) = K (g, h) (j, k)) ∧
```

```

(∀ g h j : G, ∀ q : G × G,
 K (g, h) q + K (h, j) q = K (g, j) q) ∧
∀ q r,
Tendsto
  (fun n =>
    ⟨normalizedPairDisplacement (p n) (f n) (σ n) q,
     normalizedPairDisplacement (p n) (f n) (σ n) r⟩_C)
(Filter.hyperfilter N) (N (K q r)) := by
-- proof: 16 lines, omitted

```

This theorem performs the limit extraction for the specific vectors of interest. Given a sequence of finite models: generator permutations  $p_n$ , zero-one indicators  $f_n$  with nonzero Markov defect, evaluations  $\sigma_n$  given as word products, and the rootedness hypothesis that for each pair  $a, g$  the multiplicativity  $\sigma_n(ag) = \sigma_n(a)\sigma_n(g)$  eventually holds on the support of the indicator, it produces a single matrix  $K$  indexed by  $(G \times G) \times (G \times G)$  that is positive semidefinite, invariant under diagonal left translation of both pair arguments, additive in the Chasles sense, and is the hyperfilter limit of the Gram data of the vectors `normalizedPairDisplacement`. The proof simply feeds three earlier results into the abstract `exists_hyperfilter_diagonal_positive_pair_kernel`: the word-length bound `norm_normalizedPairDisplacement_le_of_words` supplies the uniform pointwise bound required for compactness, `normalizedPairDisplacement_inner_diagonal` supplies the eventual diagonal invariance, and `normalizedPairDisplacement_add` the exact additivity. It matters because  $K$  is all the information about the model sequence that survives to the limit; everything downstream, including the Kazhdan contradiction, is played out on the Hilbert space that  $K$  generates.

#### **exists\_equivariantHilbertKernelRealization** thm

```

theorem exists_equivariantHilbertKernelRealization
  {G : Type u} [Group G]
  (K : Matrix (G × G) (G × G) ℂ)
  (hpositive : K.PosSemidef)
  (hdiagonal : ∀ a g h j k : G,
    K (a * g, a * h) (a * j, a * k) = K (g, h) (j, k)) :
  Nonempty (EquivariantHilbertKernelRealization K) := by
-- proof: 22 lines, omitted

```

This is an inline GNS-type construction: any positive semidefinite kernel  $K$  on pairs that is invariant under diagonal left translation admits an `EquivariantHilbertKernelRealization`, that is, a complete Hilbert space, vectors  $v_q$  with  $\langle v_q, v_r \rangle = K(q, r)$ , and a unitary representation  $\pi$  of  $G$  with  $\pi_a v_{(g,h)} = v_{(ag,ah)}$ . The construction routes through the file's RKHS machinery: `scalarOperatorKernel` turns  $K$  into an operator-valued kernel, positivity transfers by `scalarOperatorKernel_posSemidef`, and the carrier is `RKHS.OfKernel` with vectors the kernel functions `RKHS.kerFun` evaluated at the scalar one; the Gram identity is the reproducing property. The representation is `diagonalPairKernelUnitaryRepresentation`: translation of finitely supported pre-vectors along the diagonal permutation action preserves inner products because  $K$  is invariant (`actionPreKernelTranslation_inner`), extends to the completion as a unitary, and is checked to be a homomorphism by density. Equivariance on kernel vectors is `diagonalPairKernelUnitaryRepresentation_kerFun_one`. This converts the combinatorial limit kernel into representation-theoretic data on which property (T) arguments can act.

#### **exists\_equivariant\_hilbert\_normalized\_rooted\_pair\_realization** thm

```

theorem exists_equivariant_hilbert_normalized_rooted_pair_realization
  {G ι : Type u} [Group G] [Fintype ι] [Nonempty ι]
  {V : ℕ → Type u} [V n, Fintype (V n)]
  (p : V n, ι → Equiv.Perm (V n))
  (f : V n, V n → ℝ)
  (σ : V n, G → Equiv.Perm (V n))
  (s : ι → G) (w : G → List ι)
  (hf : ∀ n x, f n x = 0 ∨ f n x = 1)
  (hdefect : V n, permutationMarkov (p n) (indicatorVector (f n)) ≠
    indicatorVector (f n))
  (hw : ∀ n g, σ n g = ((w g).map (p n)).prod)
  (hone : ∀ n, σ n 1 = 1)
  (hgenerator : ∀ n i, σ n (s i) = p n i)

```

```

(hroot : ∀ a g : G, ∀ n in Filter.atTop,
  ∀ x : V n, indicatorVector (f n) x ≠ 0 →
    σ n (a * g) x = (σ n a * σ n g) x) :
  ∃ (K : Matrix (G × G) (G × G) ℂ)
    (R : EquivariantHilbertKernelRealization K),
  K.PosSemidef ∧
  (∀ a g h j k : G,
    K (a * g, a * h) (a * j, a * k) = K (g, h) (j, k)) ∧
  (∀ a g h : G,
    R.representation a (R.realization.vector (g, h)) =
      R.realization.vector (a * g, a * h)) ∧
  (∀ g h j : G,
    R.realization.vector (g, h) +
      R.realization.vector (h, j) =
        R.realization.vector (g, j)) ∧
  (∀ a g : G,
    equivariantPairCocycle R (a * g) =
      equivariantPairCocycle R a +
        R.representation a (equivariantPairCocycle R g)) ∧
  ||(Fintype.card ι : ℂ)-1 •
    ∑ i, equivariantPairCocycle R (s i)|| = 1 ∧
  ∀ q r,
    Tendsto
      (fun n =>
        ⟨normalizedPairDisplacement (p n) (f n) (σ n) q,
          normalizedPairDisplacement (p n) (f n) (σ n) r⟩_C)
      (Filter.hyperfilter N)
-- statement truncated; proof: 24 lines, omitted

```

The capstone of the segment assembles everything: from a rooted sequence of indicator models with generators  $s : \iota \rightarrow G$ , words  $w$ , zero-one indicators with nonzero Markov defect, it produces a kernel  $K$  and an equivariant realization  $R$  satisfying seven properties at once: positivity, diagonal invariance, equivariance of the representation, exact Chasles additivity of the vectors  $v_{(g,h)} + v_{(h,j)} = v_{(g,j)}$ , the cocycle identity for equivariant `PairCocycle` (the map  $g \mapsto v_{(g,1)}$ ), the crucial normalization that the generator average  $(|\iota|)^{-1} \sum_i b(s_i)$  of the cocycle has norm exactly one, and hyperfilter Gram convergence of the finite pair displacements to the limit vectors. The norm-one statement is where the finite combinatorics enters: at every stage  $n$  the corresponding average equals the normalized indicator defect, which has norm one by construction (`norm_average_normalized PairDisplacement_of_model`), and `norm_smul_finet_sum_eq_one_of_gram` transports this exactly to the limit. This norm-one averaged cocycle is precisely the object that the Kazhdan Markov contraction of the next segment will show must be small, setting up the eventual contradiction with soficity.

*Remaining declarations in this section:*

|                                                                             |     |                                                                                                                                           |
|-----------------------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sum_sq_le_sq_sum_of_same_sign</code>                                  | THM | If all terms of a finite family share one sign, the sum of squares is at most the square of the sum.                                      |
| <code>indicator_displacements_same_sign</code>                              | THM | For a zero-one valued $f$ , the displacements $f(p_i x) - f(x)$ over all $i$ share one sign at each point $x$ .                           |
| <code>sum_indicator_displacement_sq_le</code>                               | THM | Pointwise consequence: the sum of squared indicator displacements is bounded by the square of their sum.                                  |
| <code>sum_sum_indicator_displacement_sq_le</code>                           | THM | Sums the pointwise same-sign bound over all vertices after swapping the order of summation.                                               |
| <code>sum_sum_indicator_displacement_sq_le_card_sq_mul_markov_defect</code> | THM | Total squared displacement over all generators is at most $ \iota ^2$ times the summed squared Markov-average defect.                     |
| <code>permutationUnitary</code>                                             | DEF | The unitary on $\ell^2(V)$ over $\mathbb{C}$ permuting coordinates by a permutation, via <code>LinearIsometryEquiv.piLpCongrLeft</code> . |
| <code>permutationUnitary_mul</code>                                         | THM | <code>permutationUnitary</code> turns products of permutations into compositions of the corresponding unitaries.                          |
| <code>permutationUnitary_one</code>                                         | THM | <code>permutationUnitary</code> of the identity permutation is the identity operator.                                                     |

|                                                             |     |                                                                                                                                          |
|-------------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------|
| permutationUnitary_eq_of_agree_on_support                   | THM | Two permutations agreeing wherever a vector is nonzero produce the same unitary image of that vector.                                    |
| permutationUnitary_model_mul_of_agree_on_support            | THM | If the model $\sigma$ is multiplicative on the support of $\xi$ , the associated unitaries compose on $\xi$ .                            |
| indicatorVector                                             | DEF | Casts a real-valued function on the vertex set into Euclidean Space over $\mathbb{C}$ .                                                  |
| permutationMarkov                                           | DEF | The Markov averaging operator: the mean of the permutation unitaries over all generators.                                                |
| permutationMarkov_apply                                     | THM | Computes permutationMarkov pointwise as the average of pull-backs along generator inverses.                                              |
| permutation_indicator_displacement_norm_sq                  | THM | The squared norm of a unitary displacement of an indicator vector equals the real sum of squared displacements.                          |
| permutation_indicator_markov_defect_norm_sq                 | THM | Expresses the squared Markov defect norm of an indicator vector as a real sum over vertices.                                             |
| sum_permutation_indicator_displacement_norm_sq_le           | THM | The sum of squared generator displacement norms is at most $ \iota ^2$ times the squared Markov defect norm.                             |
| normalizedIndicatorDisplacement                             | DEF | The displacement of the indicator vector under a permutation, rescaled by the inverse of the Markov defect norm.                         |
| norm_normalizedIndicatorDisplacement_generator_le           | THM | The normalized displacement of each generator has norm at most $ \iota $ , given a nonzero defect.                                       |
| normalizedIndicatorDisplacement_mul                         | THM | Cocycle identity: the displacement of a product splits as a displacement plus a unitary-translated displacement.                         |
| norm_normalizedIndicatorDisplacement_list_prod_le           | THM | The normalized displacement of a word of generators grows at most linearly in the word length.                                           |
| normalizedIndicatorDefect                                   | DEF | The Markov defect of the indicator vector, rescaled by its own norm.                                                                     |
| norm_normalizedIndicatorDefect                              | THM | When the defect is nonzero, the normalized indicator defect has norm exactly one.                                                        |
| average_permutation_indicator_displacement                  | THM | Averaging the generator displacements recovers exactly the Markov defect vector.                                                         |
| average_normalizedIndicatorDisplacement                     | THM | Averaging the normalized generator displacements gives exactly the normalized indicator defect.                                          |
| normalizedIndicatorDisplacement_cocycle_of_agree_on_support | THM | Cocycle identity for $\sigma(ag)$ assuming multiplicativity of $\sigma$ only on the indicator's support.                                 |
| normalizedPairDisplacement                                  | DEF | The difference of normalized displacements at the two coordinates of a pair of group elements.                                           |
| normalizedPairDisplacement_diagonal                         | THM | Diagonal left translation of a pair acts on pair displacements by the unitary of $\sigma(a)$ , given rootedness.                         |
| normalizedPairDisplacement_inner_diagonal                   | THM | Inner products of pair displacements are invariant under diagonal left translation, given support-multiplicativity at the four products. |
| normalizedPairDisplacement_add                              | THM | Chasles relation: the displacements of $(g, h)$ and $(h, j)$ sum to the displacement of $(g, j)$ .                                       |
| norm_normalizedPairDisplacement_le_of_words                 | THM | Bounds a pair displacement norm by $ \iota $ times the sum of the word lengths of both entries.                                          |
| exists_hyperfilter_gram_limit_of_pointwise_bound            | THM | Ultrafilter compactness: uniformly bounded vector families have Gram inner products converging along the hyperfilter to a limit matrix.  |
| gram_posSemidef_infinite                                    | THM | The Gram matrix of any, possibly infinite, family of Hilbert-space vectors is positive semidefinite.                                     |



|                                                            |        |                                                                                                                              |
|------------------------------------------------------------|--------|------------------------------------------------------------------------------------------------------------------------------|
| exists_hyperfilter_positive_gram_kernel_of_pointwise_bound | THM    | The hyperfilter limit of Gram matrices of a bounded family is Hermitian and positive semidefinite.                           |
| exists_hyperfilter_diagonal_positive_pair_kernel           | THM    | Extracts a limit kernel that is positive, diagonally invariant, and Chasles-additive from eventual properties of the family. |
| scalarOperatorKernel                                       | DEF    | Converts a complex matrix kernel into an operator-valued kernel via scalar multiplication operators.                         |
| scalarOperatorKernel_posSemidef                            | THM    | Positive semidefiniteness transfers from a complex kernel to its operator-valued version, using the RKHS characterization.   |
| preKernel_inner_single                                     | THM    | Computes the RKHS pre-inner product of two single-point functions in terms of the kernel.                                    |
| actionPreKernelTranslation                                 | DEF    | Translation of finitely supported RKHS pre-vectors along a permutation action on the index set.                              |
| actionPreKernelTranslation_inner                           | THM    | Pre-kernel translation preserves inner products whenever the kernel is invariant under the action.                           |
| actionPreKernelTranslationIsometry                         | DEF    | Packages the inner-product-preserving pre-kernel translation as a linear isometry equivalence of the pre-space.              |
| actionKernelTranslationMap                                 | DEF    | Extends the pre-kernel translation to the RKHS completion as a continuous linear map.                                        |
| actionKernelTranslationMap_coe                             | THM    | The completed translation map agrees with the pre-kernel translation on the dense finitely supported vectors.                |
| actionKernelTranslationMap_isometry                        | THM    | The completed translation map is an isometry of the RKHS completion.                                                         |
| actionPreKernelTranslation_mul_apply                       | THM    | Pre-kernel translation is multiplicative in the group element, checked on single-point generators.                           |
| actionPreKernelTranslation_one_apply                       | THM    | Pre-kernel translation by the identity element is the identity map.                                                          |
| actionKernelTranslationMap_mul_apply                       | THM    | Multiplicativity of the completed translation map, proved by density from the pre-kernel case.                               |
| actionKernelTranslationMap_one_apply                       | THM    | The completed translation by the identity element is the identity map.                                                       |
| actionKernelTranslationLinearEquiv                         | DEF    | Upgrades the translation map to a linear equivalence, with inverse given by the inverse group element.                       |
| actionKernelTranslationUnitary                             | DEF    | The translation by a group element as a unitary, that is a linear isometry equivalence, of the RKHS.                         |
| actionKernelUnitaryRepresentation                          | DEF    | Assembles the translation unitaries into a unitary representation of $G$ on the RKHS of an invariant kernel.                 |
| diagonalPairAction                                         | DEF    | The diagonal left-multiplication action of $G$ on $G \times G$ as a homomorphism into permutations.                          |
| diagonalPairKernelUnitaryRepresentation                    | DEF    | Specializes the kernel unitary representation to the diagonal action on pairs.                                               |
| ofKernel_kerFun_one_eq_coe_single                          | THM    | Identifies the RKHS kernel function at scalar one with the image of a single-point pre-vector.                               |
| actionKernelUnitaryRepresentation_kerFun_one               | THM    | The kernel representation translates kernel functions along the index action.                                                |
| diagonalPairKernelUnitaryRepresentation_kerFun_one         | THM    | For the diagonal pair action, kernel vectors at $(g, h)$ map to kernel vectors at $(ag, ah)$ .                               |
| HilbertKernelRealization                                   | STRUCT | Structure: a complete complex Hilbert space with a family of vectors whose Gram matrix equals a given kernel.                |
| hilbertKernelRealizationNormedAddCommGroup                 | INST   | Instance exposing the bundled normed additive group structure of a realization's carrier.                                    |

|                                                               |        |                                                                                                                               |
|---------------------------------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>hilbertKernelRealizationInnerProduct Space</code>       | INST   | Instance exposing the bundled complex inner product space structure of the carrier.                                           |
| <code>hilbertKernelRealizationComplete Space</code>           | INST   | Instance exposing the bundled completeness of the realization carrier.                                                        |
| <code>hilbertKernelRealization_pair_vector_add</code>         | THM    | Kernel Chasles additivity forces exact additivity of realized vectors, by pairing the defect against all kernel vectors.      |
| <code>EquivariantHilbertKernelRealization</code>              | STRUCT | Structure: a kernel realization plus a unitary representation intertwining the vectors with diagonal translation of pairs.    |
| <code>equivariantPairCocycle</code>                           | DEF    | The candidate cocycle sending $g$ to the realized vector of the pair $(g, 1)$ .                                               |
| <code>equivariantPairCocycle_mul</code>                       | THM    | The pair cocycle satisfies the representation cocycle identity, derived from equivariance and vector additivity.              |
| <code>normalizedPairDisplacement_generator_of_model</code>    | THM    | At a generator paired with the identity, the pair displacement reduces to that generator's normalized indicator displacement. |
| <code>norm_average_normalizedPairDisplacement_of_model</code> | THM    | The average of the generator pair displacements has norm one, being the normalized indicator defect.                          |
| <code>inner_smul_finset_sum_self</code>                       | THM    | Expands the self inner product of a scaled finite sum into a double sum of inner products.                                    |
| <code>tendsto_norm_sq_smul_finset_sum_of_gram</code>          | THM    | Gram convergence of a family implies convergence of squared norms of scaled finite sums of its vectors.                       |
| <code>norm_smul_finset_sum_eq_one_of_gram</code>              | THM    | If every approximant's scaled sum has norm one, the limit realization's scaled sum also has norm one.                         |

## 6.2 Kazhdan contraction on orthogonal complement (KunThomInvariantOrthogonal)

This segment develops the property (T) spectral-gap machinery in the form used later. For a unitary representation  $\pi$  of  $G$  on a Hilbert space, it isolates the submodule of invariant vectors and restricts  $\pi$  to its orthogonal complement, where no nonzero fixed vectors remain. A Kazhdan pair then guarantees that some generator moves every such vector by  $\kappa\|x\|$ ; combined with a parallelogram estimate and a pair-extraction bound for sums of equal-norm vectors, the sum of  $\pi_g x$  over any finite symmetric superset  $S$  of the generators containing the identity loses a fixed amount  $\kappa^2/4$ . Consequently the averaging operator `unitaryFinsetMarkov` contracts the orthogonal complement by the explicit factor `kazhdanMarkovContractionFactor`, strictly less than one, and its iterates contract geometrically. The segment also proves that a cocycle summed over a symmetric set is orthogonal to every invariant vector, which is exactly what qualifies the averaged cocycle from the previous segment for this contraction.

**kazhdan\_generator\_sum\_contraction** thm

```
theorem kazhdan_generator_sum_contraction
  [CompleteSpace H] (P : SoficGroups.KazhdanPair.{u, v} G)
  (π : SoficGroups.UnitaryRepresentation G H)
  (S : Finset G) (h1 : 1 ∈ S)
  (hsub : P.generators ⊆ S) (x : H)
  (horth : ∀ y : H,
    (∀ g : G, π g y = y) → @inner ℂ H _ y x = 0) :
  ‖∑ g ∈ S, π g x‖ ≤
    ((S.card : ℝ) - P.kazhdanConstant ^ 2 / 4) * ‖x‖ := by
-- proof: 44 lines, omitted
```

This is the quantitative heart of the property (T) argument. Let  $P$  be a Kazhdan pair for  $G$ ,  $\pi$  a unitary representation on a complete space, and  $S$  a finite set containing 1 and the Kazhdan generators. For any  $x$  orthogonal to every invariant vector, it proves  $\|\sum_{g \in S} \pi_g x\| \leq (|S| - \kappa^2/4)\|x\|$  where  $\kappa$  is the Kazhdan constant. The proof: `kazhdan_generator_displacement_of_orthogonal_invariants` lifts  $x$  into the orthogonal complement of `invariantSubmodule`, where the restricted representation `orthogonalRepresentation` has no fixed vectors, so the Kazhdan property yields a generator  $g$  with  $\|\pi_g x - x\| \geq \kappa\|x\|$ . The parallelogram estimate `norm_add_le_two_`

`sub_sq_div_four` converts separation into cancellation:  $\|x + \pi_g x\| \leq (2 - \kappa^2/4)\|x\|$ . Finally `norm_finset_sum_le_pair_add` splits the full sum into this special pair (using  $1 \neq g$ , which follows since  $\kappa > 0$ ) plus  $|S| - 2$  terms of norm  $\|x\|$ . The fixed deficit  $\kappa^2/4$ , independent of the representation, is what becomes a uniform spectral gap for all the limit spaces built from hypothetical sofic models.

#### `unitaryFinsetMarkov_iterate_norm_le` thm

```
theorem unitaryFinsetMarkov_iterate_norm_le
  [CompleteSpace H] (P : SoficGroups.KazhdanPair.{u, v} G)
  (π : SoficGroups.UnitaryRepresentation G H)
  (S : Finset G) (hone : 1 ∈ S)
  (hsub : P.generators ⊆ S) (x : H)
  (horth : ∀ y : H,
    (∀ g : G, π g y = y) → @inner ℂ H _ y x = 0)
  (n : ℕ) :
  ||(unitaryFinsetMarkov π S)^[n] x|| ≤
    kazhdanMarkovContractionFactor P S ^ n * ||x|| := by
  -- proof: 27 lines, omitted
```

This theorem upgrades the one-step contraction to geometric decay: for  $x$  orthogonal to all invariant vectors, the  $n$ -th iterate of `unitaryFinsetMarkov` (the average  $(|S|)^{-1} \sum_{g \in S} \pi_g$ ) satisfies  $\|T^n x\| \leq c^n \|x\|$  with  $c = \text{kazhdanMarkovContractionFactor} < 1$ . Two ingredients make the induction work. First, `unitaryFinsetMarkov_norm_le` rescales the sum bound of `kazhdan_generator_sum_contraction` by  $|S|^{-1}$ , giving the factor  $1 - \kappa^2/(4|S|)$ , clipped at zero for safety. Second, `unitaryFinsetMarkov_orthogonal_invariants` shows the operator maps the orthogonal complement of the invariants into itself (each  $\pi_g$  preserves it by `map_mem_invariantSubmodule_orthogonal`, and the complement is a submodule), so the hypothesis propagates through the iteration; an inner induction establishes orthogonality of every iterate. The result matters because the next segment identifies the iterated Markov defect vectors of the finite models, in the Gram limit, with exactly such iterates  $T^k z$  of a norm-one vector  $z$  orthogonal to invariants; geometric decay there contradicts the norm-one normalization, giving the uniform contraction estimate that soficity will eventually violate.

*Remaining declarations in this section:*

|                                                          |     |                                                                                                                                 |
|----------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------|
| <code>invariantSubmodule</code>                          | DEF | The submodule of vectors fixed by every operator of a unitary representation.                                                   |
| <code>map_mem_invariantSubmodule_orthogonal</code>       | THM | The orthogonal complement of the invariant vectors is preserved by every representation operator.                               |
| <code>orthogonalLinearIsometryEquiv</code>               | DEF | Restriction of each representation operator to the orthogonal complement, as a linear isometry equivalence.                     |
| <code>orthogonalRepresentation</code>                    | DEF | The restricted unitary representation of $G$ on the orthogonal complement of the invariant vectors.                             |
| <code>orthogonalRepresentation_no_fixed</code>           | THM | The restricted representation has no nonzero fixed vectors, since such vectors would be invariant and orthogonal to themselves. |
| <code>cocycle_one</code>                                 | THM | Any cocycle for a unitary representation vanishes at the identity element.                                                      |
| <code>inner_cocycle_inv_of_invariant</code>              | THM | Against an invariant vector, the cocycle at $g^{-1}$ pairs to minus its value at $g$ .                                          |
| <code>inner_cocycle_sum_eq_zero_of_invariant</code>      | THM | A cocycle summed over a symmetric finite set is orthogonal to every invariant vector.                                           |
| <code>inner_smul_cocycle_sum_eq_zero_of_invariant</code> | THM | Scalar multiples of symmetric cocycle sums remain orthogonal to every invariant vector.                                         |
| <code>norm_add_le_two_sub_sq_div_four</code>             | THM | Parallelogram law: equal-norm vectors that are $\delta$ -separated have sum of norm at most $(2 - \delta^2/4)\ x\ $ .           |
| <code>norm_finset_sum_le_pair_add</code>                 | THM | Bounds a finite sum of equal-norm vectors by one chosen pair's sum plus the remaining terms.                                    |

|                                                         |     |                                                                                                        |
|---------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------|
| kazhdan_generator_displacement_of_orthogonal_invariants | THM | A Kazhdan pair moves any vector orthogonal to all invariants by $\kappa\ x\ $ under some generator.    |
| unitaryFinsetMarkov                                     | DEF | The Markov operator averaging a unitary representation over a finite subset of the group.              |
| kazhdanMarkovContractionFactor                          | DEF | The contraction factor $\max(0, 1 - \kappa^2/(4 S ))$ attached to a Kazhdan pair and finite set.       |
| kazhdanMarkovContractionFactor_lt_one                   | THM | The Kazhdan contraction factor is strictly less than one for nonempty $S$ .                            |
| kazhdanMarkovContractionFactor_nonneg                   | THM | The contraction factor is nonnegative, being a maximum with zero.                                      |
| unitaryFinsetMarkov_norm_le                             | THM | On vectors orthogonal to invariants, the finite Markov operator contracts norms by the Kazhdan factor. |
| unitaryFinsetMarkov_orthogonal_invariants               | THM | The finite Markov operator preserves orthogonality to the invariant vectors.                           |

### 6.3 Word powers and uniform rootedness radius (KunRootedWordPower)

This segment connects Markov iterates on the finite models to the limit realization and extracts a uniform finitary contraction. `pairedWordAverage` abstracts  $k$ -fold averaging of a pair-indexed vector family over left generator translation, and `pairedDefectWordAverage` its evaluation at generator-identity pairs. On rooted model sequences this recursion eventually coincides with iterates of `permutationMarkov` applied to the normalized indicator defect, and Gram convergence transports the norms to the Hilbert limit. There the defect average is exactly an iterate of `unitaryFinsetMarkov` applied to the norm-one averaged cocycle, which is orthogonal to all invariant vectors by the symmetric cocycle lemma, so Kazhdan contraction applies. The structures `RootedIndicatorMarkovModel`, `IsGenerated`, and `IsRootedAtRadius` then package single finite models, and a compactness argument produces a radius  $r$  such that every model rooted at radius  $r$  satisfies the strict contraction, finally denormalized into a bound on consecutive Markov iterates of the raw indicator vector.

**eventually\_normalizedIndicatorDefect\_markov\_pow\_lt\_of\_rooted** thm

```

theorem eventually_normalizedIndicatorDefect_markov_pow_lt_of_rooted
  {G : Type u} [Group G]
  (P : SoficGroups.KazhdanPair.{u, u} G)
  (S : Finset G) (honeS : 1 ∈ S)
  (hcover : P.generators ≤ S)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  {V : ℕ → Type u} [∀ n, Fintype (V n)]
  (p : ∀ n, ιS → Equiv.Perm (V n))
  (f : ∀ n, V n → ℝ)
  (σ : ∀ n, G → Equiv.Perm (V n))
  (w : G → List ιS)
  (hf : ∀ n x, f n x = 0 ∨ f n x = 1)
  (hdefect : ∀ n,
    permutationMarkov (p n) (indicatorVector (f n)) ≠
    indicatorVector (f n))
  (hw : ∀ n g, σ n g = ((w g).map (p n)).prod)
  (hone : ∀ n, σ n 1 = 1)
  (hgenerator : ∀ n (i : ιS), σ n (i : G) = p n i)
  (hroot : ∀ a g : G, ∀f n in Filter.atTop,
    ∀ x : V n, indicatorVector (f n) x ≠ 0 →
    σ n (a * g) x = (σ n a * σ n g) x)
  (k : ℕ) (ε : ℝ) (hε : 0 < ε) :
  ∀f n in (Filter.hyperfilter ℕ : Filter ℕ),
    ||((permutationMarkov (p n))[k])
      (normalizedIndicatorDefect (p n) (f n))|| <
      kazhdanMarkovContractionFactor P S ^ k + ε := by
-- proof: 50 lines, omitted

```

This is where property (T) meets the finite models. Fix a Kazhdan pair  $P$  and a finite symmetric  $S$

containing 1 and covering the generators, and a rooted sequence of indicator models over  $S$ . The conclusion: for every  $k$  and  $\varepsilon > 0$ , hyperfilter-eventually in  $n$ ,  $\|M_n^k(d_n)\| < c^k + \varepsilon$ , where  $M_n$  is `permutationMarkov`,  $d_n$  the normalized indicator defect, and  $c$  the `kazhdanMarkovContractionFactor`. The proof composes the whole architecture: `exists_equivariant_hilbert_normalized_rooted_pair_realization` yields the limit realization  $R$  with norm-one averaged cocycle  $z$ ; `inner_smul_cocycle_sum_eq_zero_of_invariant` shows  $z$ , a scalar multiple of the cocycle summed over the symmetric set  $S$ , is orthogonal to all invariant vectors; `pairedDefectWordAverage_eq_unitaryFinsetMarkov_iterate` identifies the limit depth- $k$  defect average with the  $k$ -th `unitaryFinsetMarkov` iterate of  $z$ ; `unitaryFinsetMarkov_iterate_norm_le` bounds this by  $c^k$ ; and `tendsto_norm_normalizedIndicatorDefect_markov_pow` transports the finite-model iterate norms to this limit quantity along the hyperfilter, so eventually they fall below  $c^k + \varepsilon$ . This asymptotic bound is not yet usable on a single model; the next theorems make it uniform.

#### `exists_rooted_word_radius_normalized_markov_contraction` thm

```

theorem exists_rooted_word_radius_normalized_markov_contraction
  {G : Type u} [Group G]
  (P : SoficGroups.KazhdanPair.{u, u} G)
  (S : Finset G) (honeS : 1 ∈ S)
  (hcover : P.generators ⊆ S)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (w : G → List ιS)
  (k : ℕ) (ε : ℝ) (hε : 0 < ε) :
  ∃ r : ℕ, ∀ X : RootedIndicatorMarkovModel G ιS,
    X.IsGenerated (fun i : ιS => (i : G)) w →
    X.IsRootedAtRadius w r →
    permutationMarkov X.generator (indicatorVector X.indicator) ≠
      indicatorVector X.indicator →
    ||((permutationMarkov X.generator)^[k])
      (normalizedIndicatorDefect X.generator X.indicator)|| <
      kazhdanMarkovContractionFactor P S ^ k + ε := by
-- proof: 38 lines, omitted

```

This theorem converts the asymptotic bound of `eventually_normalizedIndicatorDefect_markov_pow_lt_of_rooted` into a uniform finitary criterion via compactness by contradiction. It asserts: for any Kazhdan pair  $P$ , symmetric  $S$  with  $1 \in S$  covering the generators, word assignment  $w$ , exponent  $k$ , and  $\varepsilon > 0$ , there exists a radius  $r$  such that EVERY single `RootedIndicatorMarkovModel` over  $S$  that `IsGenerated` by the inclusion and `IsRootedAtRadius`  $w$   $r$ , and has nonzero Markov defect, satisfies the strict normalized contraction  $\|M^k d\| < c^k + \varepsilon$ . The proof: if no  $r$  works, choose for each  $n$  a counterexample model  $X_n$  rooted at radius  $n$ . By `eventually_rooted_of_growing_word_radius` the growing radii deliver every fixed multiplicativity instance eventually, so the sequence satisfies the rootedness hypothesis of 919; the resulting eventual strict bound along the (nontrivial) hyperfilter must meet some index where the counterexample property gives the reverse inequality, a contradiction. This uniformization is essential: soficity produces individual finite models, not convergent sequences, and only a per-model criterion with an explicit radius can be checked against a sofic approximation. Theorem `exists_rooted_word_radius_markov_iterate_contraction` then removes the normalization, bounding consecutive raw Markov iterates.

*Remaining declarations in this section:*

|                                                           |     |                                                                                                                     |
|-----------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------|
| <code>pairedWordAverage</code>                            | DEF | Recursively averages a pair-indexed vector family over left translation by the generators, $k$ times.               |
| <code>pairedDefectWordAverage</code>                      | DEF | Averages the depth- $k$ word averages over generator-identity pairs, abstracting the iterated Markov defect.        |
| <code>inner_smul_fintype_sum</code>                       | THM | Expands the inner product of two scaled finite-type sums into a double sum of inner products.                       |
| <code>tendsto_inner_pairedWordAverage_of_gram</code>      | THM | Gram convergence of a family passes to all inner products of its word averages, by induction on depth.              |
| <code>tendsto_norm_pairedDefectWordAverage_of_gram</code> | THM | Norms of defect word averages converge along the filter under Gram convergence, via squared norms and square roots. |

|                                                                        |        |                                                                                                                               |
|------------------------------------------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>permutationMarkovLinear</code>                                   | DEF    | The permutation Markov operator packaged as a linear endomorphism of <code>EuclideanSpace</code> .                            |
| <code>permutationMarkovLinear_apply</code>                             | THM    | The linear Markov operator agrees pointwise with <code>permutationMarkov</code> .                                             |
| <code>permutationMarkovLinear_pow_apply</code>                         | THM    | Powers of the linear Markov operator compute iterates of <code>permutationMarkov</code> .                                     |
| <code>permutationMarkov_pair_of_rooted</code>                          | THM    | Under rootedness at both coordinates, applying the Markov operator to a pair displacement equals the depth-one word average.  |
| <code>eventually_permutationMarkov_pair_of_rooted</code>               | THM    | The Markov-equals-word-average identity holds eventually along <code>atTop</code> for rooted model sequences.                 |
| <code>eventually_linear_pairedWordAverage_pow</code>                   | THM    | Eventually, powers of linear operators satisfying the depth-one identity compute depth- $k$ word averages.                    |
| <code>eventually_linear_pairedDefectWordAverage_pow</code>             | THM    | Eventually, the $k$ -th operator power of the depth-zero defect average equals the depth- $k$ defect average.                 |
| <code>pairedDefectWordAverage_zero_eq_normalizedIndicatorDefect</code> | THM    | At depth zero the defect word average is exactly the normalized indicator defect.                                             |
| <code>tendsto_norm_normalizedIndicatorDefect_markov_pow</code>         | THM    | Norms of Markov iterates of the normalized defect converge along the hyperfilter to limit defect average norms.               |
| <code>representationFamilyMarkovLinear</code>                          | DEF    | The linear operator averaging a unitary representation over a finite family of group elements.                                |
| <code>representationFamilyMarkovLinear_subtype_apply</code>            | THM    | For subtype-indexed families this averaged operator equals <code>unitaryFinsetMarkov</code> .                                 |
| <code>representationFamilyMarkovLinear_subtype_pow_apply</code>        | THM    | Powers of the subtype-indexed averaged operator compute iterates of <code>unitaryFinsetMarkov</code> .                        |
| <code>representationFamilyMarkovLinear_pair_of_equivariant</code>      | THM    | For equivariant vector families, the averaged representation operator realizes the depth-one word average.                    |
| <code>linear_pairedWordAverage_pow</code>                              | THM    | Exact version: operator powers compute depth- $k$ word averages when the base identity holds everywhere.                      |
| <code>pairedDefectWordAverage_eq_unitaryFinsetMarkov_iterate</code>    | THM    | Identifies limit defect word averages with iterates of the representation's finite Markov operator on the depth-zero average. |
| <code>RootedIndicatorMarkovModel</code>                                | STRUCT | Structure bundling a finite carrier, generator permutations, a real indicator, and a full evaluation map on $G$ .             |
| <code>rootedIndicatorMarkovModelFintype</code>                         | INST   | Instance exposing the bundled finiteness of the model carrier.                                                                |
| <code>RootedIndicatorMarkovModel.IsGenerated</code>                    | STRUCT | Predicate: the indicator is zero-one and the evaluation is the word product, respecting identity and generators exactly.      |
| <code>RootedIndicatorMarkovModel.IsRootedAtRadius</code>               | STRUCT | Predicate: the evaluation is multiplicative on the indicator support whenever the three word lengths total at most $r$ .      |
| <code>eventually_rooted_of_growing_word_radius</code>                  | THM    | A sequence of models rooted at growing radius eventually satisfies each fixed multiplicativity instance.                      |
| <code>permutationMarkov_iterate_difference</code>                      | THM    | Consecutive Markov iterates differ by the $k$ -th operator power applied to the single-step defect.                           |
| <code>norm_normalizedIndicatorDefect_markov_pow</code>                 | THM    | Rewrites the iterated normalized defect norm as a ratio of unnormalized iterate difference over defect norm.                  |
| <code>exists_rooted_word_radius_markov_iterate_contraction</code>      | THM    | Denormalized: a radius forcing consecutive Markov iterates of the indicator vector to contract with the Kazhdan factor.       |

## 6.4 Sofic approximations yield rooted radii (`KunActualSoficRootRadius`)

This segment shows that a genuine sofic approximation supplies, at almost every point, exactly the rootedness data that the previous segment's contraction theorem demands. For a finite window  $F$  it defines the bad sets

where the model action fails multiplicativity or fails to separate distinct elements, and proves their densities tend to zero. Since the raw sofic action need not be a word product, `chosenWordEvaluation` re-evaluates each  $g$  as the product of generator actions along a chosen word; it converges to the true action in normalized Hamming distance and is asymptotically multiplicative. `symmetricGeneratorWord` fixes canonical words, empty at the identity and single letters on generators, so identity and generator evaluations hold exactly. Specializing the window to the ball  $S^r$  yields `chosenCayleyRadiusBad`: away from a vanishing-density set of points, the chosen evaluation is multiplicative for all pairs of total word length at most  $r$  and injective on the ball, precisely the `IsRootedAtRadius` and separation hypotheses for models built at good points.

**chosenWordEvaluation\_multiplicative\_tendsto** *thm*

```
theorem chosenWordEvaluation_multiplicative_tendsto
  {G 1 : Type*} [Group G]
  (A : SoficGroups.SoficApproximation G)
  (s : 1 → G) (w : G → List 1)
  (hw : ∀ g, ((w g).map s).prod = g) (g h : G) :
  Tendsto
    (fun n =>
      SoficGroups.normalizedHamming
        (chosenWordEvaluation A s w n (g * h))
        (chosenWordEvaluation A s w n g *
          chosenWordEvaluation A s w n h))
    atTop (N 0) := by
-- proof: 49 lines, omitted
```

The raw action of a sofic approximation is only approximately multiplicative and is not literally a product of generator permutations, but `RootedIndicatorMarkovModel.IsGenerated` demands an evaluation that IS a word product. `chosenWordEvaluation` therefore re-evaluates each  $g$  as  $\prod$  of the actions of the letters of a chosen word for  $g$ . This theorem shows nothing is lost: the normalized Hamming distance between the evaluation of  $gh$  and the product of the evaluations of  $g$  and  $h$  tends to zero as the model index grows. The proof is a chain of triangle inequalities routed through the true action: `chosenWordEvaluation_tendsto_action` (built on `action_list_prod_tendsto`, an induction using soficity at each letter) says each evaluation is Hamming-close to the true action; the sofic multiplicativity axiom handles the middle step  $\text{action}(gh) \approx \text{action}(g)\text{action}(h)$ ; and the submultiplicativity `normalizedHamming_mul_le`, itself proved from left and right invariance of the Hamming distance, transfers the two word-level errors across the product. A `squeeze_zero` argument closes the limit. This is the bridge from the analytic definition of soficity to the exact word-product format the contraction machinery consumes.

**symmetricGeneratorWord** *def*

```
def symmetricGeneratorWord
  {G : Type*} [Group G] [DecidableEq G]
  (S : Finset G)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set G) = T)
  (g : G) : List 1S := by
classical
by_cases hone : g = 1
· exact []
· by_cases hg : g ∈ S
  · exact [(g, hg)]
  · exact (exists_generator_word_of_symmetric_generates
    S hsymmetric hgenerates g).choose
```

`symmetricGeneratorWord` is a small but load-bearing definition: for a finite symmetric generating set  $S$  of  $G$  it selects, for every  $g$ , a concrete list of letters of  $S$  multiplying to  $g$ . The existence content is `exists_generator_word_of_symmetric_generates`, proved by `Subgroup.closure_induction`: letters give singleton words, products concatenate, and inverses reverse the word after replacing each letter by its inverse, using symmetry of  $S$ . The definition then overrides the choice in two special cases: the identity gets the empty word, and any  $g \in S$  that is not the identity gets the single-letter word  $[g]$ . These overrides are exactly what make `chosen_`



`symmetric_wordEvaluation_one` and `chosen_symmetric_wordEvaluation_generator` hold on the nose: the chosen-word evaluation of 1 is the identity permutation, and the evaluation of each generator is exactly the model action of that generator. The structure `RootedIndicatorMarkovModel.IsGenerated` requires these two equations exactly, not approximately, so without the special cases the whole packaging of a sofic model into a rooted Markov model would fail. The generic branch uses classical choice, which is harmless since only the product identity `symmetricGeneratorWord_prod` is ever used.

#### **chosenCayleyRadiusBad\_rooted thm**

```
theorem chosenCayleyRadiusBad_rooted
  {G : Type*} [Group G] [DecidableEq G]
  (A : SoficGroups.SoficApproximation G)
  (S : Finset G) (hS : 1 ∈ S)
  (w : G → List ιS)
  (hw : ∀ g, ((w g).map fun i : ιS => (i : G)).prod = g)
  (n r : ℕ) (a g : G)
  (hword : (w a).length + (w g).length +
    (w (a * g)).length ≤ r)
  {x : Fin (A.model n).size}
  (hx : x ∉ chosenCayleyRadiusBad A S w n r) :
  chosenWordEvaluation A (fun i : ιS => (i : G)) w n (a * g) x =
    (chosenWordEvaluation A (fun i : ιS => (i : G)) w n a *
      chosenWordEvaluation A (fun i : ιS => (i : G)) w n g) x := by
  -- proof: 11 lines, omitted
```

This theorem delivers the rootedness certificate. Fix the Cayley-ball bad set `chosenCayleyRadiusBad`, which is `chosenFiniteRootBad` with window  $F = S^r$ ; its density vanishes by 968. The statement: if  $x$  avoids this set and  $a, g$  satisfy the length budget  $(|w(a)| + |w(g)| + |w(ag)|) \leq r$ , then the chosen-word evaluation is multiplicative at  $x$ : the evaluation of  $ag$  at  $x$  equals the composite of the evaluations of  $a$  and  $g$  at  $x$ . The proof observes that the length budget puts both  $a$  and  $g$  inside the ball  $S^r$  (via `mem_generator_pow_of_chosen_word_length`, which needs  $1 \in S$  to pad shorter words), so  $x$  lying outside the chosen finite multiplication-bad set for that window gives the equality by `chosenFiniteMultiplicationBad_rooted`. Together with `chosenCayleyRadiusBad_injective_ball`, which gives injectivity of  $g \mapsto \text{action}(g)(x)$  on the ball, a good point  $x$  satisfies precisely the hypotheses `IsRootedAtRadius` and the separation needed to build a `RootedIndicatorMarkovModel` from the sofic approximation. Since the bad density vanishes, most points are good, which is what lets the later argument find indicator supports avoiding the bad set and run the Kazhdan contraction against soficity.

*Remaining declarations in this section:*

|                                                          |     |                                                                                                                       |
|----------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------|
| <code>modelMultiplicationBad</code>                      | DEF | The set of points where a permutation model fails multiplicity at a fixed pair of group elements.                     |
| <code>mem_modelMultiplicationBad</code>                  | THM | Membership criterion for the multiplication-bad set, unfolding the filter definition.                                 |
| <code>modelMultiplicationBad_density_tendsto_zero</code> | THM | In a sofic approximation, multiplication-bad densities vanish, restating the multiplicativity axiom in counting form. |
| <code>modelSeparationBad</code>                          | DEF | The set of points where the actions of two group elements agree, via <code>agreementSet</code> .                      |
| <code>mem_modelSeparationBad</code>                      | THM | Membership criterion for the separation-bad, that is agreement, set.                                                  |
| <code>modelSeparationBad_density_tendsto_zero</code>     | THM | Agreement densities for distinct elements vanish, from the normalized Hamming separation of sofic approximations.     |
| <code>finiteMultiplicationBad</code>                     | DEF | Union of the multiplication-bad sets over all pairs from a finite set $F$ .                                           |
| <code>finiteSeparationBad</code>                         | DEF | Union of the agreement sets over all distinct pairs from $F$ .                                                        |
| <code>finiteRootBad</code>                               | DEF | The combined bad set: multiplication failures or separation failures over the window $F$ .                            |

|                                                    |     |                                                                                                                  |
|----------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------|
| finiteMultiplicationBad_density_tendsto_zero       | THM | The finite multiplication-bad density vanishes, by the finite-union bad-density lemma.                           |
| finiteSeparationBad_density_tendsto_zero           | THM | The finite separation-bad density vanishes, again by the finite-union lemma.                                     |
| finiteRootBad_density_tendsto_zero                 | THM | The combined bad-set density vanishes, by squeezing against the sum of the two densities.                        |
| finiteRootBad_multiplicative                       | THM | Off the combined bad set, the model action is multiplicative for every pair from $F$ .                           |
| finiteRootBad_separated                            | THM | Off the combined bad set, distinct elements of $F$ act differently at the point.                                 |
| finiteRootBad_injective_word_evaluation            | THM | Off the combined bad set, evaluation of the action at the point is injective on $F$ .                            |
| normalizedHamming_mul_le                           | THM | Normalized Hamming distance is submultiplicative: the distance of products is bounded by the sum of distances.   |
| action_list_prod_tendsto                           | THM | Model actions of list products converge to products of model actions in normalized Hamming distance.             |
| chosenWordEvaluation                               | DEF | Evaluates $g$ in the model by multiplying generator actions along the chosen word for $g$ .                      |
| chosenWordEvaluation_tendsto_action                | THM | Chosen-word evaluations converge in normalized Hamming distance to the direct model action.                      |
| chosenWordMultiplicationBad                        | DEF | The set of points where the chosen-word evaluation fails multiplicativity at a fixed pair.                       |
| mem_chosenWordMultiplicationBad                    | THM | Membership criterion for the chosen-word multiplication-bad set.                                                 |
| chosenWordMultiplicationBad_density_tendsto_zero   | THM | Densities of chosen-word multiplication failures vanish, translating the Hamming convergence into counting form. |
| chosenFiniteMultiplicationBad                      | DEF | Union of the chosen-word multiplication-bad sets over all pairs from $F$ .                                       |
| chosenFiniteMultiplicationBad_density_tendsto_zero | THM | Its density tends to zero, by the finite-union bad-density lemma.                                                |
| chosenFiniteMultiplicationBad_rooted               | THM | Off this bad set, the chosen-word evaluation is multiplicative for every pair from $F$ .                         |
| chosenFiniteRootBad                                | DEF | The combined bad set joining model root failures and chosen-word multiplication failures.                        |
| chosenFiniteRootBad_density_tendsto_zero           | THM | Its density tends to zero, combining the two vanishing density statements.                                       |
| chosenFiniteRootBad_injective_word_evaluation      | THM | Off the combined set, point evaluation of the model action is injective on $F$ .                                 |
| exists_generator_word_of_symmetric_generates       | THM | Every element of a group generated by a symmetric finite set is a product of a list of its letters.              |
| symmetricGeneratorWord_prod                        | THM | The chosen generator word multiplies back to the element it was chosen for.                                      |
| symmetricGeneratorWord_one                         | THM | The chosen word of the identity element is the empty list.                                                       |
| symmetricGeneratorWord_generator                   | THM | The chosen word of a nonidentity generator is the corresponding single-letter word.                              |
| chosen_symmetric_wordEvaluation_one                | THM | With symmetric generator words, the chosen evaluation of the identity is the identity permutation.               |
| chosen_symmetric_wordEvaluation_generator          | THM | The chosen evaluation of each generator equals the model action of that generator.                               |
| generator_word_prod_mem_pow                        | THM | A product of $n$ letters from $S$ lies in the pointwise power $S^n$ .                                            |

|                                                         |     |                                                                                     |
|---------------------------------------------------------|-----|-------------------------------------------------------------------------------------|
| <code>mem_generator_pow_of_chosen_word_length</code>    | THM | If the chosen word of $g$ has length at most $r$ and $1 \in S$ , then $g \in S^r$ . |
| <code>chosenCayleyRadiusBad</code>                      | DEF | The chosen-root bad set specialized to the Cayley ball $S^r$ as the window.         |
| <code>chosenCayleyRadiusBad_density_tendsto_zero</code> | THM | Its density tends to zero, directly from the finite chosen-root bad estimate.       |
| <code>chosenCayleyRadiusBad_injective_ball</code>       | THM | Off the bad set, distinct elements of the ball $S^r$ act differently at the point.  |

## 6.5 Indicator displacement energy and Jensen bound (KunDirectedIndicatorJensen)

Sets up the real-valued machinery relating the permutation Markov averaging operator to edge boundaries of finite sets. `realIndicator` is the 0/1 indicator of a finset and `realPermutationMarkov` averages a function over the inverses of a finite permutation family. Exact identities evaluate the total squared displacement of an indicator along a permutation, and along its inverse, as twice the number of exiting points, hence, summed over generators, as twice `SoficGroups.boundary`. A Cauchy-Schwarz step bounds the squared defect of the average by the average of squared displacements. The punchline `realPermutationMarkov_indicator_defect_sq_le_boundary` bounds the Dirichlet energy of the Markov defect of an indicator by  $2\partial T/|\iota|$ ; this combinatorial input is later transported to the complex Hilbert space and played against the Kazhdan spectral gap.

**`realPermutationMarkov_indicator_defect_sq_le_boundary`** *thm*

```

theorem realPermutationMarkov_indicator_defect_sq_le_boundary
  {V ι : Type*} [Fintype V] [DecidableEq V]
  [Fintype ι] [Nonempty ι]
  (σ : ι → Equiv.Perm V) (T : Finset V) :
  (∑ x : V,
    (realPermutationMarkov σ (realIndicator T) x -
     realIndicator T x) ^ 2) ≤
    2 * (SoficGroups.boundary σ T : ℝ) /
    (Fintype.card ι : ℝ) := by
  -- proof: 21 lines, omitted

```

The theorem states that for a family  $\sigma$  of permutations of a finite set  $V$ , indexed by a nonempty finite type  $\iota$ , and any  $T \subseteq V$ , the  $\ell^2$  energy of the Markov defect of the indicator,  $\sum_x (M1_T(x) - 1_T(x))^2$ , is at most  $2 \cdot \partial_\sigma T / |\iota|$ , where `SoficGroups.boundary` counts, summed over generators, the points of  $T$  mapped outside  $T$ . The proof is pointwise Jensen followed by an exact evaluation: `realPermutationMarkov_sub_sq_le_average_sq` (via `sq_sum_le_card_mul_sum_sq`) bounds the averaged defect squared at each  $x$  by the average of the squared single-generator displacements; summing over  $x$  and swapping the two sums reduces to `sum_sq_realIndicator_inverse_displacement`, which evaluates the total squared inverse displacement as exactly twice the boundary, using the coarea count `card_entering_eq_card_exiting` to identify entering and exiting points. This is the combinatorial half of the eventual spectral-gap contradiction: later, through `KunRealComplexMarkovBridge` and `rooted_indicator_defect_sq_le_boundary`, the inequality is transported verbatim to the complex Hilbert space, where property (T) forces the Markov operator to contract sharply, so sets of small boundary yield almost-invariant vectors.

*Remaining declarations in this section:*

|                                                |     |                                                                                                                         |
|------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------|
| <code>realIndicator</code>                     | DEF | Real-valued 0/1 indicator function of a finite set $T$ .                                                                |
| <code>realPermutationMarkov</code>             | DEF | Averages $f$ over inverse images under the permutations $\sigma_i$ : the degree-normalized permutation Markov operator. |
| <code>sq_realIndicator_displacement</code>     | THM | The squared indicator displacement along $p$ equals the indicator of leaving $T$ plus the indicator of entering $T$ .   |
| <code>sum_sq_realIndicator_displacement</code> | THM | Summing squared displacements over all vertices gives twice the number of points of $T$ that $p$ moves out.             |

|                                                            |     |                                                                                                                             |
|------------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------|
| <code>sum_sq_realIndicator_inverse_displacement</code>     | THM | The same identity for displacement along $p^{-1}$ , obtained by reindexing the vertex sum by $p$ .                          |
| <code>sum_sum_sq_realIndicator_inverse_displacement</code> | THM | Summing over all generators: the total squared inverse displacement equals twice <code>SoficGroups.boundary</code> of $T$ . |
| <code>realPermutationMarkov_sub_sq_le_average_sq</code>    | THM | Cauchy-Schwarz step: the squared defect of the Markov average at a point is at most the average squared displacement.       |

## 6.6 Markov averaging preserves mass and range (KunFinitePermutationMarkovMass)

Re-develops the permutation Markov operator, normalized as inverse cardinality times a sum, and proves its conservation laws. The operator preserves pointwise nonnegativity, the pointwise bound one, and total sums; by induction all three properties persist under every iterate. Applied to the indicator of a finset  $T$ , each iterate keeps total mass  $|T|$  and takes values in the unit interval. These facts complement the energy bound of the previous segment: iterated smoothing spreads the indicator's mass without creating or destroying it, so when the boundary of  $T$  is small the smoothed function stays close to the original indicator while conserving mass. The rooted word-power arguments at the end of this chunk consume these statements through the real/complex Markov bridge.

### `sum_realPermutationMarkov` thm

```
theorem sum_realPermutationMarkov {V ι : Type*}
  [Fintype V] [Fintype ι] [Nonempty ι]
  (σ : ι → Equiv.Perm V) (f : V → ℝ) :
  (∑ x : V, realPermutationMarkov σ f x) = ∑ x : V, f x := by
-- proof: 18 lines, omitted
```

`sum_realPermutationMarkov` states that for any real function  $f$  on a finite vertex set, the Markov average `realPermutationMarkov` has the same total sum as  $f$ : the operator is stochastic in both directions, since each row and column of its matrix is a uniform average of permutation matrices. The proof swaps the sum over vertices with the sum over generators, then for each generator  $i$  reindexes the vertex sum along the bijection  $(\sigma_i)^{-1}$  using `Equiv.sum_comp`, so every inner sum equals  $\sum_x f(x)$ ; the  $|\iota|$  identical copies cancel against the normalization  $|\iota|^{-1}$ . Combined with `realPermutationMarkov_nonneg` and `realPermutationMarkov_le_one`, and their inductive iterate versions, this yields the two facts the later argument needs: `sum_realPermutationMarkov_iterate_indicator`, saying every smoothing iterate of  $1_T$  still carries mass exactly  $|T|$ , and `realPermutationMarkov_iterate_indicator_mem_unitInterval`, saying its values stay in  $[0, 1]$ . In the rooted word-power analysis these conservation laws mean the iterated smoothing of an indicator is a bounded mass distribution of fixed total weight, whose displacement is controlled by the boundary; the tension between conserved localized mass and forced spreading is what the expander estimates exploit.

*Remaining declarations in this section:*

|                                                   |     |                                                                                                        |
|---------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------|
| <code>realIndicator</code>                        | DEF | Duplicate real 0/1 indicator definition, local to this namespace.                                      |
| <code>realPermutationMarkov</code>                | DEF | The permutation Markov operator, written as inverse cardinality times the sum over generator inverses. |
| <code>realPermutationMarkov_nonneg</code>         | THM | The Markov operator preserves pointwise nonnegativity of functions.                                    |
| <code>realPermutationMarkov_le_one</code>         | THM | The Markov operator preserves the pointwise upper bound one.                                           |
| <code>realPermutationMarkov_iterate_nonneg</code> | THM | Iterates of the Markov operator preserve nonnegativity, by induction on the iterate count.             |
| <code>realPermutationMarkov_iterate_le_one</code> | THM | Iterates of the Markov operator preserve the pointwise bound one, by induction.                        |
| <code>sum_realPermutationMarkov_iterate</code>    | THM | Iterates preserve the total sum, by induction from <code>sum_realPermutationMarkov</code> .            |

|                                                                       |     |                                                                         |
|-----------------------------------------------------------------------|-----|-------------------------------------------------------------------------|
| <code>sum_realIndicator</code>                                        | THM | The indicator of $T$ sums to $ T $ over the vertex set.                 |
| <code>sum_realPermutationMarkov_iterate_indicator</code>              | THM | Every Markov iterate of an indicator still has total mass $ T $ .       |
| <code>realPermutationMarkov_iterate_indicator_mem_unitInterval</code> | THM | Every Markov iterate of an indicator takes values in the unit interval. |

## 6.7 Partitioning residual graphs into expanders (KunResidualExpanderDecomposition)

The combinatorial core of the chunk: a recursive expander decomposition for permutation-family graphs. After boundary bookkeeping (indicator formulas, complement invariance, the degree bound, and a submodularity-style inequality), it shows that a minimum-cardinality sparse cut, once replaced by a nearby low-boundary improvement set  $U$ , isolates a part  $R \cap U$  that  $\gamma$ -expands on half-size subsets. Strong induction yields `exists_expanding_residual_finpartition`: any residual set  $R$  partitions into  $\gamma$ -half-expanding parts with total boundary at most  $\partial R + 4\alpha|R|$ . Adjoining singletons of the bad set  $B$  produces full-vertex partitions with budget  $2|\iota||B| + 4\alpha|V|$ , and a sequence version drives the normalized budget to zero. Finally, completing a part's internal dynamics into honest permutations of the part costs only the part's own boundary, giving additive expansion for completed components.

### `exists_expanding_residual_finpartition` thm

```

theorem exists_expanding_residual_finpartition
  {V ι : Type*} [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (B : Finset V) (γ α : ℝ)
  (hα : 0 ≤ α)
  (hImprove : ∀ T : Finset V, T ⊆ Finset.univ \ B →
    (SoficGroups.boundary σ T : ℝ) < γ * (T.card : ℝ) →
    ∃ U : Finset V, 3 * (U ∆ T).card < T.card ∧
      (SoficGroups.boundary σ U : ℝ) ≤ α * (U.card : ℝ))
  (R : Finset V) (hRB : R ⊆ Finset.univ \ B) :
  ∃ P : Finpartition R,
    (∀ C ∈ P.parts, ∀ E : Finset V, E ⊆ C →
      2 * E.card ≤ C.card →
        γ * (E.card : ℝ) ≤ (SoficGroups.boundary σ E : ℝ)) ∧
    (∑ C ∈ P.parts, (SoficGroups.boundary σ C : ℝ)) ≤
      (SoficGroups.boundary σ R : ℝ) +
        4 * α * (R.card : ℝ) := by
-- proof: 124 lines, omitted

```

Fix generators  $\sigma$ , a bad set  $B$ , thresholds  $\gamma$  and  $\alpha \geq 0$ , and the improvement hypothesis: every sparse cut  $T$  inside  $\text{univ} \setminus B$  (meaning  $\partial T < \gamma|T|$ ) admits a nearby set  $U$  with  $3|U \Delta T| < |T|$  and  $\partial U \leq \alpha|U|$ . The theorem produces, for any residual  $R \subseteq \text{univ} \setminus B$ , a finpartition of  $R$  whose parts  $\gamma$ -expand on subsets of at most half the part, with total part boundary at most  $\partial R + 4\alpha|R|$ . The proof is strong induction on  $R$  via `Finset.strongInductionOn`: if no sparse cut exists, the indiscrete partition already expands; otherwise take a minimum-cardinality sparse cut  $T$  (`exists_minimum_sparse_residual_cut`), improve it to  $U$ , and set  $C = R \cap U$ . By `close_residual_inter_properties`  $C$  is nonempty with  $|U| \leq 2|C|$ , and by `minimum_sparse_cut_expands_close_residual_part` minimality forces  $C$  to expand. Recurse on  $R \setminus C$  and extend the resulting partition by  $C$  using `Finpartition.extend`. The budget uses `boundary_inter_add_boundary_sdiff_le`:  $\partial C + \partial(R \setminus C) \leq \partial R + 2\partial U \leq \partial R + 4\alpha|C|$ , so the  $\alpha$ -charges accumulate additively against  $|R|$ . This engine converts the almost-centralizer improvement property of hypothetical sofic approximations into an expander decomposition.

### `exists_expanding_full_finpartition_sequence` thm

```

theorem exists_expanding_full_finpartition_sequence
  (ι : Type*) [Fintype ι]
  (V : ℕ → Type*)
  [∀ n, Fintype (V n)]
  [∀ n, Nonempty (V n)]
  [∀ n, DecidableEq (V n)]
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))

```

```

(B : (n : ℕ) → Finset (V n))
(γ : ℝ) (hγ : 0 < γ)
(α : ℕ → ℝ)
(hα : ∀ n, 0 ≤ α n)
(hazero : Tendsto α atTop (nhds 0))
(hbad : Tendsto
  (fun n => ((B n).card : ℝ) / Fintype.card (V n))
  atTop (nhds 0))
(himprove : ∀ n (T : Finset (V n)),
  T ⊆ Finset.univ \ B n →
  (SoficGroups.boundary (σ n) T : ℝ) <
  γ * (T.card : ℝ) →
  ∃ U : Finset (V n), 3 * (U ∆ T).card < T.card ∧
  (SoficGroups.boundary (σ n) U : ℝ) ≤
  α n * (U.card : ℝ)) :
  ∃ P : (n : ℕ) → Finpartition (Finset.univ : Finset (V n)),
  0 < γ ∧
  (∀ n, ∀ C ∈ (P n).parts, ∀ E : Finset (V n), E ⊆ C →
    2 * E.card ≤ C.card →
    γ * (E.card : ℝ) ≤
    (SoficGroups.boundary (σ n) E : ℝ)) ∧
  Tendsto
  (fun n =>
    (∑ C ∈ (P n).parts,
      (SoficGroups.boundary (σ n) C : ℝ)) /
      Fintype.card (V n))
  atTop (nhds 0) := by
-- proof: 57 lines, omitted

```

`exists_expanding_full_finpartition_sequence` is the asymptotic form actually consumed downstream. Given a sequence of finite vertex types  $V_n$  with permutation families  $\sigma_n$ , bad sets  $B_n$  whose densities  $|B_n|/|V_n|$  tend to zero, improvement tolerances  $\alpha_n \rightarrow 0$ , and the improvement hypothesis at every  $n$ , it produces a choice of full-vertex finpartitions  $P_n$  such that every part  $\gamma$ -expands on half-size subsets and the normalized total boundary  $\sum_{C \in P_n} \partial C / |V_n|$  converges to zero. The construction simply invokes `exists_expanding_full_finpartition` at each  $n$  and extracts the witnesses with `choose`; that theorem supplies the budget  $2|\iota|(|B_n|/|V_n| + 4\alpha_n)$ . Dividing by  $|V_n|$  bounds the normalized sum by  $2|\iota| \cdot (|B_n|/|V_n|) + 4\alpha_n$ , which tends to zero by the two density hypotheses; `squeeze_zero` concludes, the lower bound being nonnegativity of the boundary counts. The significance is that a sofic approximation sequence enjoying the almost-centralizer improvement property yields, for each  $n$ , a partition into genuine  $\gamma$ -expanders whose total cut is asymptotically negligible: this is the geometric structure against which the Kazhdan spectral gap and the transported Markov estimates are later played.

#### **completed\_component\_additive\_expansion** *thm*

```

theorem completed_component_additive_expansion
  {V ι : Type*} [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (C : Finset V)
  (τ : ι → Equiv.Perm {x : V // x ∈ C})
  (hτ : ∀ i (x : V) (hx : x ∈ C) (_hy : σ i x ∈ C),
    ((τ i (x, hx) : {x : V // x ∈ C}) : V) = σ i x)
  (γ : ℝ)
  (hexpand : ∀ E : Finset V, E ⊆ C →
    2 * E.card ≤ C.card →
    γ * (E.card : ℝ) ≤ (SoficGroups.boundary σ E : ℝ)) :
  ∀ A : Finset {x : V // x ∈ C},
  γ * min (A.card : ℝ) ((C.card : ℝ) - A.card) -
  (SoficGroups.boundary σ C : ℝ) ≤
  (SoficGroups.boundary τ A : ℝ) := by
-- proof: 65 lines, omitted

```

`completed_component_additive_expansion` transfers expansion from a part of the ambient graph to a self-contained finite dynamical system on that part. Let  $C$  be a part, and  $\tau$  a family of permutations of the subtype of  $C$  that agree with  $\sigma$  at every point whose image stays inside  $C$ ; suppose  $C$   $\gamma$ -expands under  $\sigma$  in the half-size form. The conclusion is that for every subset  $A$  of the subtype,  $\gamma \cdot \min(|A|, |C| - |A|) - \partial_\sigma C \leq \partial_\tau A$ .

For  $A$  at most half of  $C$ , embed  $A$  as  $E \subseteq C$ ; `original_boundary_le_completed_add_component_boundary` gives  $\partial_\sigma E \leq \partial_\tau A + \partial_\sigma C$ , since every  $\sigma$ -exit from  $E$  either has a  $\tau$ -counterpart leaving  $A$  or leaves  $C$  altogether; the expansion hypothesis  $\gamma|E| \leq \partial_\sigma E$  then rearranges to the claim. For large  $A$ , pass to the complement inside the subtype and use `boundary_complement` to equate the two boundaries. The point is that completing a component's partial dynamics into honest permutations costs only the component's own boundary, uniformly over test sets: parts with cheap cuts become genuine additive expanders, which is precisely the hypothesis format of `exists_completed_pruned_expander` in the next segments.

*Remaining declarations in this section:*

|                                                                    |     |                                                                                                                     |
|--------------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------|
| <code>boundary_eq_sum_indicator</code>                             | THM | Rewrites <code>SoficGroups.boundary</code> as a double sum of exit indicators over generators and vertices.         |
| <code>boundary_eq_sum_entering_indicator</code>                    | THM | Rewrites the boundary as a sum of entering indicators, via the coarea bijection.                                    |
| <code>boundary_complement</code>                                   | THM | The boundary of the complement of $A$ equals the boundary of $A$ .                                                  |
| <code>boundary_le_degree_mul_card</code>                           | THM | Trivial degree bound: the boundary of $A$ is at most $ \iota  \cdot  A $ .                                          |
| <code>boundary_inter_add_boundary_sdiff_le</code>                  | THM | Submodularity-style estimate: cutting $R$ along $U$ costs at most twice the boundary of $U$ extra.                  |
| <code>exists_minimum_sparse_residual_cut</code>                    | THM | Among the sparse cuts contained in a residual set there is one of minimum cardinality.                              |
| <code>close_residual_inter_properties</code>                       | THM | If $3 U \triangle T  <  T $ , then $R \cap U$ is nonempty, at least half of $U$ , and smaller than $2 T $ .         |
| <code>minimum_sparse_cut_expands_close_residual_part</code>        | THM | Minimality of the sparse cut forces the piece $R \cap U$ to $\gamma$ -expand on half-size subsets.                  |
| <code>exists_expanding_clean_finpartition</code>                   | THM | Applies the decomposition to $\text{univ} \setminus B$ , bounding the budget via the degree bound on $\partial B$ . |
| <code>completeCleanFinpartition</code>                             | DEF | Extends a partition of $\text{univ} \setminus B$ to the whole vertex set by adjoining singleton parts for $B$ .     |
| <code>completeCleanFinpartition_parts</code>                       | THM | The parts of the completed partition are exactly the clean parts together with the singleton bad parts.             |
| <code>disjoint_clean_parts_singleton_bad_parts</code>              | THM | The clean parts and the singleton bad parts form disjoint families.                                                 |
| <code>sum_singleton_bad_boundary_le</code>                         | THM | The singleton bad parts contribute at most $ \iota  \cdot  B $ total boundary.                                      |
| <code>completeCleanFinpartition_half_expansion</code>              | THM | The completed partition still half-expands: singleton parts only admit the empty test set.                          |
| <code>exists_expanding_full_finpartition</code>                    | THM | Full-vertex version: an expanding partition of all of $V$ with budget $2 \iota  B  + 4\alpha V $ .                  |
| <code>original_boundary_le_completed_add_component_boundary</code> | THM | The boundary of a subset embedded from a component is at most its internal boundary plus the component boundary.    |

## 6.8 Good permutation graph after pruning bad points (KunDiagonalGoodRootGraphLoss)

Studies the graph of an auxiliary permutation  $p$ , a subset of  $V \times V$  acted on diagonally by the pairs  $(\sigma_i, \sigma_i)$ . The good graph discards pairs meeting a bad set  $B$  in either coordinate. Bijective counting shows exactly  $|B|$  pairs are lost through each coordinate, so at most  $2|B|$  in total, and the good graph keeps at least  $|V| - 2|B|$  of the  $|V|$  pairs. The main estimate bounds the diagonal boundary of the good graph by `SoficGroups.permutationCommutationDefect` plus  $2|\iota||B|$ : the boundary of the full graph equals the commutation defect exactly, and pruning contributes only degree times the discarded pairs. Thus an almost-commuting  $p$  supplies a large subset of the pair space with a sparse diagonal cut, a test set to be played against pair expanders.

### boundary\_goodPermutationGraph\_le\_commutationDefect\_add thm

```

theorem boundary_goodPermutationGraph_le_commutationDefect_add {V ι : Type*}
  [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (p : Equiv.Perm V) (B : Finset V) :
  SoficGroups.boundary
    (fun i => (σ i).prodCongr (σ i)) (goodPermutationGraph p B) ≤
    SoficGroups.permutationCommutationDefect σ p +
    2 * Fintype.card ι * B.card := by
-- proof: 23 lines, omitted

```

`boundary_goodPermutationGraph_le_commutationDefect_add` considers the diagonal action of the generator family on pairs,  $i \mapsto (\sigma_i, \sigma_i)$  via `Equiv.Perm.prodCongr`, and the good graph of  $p$  relative to a bad set  $B$ . It bounds the boundary of the good graph under this action by `SoficGroups.permutationCommutationDefect` of  $\sigma$  and  $p$ , plus  $2|\iota||B|$ . The proof combines two earlier facts: `boundary_le_boundary_add_sdiff` from the `KunThomFiberCoarea` namespace, which says the boundary of a subset is at most the boundary of a superset plus degree times the cardinality of the difference, and `boundary_permutationGraph_eq_commutationDefect`, which identifies the diagonal boundary of the full permutation graph exactly with the commutation defect, i.e. the total count of points where  $p$  fails to commute with some  $\sigma_i$ . The discarded difference has at most  $2|B|$  elements by `card_permutationGraph_sdiff_goodPermutationGraph_le`. The significance: the graph of  $p$  serves as a test set inside the pair space; after pruning the bad set the test set is still large (`card_goodPermutationGraph_ge_card_sub_twice_bad`) yet its diagonal cut is controlled by how well  $p$  almost-commutes with the generators. This is exactly how an almost-central element manufactures a sparse cut that the expander decomposition of the diagonal action must forbid.

*Remaining declarations in this section:*

|                                                                                   |     |                                                                                                                         |
|-----------------------------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------|
| <code>goodPermutationGraph</code>                                                 | DEF | The subgraph of <code>SoficGroups.permutationGraph</code> of $p$ whose pairs avoid the bad set $B$ in both coordinates. |
| <code>goodPermutationGraph_subset</code>                                          | THM | The good graph is contained in the full permutation graph.                                                              |
| <code>card_filter_permutationGraph_fst_mem</code>                                 | THM | Exactly $ B $ graph pairs have first coordinate in $B$ , by the bijection sending a pair to its source.                 |
| <code>card_filter_permutationGraph_snd_mem</code>                                 | THM | Exactly $ B $ graph pairs have second coordinate in $B$ , by the bijection onto targets.                                |
| <code>permutationGraph_sdiff_good</code><br><code>PermutationGraph_subset</code>  | THM | Every discarded pair lies in one of the two coordinate-in- $B$ filtered subgraphs.                                      |
| <code>card_permutationGraph_sdiff_good</code><br><code>PermutationGraph_le</code> | THM | At most $2 B $ pairs of the permutation graph are discarded when forming the good graph.                                |
| <code>card_goodPermutationGraph_add_graph_loss</code>                             | THM | Discarded plus good pairs total $ V $ , the cardinality of the full permutation graph.                                  |
| <code>card_goodPermutationGraph_ge_card_sub_twice_bad</code>                      | THM | The good graph retains at least $ V  - 2 B $ pairs, in real form.                                                       |

## 6.9 Pruned components completed into honest expanders (KunCompletedPrunedComponent)

Converts expansion with an additive error  $a|V|$  into an honest expander on a pruned vertex set. The segment first notes that the tolerance-zero almost-centralizer improvement property holds trivially, re-proves complement invariance of the boundary, and shows two structural lemmas: half-size expansion upgrades to the  $\min(|A|, |V| - |A|)$  form by passing to complements, and the boundary of completed internal permutations dominates the induced boundary of embedded subsets. The main theorem `exists_completed_pruned_expander` then produces a bad set  $B$ , of size at most  $a|V|/(\gamma - \ell)$  and at most half the space, together with completions  $\tau$  of the generators on the survivors satisfying the min-form  $\ell$ -expansion. This is the finite expander stage on which the completed almost-commuting family is subsequently placed.



**exists\_completed\_pruned\_expander** thm

```

theorem exists_completed_pruned_expander
  {V ι : Type*} [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (γ ell a : ℝ)
  (hgap : ell < γ)
  (hadd : ∀ A : Finset V,
    γ * min (A.card : ℝ)
      ((Fintype.card V : ℝ) - (A.card : ℝ)) -
    a * (Fintype.card V : ℝ) ≤
    (SoficGroups.boundary σ A : ℝ))
  (hsmall :
    2 * a * (2 * (γ - ell) + (Fintype.card ι : ℝ)) ≤
    (γ - ell) ^ 2) :
  ∃ (B : Finset V)
    (τ : ι → Equiv.Perm
      {x : V // x ∈ (Finset.univ \ B)}),
  2 * B.card ≤ Fintype.card V ∧
  (B.card : ℝ) ≤
    a * (Fintype.card V : ℝ) / (γ - ell) ∧
  (∀ i (x : V) (hx : x ∈ Finset.univ \ B)
    (_hy : σ i x ∈ Finset.univ \ B),
    ((τ i (x, hx) :
      {x : V // x ∈ Finset.univ \ B}) : V) = σ i x) ∧
  ∀ A : Finset {x : V // x ∈ Finset.univ \ B},
    ell * min (A.card : ℝ)
      ((Fintype.card {x : V // x ∈ Finset.univ \ B} : ℝ) -
        A.card) ≤ (SoficGroups.boundary τ A : ℝ) := by
-- proof: 51 lines, omitted

```

`exists_completed_pruned_expander` is the segment's goal. Hypotheses: a spectral-type additive expansion bound  $\gamma \cdot \min(|A|, |V| - |A|) - a|V| \leq \partial A$  for all  $A$ , a gap  $\ell < \gamma$ , and a smallness condition  $2a(2(\gamma - \ell) + |\iota|) \leq (\gamma - \ell)^2$ . Conclusion: there is a bad set  $B$  with  $2|B| \leq |V|$  and  $|B| \leq a|V|/(\gamma - \ell)$ , and permutations  $\tau_i$  of the subtype  $\text{univ} \setminus B$  agreeing with  $\sigma_i$  at points mapped internally, such that every subset  $A$  of the subtype satisfies the honest min-form expansion  $\ell \cdot \min(|A|, |Z| - |A|) \leq \partial_\tau A$ . The proof calls `SoficGroups.exists_pruned_expander` to obtain  $B$  with an induced-boundary expansion on survivors, then completes each  $\sigma_i$  to a permutation of the subtype via `SoficGroups.exists_completion_of_internal_permutation`. `inducedBoundary_le_completed_boundary` shows the completed boundary dominates the induced boundary of the embedded subset, so the induced expansion transfers; `boundary_expansion_of_half` then upgrades half-size expansion to min form, using the cardinality computation  $|Z| = |V| - |B|$ . This theorem is the pivot from the decomposition machinery to genuine finite expanders: it manufactures the closed dynamical system on which the completed restrictions of the almost-commuting family will be analyzed.

*Remaining declarations in this section:*

|                                                    |     |                                                                                                                 |
|----------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------|
| <code>hasAlmostCentralizerImprovement_zero</code>  | THM | The almost-centralizer improvement property at tolerance zero holds trivially, witnessed by the element itself. |
| <code>boundary_complement</code>                   | THM | Complementary sets have equal boundary, proved again locally via the entering-exiting coarea count.             |
| <code>boundary_expansion_of_half</code>            | THM | Expansion on half-size sets upgrades to the $\min( A ,  V  -  A )$ form by passing to complements.              |
| <code>inducedBoundary_le_completed_boundary</code> | THM | The induced boundary of an embedded subset is at most its boundary under the completed permutations.            |

## 6.10 Completing restrictions with bad-set control (MatchedComponentCompletion)

Builds the completed restriction of an almost-action to a survivor set  $Z$  and quantifies its defects. `completedRestriction` chooses, via the completion existence lemma, a permutation of the subtype of  $Z$  agreeing with  $p$  wherever  $p$  stays inside  $Z$ . The finset `sourceCompletionBad` collects every failure point: generator exits from  $Z$ , exits under tracked words, commutation failures with the generators, composite exits, multiplicativity failures

over the window  $F$ , and collisions of distinct window elements. Off this one set the completed restrictions are multiplicative, pairwise distinct at every point, and commute with the completed generators. Consequently the Hamming distance from the restriction of a product to the product of restrictions, the separation of distinct restrictions, and the commutation defect against the completed generators are all controlled by the cardinality of the bad set.

**sourceCompletionBad** *def*

```

noncomputable def sourceCompletionBad
  {V ι J : Type*} [DecidableEq V]
  [Fintype ι] [Group J]
  (σ : ι → Equiv.Perm V) (p : J → Equiv.Perm V)
  (F : Finset J) (Z : Finset V) : Finset V := by
classical
let T := SoficGroups.CompressionCriterion.productTrackedTable F
exact
(Finset.univ.biUnion fun i : ι =>
  Z.filter fun z => σ i z ∉ Z) ∪
(T.biUnion fun j =>
  Z.filter fun z => p j z ∉ Z) ∪
(Finset.univ.biUnion fun i : ι =>
  T.biUnion fun j =>
    Z.filter fun z => p j (σ i z) ≠ σ i (p j z)) ∪
(Finset.univ.biUnion fun i : ι =>
  T.biUnion fun j =>
    Z.filter fun z => p j (σ i z) ∉ Z) ∪
(F.biUnion fun j =>
  F.biUnion fun k =>
    Z.filter fun z => p (j * k) z ≠ p j (p k z)) ∪
(F.biUnion fun j =>
  F.biUnion fun k =>
    Z.filter fun z => j ≠ k ∧ p j z = p k z)

```

`sourceCompletionBad` is the bookkeeping device of the whole matched-completion argument. Given the generator family  $\sigma$ , an almost-action  $p$  of a group  $J$  by permutations of  $V$ , a finite window  $F \subseteq J$ , and a survivor set  $Z$ , it is defined as the union of six filtered subsets of  $Z$ : points that some generator  $\sigma_i$  throws out of  $Z$ ; points thrown out by  $p_j$  for  $j$  in `SoficGroups.CompressionCriterion.productTrackedTable` of  $F$  (a finite table of tracked products of window elements); points where  $p_j$  and  $\sigma_i$  fail to commute; points where the composite  $p_j(\sigma_i z)$  escapes  $Z$ ; points witnessing a multiplicativity failure  $p_{jk}z \neq p_j(p_k z)$  for  $j, k \in F$ ; and points where two distinct window elements collide,  $p_j z = p_k z$ . Its role, established by `sourceCompletionBad_good`, is that outside this single set every algebraic property needed of the completed restrictions holds simultaneously, so the subsequent theorems 1033 to 1038 are uniform over one bad set. The companion segment `MatchedComponentExitBudget` then shows this set has vanishing density, so the completion of  $p$  to the pruned component behaves like a genuine multiplicative, separated,  $\sigma$ -commuting family except on a negligible fraction of points.

**completedRestriction\_mul\_of\_not\_mem\_sourceCompletionBad** *thm*

```

theorem completedRestriction_mul_of_not_mem_sourceCompletionBad
  {V ι J : Type*} [Fintype V] [DecidableEq V]
  [Fintype ι] [Group J]
  (σ : ι → Equiv.Perm V) (p : J → Equiv.Perm V)
  (F : Finset J) (Z : Finset V)
  {j k : J} (hj : j ∈ F) (hk : k ∈ F)
  (z : {x : V // x ∈ Z})
  (hz : (z : V) ∉ sourceCompletionBad σ p F Z) :
  completedRestriction (p (j * k)) Z z =
    (completedRestriction (p j) Z * completedRestriction (p k) Z) z := by
-- proof: 41 lines, omitted

```

`completedRestriction_mul_of_not_mem_sourceCompletionBad` states that for window elements  $j, k \in F$  and a subtype point  $z$  whose underlying element avoids `sourceCompletionBad`, the completed restriction of  $p_{jk}$  at  $z$  equals the composition of the completed restrictions of  $p_j$  and  $p_k$ . The proof extracts from `sourceCompletion`

Bad\_good that  $p_k z$  and  $p_{jk} z$  remain in  $Z$ , since  $k$  and  $jk$  lie in the tracked table by `mem_productTrackedTable` and `mul_mem_productTrackedTable`, and that the multiplicativity identity  $p_{jk} z = p_j(p_k z)$  holds at  $z$ . Each completed Restriction is then evaluated through its defining property `completedRestriction_apply_of_mem`: the left side computes  $p_{jk} z$ ; the right side first computes  $p_k z$ , and since  $p_j$  of that value stays in  $Z$  it computes  $p_j(p_k z)$ ; `Subtype.ext` closes the goal. This is the pointwise engine behind `completedRestriction_mul_distance_le_sourceCompletionBad`, which converts multiplicativity off a small set into a Hamming-distance bound between the restriction of a product and the product of restrictions: exactly the approximate-multiplicativity clause that the completed family must satisfy to serve as a sofic-style model on the pruned component.

**completedRestriction\_commutationDefect\_le\_sourceCompletionBad thm**

```
theorem completedRestriction_commutationDefect_le_sourceCompletionBad
  {V ι J : Type*} [Fintype V] [DecidableEq V]
  [Fintype ι] [Group J]
  (σ : ι → Equiv.Perm V) (p : J → Equiv.Perm V)
  (F : Finset J) (Z : Finset V)
  (σZ : ι → Equiv.Perm {x : V // x ∈ Z})
  (hσZ : ∀ i (x : V) (hx : x ∈ Z) (_hi : σ i x ∈ Z),
    ((σZ i (x, hx) : {x : V // x ∈ Z}) : V) = σ i x)
  {j : J}
  (hj : j ∈ SoficGroups.CompressionCriterion.productTrackedTable F) :
  SoficGroups.permutationCommutationDefect
    σZ (completedRestriction (p j) Z) ≤
    Fintype.card ι * (sourceCompletionBad σ p F Z).card := by
  -- proof: 20 lines, omitted
```

`completedRestriction_commutationDefect_le_sourceCompletionBad` bounds `SoficGroups.permutationCommutationDefect` of the completed generator family  $\sigma_Z$  against the completed restriction of  $p_j$ , for any  $j$  in the tracked table, by  $|\iota|$  times the cardinality of `sourceCompletionBad`. The defect is by definition a sum over generators  $i$  of the number of subtype points  $x$  where the two compositions disagree,  $\text{res}(p_j)(\sigma_Z i x) \neq \sigma_Z i(\text{res}(p_j)x)$ . The proof shows each such  $x$  must project into the bad set: by `completedRestriction_commute_of_not_mem_sourceCompletionBad`, at any good point the completed restriction commutes with each completed generator, because all four relevant images stay in  $Z$  and the underlying commutation identity holds there. Each generator's count is therefore at most the subtype-bad cardinality, which equals the bad-set cardinality by `card_subtype_sourceCompletionBad`. Together with the distance bound 1036 and the separation bound 1037, this packages the completed family as an almost-commuting, almost-multiplicative, well-separated tuple on the pruned expander. The commutation defect is precisely the quantity that `boundary_goodPermutationGraph_le_commutationDefect_add` feeds into the diagonal expander as a sparse cut, so bounding it by the vanishing bad set is what makes the eventual contradiction quantitative.

*Remaining declarations in this section:*

|                                                                 |     |                                                                                                                       |
|-----------------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------|
| <code>completedRestriction</code>                               | DEF | Chooses a permutation of the subtype of $Z$ agreeing with $p$ wherever $p$ maps into $Z$ .                            |
| <code>completedRestriction_apply_of_mem</code>                  | THM | Specification lemma: the completed restriction computes $p$ at points whose image stays inside $Z$ .                  |
| <code>subtypeBad</code>                                         | DEF | The subset of the subtype of $Z$ whose underlying elements lie in $E$ ; also, the identity completes to the identity. |
| <code>card_subtypeBad</code>                                    | THM | The bad subtype set has cardinality $ Z \cap E $ ; membership unfolds to underlying-element membership.               |
| <code>permutationDistance_le_subtypeBad</code>                  | THM | Permutations of the subtype agreeing off $E$ have Hamming distance at most the bad subtype cardinality.               |
| <code>card_lt_five_mul_permutationDistance_of_subtypeBad</code> | THM | Permutations disagreeing everywhere off a small $E$ are far apart: $ Z  < 5 \cdot \text{dist}(p, q)$ .                |
| <code>sourceCompletionBad_subset</code>                         | THM | The bad set consists of surviving points: it is contained in $Z$ .                                                    |
| <code>card_subtype_sourceCompletionBad</code>                   | THM | The subtype version of the bad set has the same cardinality as the bad set.                                           |

|                                                                 |     |                                                                                                                                     |
|-----------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------------|
| sourceCompletionBad_good                                        | THM | Off the bad set all six goodness clauses hold: closure, word closure, commutation, composite closure, multiplicativity, separation. |
| completedRestriction_ne_of_not_mem_<br>sourceCompletionBad      | THM | At good points the restrictions of distinct window elements $j \neq k$ act differently.                                             |
| completedRestriction_commute_of_not_<br>mem_sourceCompletionBad | THM | At good points the completed restriction of $p_j$ commutes with the completed generators.                                           |
| completedRestriction_mul_distance_<br>le_sourceCompletionBad    | THM | The distance from the restriction of a product to the product of restrictions is at most the bad-set size.                          |
| completedRestriction_separated_of_<br>sourceCompletionBad       | THM | If the bad set is at most a fifth of $Z$ , distinct restrictions are separated on most points.                                      |

## 6.11 Bad-set density vanishes under pruning (MatchedComponentExitBudget)

Shows the bad set of the previous segment is asymptotically negligible. Injectivity of permutations bounds every exit family: points thrown out of the survivor set  $Z$  by a generator, a tracked word, or a composite inject into the deleted set, so unions over index sets cost only multiplicative factors. The  $Z$ -independent failures (commutation, multiplicativity, and separation over the window  $F$ , measured on the whole space) are collected into `sourceWordTestBad`. Splitting `sourceCompletionBad` accordingly gives a linear budget: a constant times the deleted cardinality plus the word-test size. Hence, along a sequence where deletion densities and word-test densities vanish, the bad set's density vanishes relative to  $|V_n|$  and, via a retained-density transfer lemma, relative to the surviving set itself; moreover the surviving sets grow without bound.

**card\_sourceCompletionBad\_le\_deleted\_add\_wordBad thm**

```
theorem card_sourceCompletionBad_le_deleted_add_wordBad
  {V ι J : Type*} [Fintype V] [DecidableEq V]
  [Fintype ι] [Group J]
  (σ : ι → Equiv.Perm V) (p : J → Equiv.Perm V)
  (F : Finset J) (Z : Finset V) :
  (SoficGroups.MatchedComponentCompletion.sourceCompletionBad
   σ p F Z).card ≤
    (Fintype.card ι +
     (SoficGroups.CompressionCriterion.productTrackedTable F).card +
     Fintype.card ι *
     (SoficGroups.CompressionCriterion.productTrackedTable F).card) *
    (Finset.univ \ Z).card +
    (sourceWordTestBad σ p F).card := by
  -- proof: 53 lines, omitted
```

`card_sourceCompletionBad_le_deleted_add_wordBad` gives the master budget:  $|\text{sourceCompletionBad}| \leq (|\iota| + |T| + |\iota||T|) \cdot |\text{univ} \setminus Z| + |\text{sourceWordTestBad}|$ , where  $T$  is the tracked product table of the window  $F$ . The proof rests on the covering lemma `sourceCompletionBad_subset_exit_union_wordBad`: of the six failure families defining the bad set, three are exit families, namely generator exits, tracked-word exits, and composite exits, while the commutation, multiplicativity, and separation failures are subsumed by the  $Z$ -free set `sourceWordTestBad`, which filters over the whole vertex set. Each exit family is bounded via injectivity: a point of  $Z$  whose image leaves  $Z$  injects, through the permutation itself (`card_permutation_exit_le_deleted`, `card_composite_exit_le_deleted`), into the deleted set, so the `biUnion` bounds 1041 and 1042 multiply by the index cardinalities. A union bound over the four pieces assembles the estimate. Its importance is that it separates the two sources of badness with entirely different asymptotics: exits are proportional to how much was deleted (controlled by the pruning bound of `exists_completed_pruned_expander`), while word-test failures are exactly the defect statistics of the original almost-action on the full space, which vanish by hypothesis. This split powers both density limits 1047 and 1048.

**sourceCompletionBad\_surviving\_density\_tendsto\_zero thm**

```
theorem sourceCompletionBad_surviving_density_tendsto_zero
```

```

(V : ℕ → Type*) [∀ n, Fintype (V n)]
[∀ n, DecidableEq (V n)]
{ι J : Type*} [Fintype ι] [Group J]
(σ : (n : ℕ) → ι → Equiv.Perm (V n))
(p : (n : ℕ) → J → Equiv.Perm (V n))
(F : Finset J)
(B : (n : ℕ) → Finset (V n))
(hZ : ∀ n, (Finset.univ \ B n : Finset (V n)).Nonempty)
(hdeleted : Tendsto
  (fun n => ((B n).card : ℝ) / Fintype.card (V n))
  atTop (nhds 0))
(hword : Tendsto
  (fun n =>
    ((sourceWordTestBad (σ n) (p n) F).card : ℝ) /
    Fintype.card (V n))
  atTop (nhds 0)) :
Tendsto
  (fun n =>
    ((SoficGroups.MatchedComponentCompletion.sourceCompletionBad
      (σ n) (p n) F (Finset.univ \ B n)).card : ℝ) /
    (Finset.univ \ B n : Finset (V n)).card)
  atTop (nhds 0)) := by
-- proof: 40 lines, omitted

```

`sourceCompletionBad_surviving_density_tendsto_zero` is the renormalized limit statement the pruned-component argument needs. Along a sequence with survivor sets  $Z_n = \text{univ} \setminus B_n$  nonempty, deletion densities  $|B_n|/|V_n| \rightarrow 0$ , and word-test densities  $|\text{sourceWordTestBad}|/|V_n| \rightarrow 0$ , it concludes that the bad-set cardinality divided by  $|Z_n|$ , not merely by  $|V_n|$ , tends to zero. The proof first invokes `sourceCompletionBad_original_density_tendsto_zero`, which squeezes the global density against the linear budget of 1045; then it computes that the cover ratio  $|Z_n|/|V_n|$  tends to one, directly from the deletion density; finally the generic transfer lemma `SoficGroups.retained_bad_density_tendsto_zero` converts vanishing global density plus full asymptotic cover into vanishing relative density, after rewriting  $Z_n \cap E_n = E_n$  using `sourceCompletionBad_subset_survivors`. The point of the renormalization is that the eventual contradiction lives entirely on the pruned components: the completed restrictions form permutations of  $Z_n$ , so their defect statistics, multiplicativity distance, separation, and commutation defect, must be small compared to  $|Z_n|$ . Together with `pruned_component_card_tendsto_atTop`, which guarantees the survivors grow without bound, this shows the completed family is asymptotically a genuine sofic-style approximation on the pruned expanders.

*Remaining declarations in this section:*

|                                                                |     |                                                                                                                              |
|----------------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------|
| <code>card_permutation_exit_le_deleted</code>                  | THM | Points of $Z$ exiting under a permutation inject into the deleted set $\text{univ} \setminus Z$ .                            |
| <code>card_composite_exit_le_deleted</code>                    | THM | Points of $Z$ exiting under a composite of two permutations also inject into the deleted set.                                |
| <code>card_biUnion_permutation_exit_le</code>                  | THM | A union of exit sets over an index set $I$ has at most $ I $ times the deleted cardinality.                                  |
| <code>card_biUnion_composite_exit_le</code>                    | THM | A double union of composite exit sets is at most $ I  K $ times the deleted cardinality.                                     |
| <code>sourceWordTestBad</code>                                 | DEF | The $Z$ -independent failure set: commutation, multiplicativity, and separation failures measured over the whole vertex set. |
| <code>sourceCompletionBad_subset_exit_union_wordBad</code>     | THM | The completion bad set is covered by three exit unions plus the word-test failure set.                                       |
| <code>sourceCompletionBad_subset_survivors</code>              | THM | The completion bad set is contained in the surviving set $Z$ .                                                               |
| <code>sourceCompletionBad_original_density_tendsto_zero</code> | THM | The bad-set density relative to $ V_n $ tends to zero when deletion and word-failure densities vanish.                       |
| <code>pruned_component_card_tendsto_atTop</code>               | THM | If $ V_n  \rightarrow \infty$ and the deletion density vanishes, the surviving-set cardinalities tend to infinity.           |

## 6.12 Real Markov operators as unitary averages (KunRealComplexMarkovBridge)

Transfers the real Markov analysis into the complex Hilbert space  $\ell^2(V, \mathbb{C})$ . Each permutation induces a coordinate-relabeling unitary, `permutationUnitary`, and `permutationMarkov` is the average of these unitaries; `indicatorVector` complexifies a real function on  $V$ . The key intertwining lemma `indicatorVector_realMarkov` states that complexification commutes with Markov averaging, and by induction with all iterates. Norm computations then identify the squared Hilbert norm of Markov defects, whether one step,  $k$  steps, or between successive iterates, with the corresponding real sums of squared differences. This dictionary lets the property (T) spectral-gap estimates, which are native to averages of unitaries in a representation, be applied verbatim to the combinatorial indicator data developed in the earlier segments.

**permutationMarkov** *def*

```
def permutationMarkov {ι V : Type*} [Fintype ι] [Fintype V]
  (p : ι → Equiv.Perm V) (ξ : EuclideanSpace ℂ V) :
  EuclideanSpace ℂ V :=
  (Fintype.card ι : ℂ)-1 • ∑ i, permutationUnitary (p i) ξ
```

`permutationMarkov` is defined, for a finite family  $p$  of permutations of  $V$ , as  $(|\iota|)^{-1}$  times the sum of `permutationUnitary`  $(p_i)\xi$  over the index type, an operator on `EuclideanSpace` over  $\mathbb{C}$ , where each `permutationUnitary` is the coordinate-relabeling linear isometry `LinearIsometryEquiv.piLpCongrLeft`. Pointwise, by `permutationMarkov_apply`, it sends  $\xi$  to the average of the values  $\xi((p_i)^{-1}x)$ , matching the real operator `realMarkov` exactly. This operator is the transfer-operator avatar of a finite permutation model inside the Hilbert space where the Kazhdan machinery operates: an average of unitaries drawn from a representation with spectral gap contracts the orthogonal complement of the invariant vectors at a definite rate, quantified elsewhere by `kazhdanMarkovContractionFactor`. The surrounding lemmas 1058 to 1061 compute its action on complexified real data in purely real terms, so every spectral estimate proved for this operator translates without loss into a statement about boundaries and masses of finite sets. It is the object iterated in the rooted word-power segment, and the bridge through which property (T) is brought to bear on the combinatorial expander decomposition.

**indicatorVector\_realMarkov** *thm*

```
theorem indicatorVector_realMarkov
  {ι V : Type*} [Fintype ι] [Fintype V]
  (p : ι → Equiv.Perm V) (f : V → ℝ) :
  indicatorVector (realMarkov p f) =
  permutationMarkov p (indicatorVector f) := by
  -- proof: 8 lines, omitted
```

`indicatorVector_realMarkov` is the intertwining identity: complexifying the real Markov average of  $f$  equals applying the complex Markov operator to the complexification of  $f$ , i.e. `indicatorVector (realMarkov p f) = permutationMarkov p (indicatorVector f)`. The proof is a pointwise computation: both sides at  $x$  are the average of the values  $f((p_i)^{-1}x)$ , one computed in  $\mathbb{R}$  and then cast to  $\mathbb{C}$ , the other computed in  $\mathbb{C}$  from cast values; `norm_cast` identifies them. Induction extends it to all iterates (`indicatorVector_iterate_realMarkov`), and combined with the norm computation `norm_indicatorVector_sub_sq`, which evaluates squared Hilbert norms of differences of complexified functions as real sums of squares, it yields the full dictionary 1059 to 1061: Hilbert-space norms of Markov defects, at any iterate, equal real Dirichlet-type sums. This lemma is the bridge's whole point. The property (T) contraction estimates are proved for the complex operator, whereas the boundary and mass estimates of the first two segments concern the real operator; the intertwining makes every inequality transfer with no loss, as exercised by `rooted_indicator_defect_sq_le_boundary` at the end of the chunk.

*Remaining declarations in this section:*

|                                 |     |                                                                                                                   |
|---------------------------------|-----|-------------------------------------------------------------------------------------------------------------------|
| <code>realMarkov</code>         | DEF | The real averaging operator over generator inverses, the real shadow of the complex Markov operator.              |
| <code>permutationUnitary</code> | DEF | The linear isometry equivalence of $\ell^2(V, \mathbb{C})$ induced by relabeling coordinates along a permutation. |

|                                                                      |     |                                                                                                   |
|----------------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------|
| <code>indicatorVector</code>                                         | DEF | Embeds a real function on $V$ as a vector of the complex Euclidean space.                         |
| <code>indicatorVector_apply</code>                                   | THM | Simp lemma: the indicator vector evaluates pointwise to the complexified function value.          |
| <code>permutationMarkov_apply</code>                                 | THM | Pointwise formula: the Markov image at $x$ averages the vector over generator inverse images.     |
| <code>indicatorVector_iterate_realMarkov</code>                      | THM | The intertwining persists under all iterates, by induction.                                       |
| <code>norm_indicatorVector_sub_sq</code>                             | THM | The squared norm of a difference of indicator vectors equals the real sum of squared differences. |
| <code>norm_permutationMarkov_indicator_sub_sq</code>                 | THM | The single-step Markov defect norm squared equals the real defect energy.                         |
| <code>norm_iterate_permutationMarkov_indicator_sub_sq</code>         | THM | The iterate-versus-original defect norm squared equals the corresponding real sum of squares.     |
| <code>norm_iterate_permutationMarkov_indicator_sub_iterate_sq</code> | THM | The successive-iterate difference norm squared equals the corresponding real sum of squares.      |

### 6.13 Geometric iterate bounds via Kazhdan contraction (KunRootedWordPower)

Combines the Kazhdan contraction factor with the Markov bridge to control iterated smoothing in rooted models. After a monotonicity lemma for the rootedness radius, one radius is chosen so that all successive iterate differences up to step  $k$  contract like  $q^j + \epsilon$ , where  $q$  is `kazhdanMarkovContractionFactor`. A telescoping inequality plus an explicit geometric budget bound the total displacement, giving the main theorem: suitably rooted models satisfy  $\|M^k \xi - \xi\| \leq (2/(1-q))\|M\xi - \xi\|$ . The closing lemmas identify the bridge operators with the rooted-crossing and mass-namespace versions and import the Jensen estimate of the first segment, so the one-step defect of an indicator is bounded by  $2\partial T/|\iota|$ . Together, low-boundary sets produce vectors that remain almost Markov-invariant at every iterate.

**exists\_rooted\_word\_radius\_geometric\_markov\_displacement** thm

```

theorem exists_rooted_word_radius_geometric_markov_displacement
  {G : Type u} [Group G]
  (P : SoficGroups.KazhdanPair.{u, u} G)
  (S : Finset G) (honeS : 1 ∈ S)
  (hcover : P.generators ⊆ S)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (w : G → List ιS)
  (k : ℕ) :
  ∃ r : ℕ, ∀ X : RootedIndicatorMarkovModel G ιS,
    X.IsGenerated (fun i : ιS => (i : G)) w →
    X.IsRootedAtRadius w r →
    ‖((permutationMarkov X.generator)^[k])
      (indicatorVector X.indicator) -
      indicatorVector X.indicator‖ ≤
    (2 / (1 - kazhdanMarkovContractionFactor P S)) *
    ‖permutationMarkov X.generator
      (indicatorVector X.indicator) -
      indicatorVector X.indicator‖ := by
-- proof: 52 lines, omitted

```

`exists_rooted_word_radius_geometric_markov_displacement` is the main theorem of the segment. Fix a Kazhdan pair  $P$  for  $G$ , a finite symmetric window  $S$  containing the identity and the Kazhdan generators, a word assignment  $w$ , and an iterate count  $k$ . Then there is a radius  $r$  such that every `RootedIndicatorMarkovModel` that is generated by  $S$  along  $w$  and rooted at radius  $r$  satisfies  $\|M^{[k]}\xi - \xi\| \leq (2/(1-q))\|M\xi - \xi\|$ , where  $M$  is the model's permutation Markov operator,  $\xi$  its indicator vector, and  $q = \text{kazhdanMarkovContractionFactor}(P, S) < 1$ . The proof sets  $\epsilon = ((k+1)(1-q))^{-1}$  and invokes `exists_rooted_word_radius_all_markov_iterate_contractions`, which sums per-step radii from `exists_rooted_word_radius_markov_iterate_contraction` to make every successive difference at most  $(q^j + \epsilon)\|M\xi - \xi\|$ ; telescoping (`norm_iterate_sub_le_sum_successive`)



and the budget computation (`sum_geometric_budget_le`) then sum the series to  $2/(1-q)$ . The significance: the local spectral gap of property (T) becomes a radius-uniform statement about finite models, namely that iterated smoothing never moves an indicator much farther than a single smoothing step. Downstream, this pins the mass of smoothed indicators near their support, in tension with the transport behavior that the expander decomposition forces on a hypothetical sofic approximation.

**rooted\_indicator\_defect\_sq\_le\_boundary** thm

```
theorem rooted_indicator_defect_sq_le_boundary
  {ι V : Type*} [Fintype ι] [Nonempty ι]
  [Fintype V] [DecidableEq V]
  (p : ι → Equiv.Perm V) (T : Finset V) :
  ||permutationMarkov p
    (indicatorVector
      (SoficGroups.KunFinitePermutationMarkovMass.realIndicator T)) -
    indicatorVector
      (SoficGroups.KunFinitePermutationMarkovMass.realIndicator T)|| ^ 2 ≤
  2 * (SoficGroups.boundary p T : ℝ) /
    (Fintype.card ι : ℝ) := by
-- proof: 40 lines, omitted
```

`rooted_indicator_defect_sq_le_boundary` states that for any finite family  $p$  of permutations of a finite  $V$  and any  $T \subseteq V$ , the squared Hilbert norm of the one-step Markov defect of the indicator vector of  $T$  is at most  $2 \cdot \partial_p T / |\iota|$ , phrased for the `KunRootedIndicatorCrossing` operators that the rooted models use. The proof is bookkeeping around one substantive input: the operators of `KunRealComplexMarkovBridge` are definitionally equal to the rooted-crossing ones (recorded as `rfl` identities), the two indicator definitions from `KunFinitePermutationMarkovMass` and `KunDirectedIndicatorJensen` coincide by `funext`, `norm_permutationMarkov_indicator_sub_sq` rewrites the Hilbert norm as the real defect energy, and `realPermutationMarkov_indicator_defect_sq_le_boundary` bounds that energy by the boundary. Its role is to close the loop within this chunk: the geometric control of `exists_rooted_word_radius_geometric_markov_displacement` says every iterated defect is at most a constant multiple of the one-step defect, and this lemma says the one-step defect is small whenever the test set has small edge boundary. Together, indicators of low-boundary sets in rooted models are almost invariant under all Markov iterates, exactly the configuration that the property (T) spectral gap will forbid at the densities produced by the expander decomposition.

*Remaining declarations in this section:*

|                                                  |     |                                                                                                                      |
|--------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------|
| <code>RootedIndicatorMarkovModel.IsRooted</code> | THM | Rootedness at radius $R$ implies rootedness at every smaller radius.                                                 |
| <code>AtRadius.mono</code>                       |     |                                                                                                                      |
| <code>exists_rooted_word_radius_all_</code>      | THM | One radius works for all steps $j \leq k$ : successive Markov differences contract like $q^j + \epsilon$ .           |
| <code>markov_iterate_contractions</code>         |     |                                                                                                                      |
| <code>norm_iterate_sub_le_sum_successive</code>  | THM | Telescoping: the iterate displacement is at most the sum of the successive-step norms.                               |
| <code>sum_geometric_budget_le</code>             | THM | The geometric-plus-noise budget $\sum_{j < k} (q^j + ((k+1)(1-q))^{-1})$ is at most $2/(1-q)$ .                      |
| <code>rooted_realMarkov_eq_mass_real</code>      | THM | The bridge and mass namespaces define the same real Markov operator.                                                 |
| <code>PermutationMarkov</code>                   |     |                                                                                                                      |
| <code>rooted_markov_real_iterate_sq_error</code> | THM | Transports the iterate defect energy of an indicator across the namespace identifications, as a real sum of squares. |

## 7 Transporting and completing sofic approximations

The technical heart of the transport argument: sharp threshold cuts via coarea, matching of compression ranks between parts, crossing-density control under conjugated actions, selection of large retained components, and the two-stage completion that turns restricted almost-actions into genuine permutation actions with prescribed spectral radius schedules.



## 7.1 Sofic Models as Rooted Markov Models (KunActualRootedModelBridge)

This short bridge section converts a hypothetical sofic approximation of a group  $G$  into the abstract rooted-model interface. `sourceFiniteIndicator` is the real-valued 0/1 indicator of a finite set. `sourceRootedIndicator` `MarkovModel` packages level  $n$  of the approximation together with a candidate vertex set  $T$ : the carrier is the model's point set, the generators act by the model permutations of elements of the symmetric generating set  $S$ , the indicator is that of  $T$ , and the evaluation is the chosen word evaluation for the symmetric generator word. The two theorems verify the interface: the model satisfies `IsGenerated` (the indicator is 0/1-valued and the evaluation respects the identity and the generators), and it satisfies `IsRootedAtRadius`  $r$  whenever  $T$  avoids the chosen Cayley-radius bad set. Thus the spectral machinery developed for rooted indicator Markov models becomes applicable to each finite level of a sofic approximation.

**sourceRootedIndicatorMarkovModel** `def`

```
def sourceRootedIndicatorMarkovModel
  {G : Type} [Group G] [DecidableEq G]
  (A : SoficGroups.SoficApproximation G)
  (S : Finset G)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set G) = T)
  (n : ℕ) (T : Finset (Fin (A.model n).size)) :
  RootedIndicatorMarkovModel G ιS where
  carrier := Fin (A.model n).size
  fintype := inferInstance
  generator i := (A.model n).action (i : G)
  indicator := sourceFiniteIndicator T
  evaluation :=
    chosenWordEvaluation A (fun i : ιS => (i : G))
    (symmetricGeneratorWord S hsymmetric hgenerates) n
```

Given a sofic approximation  $A$  of a group  $G$ , a finite symmetric generating set  $S$  whose closure is the whole group, a level  $n$ , and a subset  $T$  of the model's points, this definition assembles a `RootedIndicatorMarkovModel` indexed by the elements of  $S$ . The carrier is the finite point set of the  $n$ -th model, the generator assigned to  $i \in S$  is the model's permutation action of  $i$ , the indicator is `sourceFiniteIndicator` of  $T$ , and the evaluation is `chosenWordEvaluation` applied to the canonical `symmetricGeneratorWord` for  $S$ . The point of the construction is architectural: all the heavy spectral analysis (Kazhdan-pair contraction of the permutation Markov operator, sparse level-set cuts) is developed abstractly for rooted indicator Markov models, and this bridge is the unique place where a hypothetical sofic approximation is converted into such a model. Each finite level of the approximation, paired with any candidate set  $T$ , thus becomes a legitimate input to `exists_rooted_word_radius_sparse_cut_of_boundary` and its relatives, provided the accompanying interface theorems (`IsGenerated`, `IsRootedAtRadius`) are verified, which is precisely what the next two theorems do.

**sourceRootedIndicatorMarkovModel\_isRootedAtRadius** `thm`

```
theorem sourceRootedIndicatorMarkovModel_isRootedAtRadius
  {G : Type} [Group G] [DecidableEq G]
  (A : SoficGroups.SoficApproximation G)
  (S : Finset G) (hone : 1 ∈ S)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set G) = T)
  (n r : ℕ) (T : Finset (Fin (A.model n).size))
  (hgood : Disjoint T
    (chosenCayleyRadiusBad A S
      (symmetricGeneratorWord S hsymmetric hgenerates) n r)) :
  (sourceRootedIndicatorMarkovModel
    A S hsymmetric hgenerates n T).IsRootedAtRadius
    (symmetricGeneratorWord S hsymmetric hgenerates) r := by
  -- proof: 21 lines, omitted
```

States that the bridge model is rooted at radius  $r$  provided the chosen set  $T$  is disjoint from `chosenCayleyRadiusBad`, the set of model points at which the word evaluation fails to track Cayley-ball geometry up to

radius  $r$ . Rootedness (`IsRootedAtRadius`) demands that whenever a word within the radius budget evaluates to a group element  $g$  and the indicator is nonzero at a point  $x$ , the word evaluation behaves correctly at  $x$ . The proof unfolds the indicator: a point with nonzero indicator must lie in  $T$ , hence outside the bad set by the disjointness hypothesis, and `chosenCayleyRadiusBad_rooted` supplies the required tracking property. The significance is quantitative: earlier sections show the bad set has vanishing density as  $n \rightarrow \infty$ , so for large  $n$  candidate sets  $T$  can be arranged or pruned to satisfy the disjointness, making rootedness available at any fixed radius on a tail of the approximation. This is exactly the hypothesis under which the geometric Markov contraction of the `KunRootedWordPower` section applies to the concrete models built from a sofic approximation.

*Remaining declarations in this section:*

|                                                           |     |                                                                                                                                                         |
|-----------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sourceFiniteIndicator</code>                        | DEF | Real-valued 0/1 indicator function of a finite set $T$ , used as the model indicator.                                                                   |
| <code>sourceRootedIndicatorMarkovModel_isGenerated</code> | THM | The bridge model satisfies <code>IsGenerated</code> : 0/1 indicator, correct generator actions, and word evaluation respecting identity and generators. |

## 7.2 Sharp Threshold Cut via Coarea (`KunSharpThresholdCut`)

A finite coarea and mean-value argument producing a sharp threshold cut. `upperLevel` is the superlevel set  $\{f > t\}$ , and `highCrossingProfile` counts, for each threshold  $t$ , the generator-vertex pairs whose edge crosses  $t$ ; it equals the combinatorial boundary of the superlevel set exactly when  $a < t$ . Integrability lemmas justify integrating the profile over thresholds, and the layer-cake computation bounds that integral by the restricted variation  $\sum_i \sum_{f(x) > 1/3} |f(\sigma_i x) - f(x)|$ . Separately, for any  $t \in [1/3, 2/3]$  each point of `upperLevel` contributes at least  $1/9$  of its squared error, giving  $|\text{upperLevel} \Delta T| \leq 9 \sum_x (f(x) - \mathbf{1}_T(x))^2$ . Averaging over  $t \in (1/3, 2/3)$  selects one threshold satisfying both a boundary bound (three times the restricted variation) and the symmetric-difference bound, converting analytic smallness into a sparse combinatorial cut near  $T$ .

**`setIntegral_highCrossingProfile_le_variation` thm**

```
theorem setIntegral_highCrossingProfile_le_variation {V ι : Type*}
  [Fintype V] [Fintype ι]
  (σ : ι → Equiv.Perm V) (f : V → ℝ) (a c d : ℝ) :
  (f t in Ioo c d, highCrossingProfile σ f a t) ≤
    ∑ i : ι, ∑ x ∈ Finset.univ.filter (fun x : V => a < f x),
      |f (σ i x) - f x| := by
-- proof: 36 lines, omitted
```

The layer-cake (coarea) inequality in this finite setting. `highCrossingProfile` at threshold  $t$  sums, over generators  $i$  and vertices  $x$  with  $f(x) > a$ , the indicator that  $t$  lies in the interval from  $f(\sigma_i x)$  to  $f(x)$ . Integrating in  $t$  over  $(c, d)$  and exchanging the integral with the finite sums (justified by `integrableOn_crossingIndicator`) reduces the claim to a single interval indicator, whose integral is at most the interval length  $|f(\sigma_i x) - f(x)|$  by `setIntegral_crossingIndicator_le_abs`. Hence the total threshold mass of level-set boundary edges is dominated by the restricted variation  $\sum_i \sum_{f(x) > a} |f(\sigma_i x) - f(x)|$ . Since `highCrossingProfile_eq_boundary` identifies the profile at  $t$  with the boundary of `upperLevel` at  $t$  whenever  $a < t$ , this is exactly the estimate that lets a first-moment argument find a single good threshold: a function of small variation must have some level set with small boundary. It is the finite, permutation-graph analogue of the classical coarea formula used in Cheeger-type inequalities.

**`exists_upperLevel_boundary_and_symmDiff_le` thm**

```
theorem exists_upperLevel_boundary_and_symmDiff_le {V ι : Type*}
  [Fintype V] [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (T : Finset V) (f : V → ℝ) :
  ∃ t ∈ Ioo (1 / 3 : ℝ) (2 / 3 : ℝ),
    (SoficGroups.boundary σ (upperLevel f t) : ℝ) ≤
      3 * ∑ i : ι,
```

```

      ∑ x ∈ Finset.univ.filter
        (fun x : V => (1 / 3 : ℝ) < f x),
        |f (σ i x) - f x| ∧
      ((upperLevel f t Δ T).card : ℝ) ≤
      9 * ∑ x : V,
        (f x - if x ∈ T then (1 : ℝ) else 0) ^ 2 := by
-- proof: 29 lines, omitted

```

The section's main export. For any function  $f$  on a finite vertex set with permutations  $\sigma$  and any reference set  $T$ , there is a threshold  $t \in (1/3, 2/3)$  such that simultaneously the combinatorial boundary of `upperLevel` at  $t$  is at most  $3 \sum_i \sum_{f(x) > 1/3} |f(\sigma_i x) - f(x)|$ , and  $|\text{upperLevel} \Delta T| \leq 9 \sum_x (f(x) - \mathbf{1}_T(x))^2$ . The proof is a first-moment argument over the threshold: `exists_le_setAverage` produces a point  $t$  where `highCrossingProfile` is at most its average over  $(1/3, 2/3)$ ; the average equals three times the integral (the interval has length  $1/3$ ), and the integral is bounded by the restricted total variation via `setIntegral_highCrossingProfile_le_variation`. The symmetric-difference bound holds for every  $t$  in the window by `card_symmDiff_upperLevel_le_nine_sq_error`, a pointwise estimate: a point in the symmetric difference has  $f$ -value at distance at least  $1/3$  from its  $T$ -indicator value, so contributes at least  $1/9$  of its own squared error, verified by `nlinarith` on  $(3f(x) - 1)^2$  and  $(3f(x) - 2)^2$ . This converts smallness of  $f - \mathbf{1}_T$  and of the restricted variation into a genuine sparse combinatorial cut close to  $T$ .

*Remaining declarations in this section:*

|                                                        |     |                                                                                                                                   |
|--------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>upperLevel</code>                                | DEF | Superlevel set <code>upperLevel</code> : the vertices where $f$ exceeds a threshold $t$ .                                         |
| <code>highCrossingProfile</code>                       | DEF | Counts, over generators and vertices with $f(x) > a$ , thresholds $t$ crossed between $f(\sigma_i x)$ and $f(x)$ .                |
| <code>highCrossingProfile_eq_boundary</code>           | THM | For $a < t$ , the crossing profile at $t$ equals the boundary of the superlevel set at $t$ .                                      |
| <code>integrableOn_crossingIndicator</code>            | THM | Interval indicator functions are integrable on bounded open intervals.                                                            |
| <code>integrableOn_highCrossingProfile</code>          | THM | The high crossing profile, a finite sum of interval indicators, is integrable on $(c, d)$ .                                       |
| <code>setIntegral_crossingIndicator_le_abs</code>      | THM | The integral of an interval indicator over $(c, d)$ is at most the interval length $ b - a $ .                                    |
| <code>card_symmDiff_upperLevel_le_nine_sq_error</code> | THM | For $1/3 \leq t \leq 2/3$ , the symmetric difference of <code>upperLevel</code> with $T$ is at most nine times the squared error. |

### 7.3 Markov Residual Controls Restricted Variation (KunActualFinalRestrictedVariation)

This section proves that a small Markov residual forces a small restricted variation. `realMarkov` averages  $f$  over inverse generator permutations. Exact identities (change of variables, expansion of squares) show the Dirichlet energy  $\sum_i \sum_x (f(\sigma_i x) - f(x))^2$  equals twice the generator count times the pairing  $\sum_x f(x)(f(x) - Mf(x))$ . Cauchy-Schwarz bounds that pairing by  $\sqrt{|T|}$  times the residual norm, provided  $f$  takes values in  $[0, 1]$  with total mass  $|T|$ . Markov's inequality bounds the high support  $\{f > 1/3\}$  by  $3|T|$ , and Cauchy-Schwarz again bounds the restricted variation by the energy. A closing arithmetic lemma assembles these square bounds: under the explicit smallness condition  $486d^2\eta \leq 4\alpha^2$ , the high-support variation  $H$  satisfies  $9H \leq 2\alpha|T|$ . This estimate, with its explicit constants, is exactly what the threshold-cut section needs to produce level sets with sparse boundary in the rooted-model argument.

**`sum_sum_sq_displacement_eq_twice_card_mul_realMarkov_residual` thm**

```

theorem sum_sum_sq_displacement_eq_twice_card_mul_realMarkov_residual
  {V : Type*} [Fintype V] [Fintype ι] [Nonempty ι]
  (σ : ι → Equiv.Perm V) (f : V → ℝ) :
  (∑ i : ι, ∑ x : V, (f (σ i x) - f x) ^ 2) =
    2 * (Fintype.card ι : ℝ) *

```

```

       $\sum x : V, f x * (f x - \text{realMarkov } \sigma f x) := \text{by}$ 
-- proof: 24 lines, omitted

```

An exact discrete summation-by-parts identity: for a finite family  $\sigma$  of permutations of  $V$  and  $f : V \rightarrow \mathbb{R}$ , the Dirichlet energy  $\sum_i \sum_x (f(\sigma_i x) - f(x))^2$  equals  $2|V| \sum_x f(x)(f(x) - Mf(x))$ , where  $M$  is the `realMarkov` averaging operator sending  $x$  to the mean of  $f(\sigma_i^{-1}x)$ . The proof works one permutation at a time: `sum_sq_displacement_eq_twice_inverse_residual` expands the square and uses the change of variables  $\sum_x f(px)f(x) = \sum_x f(x)f(p^{-1}x)$  (`sum_permutation_mul_eq_sum_inverse_mul`) together with invariance of  $\sum f^2$  under permutations, then sums over  $i$  and swaps the order of summation. Its role is to convert smallness of the Markov residual  $f - Mf$ , which the Kazhdan spectral gap supplies for iterates of the Markov operator, into smallness of the edge-wise Dirichlet energy of  $f$ , the quantity the coarea argument understands. Note that the inverse permutations built into `realMarkov` are exactly what makes the cross terms match without any self-adjointness normalization.

#### `highSupport_variation_le_of_realMarkov_residual` thm

```

theorem highSupport_variation_le_of_realMarkov_residual
  {V ι : Type*} [Fintype V]
  [Fintype ι] [Nonempty ι]
  (σ : ι → Equiv.Perm V) (T : Finset V) (f : V → ℝ)
  (α η b : ℝ)
  (hfzero : ∀ x, 0 ≤ f x) (hfone : ∀ x, f x ≤ 1)
  (hmass : ( $\sum x : V, f x$ ) = (T.card : ℝ))
  (hα : 0 ≤ α) (hη : 0 ≤ η)
  (hbase : b ^ 2 ≤ (T.card : ℝ))
  (hresidual :
    Real.sqrt ( $\sum x : V,$ 
      (realMarkov σ f x - f x) ^ 2) ≤ η * b)
  (hsmall :
    486 * (Fintype.card ι : ℝ) ^ 2 * η ≤ 4 * α ^ 2) :
  9 * ( $\sum i : \iota,$ 
     $\sum x \in \text{Finset.univ.filter}$ 
      (fun x : V => (1 / 3 : ℝ) < f x),
    |f (σ i x) - f x|) ≤ 2 * α * (T.card : ℝ) := by
-- proof: 5 lines, omitted

```

The section's final estimate, consumed verbatim in `exists_rooted_word_radius_sparse_cut_of_boundary`. Suppose  $f : V \rightarrow [0, 1]$  has total mass  $|T|$ ,  $b^2 \leq |T|$ , the Markov residual satisfies  $\sqrt{\sum_x (Mf(x) - f(x))^2} \leq \eta b$ , and the smallness condition  $486d^2\eta \leq 4\alpha^2$  holds with  $d$  the number of generators. Then the variation of  $f$  restricted to the high support  $\{f > 1/3\}$  obeys  $9 \sum_i \sum_{f(x) > 1/3} |f(\sigma_i x) - f(x)| \leq 2\alpha|T|$ . It is proved by feeding `card_highSupport_le_three_card` (Markov's inequality bounding the high support by  $3|T|$ ) into `highSupport_variation_le_of_realMarkov_residual_and_highSupport`, which itself instantiates the purely arithmetic lemma `high_variation_le_of_markov_square_bounds` with the energy identity, the Cauchy-Schwarz pairing bound, and the restricted Cauchy-Schwarz variation bound. The factor 9 matches the factor from the symmetric-difference estimate of the threshold-cut section, and the explicit constant 486 propagates into the choice of  $\eta = 4\alpha^2/(486|S|^2)$  made later, so the numerology here directly dictates how many Markov iterations the rooted-model argument must take.

*Remaining declarations in this section:*

|                                                            |     |                                                                                                       |
|------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------|
| <code>realMarkov</code>                                    | DEF | Averaging operator <code>realMarkov</code> : the mean of $f$ over the inverse generator permutations. |
| <code>sum_permutation_mul_eq_sum_inverse_mul</code>        | THM | Change of variables: $\sum_x f(px)f(x) = \sum_x f(x)f(p^{-1}x)$ for a permutation $p$ .               |
| <code>sum_sq_displacement_eq_twice_inverse_residual</code> | THM | Squared displacement identity: $\sum_x (f(px) - f(x))^2 = 2 \sum_x f(x)(f(x) - f(p^{-1}x))$ .         |
| <code>sum_sq_le_card_of_unit_interval_mass</code>          | THM | For $[0, 1]$ -valued $f$ of mass $ T $ , the sum of squares is at most $ T $ .                        |

|                                      |     |                                                                                                                 |
|--------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------|
| realMarkov_residual_pairing_le_sqrt_ | THM | Cauchy-Schwarz: the residual pairing is at most $\sqrt{ T }$ times the residual norm.                           |
| card_mul_residual                    |     |                                                                                                                 |
| sum_sum_sq_displacement_le_twice_    | THM | Energy bound: the total squared displacement is at most $2 \iota \sqrt{ T }$ times the residual norm.           |
| card_mul_sqrt_card_mul_residual      |     |                                                                                                                 |
| card_highSupport_le_three_card       | THM | Markov inequality: the high support where $f > 1/3$ has at most $3 T $ elements.                                |
| filtered_variation_sq_le_card_mul_   | THM | Cauchy-Schwarz on $H$ : the squared filtered variation is at most $ \iota  H $ times the filtered energy.       |
| filtered_energy                      |     |                                                                                                                 |
| filtered_variation_sq_le_card_mul_   | THM | Extends the filtered variation bound to the full Dirichlet energy by positivity of the added terms.             |
| energy                               |     |                                                                                                                 |
| highSupport_variation_sq_le_three_   | THM | Combines the high-support cardinality bound to control the squared variation by $3 \iota  T $ times the energy. |
| mul_reference_energy                 |     |                                                                                                                 |
| high_variation_le_of_markov_square_  | THM | Abstract real-arithmetic lemma: the listed square bounds force $9H \leq 2\alpha t$ .                            |
| bounds                               |     |                                                                                                                 |
| highSupport_variation_le_of_real     | THM | Instantiates the abstract lemma with the concrete energy, residual, and high-support quantities.                |
| Markov_residual_and_highSupport      |     |                                                                                                                 |

## 7.4 Bad Density Transfer to Restrictions (MatchedFirstStageWordRadiusTransfer)

Transfers vanishing bad-set densities from ambient permutation systems to their completed restrictions on parts  $P_n$ . The word-test bad set of the restricted system, consisting of points where the restricted permutations fail commutation, multiplication, or separation tests, injects into the source completion bad set of the ambient system (1097), so its cardinality is dominated. If for every  $n$  the completion-bad set is captured inside the intersection of  $P_n$  with an exceptional set of vanishing relative density, the word-test bad density vanishes (1098); the section specializes this to exceptional sets of the form `matchedRadiusBad` at a radius  $r(n)$  (1099). Finally, after deleting a further vanishing-density set  $D_n$  of points, the completion-bad set of the restricted system relative to the surviving points still has vanishing density (1100). These statements let the density hypotheses established for the ambient sofic models survive both restriction to matched parts and the pruning surgery performed later.

**pruned\_sourceCompletionBad\_density\_tendsto\_zero\_of\_matchedRadius thm**

```

theorem pruned_sourceCompletionBad_density_tendsto_zero_of_matchedRadius
  (V : ℕ → Type*) [V n, Fintype (V n)]
  [V n, DecidableEq (V n)]
  {ι κ J : Type*} [Fintype ι] [Group J]
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))
  (p : (n : ℕ) → J → Equiv.Perm (V n))
  (F : Finset J)
  (U P : (n : ℕ) → Finset (V n))
  (Q : (n : ℕ) → Finpartition (U n))
  (I : ℕ → Finset κ)
  (w : (n : ℕ) → κ → Equiv.Perm (V n))
  (r : ℕ → ℕ)
  (B : (n : ℕ) → ℕ → Finset (V n))
  (D : (n : ℕ) → Finset {x : V n // x ∈ P n})
  (hcapture : ∀ n,
    SoficGroups.MatchedComponentCompletion.sourceCompletionBad
      (σ n) (p n) F (P n) ⊆
      P n n SoficGroups.matchedRadiusBad
        (Q n) (I (r n)) (w n) (B n (r n)))
  (hradius : Tendsto
    (fun n =>
      ((P n n SoficGroups.matchedRadiusBad
        (Q n) (I (r n)) (w n) (B n (r n))).card : ℝ) / (P n).card))
    atTop (nhds 0))
  (hZ : ∀ n,
    (Finset.univ \ D n :

```

```

    Finset {x : V n // x ∈ P n}).Nonempty)
(hdeleted : Tendsto
  (fun n => ((D n).card : ℝ) / (P n).card)
  atTop (nhds 0)) :
Tendsto
  (fun n =>
    ((SoficGroups.MatchedComponentCompletion.sourceCompletionBad
      (fun i => SoficGroups.MatchedComponentCompletion.completedRestriction
        (σ n i) (P n))
      (fun j => SoficGroups.MatchedComponentCompletion.completedRestriction
        (p n j) (P n))
      F (Finset.univ \ D n)).card : ℝ) /
    (Finset.univ \ D n :
      Finset {x : V n // x ∈ P n}).card)
  -- statement truncated; proof: 15 lines, omitted

```

Combines the two pruning mechanisms of the restricted permutation systems. The hypotheses: for every  $n$ , the completion-bad set of the ambient permutations relative to the part  $P_n$  is captured inside the intersection of  $P_n$  with a `matchedRadiusBad` set whose relative density vanishes; and a further deletion set  $D_n$  of points of  $P_n$  has vanishing density while its complement stays nonempty. The conclusion: the completion-bad set of the completed restrictions, computed relative to the surviving points (the complement of  $D_n$ ), has density tending to zero. The proof first shows the word-test failures of the restricted system have vanishing density, because they inject into the ambient completion-bad set (`card_completed_sourceWordTestBad_le_sourceCompletionBad`, whose proof maps the bad subtype points through the subtype embedding and refutes each failure mode using the completed-restriction compatibility lemmas), and then applies the generic surviving-density transfer lemma `sourceCompletionBad_surviving_density_tendsto_zero` from the `MatchedComponentExitBudget` namespace. This matters because the final construction works not with full parts but with parts after discarding a small exceptional set, and every asymptotic bad-density hypothesis must survive that surgery for the argument to reach its contradiction.

*Remaining declarations in this section:*

|                                                   |                  |                                                                                                                            |
|---------------------------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>card_completed_sourceWordTestBad_le_</code> | <code>THM</code> | Word-test bad points of completed restrictions embed into the ambient source completion bad set, comparing cardinalities.  |
| <code>sourceCompletionBad</code>                  |                  |                                                                                                                            |
| <code>completed_sourceWordTestBad_density_</code> | <code>THM</code> | If completion-bad sets are captured with vanishing relative density, the word-test bad density inside $P_n$ tends to zero. |
| <code>tendsto_zero</code>                         |                  |                                                                                                                            |
| <code>completed_sourceWordTestBad_density_</code> | <code>THM</code> | Specializes the vanishing word-test bad density to capture by <code>matchedRadiusBad</code> sets at radius $r(n)$ .        |
| <code>tendsto_zero_of_matchedRadius</code>        |                  |                                                                                                                            |

## 7.5 Compression Rank Matching of Parts (SourceCompressionMatching)

Quantitative matching between a partition part and its transported image, driven by a logarithmic compression rank. If two positive reals have equal floors of their shifted, rescaled logarithms, one is below  $e^H$  times the other (1101); applied to component sizes, preservation of the `offsetFloorRank` statistic at a single vertex forces the maximally overlapping target part  $D$  to satisfy  $|D| < e^H|C|$  (1102). The exact identity  $|C \triangle D| + 2|C \cap D| = |C| + |D|$  (1103) then converts an overlap lower bound  $(1 - \eta)|C| \leq |C \cap D|$  into  $|C \triangle D| < (e^H - 1 + 2\eta)|C|$  (1104), and when  $2(e^H - 1 + 2\eta) < 1$  into the majority property  $|D| < 2|C \cap D|$  (1105). Summing over retained parts and letting  $H, \eta \rightarrow 0$  shows the aggregated symmetric-difference density vanishes (1106), the matching input for the retained-selection machinery.

**`transported_maximumOverlapPart_card_lt_exp_mul`** `thm`

```

theorem transported_maximumOverlapPart_card_lt_exp_mul
  {V : Type*} [Fintype V] [DecidableEq V]
  (Q : Finpartition (Finset.univ : Finset V))
  (T : Equiv.Perm V) (C : Finset V) (hC : C ∈ Q.parts)
  (H r : ℝ) (hH : 0 < H) (x : V) (hx : x ∈ C)
  (hTx : T x ∈ SoficGroups.maximumOverlapPart Q
    (C.map T.toEmbedding))

```

```

(hrank :
  SoficGroups.MidrankPermutationEnergy.offsetFloorRank
  (fun z : V =>
    Real.log (SoficGroups.partitionComponentSize Q z : ℝ))
  H r (T x) =
  SoficGroups.MidrankPermutationEnergy.offsetFloorRank
  (fun z : V =>
    Real.log (SoficGroups.partitionComponentSize Q z : ℝ))
  H r x) :
  ((SoficGroups.maximumOverlapPart Q
    (C.map T.toEmbedding)).card : ℝ) <
    Real.exp H * (C.card : ℝ) := by
-- proof: 21 lines, omitted

```

The compression bookkeeping that controls how much a transported component can grow. Each vertex  $z$  carries the statistic `offsetFloorRank`, the floor of  $(\log(\text{componentSize}(z)) + r)/H$ , where the component size is the cardinality of the part of  $Q$  containing  $z$ . Suppose  $C$  is a part of  $Q$ ,  $x \in C$ , and the transported point  $T(x)$  lands in the part  $D$  of  $Q$  that maximally overlaps the image of  $C$  under  $T$  (`maximumOverlapPart`). If the floor rank at  $T(x)$  equals the floor rank at  $x$ , then since `partitionComponentSize` evaluates to  $|C|$  at  $x$  and to  $|D|$  at  $T(x)$ , the floors of  $(\log |C| + r)/H$  and  $(\log |D| + r)/H$  agree, and the elementary lemma `lt_exp_mul_of_equal_log_floor` gives  $|D| < e^H |C|$ . The point: preservation of a single coarse, integer-valued statistic, cheap to guarantee for most vertices by counting arguments elsewhere, pins the size of the receiving part to within a factor  $e^H$ , with  $H$  eventually sent to zero. Combined with the overlap-majority arithmetic that follows, this is how the source-compression matching forces transported parts to essentially coincide with parts of the target partition.

#### **`symmDiff_density_tendsto_zero_of_log_rank_bounds` thm**

```

theorem symmDiff_density_tendsto_zero_of_log_rank_bounds
  {V : ℕ → Type*} [V n, DecidableEq (V n)]
  (U : V n, Finset (V n)) (hU : V n, (U n).Nonempty)
  (P : V n, Finpartition (U n))
  (R : V n, Finset (Finset (V n)))
  (hR : V n, R n ⊆ (P n).parts)
  (D : V n, Finset (V n) → Finset (V n))
  (H eta : ℕ → ℝ)
  (hH : V n, 0 ≤ H n) (heta : V n, 0 ≤ eta n)
  (hHzero : Tendsto H atTop (nhds 0))
  (hetazero : Tendsto eta atTop (nhds 0))
  (hbound : V n C, C ∈ R n →
    ((C Δ D n C).card : ℝ) ≤
    (Real.exp (H n) - 1 + 2 * eta n) * (C.card : ℝ)) :
  Tendsto
    (fun n =>
      ((∑ C ∈ R n, (C Δ D n C).card : ℕ) : ℝ) / (U n).card)
    atTop (nhds 0) := by
-- proof: 61 lines, omitted

```

Aggregates the per-part matching estimates into the asymptotic input needed by the retained-selection machinery. For each  $n$  there is a partition  $P_n$  of the support  $U_n$ , a retained subfamily  $R_n$  of parts, and an assignment  $D_n(C)$  of a matched target part to each retained  $C$ , with the per-part bound  $|C \Delta D_n(C)| \leq (e^{H_n} - 1 + 2\eta_n)|C|$ . Since the retained parts are pairwise disjoint parts of  $P_n$ , their cardinalities sum to the cardinality of the retained support (`matchedRetainedSupport_card`), which is at most  $|U_n|$ , so the summed symmetric difference is at most  $(e^{H_n} - 1 + 2\eta_n)|U_n|$ . As  $H_n \rightarrow 0$  and  $\eta_n \rightarrow 0$ , continuity of the exponential makes the coefficient tend to zero, and a squeeze argument gives that the ratio of  $\sum_{C \in R_n} |C \Delta D_n(C)|$  to  $|U_n|$  tends to zero. This is exactly the summed symmetric-difference hypothesis of `exists_matched_slow_diagonal_large_components_with_vanishing_boundary`, tying the quantitative compression-matching bounds of this section to the diagonal selection of large good components in the next.

*Remaining declarations in this section:*

`lt_exp_mul_of_equal_log_floor`      THM      Equal floors of  $(\log x + r)/H$  and  $(\log y + r)/H$  force  $y < e^H x$ .



|                                                       |     |                                                                                                                |
|-------------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------|
| <code>card_symmDiff_add_twice_inter</code>            | THM | Counting identity: $ C \triangle D  + 2 C \cap D  =  C  +  D $ .                                               |
| <code>symmDiff_card_lt_of_overlap_and_exp_card</code> | THM | Large overlap plus exponential cardinality control bounds the symmetric difference by $(e^H - 1 + 2\eta) C $ . |
| <code>target_majority_of_overlap_and_exp_card</code>  | THM | Under the smallness condition, the overlap is a majority of $D$ : $ D  < 2 C \cap D $ .                        |

## 7.6 Crossing Densities under Conjugated Actions (SourceCompressionTransportCrossing)

Relates partition-crossing statistics of nearby permutations, so that crossing hypotheses stated for group elements apply to conjugated model permutations. `partitionWordCrossing` collects the vertices whose part changes under a permutation; membership is characterized (1107) and the crossing count is 1-Lipschitz in the Hamming permutationDistance (1108). For a sofic approximation, asymptotic multiplicativity yields that the action of  $u^{-1}$  is asymptotically inverse to the action of  $u$  (1109), and a double triangle inequality with the bi-invariance of `normalizedHamming` shows the action of  $ugu^{-1}$  is asymptotically the conjugate of the individual actions (1110). Consequently, if the crossing density of the permutations realizing  $ugu^{-1}$  vanishes, so does the crossing density of the conjugated products of actions (1111), by squeezing between the original density and the vanishing normalized Hamming defects.

### `conjugated_word_crossing_density_tendsto_zero` thm

```
theorem conjugated_word_crossing_density_tendsto_zero
  {G : Type*} [Group G] (A : SoficGroups.SoficApproximation G)
  (u g : G)
  (Q : ∀ n, Finpartition
    (Finset.univ : Finset (Fin (A.model n).size)))
  (hword : Tendsto
    (fun n =>
      ((SoficGroups.partitionWordCrossing (Q n)
        ((A.model n).action (u * g * u⁻¹))).card : ℝ) /
      (A.model n).size)
    atTop (nhds 0)) :
  Tendsto
    (fun n =>
      ((SoficGroups.partitionWordCrossing (Q n)
        ((A.model n).action u * (A.model n).action g *
          ((A.model n).action u⁻¹))).card : ℝ) /
      (A.model n).size)
    atTop (nhds 0) := by
  -- proof: 70 lines, omitted
```

The transfer of crossing statistics from group-level words to model-level products. The hypothesis says that for the partitions  $Q_n$  of the sofic models, the density of vertices whose part changes under the single permutation realizing  $ugu^{-1}$  tends to zero. The conclusion moves the crossing statement to the conjugated product of the actions of  $u$ ,  $g$ , and  $u^{-1}$ , which is the permutation the transported construction actually applies. The proof combines two ingredients: the Lipschitz estimate `card_partitionWordCrossing_le_add_distance`, showing crossing counts of two permutations differ by at most their Hamming distance, and the asymptotic homomorphism property of sofic approximations, upgraded in `tendsto_action_conjugate` to conjugation: by two triangle inequalities and the left and right invariance lemmas for `normalizedHamming`, the distance between the action of  $ugu^{-1}$  and the conjugated product is bounded by the multiplicativity defects of the pairs  $(ug, u^{-1})$  and  $(u, g)$  plus the inverse defect handled in `tendsto_action_inverse`. Dividing the Lipschitz bound by the model size and squeezing gives the result. This lemma is what lets word-crossing hypotheses, naturally stated for elements of the group, be consumed by constructions that conjugate model permutations directly.

*Remaining declarations in this section:*

|                                             |     |                                                                                                |
|---------------------------------------------|-----|------------------------------------------------------------------------------------------------|
| <code>mem_partitionWordCrossing_univ</code> | THM | A vertex crosses the partition under $p$ exactly when its part differs from that of its image. |
|---------------------------------------------|-----|------------------------------------------------------------------------------------------------|



|                                                 |     |                                                                                                       |
|-------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------|
| <code>card_partitionWordCrossing_le_add_</code> | THM | Crossing counts of two permutations differ by at most their Hamming permutationDistance.              |
| <code>tendsto_action_inverse</code>             | THM | In a sofic approximation, the action of $u^{-1}$ is asymptotically the inverse of the action of $u$ . |
| <code>tendsto_action_conjugate</code>           | THM | The action of a conjugate $ugu^{-1}$ asymptotically equals the conjugate of the individual actions.   |

## 7.7 Selecting Large Retained Components (KunResidualRetainedSelection)

The selection engine extracting one good part per level. A weighted-average pigeonhole (1112) finds a retained part whose boundary density and bad-set density are both at most the support-level averages; those averages vanish given vanishing discarded density and vanishing total boundary density (1113), yielding per- $n$  parts  $C_n$  with both densities tending to zero (1114). The main diagonal theorem (1115) also produces a slowly growing radius  $r_n \rightarrow \infty$  so that the `matchedRadiusBad` density inside  $C_n$  vanishes, and largeness  $|C_n| \rightarrow \infty$  follows from a realization hypothesis injecting the ball  $S^{r_n/2}$  of an infinite group into  $C_n$ . Tail variants (1116, 1117) re-index by  $n + N$  so that an eventually-true guard predicate holds at every index; the guard is instantiated (1118, 1119) with the source-compression scale conditions  $\eta_n \leq 1/2$  and  $2(e^{H_n} - 1 + 2\eta_n) < 1$ , producing the packaged statement used at the compression scale.

**exists\_matched\_slow\_diagonal\_large\_components\_with\_vanishing\_boundary** *thm*

```

theorem exists_matched_slow_diagonal_large_components_with_vanishing_boundary
  {K : Type*} [Group K] [Infinite K] [DecidableEq K]
  (S : Finset K) (hS : 1 ∈ S)
  (hgen : Subgroup.closure (S : Set K) = T)
  {V : ℕ → Type*} [∀ n, DecidableEq (V n)]
  {ι κ : Type*} [Fintype κ]
  (σ : (n : ℕ) → κ → Equiv.Perm (V n))
  (U : ∀ n, Finset (V n)) (hU : ∀ n, (U n).Nonempty)
  (P Q : ∀ n, Finpartition (U n))
  (R : ∀ n, Finset (Finset (V n)))
  (hR : ∀ n, R n ⊆ (P n).parts)
  (hRne : ∀ n, (R n).Nonempty)
  (D : ∀ n, Finset (V n) → Finset (V n))
  (hD : ∀ n C, C ∈ R n → D n C ∈ (Q n).parts)
  (hmajor : ∀ n C, C ∈ R n →
    (D n C).card < 2 * (C n D n C).card)
  (hdiscard : Tendsto
    (fun n =>
      (((U n \ SoficGroups.matchedRetainedSupport (R n)).card : ℝ) /
        (U n).card)) atTop (ℵ 0))
  (hsymm : Tendsto
    (fun n =>
      ((∑ C ∈ R n, (C Δ D n C).card : ℝ) : ℝ) / (U n).card)
    atTop (ℵ 0))
  (I : ℕ → Finset ι)
  (w : ∀ n, ι → Equiv.Perm (V n))
  (hword : ∀ k, Tendsto
    (fun n =>
      ((∑ i ∈ I k,
        (SoficGroups.partitionWordCrossing (Q n) (w n i)).card : ℝ) :
        ℝ) / (U n).card)
    atTop (ℵ 0))
  (B : ∀ n, ℕ → Finset (V n))
  (hbad : ∀ k, Tendsto
    (fun n => (((U n n B n k).card : ℝ) / (U n).card))
    atTop (ℵ 0))
  (hboundary : Tendsto
    (fun n =>
      (∑ C ∈ (P n).parts, (SoficGroups.boundary (σ n) C : ℝ)) /
        (U n).card) atTop (ℵ 0))
-- statement truncated; proof: 54 lines, omitted

```

The central diagonalization of this section. Inputs: retained parts  $R_n$  covering all but a vanishing fraction of  $U_n$ ; matched target parts  $D_n(C)$  with the majority property  $|D_n(C)| < 2|C \cap D_n(C)|$  and vanishing summed symmetric-difference density; word-crossing densities vanishing for every fixed radius index  $k$ ; bad-set densities vanishing for every fixed  $k$ ; vanishing total boundary density; and a realization hypothesis: any point of a retained part avoiding the radius- $k$  bad set supports an injection of the ball  $S^{k/2}$  of the infinite group  $K$  into that part. The conclusion produces a single slowly growing radius sequence  $r_n \rightarrow \infty$  together with parts  $C_n \in R_n$  such that the boundary density of  $C_n$  and the density of the radius- $r_n$  bad set inside  $C_n$  both vanish, and  $|C_n| \rightarrow \infty$ . The proof invokes the pre-existing slow-diagonal lemma `exists_matched_slow_diagonal_word_errors` to choose  $r_n$ , folds the radius-bad sets into a single bad family, selects  $C_n$  by the weighted-average pigeonhole of 1114, and derives largeness from the realization: eventually  $C_n$  contains an injected copy of  $S^{r_n/2}$ , whose cardinality grows without bound since  $S$  generates the infinite group  $K$ . This furnishes the arbitrarily large components with vanishing boundary that the expander estimates are played against downstream.

**exists\_matched\_slow\_diagonal\_large\_components\_on\_source\_scale\_tail** *thm*

```

theorem exists_matched_slow_diagonal_large_components_on_source_scale_tail
  {K : Type*} [Group K] [Infinite K] [DecidableEq K]
  (S : Finset K) (hS : 1 ∈ S)
  (hgen : Subgroup.closure (S : Set K) = T)
  {V : ℕ → Type*} [∀ n, DecidableEq (V n)]
  {ι κ : Type*} [Fintype κ]
  (σ : (n : ℕ) → κ → Equiv.Perm (V n))
  (U : ∀ n, Finset (V n)) (hU : ∀ n, (U n).Nonempty)
  (P Q : ∀ n, Finpartition (U n))
  (R : ∀ n, Finset (Finset (V n)))
  (hR : ∀ n, R n ⊆ (P n).parts)
  (D : ∀ n, Finset (V n) → Finset (V n))
  (hD : ∀ n C, C ∈ R n → D n C ∈ (Q n).parts)
  (H eta : ℕ → ℝ)
  (hH : Tendsto H atTop (ℕ 0))
  (heta : Tendsto eta atTop (ℕ 0))
  (hmajor : ∀ n,
    eta n ≤ (1 : ℝ) / 2 →
    2 * (Real.exp (H n) - 1 + 2 * eta n) < 1 →
    ∀ C, C ∈ R n →
      (D n C).card < 2 * (C n D n C).card)
  (hdiscard : Tendsto
    (fun n =>
      ((U n \ SoficGroups.matchedRetainedSupport (R n)).card : ℝ) /
      (U n).card)) atTop (ℕ 0))
  (hsymm : Tendsto
    (fun n =>
      ((∑ C ∈ R n, (C Δ D n C).card : ℝ) /
      (U n).card)
      atTop (ℕ 0))
    (I : ℕ → Finset ι)
    (w : ∀ n, ι → Equiv.Perm (V n))
    (hword : ∀ k, Tendsto
      (fun n =>
        ((∑ i ∈ I k,
          (SoficGroups.partitionWordCrossing (Q n) (w n i)).card :
          ℝ) / (U n).card)
          atTop (ℕ 0))
      (B : ∀ n, ℕ → Finset (V n))
      (hbad : ∀ k, Tendsto
-- statement truncated; proof: 14 lines, omitted

```

Packages the entire selection for consumption at the source-compression scale. The extra wrinkle relative to `exists_matched_slow_diagonal_large_components_with_vanishing_boundary` is that the majority hypothesis for matched parts is only available when the scale parameters are in range:  $\eta_n \leq 1/2$  and  $2(e^{H_n} - 1 + 2\eta_n) < 1$ . Since  $H_n \rightarrow 0$  and  $\eta_n \rightarrow 0$ , the lemma `source_compression_scale_guard_eventually` shows this guard holds for all large  $n$ , and `matchedRetained_parts_eventually_nonempty_of_discard` similarly makes the retained family

eventually nonempty. The guarded-tail theorem 1117 then re-indexes everything by  $n + N$  for a suitable  $N \geq N_0$ , composing all the vanishing-density hypotheses with the shift, so that on the tail the guard holds at every index and the unguarded selection theorem applies verbatim. The output, consisting of a tail base  $N$ , radii  $r_n \rightarrow \infty$ , and large parts  $C_n \in R_{n+N}$  with vanishing boundary and bad densities together with the recorded guard inequalities, is stated in exactly the shifted form the final assembly needs, avoiding any eventual-quantifier bookkeeping at the point of use.

*Remaining declarations in this section:*

|                                                                                        |     |                                                                                                                              |
|----------------------------------------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------|
| <code>exists_matchedRetained_part_<br/>boundary_and_bad_density_le</code>              | THM | Weighted-average selection: some retained part has boundary and bad densities both below the combined support-level average. |
| <code>matchedRetained_boundary_density_<br/>tendsto_zero</code>                        | THM | Vanishing discard and total boundary densities force the boundary density over the retained support to vanish.               |
| <code>exists_matchedRetained_parts_with_<br/>vanishing_boundary_and_bad_density</code> | THM | Produces per- $n$ retained parts $C_n$ whose boundary density and bad-set density both tend to zero.                         |
| <code>matchedRetained_parts_eventually_<br/>nonempty_of_discard</code>                 | THM | If the discarded density vanishes, the retained part family is eventually nonempty.                                          |
| <code>exists_matched_slow_diagonal_large_<br/>components_on_guarded_tail</code>        | THM | Tail-shifted version: restarting at a large index $N$ makes the guard predicate hold everywhere along the selection.         |
| <code>source_compression_scale_guard_<br/>eventually</code>                            | THM | The compression-scale guard $\eta_n \leq 1/2$ and $2(e^{H_n} - 1 + 2\eta_n) < 1$ holds eventually.                           |

## 7.8 Good-Root Cuts Give Centralizer Improvement (KunUniversalGoodRootReferenceCuts)

Derives the almost-centralizer improvement property from uniform good-root threshold cuts. For any permutation  $p$  with commutation defect at most  $2t$ , a hypothesis supplies a pair set  $U$  close in symmetric difference to a good subgraph of the permutation graph of  $p$  and with sparse boundary under the doubled permutations. Set arithmetic (1120) and the boundary control on the good subgraph convert this into  $|U \setminus \text{graph}(p)| + |\text{graph}(p) \setminus U| \leq Ct$  with the explicit constant  $C = 216/(d(1 - q)^2) + 1/d$  (1121). A private real-arithmetic lemma (1122) then extracts the three inequalities demanded by the almost-centralizer interface (closeness, boundary improvement, and the distance-versus-size inequality) from a slow-tolerance assumption. The main theorem (1123) feeds these into the rooted-reference-cut criterion of the `KunThomFiberCoarea` namespace, concluding `HasAlmostCentralizerImprovement`: the spectral-gap rigidity statement that later sections pit against the flexibility of a hypothetical sofic approximation.

**reference\_difference\_le\_of\_good\_root\_threshold** thm

```
theorem reference_difference_le_of_good_root_threshold
  {V ι : Type*} [Fintype V] [DecidableEq V]
  [Fintype ι] [Nonempty ι]
  (σ : ι → Equiv.Perm V) (p : Equiv.Perm V)
  (B : Finset V) (T U : Finset (V × V))
  (t : ℕ) (q : ℝ) (hq : q < 1)
  (hsub : T ⊆ SoficGroups.permutationGraph p)
  (hmissing :
    (SoficGroups.permutationGraph p \ T).card ≤ 2 * B.card)
  (hbad :
    2 * (Fintype.card ι : ℝ) * (B.card : ℝ) ≤ (t : ℝ))
  (hdefect : SoficGroups.permutationCommutationDefect σ p ≤ 2 * t)
  (hboundary :
    SoficGroups.boundary (fun i => (σ i).prodCongr (σ i)) T ≤
    SoficGroups.permutationCommutationDefect σ p +
    2 * Fintype.card ι * B.card)
  (hthreshold :
    ((U Δ T).card : ℝ) ≤
    72 * (SoficGroups.boundary
      (fun i => (σ i).prodCongr (σ i)) T : ℝ) /
```

```

      ((Fintype.card ι : ℝ) * (1 - q) ^ 2)) :
      (((U \ SoficGroups.permutationGraph p).card +
        (SoficGroups.permutationGraph p \ U).card : ℕ) : ℝ) ≤
        (216 / ((Fintype.card ι : ℝ) * (1 - q) ^ 2) +
          1 / (Fintype.card ι : ℝ)) * (t : ℝ) := by
-- proof: 54 lines, omitted

```

Turns the threshold-cut output into a distance bound between the candidate set  $U$  and the graph of the permutation  $p$ . The good set  $T$  is a subgraph of the permutation graph of  $p$  missing at most  $2|B|$  pairs; the threshold hypothesis bounds  $|U \Delta T|$  by  $72\partial T / (d(1 - q)^2)$ , where  $\partial T$  is the boundary of  $T$  under the doubled permutations (each  $\sigma_i$  acting diagonally on pairs) and  $d$  is the number of generators. Chaining the set-theoretic triangle inequality `reference_difference_le_symmDiff_add_missing` with the boundary hypothesis  $\partial T \leq \text{defect}(p) + 2d|B|$ , the defect bound  $\text{defect}(p) \leq 2t$ , and the bad-set bound  $2d|B| \leq t$  yields  $|U \setminus \text{graph}(p)| + |\text{graph}(p) \setminus U| \leq (216 / (d(1 - q)^2) + 1/d)t$ . The explicit constant is important: it enters the slow-tolerance condition  $5(4 + hC)t \leq hN$  of the improvement theorem, which is satisfiable because the tolerance  $t$  is fixed while the model size  $N$  grows along the sofic sequence.

#### **hasAlmostCentralizerImprovement\_of\_universal\_good\_root\_thresholds** `thm`

```

theorem hasAlmostCentralizerImprovement_of_universal_good_root_thresholds
  {V ι : Type*}
  [Fintype V] [Nonempty V] [DecidableEq V]
  [Fintype ι] [Nonempty ι]
  (σ : ι → Equiv.Perm V) (tolerance : ℝ)
  (h q : ℝ) (hpositive : 0 < h) (hq : q < 1)
  (B : Finset V)
  (good : Equiv.Perm V → Finset (V × V))
  (hexp : ∀ A : Finset V,
    h * min (A.card : ℝ)
      ((Fintype.card V : ℝ) - A.card) ≤
      (SoficGroups.boundary σ A : ℝ))
  (hbad :
    2 * (Fintype.card ι : ℝ) * (B.card : ℝ) ≤
    (tolerance : ℝ))
  (hgood_subset : ∀ p : Equiv.Perm V,
    good p ⊆ SoficGroups.permutationGraph p)
  (hgood_missing : ∀ p : Equiv.Perm V,
    (SoficGroups.permutationGraph p \ good p).card ≤
    2 * B.card)
  (hgood_boundary : ∀ p : Equiv.Perm V,
    SoficGroups.boundary
      (fun i => (σ i).prodCongr (σ i)) (good p) ≤
      SoficGroups.permutationCommutationDefect σ p +
      2 * Fintype.card ι * B.card)
  (hslow :
    5 * (4 + h *
      (216 / ((Fintype.card ι : ℝ) * (1 - q) ^ 2) +
        1 / (Fintype.card ι : ℝ))) * (tolerance : ℝ) ≤
    h * (Fintype.card V : ℝ))
  (hthreshold : ∀ p : Equiv.Perm V,
    SoficGroups.permutationCommutationDefect σ p ≤ 2 * tolerance →
    ∃ U : Finset (V × V),
      ((U Δ good p).card : ℝ) ≤
      72 * (SoficGroups.boundary
        (fun i => (σ i).prodCongr (σ i)) (good p) : ℝ) /
        ((Fintype.card ι : ℝ) * (1 - q) ^ 2) ∧
      (SoficGroups.boundary
        (fun i => (σ i).prodCongr (σ i)) U : ℝ) ≤
      (h * (tolerance : ℝ) /

```

-- statement truncated; proof: 70 lines, omitted

The section's main theorem and the bridge to the rigidity mechanism. Hypotheses: the permutations  $\sigma$  form an  $h$ -expander in the Cheeger sense (every set's boundary is at least  $h$  times the smaller of its size and its complement's size); a small exceptional set  $B$  with  $2d|B| \leq t$ ; for every permutation  $p$ , a good subgraph of its permutation graph missing at most  $2|B|$  pairs whose doubled boundary is controlled by the commutation

defect of  $p$ ; a slow-tolerance inequality making  $t$  small relative to the vertex count; and, crucially, for every  $p$  of defect at most  $2t$ , a threshold cut  $U$  close to the good graph in symmetric difference and with boundary at most a specific multiple of  $|U|$ , which is precisely what `exists_rooted_word_radius_sparse_cut_of_boundary` later provides. The proof applies `reference_difference_le_of_good_root_threshold` to bound the distance between  $U$  and the permutation graph, controls  $|U|$  by the vertex count plus that distance, and feeds everything to the arithmetic lemma `rooted_reference_cut_bounds_of_slow_tolerance`, whose three conclusions instantiate the rooted-reference-cut criterion `hasAlmostCentralizerImprovement_of_rooted_reference_cuts` from the `KunThomFiberCoarea` namespace. The conclusion `HasAlmostCentralizerImprovement` is the quantitative rigidity property that the surrounding Kun argument extracts from the Kazhdan spectral gap and ultimately turns against the hypothetical sofic approximation.

*Remaining declarations in this section:*

|                                                   |     |                                                                                                                               |
|---------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------|
| <code>reference_difference_le_symmDiff_</code>    | THM | Set arithmetic: the two-sided difference of $U$ from $W$ is bounded by $ U\Delta T  +  W \setminus T $ when $T \subseteq W$ . |
| <code>add_missing</code>                          |     |                                                                                                                               |
| <code>rooted_reference_cut_bounds_of_slow_</code> | THM | Private real-arithmetic lemma deriving the three almost-centralizer inequalities from slow-tolerance and boundary hypotheses. |
| <code>tolerance</code>                            |     |                                                                                                                               |

## 7.9 Sparse Cuts in Rooted Models (KunRootedWordPower)

The payoff of the rooted-model spectral development. For a Kazhdan pair with generators inside the symmetric set  $S$ , the contraction factor  $q$  of the permutation Markov operator is below one, and rootedness at a sufficiently large word radius makes the contraction usable on concrete finite models. Theorem 1125 supplies a uniform radius after which the  $k$ -th Markov iterate  $f$  of an indicator  $\mathbf{1}_T$  satisfies  $\sum_x (f - \mathbf{1}_T)^2 \leq 8\partial T/(|S|(1-q)^2)$ . Definitional bridges (1126, 1127) identify the two Markov operators in play and express the residual norm as a consecutive-iterate difference. The main theorem 1128 then chooses  $k$  with  $q^k$  small and combines the residual bound, the restricted-variation estimate, and the sharp threshold cut to show: any nonempty  $T$  with boundary below  $\gamma|T|$  admits an upper-level cut  $U$  with  $|U\Delta T| \leq 72\partial T/(|S|(1-q)^2)$ , majority  $3|U\Delta T| < |T|$ , and  $\partial U \leq \alpha|U|$ , the universal sparse-cut engine behind the good-root thresholds.

**`exists_rooted_word_radius_real_markov_sq_error_le_boundary`** thm

```
theorem exists_rooted_word_radius_real_markov_sq_error_le_boundary
  {G : Type u} [Group G]
  (P : SoficGroups.KazhdanPair.{u, u} G)
  (S : Finset G) (hOneS : 1 ∈ S)
  (hcover : P.generators ⊆ S)
  (hsymmetric : ∀ g ∈ S, g-1 ∈ S)
  (w : G → List ιS)
  (k : ℕ) :
  ∃ r : ℕ, ∀ X : RootedIndicatorMarkovModel G ιS,
    X.IsGenerated (fun i : ιS => (i : G)) w →
    X.IsRootedAtRadius w r →
    ∀ T : Finset X.carrier,
      (∀ x : X.carrier,
        X.indicator x = if x ∈ T then (1 : ℝ) else 0) →
      (∑ x : X.carrier,
        (((SoficGroups.KunFinitePermutationMarkovMass.realPermutationMarkov
          X.generator)[k])
          (SoficGroups.KunFinitePermutationMarkovMass.realIndicator T)) x -
        if x ∈ T then (1 : ℝ) else 0) ^ 2) ≤
      8 * (SoficGroups.boundary X.generator T : ℝ) /
      ((S.card : ℝ) *
        (1 - kazhdanMarkovContractionFactor P S) ^ 2) := by
-- proof: 69 lines, omitted
```

The uniform spectral estimate for rooted models. Fix a Kazhdan pair  $P$  with generators inside the finite symmetric set  $S$  containing the identity, and a word assignment  $w$ . For each iteration count  $k$  there is a

radius  $r$  such that for every rooted indicator Markov model over  $S$  that is generated compatibly with  $w$  and rooted at radius  $r$ , with indicator the characteristic function of a set  $T$ , the  $k$ -th iterate  $f$  of the real permutation Markov operator applied to  $\mathbf{1}_T$  satisfies  $\sum_x (f(x) - \mathbf{1}_T(x))^2 \leq 8 \partial T / (|S|(1-q)^2)$ , where  $q$  is the `kazhdanMarkovContractionFactor` and  $\partial T$  is the boundary of  $T$  under the model generators. The proof combines the geometric-series displacement bound `exists_rooted_word_radius_geometric_markov_displacement`, by which rootedness at large radius lets the Kazhdan contraction apply to each Markov step so the total drift of the iterates is at most  $2/(1-q)$  times the first step, with the Jensen-type bound `rooted_indicator_defect_sq_le_boundary`, which controls the first step's squared norm by  $2 \partial T / |S|$ . Squaring and combining gives the constant 8. Crucially the radius is uniform over all models, which is what allows it to be met on the sofic side by the density arguments of the bridge section.

**exists\_rooted\_word\_radius\_sparse\_cut\_of\_boundary** thm

```

theorem exists_rooted_word_radius_sparse_cut_of_boundary
  {G : Type u} [Group G]
  (P : SoficGroups.KazhdanPair.{u, u} G)
  (S : Finset G) (hS : 1 ∈ S)
  (hcover : P.generators ⊆ S)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (w : G → List S)
  (γ : ℝ) (hγ : 0 < γ) (hα : 0 < α)
  (hysmall :
    216 * γ <
      (S.card : ℝ) *
        (1 - kazhdanMarkovContractionFactor P S) ^ 2) :
  ∃ (k r : ℕ), ∀ X : RootedIndicatorMarkovModel G S,
    X.IsGenerated (fun i : S => (i : G)) w →
    X.IsRootedAtRadius w r →
    ∀ T : Finset X.carrier,
      T.Nonempty →
      (∀ x : X.carrier,
        X.indicator x = if x ∈ T then (1 : ℝ) else 0) →
      (SoficGroups.boundary X.generator T : ℝ) <
        γ * (T.card : ℝ) →
    ∃ U : Finset X.carrier,
      ((U Δ T).card : ℝ) ≤
        9 * (∑ x : X.carrier,
          (((SoficGroups.KunFinitePermutationMarkovMass.realPermutationMarkov
            X.generator)^[k])
            (SoficGroups.KunFinitePermutationMarkovMass.realIndicator T)) x -
            if x ∈ T then (1 : ℝ) else 0) ^ 2) ∧
      ((U Δ T).card : ℝ) ≤
        72 * (SoficGroups.boundary X.generator T : ℝ) /
          ((S.card : ℝ) *
            (1 - kazhdanMarkovContractionFactor P S) ^ 2) ∧
      3 * (U Δ T).card < T.card ∧
      (SoficGroups.boundary X.generator U : ℝ) ≤
        α * (U.card : ℝ) := by
-- proof: 201 lines, omitted

```

The culmination of the rooted-model spectral development: a Cheeger-type cut improvement with universal constants. Given a Kazhdan pair with generators in  $S$ , a target boundary ratio  $\alpha > 0$ , and a sparsity scale  $\gamma$  with  $216\gamma < |S|(1-q)^2$ , the theorem produces an iteration count  $k$  and a radius  $r$  such that in any generated model rooted at radius  $r$ , any nonempty indicator set  $T$  with  $\partial T < \gamma|T|$  admits a set  $U$  satisfying  $|U \Delta T| \leq 9 \sum_x (f - \mathbf{1}_T)^2 \leq 72 \partial T / (|S|(1-q)^2)$ , the majority property  $3|U \Delta T| < |T|$ , and  $\partial U \leq \alpha|U|$ . The construction sets  $\eta = 4\alpha^2/(486|S|^2)$ , chooses  $k$  with  $q^k < \eta$ , and takes  $r$  as the maximum of the squared-error radius from 1125 and the iterate-contraction radius. The  $k$ -th Markov iterate  $f$  of  $\mathbf{1}_T$  is  $[0, 1]$ -valued of mass  $|T|$ ; the residual bound feeds `highSupport_variation_le_of_realMarkov_residual`, whose restricted-variation output combines with the threshold cut `exists_upperLevel_boundary_and_symmDiff_le` to define  $U$  as an upper level set of  $f$ . The final boundary estimate uses  $2|T| \leq 3|U|$ , extracted from the majority property. This theorem manufactures the universal good-root threshold cuts consumed by `hasAlmostCentralizerImprovement_of_universal_good_root_thresholds`.

Remaining declarations in this section:

|                                                          |                   |                                                                                                 |
|----------------------------------------------------------|-------------------|-------------------------------------------------------------------------------------------------|
| <code>rootedSparseModelDecidableEq</code>                | <code>INST</code> | Local classical decidable-equality instance for the carriers of rooted sparse models.           |
| <code>rooted_final_realMarkov_eq_mass_real</code>        | <code>THM</code>  | The two definitions of the real permutation Markov averaging operator coincide.                 |
| <code>PermutationMarkov</code>                           |                   |                                                                                                 |
| <code>rooted_real_markov_iterate_residual_sqrt_eq</code> | <code>THM</code>  | The residual norm of the iterated Markov equals the norm of the consecutive iterate difference. |

## 7.10 Retained components after compression discard (SourceCompressionRetainedDiscard)

Given two partitions  $P$  and  $Q$  of a finite vertex set, a permutation  $T$  and an integer rank labeling  $b$ , this segment classifies the parts of  $P$ . `witnesslessComponents` are parts that overlap their dominant  $Q$ -part sufficiently yet contain no overlap point  $y$  with  $b(T^{-1}y) = b(y)$ ; `retainedComponents` are the parts left after discarding these together with the insufficient-overlap components. Retained parts are certified to carry a  $(1 - \eta)$ -dominant overlap containing a rank-preservation witness. The discard is cheap: for  $\eta \leq 1/2$  a witnessless part has at least half its points in the rank-changing arc of  $T^{-1}$ , so the mass missed by the retained support is bounded by the overlap-bad mass plus twice the rank-changing arc, and its density vanishes in the limit. This underlies the source-compression step, where retained witness-carrying components anchor the transport of rank data.

**retainedComponents\_spec** `thm`

```
theorem retainedComponents_spec
  {V : Type*} [Fintype V] [DecidableEq V]
  (P Q : Finpartition (Finset.univ : Finset V))
  (T : Equiv.Perm V) (b : V → ℤ) (eta : ℝ)
  (C : Finset V)
  (hC : C ∈ retainedComponents P Q T b eta) :
  C ∈ P.parts ∧
  SoficGroups.maximumOverlapPart Q C ∈ Q.parts ∧
  (1 - eta) * (C.card : ℝ) ≤
    ((C ∩ SoficGroups.maximumOverlapPart Q C).card : ℝ) ∧
  ∃ y ∈ C ∩ SoficGroups.maximumOverlapPart Q C,
    b (T.symm y) = b y := by
-- proof: 19 lines, omitted
```

`retainedComponents_spec` unpacks membership in `retainedComponents` into the exact interface later constructions consume. Any retained component  $C$  satisfies four facts: it is a part of  $P$ ; its maximum-overlap part  $D$  computed by `SoficGroups.maximumOverlapPart` is genuinely a part of  $Q$ ; the overlap is dominant,  $(1 - \eta)|C| \leq |C \cap D|$ ; and the overlap contains a witness  $y$  with  $b(T^{-1}y) = b(y)$ . The proof is bookkeeping over the definitions: membership in the set difference shows  $C$  is a part avoiding both discard families; avoiding `insufficientOverlapComponents` yields the overlap bound via `maximumOverlapPart_overlap_of_not_mem_insufficient`; and if no witness existed,  $C$  would satisfy the filter condition defining `witnesslessComponents`, a contradiction obtained by push-negation. The significance is that a retained component comes matched to a single dominant part of the second partition together with a concrete point at which the integer rank labeling  $b$  is preserved by  $T^{-1}$ . These witnesses are what allow rank information to be transported across the compression permutation in the source-compression argument, one component at a time.

**retained\_missing\_density\_tendsto\_zero** `thm`

```
theorem retained_missing_density_tendsto_zero
  (V : ℕ → Type*)
  [V n, Fintype (V n)] [V n, DecidableEq (V n)]
  (P Q : (n : ℕ) → Finpartition
    (Finset.univ : Finset (V n)))
  (T : (n : ℕ) → Equiv.Perm (V n))
  (b : (n : ℕ) → V n → ℤ) (eta : ℕ → ℝ)
  (heta : Tendsto eta atTop (nhds 0))
```



```

(hoverlap : Tendsto
  (fun n =>
    (( $\sum$  C  $\in$  SoficGroups.insufficientOverlapComponents
      (P n) (Q n) (eta n), C.card :  $\mathbb{N}$ ) :  $\mathbb{R}$ ) /
      Fintype.card (V n))
    atTop (nhds 0))
(hrank : Tendsto
  (fun n =>
    ((SoficGroups.RankArcCharging.rankChangingArc
      (Finset.univ : Finset (V n)) (b n) (T n).symm).card :  $\mathbb{R}$ ) /
      Fintype.card (V n))
    atTop (nhds 0)) :
Tendsto
  (fun n =>
    (((Finset.univ : Finset (V n)) \
      SoficGroups.matchedRetainedSupport
        (retainedComponents (P n) (Q n) (T n) (b n)
          (eta n))).card :  $\mathbb{R}$ ) / Fintype.card (V n))
    atTop (nhds 0)) := by
-- proof: 55 lines, omitted

```

The limit theorem for the whole segment. For sequences of finite vertex types  $V_n$ , partitions  $P_n, Q_n$ , permutations  $T_n$ , labelings  $b_n$ , and tolerances  $\eta_n \rightarrow 0$ , assume the density of the total mass of insufficientOverlapComponents and the density of the rank-changing arc RankArcCharging.rankChangingArc both tend to zero. Then the density of points outside matchedRetainedSupport of the retained components tends to zero. The quantitative input is retained\_missing\_card\_le\_overlap\_add\_twice\_rankChanging: the missing points are exactly the mass of the discarded parts, which splits into insufficient-overlap parts plus witnessless parts; by witnessless\_component\_card\_le\_twice\_rankChanging, once  $\eta \leq 1/2$  a witnessless part has overlap at least  $|C|/2$  consisting entirely of rank-changing points, so its size is at most twice its rank-changing intersection, and summing against the partition gives the factor-two bound. The limit statement then follows by a squeeze once  $\eta_n < 1/2$  eventually holds. This guarantees the compression-retained decomposition covers asymptotically all points of the sofic model, so subsequent estimates carried out on retained components lose nothing in density.

*Remaining declarations in this section:*

|                                                         |     |                                                                                                                    |
|---------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------|
| witnesslessComponents                                   | DEF | Parts of P with adequate Q-overlap whose overlap contains no point where $b$ is preserved by $T^{-1}$ .            |
| retainedComponents                                      | DEF | Parts of P surviving removal of the insufficient-overlap and witnessless components.                               |
| witnesslessComponents_subset_parts                      | THM | Witnessless components form a subset of the parts of P.                                                            |
| overlapBad_disjoint_witnesslessComponents               | THM | The insufficient-overlap and witnessless component families are disjoint.                                          |
| witnessless_component_card_le_twice_rankChanging        | THM | A witnessless component has at most twice as many points as its rank-changing intersection, when $\eta \leq 1/2$ . |
| witnesslessComponents_mass_le_twice_rankChanging        | THM | Total cardinality of the witnessless components is at most twice the rank-changing arc's cardinality.              |
| retained_missing_card_le_overlap_add_twice_rankChanging | THM | Points outside the retained support are bounded by insufficient-overlap mass plus twice the rank-changing arc.     |

## 7.11 Crossing densities of generated words (SourceGeneratedWordCrossing)

This segment controls partitionWordCrossing, the set of model points whose part in a partition changes under a permutation. Crossing counts are Lipschitz in permutation distance, subadditive under products, and sum over the generators to the total partition boundary. Consequently, if the boundary density of a partition sequence vanishes, then so does the crossing density of each generator, of every word product of generators, and, using that a sofic approximation evaluates products of group elements asymptotically correctly (action\_list\_prod\_tendsto), of the model action of any fixed element of the subgroup generated by a symmetric finite set. The segment closes by defining sourceAlphaInclusion, the inclusion of the alpha-prefix elementary group

into the nine-prefix elementary group, and specializing: partitions almost invariant under the included alpha generators are almost invariant under the action of every alpha element.

**action\_list\_prod\_tendsto** thm

```
theorem action_list_prod_tendsto
  {G : Type*} [Group G]
  (A : SoficGroups.SoficApproximation G) (l : List G) :
  Tendsto
    (fun n =>
      SoficGroups.normalizedHamming
        ((A.model n).action l.prod)
        ((l.map (A.model n).action).prod))
    atTop (nhds 0) := by
  -- proof: 27 lines, omitted
```

A `SoficGroups.SoficApproximation` is only asymptotically multiplicative: for each pair  $g, h$  the normalized Hamming distance between the action of  $gh$  and the product of the actions tends to zero. `action_list_prod_tendsto` iterates this along an arbitrary list  $l$  of group elements: the distance between the action of the product of  $l$  and the product of the individual letter actions tends to zero. The proof is induction on the list; the `cons` step compares the action of  $g$  times the tail product first to the product of the action of  $g$  with the action of the tail product (the multiplicativity hypothesis of the approximation) and then to the product of the letter actions (the inductive hypothesis), using the triangle inequality and left invariance `normalizedHamming_mul_left` to combine the two errors. This is the basic mechanism that lets word-based, local arguments run inside the finite models of a hypothetical sofic approximation: any identity between words in the group holds in the model up to a vanishing fraction of points, so word evaluations may be substituted for actions of products throughout the segment.

**fixed\_generated\_word\_crossing\_density\_tendsto\_zero** thm

```
theorem fixed_generated_word_crossing_density_tendsto_zero
  {G H : Type*} [Group G] [Group H]
  (A : SoficGroups.SoficApproximation G)
  (φ : H →* G) (S : Finset H)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set H) = T)
  (Q : ∀ n, Finpartition
    (Finset.univ : Finset (Fin (A.model n).size)))
  (hboundary : Tendsto
    (fun n =>
      (∑ C ∈ (Q n).parts,
        (SoficGroups.boundary
          (fun i : iS => (A.model n).action (φ (i : H))) C : ℝ)) /
        (A.model n).size)
    atTop (nhds 0))
  (g : H) :
  Tendsto
    (fun n =>
      ((SoficGroups.partitionWordCrossing (Q n)
        ((A.model n).action (φ g))).card : ℝ) /
        (A.model n).size)
    atTop (nhds 0) := by
  -- proof: 55 lines, omitted
```

The central theorem of the segment. Setting: a sofic approximation  $A$  of  $G$ , a homomorphism  $\varphi : H \rightarrow G$ , a symmetric finite set  $S$  generating  $H$ , and a partition sequence  $Q_n$  of the model points whose total boundary density with respect to the generator actions (the actions of  $\varphi(i)$  for  $i \in S$ ) tends to zero. Conclusion: for every fixed  $g \in H$ , the density of points that change  $Q_n$ -part under the action of  $\varphi(g)$  tends to zero. The proof chains the segment's lemmas: `exists_word_of_symmetric_generators` writes  $g$  as a product of a list of letters of  $S$  by closure induction; `generator_crossing_density_tendsto_zero` extracts vanishing crossing density for each generator from the boundary hypothesis via `sum_generator_crossing_eq_sum_partition_boundary`; `list_generator_crossing_density_tendsto_zero` handles the word product by subadditivity; and `action_list_prod`

tendsto together with the Lipschitz comparison `crossing_density_tendsto_zero_of_normalizedHamming` transfers the estimate from the product of letter actions to the action of  $\varphi(g)$  itself. This upgrades almost-invariance of the expander-decomposition partitions from the finitely many generators to every element of the generated subgroup, which is what the overlap and transport arguments downstream require.

*Remaining declarations in this section:*

|                                                                 |     |                                                                                                                                |
|-----------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------|
| <code>mem_partitionWordCrossing_univ</code>                     | THM | A point crosses under $p$ exactly when its $Q$ -part differs from that of its image.                                           |
| <code>card_partitionWordCrossing_le_add_distance</code>         | THM | Crossing count of $p$ exceeds that of $q$ by at most their permutation distance.                                               |
| <code>card_partitionWordCrossing_mul_le</code>                  | THM | Crossing count of a product is at most the sum of the factors' crossing counts.                                                |
| <code>sum_generator_crossing_eq_sum_partition_boundary</code>   | THM | Summing generator crossing counts over generators equals summing <code>SoficGroups.boundary</code> over the partition's parts. |
| <code>generator_crossing_density_tendsto_zero</code>            | THM | Vanishing total boundary density forces each generator's crossing density to vanish.                                           |
| <code>list_generator_crossing_density_tendsto_zero</code>       | THM | Crossing density of any fixed word product of the generators tends to zero.                                                    |
| <code>exists_word_of_symmetric_generators</code>                | THM | Every element of a group generated by a symmetric finite set is a product of a list of generators.                             |
| <code>crossing_density_tendsto_zero_of_normalizedHamming</code> | THM | Vanishing crossing density transfers between permutation sequences at vanishing normalized Hamming distance.                   |
| <code>sourceAlphaInclusion</code>                               | DEF | Inclusion homomorphism from the alpha-prefix elementary group into the nine-prefix elementary group.                           |
| <code>source_alpha_word_crossing_density_tendsto_zero</code>    | THM | Specializes the fixed-element crossing theorem to alpha-prefix elements via <code>sourceAlphaInclusion</code> .                |

## 7.12 Slow tolerance numeric inequalities (KunRootedUniversalToleranceNumerics)

Two purely arithmetic real-number lemmas isolating the numeric bookkeeping of the Kun rooted-graph argument. `rooted_graph_boundary_lt_of_slow_tolerance` shows that if the bad set satisfies  $2db \leq t$ , the good graph has size at least  $N - 2b$ , its boundary is at most  $3t$ , and the tolerance obeys the slow-growth inequality  $5(4 + h(216/(d(1-q)^2) + 1/d))t \leq hN$ , then the boundary is strictly smaller than  $\gamma c$  for the coefficient  $\gamma = d(1-q)^2/288$ . `rooted_graph_coefficient_small` records that this  $\gamma$  is positive and satisfies  $216\gamma < d(1-q)^2$ , exactly the smallness the sparse-cut existence theorem requires. Both are consumed verbatim by the uniform-radius almost-centralizer improvement theorem, which uses them to verify the sparsity hypothesis of the diagonal sparse-cut machinery.

**`rooted_graph_boundary_lt_of_slow_tolerance` thm**

```
theorem rooted_graph_boundary_lt_of_slow_tolerance
  {h d q t N b c e : ℝ}
  (hh : 0 < h) (hd : 0 < d) (hq : q < 1)
  (ht : 0 < t)
  (hbad : 2 * d * b ≤ t)
  (hmissing : N - 2 * b ≤ c)
  (hboundary : e ≤ 3 * t)
  (hslow :
    5 * (4 + h *
      (216 / (d * (1 - q) ^ 2) + 1 / d)) * t ≤ h * N) :
  e < (d * (1 - q) ^ 2 / 288) * c := by
-- proof: 48 lines, omitted
```

A quantifier-free real inequality with named parameters:  $h$  an expansion constant,  $d$  the generator count,  $q < 1$  a Markov contraction factor,  $t$  the tolerance,  $N$  the model size,  $b$  the bad-set size,  $c$  the

good-graph size, and  $e$  its boundary. From  $2db \leq t$ ,  $N - 2b \leq c$ ,  $e \leq 3t$ , and the slow-tolerance hypothesis  $5(4 + h(216/(d(1-q)^2) + 1/d))t \leq hN$ , it concludes  $e < (d(1-q)^2/288)c$ . The proof abbreviates  $\delta = d(1-q)^2$ ,  $\gamma = \delta/288$ ,  $C = 216/\delta + 1/d$ , cancels  $h$  to obtain  $5Ct \leq N$ , converts the bad-set bound into  $N - t/d \leq c$ , and finishes with the exact algebraic identity  $\gamma(5Ct) - \gamma(t/d) = (15/4)t + 4(\gamma/d)t$ , which yields  $3t < \gamma c$ ; the boundary bound  $e \leq 3t$  then gives the strict inequality. The point of the specific constants:  $\gamma$  is precisely the sparse-cut coefficient of the Kun machinery, so this lemma certifies that whenever the tolerance grows slowly relative to the model size, the good permutation graph's boundary is  $\gamma$ -sparse and the sparse-cut theorem applies.

*Remaining declarations in this section:*

|                                |     |                                                                                       |
|--------------------------------|-----|---------------------------------------------------------------------------------------|
| rooted_graph_coefficient_small | THM | The coefficient $d(1-q)^2/288$ is positive, and 216 times it stays below $d(1-q)^2$ . |
|--------------------------------|-----|---------------------------------------------------------------------------------------|

### 7.13 Compression elements normalize alpha subgroup (SourceBothCompressionNormalization)

Packages the two compression elements as members of the nine-prefix elementary group and proves they normalize the embedded alpha-prefix subgroup. `sourceCompressionUElement` and `sourceCompressionVElement` wrap `compressionU` and `compressionV`; `sourceAlphaElement` includes an alpha-prefix element into the nine-prefix group. The constructions `sourceConjugatedAlphaUElement` and `sourceConjugatedAlphaVElement` show that conjugating an alpha element by either compression element lands back in the alpha-prefix elementary group, using the previously established compression conjugation images and the containment of the alpha-zero group in the alpha group. Finally `sourceCompressionTable` lists the two elements as a family indexed by `Fin 2`, and the closing theorem states that each table entry conjugates every included alpha element to another included alpha element, the normalization later exploited when the ambient sofic model is conjugated by the actions of these elements.

#### `sourceCompressionTable_conjugates_alpha` thm

```
theorem sourceCompressionTable_conjugates_alpha
  (i : Fin 2)
  (g : SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode) :
  ∃ k : SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode,
    sourceCompressionTable i * sourceAlphaElement g *
      (sourceCompressionTable i)-1 =
        sourceAlphaElement k := by
  -- proof: 6 lines, omitted
```

`sourceCompressionTable` is the pair of compression elements viewed inside `prefixElementaryGroup` applied to `ninePrefixCode`, and this theorem asserts that each of them normalizes the embedded alpha subgroup: for every alpha-prefix element  $g$  there is an alpha-prefix element  $k$  with  $\text{table}(i) \cdot \iota(g) \cdot \text{table}(i)^{-1} = \iota(k)$ , where  $\iota$  is `sourceAlphaElement`. The proof is a two-case split reducing to `sourceCompressionUElement_conjugates_alpha` and its  $V$  analogue, both proved by `Subtype.ext` and definitional equality; the substance sits in the constructions `sourceConjugatedAlphaUElement` and `sourceConjugatedAlphaVElement`, which use `compressionU_map_alphaPrefixElementaryGroup` (conjugation by the compression carries the alpha subgroup into the alpha-zero subgroup) and `alphaZero_prefixElementaryGroup_le` to land back inside the alpha-prefix elementary group. This normalization is what makes the doubling trick work downstream: the ambient sofic model is conjugated by the model actions of the two compression elements, and the transported-overlap machinery (with transport index `Fin 2`) needs the conjugated alpha subgroup to remain within the alpha subgroup so that its almost-invariant partitions can be compared against the original one.

*Remaining declarations in this section:*

|                                        |     |                                                                                       |
|----------------------------------------|-----|---------------------------------------------------------------------------------------|
| <code>sourceCompressionUElement</code> | DEF | Packages <code>compressionU</code> as an element of the nine-prefix elementary group. |
| <code>sourceCompressionVElement</code> | DEF | Packages <code>compressionV</code> as an element of the nine-prefix elementary group. |

|                                                         |     |                                                                                                                                     |
|---------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>sourceAlphaElement</code>                         | DEF | Includes an alpha-prefix element into the nine-prefix elementary group.                                                             |
| <code>sourceConjugatedAlphaUElement</code>              | DEF | Conjugation of an alpha element by <code>compressionU</code> , landing back in the alpha-prefix elementary group.                   |
| <code>sourceConjugatedAlphaVElement</code>              | DEF | Conjugation of an alpha element by <code>compressionV</code> , landing back in the alpha-prefix elementary group.                   |
| <code>sourceCompressionUElement_conjugates_alpha</code> | THM | Conjugating <code>sourceAlphaElement</code> by <code>sourceCompressionUElement</code> equals the included conjugated alpha element. |
| <code>sourceCompressionVElement_conjugates_alpha</code> | THM | Conjugating <code>sourceAlphaElement</code> by <code>sourceCompressionVElement</code> equals the included conjugated alpha element. |
| <code>sourceCompressionTable</code>                     | DEF | Length-two table listing the two compression elements.                                                                              |

## 7.14 Diagonal rooted model sparse cuts (KunCompletedProductDiagonalSparseCut)

Builds the rooted-model interface on the product space  $V \times V$  and derives the sparse-cut theorem for good permutation graphs. `completedDiagonalPermutationHom` sends  $p$  to  $p \times p$ ; `completedGoodPermutationGraphRootedModel` equips  $V \times V$  with diagonal generators, the zero-one indicator of `goodPermutationGraph` (pairs in the graph of an almost-commuting permutation whose coordinates avoid the bad set  $B$ ), and the diagonal evaluation of a word-generated map  $\varphi$ . Two verification lemmas check the generation axioms and rootedness at radius  $r$  from local multiplicativity outside  $B$ . The main theorem produces, from a Kazhdan pair and the smallness  $216\gamma < |S|(1-q)^2$ , a uniform radius such that any nonempty good permutation graph with boundary below  $\gamma|T|$  admits a nearby cut  $U$  with quantitatively small symmetric difference and boundary at most  $\alpha|U|$ .

**completedGoodPermutationGraphRootedModel** `def`

```
def completedGoodPermutationGraphRootedModel
  {G : Type u} [Fintype V] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (φ : G → Equiv.Perm V)
  (p : Equiv.Perm V) (B : Finset V) :
  RootedIndicatorMarkovModel G ι where
  carrier := V × V
  fintype := inferInstance
  generator i := completedDiagonalPermutationHom (σ i)
  indicator := completedGoodPermutationGraphIndicator
    (goodPermutationGraph p B)
  evaluation g := completedDiagonalPermutationHom (φ g)
```

This definition packages the data needed to run the Kun rooted spectral machinery on the product space. The carrier is  $V \times V$ ; each generator  $i$  acts diagonally by  $\sigma_i \times \sigma_i$  through `completedDiagonalPermutationHom`; the real indicator is the characteristic function of `goodPermutationGraph`, the graph of an almost-commuting permutation  $p$  restricted to pairs whose coordinates avoid the bad set  $B$ ; and the evaluation of a group element  $g$  is the diagonal action of  $\varphi(g)$ . The companion lemmas verify the two interfaces. `completedGoodPermutationGraphRootedModel_isGenerated` checks the indicator is Boolean and the evaluation is exactly the word product of the diagonal generators, with correct unit and generator compatibility, pushed through the homomorphism property of the diagonal map. `completedGoodPermutationGraphRootedModel_isRootedAtRadius` shows that when  $\varphi$  is multiplicative outside  $B$  for all words of combined length at most  $r$ , the evaluation is multiplicative at every point where the indicator is nonzero, coordinate by coordinate, since such points have both coordinates outside  $B$ . The construction reduces the problem of improving an almost-centralizing permutation to a statement about a rooted indicator Markov model, where the property (T) spectral estimates apply.

**exists\_completed\_goodPermutationGraph\_sparse\_cut** `thm`

```
theorem exists_completed_goodPermutationGraph_sparse_cut
  {G : Type u} [Group G]
  (P : SoficGroups.KazhdanPair.{u, u} G)
  (S : Finset G) (honeS : 1 ∈ S)
```

```

(hcover : P.generators ≤ S)
(hsymmetric : ∀ g ∈ S, g-1 ∈ S)
(w : G → List ιS)
(γ α : ℝ) (hγ : 0 < γ) (hα : 0 < α)
(hysmall :
  216 * γ <
    (S.card : ℝ) *
      (1 - kazhdanMarkovContractionFactor P S) ^ 2) :
∃ r : ℕ,
  ∀ (V : Type u) [Fintype V] [DecidableEq V]
    (σ : ιS → Equiv.Perm V) (φ : G → Equiv.Perm V),
  (∀ g : G, φ g = (w g).map σ).prod →
  φ 1 = 1 →
  (∀ i : ιS, φ (i : G) = σ i) →
  ∀ (p : Equiv.Perm V) (B : Finset V),
  (∀ a g : G,
    (w a).length + (w g).length + (w (a * g)).length ≤ r →
    ∀ x : V, x ∉ B →
      φ (a * g) x = (φ a * φ g) x) →
  let T := goodPermutationGraph p B
  T.Nonempty →
  (SoficGroups.boundary
    (fun i : ιS => (σ i).prodCongr (σ i)) T : ℝ) <
    γ * (T.card : ℝ) →
  ∃ U : Finset (V × V),
    (((U Δ T).card : ℝ) ≤
      72 * (SoficGroups.boundary
        (fun i : ιS => (σ i).prodCongr (σ i)) T : ℝ) /
        ((S.card : ℝ) *
          (1 - kazhdanMarkovContractionFactor P S) ^ 2) ∧
      3 * (U Δ T).card < T.card ∧
      (SoficGroups.boundary
        (fun i : ιS => (σ i).prodCongr (σ i)) U : ℝ) ≤
        α * (U.card : ℝ) := by
-- proof: 57 lines, omitted

```

The segment's main theorem. Given a Kazhdan pair  $P$  for  $G$ , a symmetric finite set  $S$  containing the identity and covering the Kazhdan generators, chosen words  $w$ , and constants  $\gamma, \alpha > 0$  with  $216\gamma < |S|(1-q)^2$ , where  $q$  is `kazhdanMarkovContractionFactor`, there is a uniform word radius  $r$  with the following property in every finite model: if  $\varphi$  is the word evaluation of the generators  $\sigma$ , multiplicative outside a bad set  $B$  for words of combined length at most  $r$ , and the good permutation graph  $T$  of a permutation  $p$  is nonempty with diagonal boundary below  $\gamma|T|$ , then there is  $U \subseteq V \times V$  with  $|U \Delta T|$  at most 72 times the boundary divided by  $|S|(1-q)^2$ , with  $3|U \Delta T| < |T|$ , and with boundary at most  $\alpha|U|$ . The proof instantiates the completed rooted model of the preceding declarations and applies `exists_rooted_word_radius_sparse_cut_of_boundary`; the body is largely decidability-instance management and translation of the model-level conclusion back to concrete finsets. This is the property (T) cut-cleaning device consumed by the uniform-radius improvement theorem in the next Kun segment.

*Remaining declarations in this section:*

|                                                                        |     |                                                                                                          |
|------------------------------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------|
| <code>completedDiagonalPermutationHom</code>                           | DEF | Monoid homomorphism sending a permutation $p$ to the diagonal permutation $p \times p$ of $V \times V$ . |
| <code>completedGoodPermutationGraphIndicator</code>                    | DEF | Real-valued zero-one indicator function of a finite subset of $V \times V$ .                             |
| <code>completedGoodPermutationGraphRootedModel_isGenerated</code>      | THM | The completed diagonal model satisfies the word-generation axioms of the rooted indicator interface.     |
| <code>completedGoodPermutationGraphRootedModel_isRootedAtRadius</code> | THM | Local multiplicativity outside the bad set yields rootedness at radius $r$ for the completed model.      |

## 7.15 Transported partitions and overlap losses (KunTransportedAmbientOverlap)

Compares an expanding partition with a second partition and with its own transports by conjugating permutations. First, refinement boundaries: points exiting an intersection  $C \cap D$  exit  $C$  or exit  $D$ , so the boundary of the common refinement is bounded by the two partitions' boundary sums. Under a  $\gamma$ -expansion hypothesis for small subsets of parts, each part's deficit from its dominant overlap part converts into refinement boundary, so the total deficit density vanishes when both boundary densities do: an expanding partition is asymptotically refined by any low-boundary partition. Conjugation invariance of the boundary shows transported partitions inherit expansion and boundary bounds. The final theorem manufactures common antitone vanishing tolerance scales  $\eta$  and  $H$  with  $\eta/H \rightarrow 0$  under which the insufficient-overlap component mass of every transported partition against the original vanishes, uniformly over a finite family of transports.

**dominant\_component\_loss\_density\_tendsto\_zero** *thm*

```

theorem dominant_component_loss_density_tendsto_zero
  {V : ℕ → Type*}
  [V n, Fintype (V n)] [V n, Nonempty (V n)]
  [V n, DecidableEq (V n)]
  {ι : Type*} [Fintype ι]
  (P Q : V n, Finpartition (Finset.univ : Finset (V n)))
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))
  (gamma : ℝ) (hgamma : 0 < gamma)
  (hexp : ∀ n C, C ∈ (P n).parts →
    ∀ E : Finset (V n), E ⊆ C →
      2 * E.card ≤ C.card →
        gamma * (E.card : ℝ) ≤
          (SoficGroups.boundary (σ n) E : ℝ))
  (hsource : Tendsto
    (fun n =>
      (∑ C ∈ (P n).parts,
        (SoficGroups.boundary (σ n) C : ℝ)) /
        Fintype.card (V n))
    atTop (nhds 0))
  (htarget : Tendsto
    (fun n =>
      (∑ D ∈ (Q n).parts,
        (SoficGroups.boundary (σ n) D : ℝ)) /
        Fintype.card (V n))
    atTop (nhds 0)) :
  Tendsto
    (fun n =>
      (∑ C ∈ (P n).parts,
        ((C.card : ℝ) -
          ((C n SoficGroups.maximumOverlapPart (Q n) C).card : ℝ))) /
        Fintype.card (V n))
    atTop (nhds 0) := by
  -- proof: 57 lines, omitted

```

For partition sequences  $P_n$  (expanding) and  $Q_n$  on finite vertex types, this theorem shows the averaged dominant-overlap deficit  $\sum_{C \in P_n} (|C| - |C \cap D_C|)$ , where  $D_C$  is the maximum overlap part of  $C$  in  $Q_n$ , normalized by  $|V_n|$ , tends to zero. Hypotheses: the expansion property (every subset  $E$  of a part  $C$  with  $2|E| \leq |C|$  has boundary at least  $\gamma|E|$ ) and vanishing boundary densities for both partition sequences. The finitary core is `dominant_component_loss_le_ambient_intersection_boundaries`: within an expanding part  $C$ , every intersection with a non-dominant part of  $Q$  occupies at most half of  $C$ , because it is disjoint from the maximal intersection, so expansion converts its cardinality into refinement boundary; summing over the non-dominant parts yields  $\gamma(|C| - |C \cap D_C|) \leq \sum_E \partial(C \cap E)$ , and `sum_intersection_boundary_le_partition_boundaries` bounds all refinement boundaries by the two partitions' boundary sums. The statement expresses that an expanding partition is asymptotically refined by any low-boundary partition: each part sits almost entirely inside a single part of the other partition. This alignment principle drives the comparison of a partition with its conjugated transports in the following declarations.



## exists\_common\_slow\_overlap\_scales\_for\_transport partitions thm

```

theorem exists_common_slow_overlap_scales_for_transport partitions
  {V : ℕ → Type*}
  [V n, Fintype (V n)] [V n, Nonempty (V n)]
  [V n, DecidableEq (V n)]
  {ι κ : Type*} [Fintype ι] [Fintype κ]
  (Q : V n, Finpartition (Finset.univ : Finset (V n)))
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))
  (T : (n : ℕ) → κ → Equiv.Perm (V n))
  (gamma : ℝ) (hgamma : 0 < gamma)
  (hexp : V n C, C ∈ (Q n).parts →
    V E : Finset (V n), E ⊆ C →
      2 * E.card ≤ C.card →
        gamma * (E.card : ℝ) ≤
          (SoficGroups.boundary (σ n) E : ℝ))
  (hsource : Tendsto
    (fun n =>
      (Σ C ∈ (Q n).parts,
        (SoficGroups.boundary (σ n) C : ℝ)) /
        Fintype.card (V n))
    atTop (nhds 0))
  (htarget : V j : κ, Tendsto
    (fun n =>
      (Σ D ∈ (Q n).parts,
        (SoficGroups.boundary
          (fun i => T n j * σ n i * (T n j)-1 D : ℝ)) /
          Fintype.card (V n))
        atTop (nhds 0)) :
    ∃ eta H : ℕ → ℝ,
      (V n, 0 < eta n) ∧
        Antitone eta ∧
        Tendsto eta atTop (nhds 0) ∧
        (V n, 0 < H n) ∧
        Antitone H ∧
        Tendsto H atTop (nhds 0) ∧
        Tendsto (fun n => eta n / H n) atTop (nhds 0) ∧
        V j : κ, Tendsto
          (fun n =>
            (Σ C ∈ SoficGroups.insufficientOverlapComponents
              (SoficGroups.transportedUnivFinpartition
                (Q n) (T n j))
            )
          )
  -- statement truncated; proof: 90 lines, omitted

```

The culmination of the segment. Inputs: an expanding partition sequence  $Q_n$  with coefficient  $\gamma$  and vanishing boundary density, and a finite family of transport permutations  $T_n^j$  whose conjugated boundary densities also vanish. Output: two sequences  $\eta, H$  of positive reals, antitone, both tending to zero, with  $\eta_n/H_n \rightarrow 0$ , such that for every index  $j$  the mass density of `insufficientOverlapComponents` of the transported partition `transportedUnivFinpartition` of  $Q_n$  along  $T_n^j$ , measured against  $Q_n$  at tolerance  $\eta_n$ , tends to zero. Construction: for each  $j$ , define the loss  $L_j(n)$  as the dominant-overlap deficit density of the transported partition; it vanishes by `dominant_component_loss_density_tendsto_zero`, whose hypotheses are supplied by `transportedUnivFinpartition_half_expansion` (transported partitions inherit expansion for the conjugated generators) and `transported_partition_boundary_density_tendsto_zero`. Summing over  $j$  gives one vanishing error sequence  $e$ , which is fed to `exists_positive_antitone_slow_overlap_scales`; a Markov-type bound (`insufficientOverlapComponents_mass_le_loss`) completes the squeeze. The common schedule matters: the retained-component analysis is applied simultaneously to both compression conjugations, and all instances must share one tolerance sequence.

*Remaining declarations in this section:*

|                                                  |     |                                                                                            |
|--------------------------------------------------|-----|--------------------------------------------------------------------------------------------|
| <code>intersection_generator_exit_subset_</code> | THM | Points exiting an intersection $C \cap D$ under $p$ exit $C$ or exit $D$ .                 |
| <code>union</code>                               |     |                                                                                            |
| <code>sum_intersection_generator_exit_</code>    | THM | Total exit counts of the common refinement are bounded by the two partitions' exit counts. |
| <code>card_le</code>                             |     |                                                                                            |

|                                                                                          |     |                                                                                                   |
|------------------------------------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------|
| <code>sum_intersection_boundary_le_</code><br><code>partition_boundaries</code>          | THM | Boundary summed over the common refinement is bounded by the two partitions' boundary sums.       |
| <code>dominant_component_loss_le_ambient_</code><br><code>intersection_boundaries</code> | THM | Expansion converts a part's deficit from its dominant overlap into refinement boundary mass.      |
| <code>sum_dominant_component_losses_le_</code><br><code>partition_boundaries</code>      | THM | Sums the dominant-overlap deficits over all parts against both partitions' boundary sums.         |
| <code>boundary_conjugate_map</code>                                                      | THM | Boundary is unchanged when generators are conjugated and the set is transported by $T$ .          |
| <code>transportedUnivFinpartition_half_</code><br><code>expansion</code>                 | THM | The transported partition inherits $\gamma$ -expansion with respect to the conjugated generators. |
| <code>sum_boundary_transportUniv</code><br><code>Finpartition</code>                     | THM | Boundary sum of the transported partition equals the original partition's boundary sum.           |
| <code>transported_partition_boundary_</code><br><code>density_tendsto_zero</code>        | THM | Boundary density of transported partitions vanishes whenever the original boundary density does.  |

## 7.16 Uniform radius centralizer improvement (KunUniformCompletedRootRadiusImprovement)

A single omnibus theorem, `exists_uniform_radius_hasAlmostCentralizerImprovement`: given a Kazhdan pair  $P$ , a symmetric generating superset  $S$  containing the identity, chosen words  $w$ , an expansion constant  $h > 0$  and a target  $\alpha > 0$ , there is a word radius  $r$  valid for every finite model. Whenever the generators expand all subsets at rate  $h$ , the tolerance obeys the slow-growth inequality, and the model is exactly multiplicative outside a small bad set  $B$  for words up to radius  $r$ , the conclusion `SoficGroups.HasAlmostCentralizerImprovement` holds at that tolerance. The proof instantiates the diagonal sparse-cut theorem with the coefficient  $\gamma = |S|(1-q)^2/288$  from the numeric lemmas, checks via the slow-tolerance inequality that any almost-commuting permutation's good graph has  $\gamma$ -sparse boundary, and packages the resulting cut through the universal good-root threshold criterion.

**`exists_uniform_radius_hasAlmostCentralizerImprovement`** thm

```

theorem exists_uniform_radius_hasAlmostCentralizerImprovement
  {G : Type u} [Group G]
  (P : SoficGroups.KazhdanPair.{u, u} G)
  (S : Finset G) (honeS : 1 ∈ S)
  (hcover : P.generators ≤ S)
  (hsymmetric : ∀ g ∈ S, g-1 ∈ S)
  (w : G → List ιS)
  (h α : ℝ) (hpositive : 0 < h) (hα : 0 < α) :
  ∃ r : ℕ,
    ∀ (V : Type u) [Fintype V] [Nonempty V] [DecidableEq V]
      (σ : ιS → Equiv.Perm V) (φ : G → Equiv.Perm V)
      (tolerance : ℕ) (B : Finset V),
      (∀ g : G, φ g = ((w g).map σ).prod) →
      φ 1 = 1 →
      (∀ i : ιS, φ (i : G) = σ i) →
      (∀ A : Finset V,
        h * min (A.card : ℝ)
          ((Fintype.card V : ℝ) - A.card) ≤
          (SoficGroups.boundary σ A : ℝ)) →
      (5 * (4 + h *
        (216 / ((S.card : ℝ) *
          (1 - kazhdanMarkovContractionFactor P S) ^ 2) +
          1 / (S.card : ℝ))) * (tolerance : ℝ) ≤
        h * (Fintype.card V : ℝ)) →
      (tolerance = 0 ∨
        (0 < (tolerance : ℝ) ∧
          α ≤ h * (tolerance : ℝ) /
            (2 * (h + 8 * (S.card : ℝ)) *
              (Fintype.card V : ℝ)) ∧
            2 * (S.card : ℝ) * (B.card : ℝ) ≤

```

```

    (tolerance : ℝ) ∧
    ∀ a g : G,
    (w a).length + (w g).length +
    (w (a * g)).length ≤ r →
    ∀ x : V, x ∉ B →
    φ (a * g) x = (φ a * φ g x) →
    SoficGroups.HasAlmostCentralizerImprovement σ tolerance := by
-- proof: 113 lines, omitted

```

The only theorem of this namespace, and the payoff of the surrounding Kun development. Given a Kazhdan pair  $P$ , a symmetric generating superset  $S$  containing 1 and covering the Kazhdan generators, chosen words  $w$ , an expansion constant  $h > 0$ , and a target  $\alpha > 0$ , it produces a word radius  $r$  that works uniformly for all finite models: whenever (i)  $\varphi$  is the word evaluation of generators  $\sigma$  with correct unit and generator values, (ii) the generators expand every subset  $A$  at rate  $h \min(|A|, N - |A|)$ , (iii) the tolerance  $t$  obeys the slow-growth inequality  $5(4 + h(216/(|S|(1 - q)^2) + 1/|S|))t \leq hN$ , and (iv) either  $t = 0$  or  $t > 0$  with  $\alpha$  below  $ht/(2(h + 8|S|)N)$ , the bad set satisfying  $2|S||B| \leq t$ , and  $\varphi$  multiplicative outside  $B$  at radius  $r$ , then `SoficGroups.HasAlmostCentralizerImprovement` holds for  $\sigma$  at tolerance  $t$ . The proof takes  $\gamma = |S|(1 - q)^2/288$ , positive and small by `rooted_graph_coefficient_small`, obtains the sparse-cut radius from `exists_completed_good PermutationGraph_sparse_cut`, and for any permutation  $p$  with commutation defect at most  $2t$  shows its good graph is large, has boundary at most  $3t$ , and, by `rooted_graph_boundary_lt_of_slow_tolerance`, is  $\gamma$ -sparse; the resulting cut  $U$  is then packaged through `hasAlmostCentralizerImprovement_of_universal_good_root_thresholds`. This is the spectral-gap engine later turned against a hypothetical sofic approximation of the source group.

## 7.17 Chosen word evaluations converge (CompletedChosenWordLocalRoot)

Develops word-evaluation permutations that are exactly word products yet asymptotically indistinguishable from an approximate action. The distance bound `normalizedHamming_mul_le` gives factorwise control of products, whence letterwise-convergent permutation lists have convergent products, and any asymptotically multiplicative family  $p_n$  evaluates list products correctly in the limit. `chosenWordEvaluation` is the product of the generator permutations along the chosen word of  $g$ ; if the generators converge to  $p_n(s i)$  letterwise and the words evaluate to their elements in the group, the evaluation converges pointwise to  $p_n(g)$  and is itself asymptotically multiplicative. This supplies the exact word-generated evaluation  $\varphi$  demanded by the rooted-model hypotheses while preserving all the asymptotic estimates of the original sofic approximation.

### `chosenWordEvaluation_multiplicative_tendsto` thm

```

theorem chosenWordEvaluation_multiplicative_tendsto
  {G ι : Type*} [Group G]
  (V : ℕ → Type*)
  [V n, Fintype (V n)] [V n, DecidableEq (V n)]
  (p : V n, G → Equiv.Perm (V n))
  (hone : ∀ n, p n 1 = 1)
  (hmul : ∀ a g : G,
    Tendsto
      (fun n => SoficGroups.normalizedHamming
        (p n (a * g)) (p n a * p n g))
      atTop (nhds 0))
  (s : ι → G) (w : G → List ι)
  (hw : ∀ g : G, ((w g).map s).prod = g)
  (σ : V n, ι → Equiv.Perm (V n))
  (hletter : ∀ i : ι,
    Tendsto
      (fun n => SoficGroups.normalizedHamming
        (σ n i) (p n (s i)))
      atTop (nhds 0))
  (a g : G) :
  Tendsto
    (fun n => SoficGroups.normalizedHamming
      (chosenWordEvaluation (σ n) w (a * g))
      (chosenWordEvaluation (σ n) w a *
        chosenWordEvaluation (σ n) w g))

```

```

    atTop (nhds 0) := by
-- proof: 70 lines, omitted

```

`chosenWordEvaluation` applied to  $\sigma, w, g$  is the product of the generator permutations along the chosen word of  $g$ : by construction an exact word product, which is what the rooted-model interfaces require of their evaluation maps. The two theorems of this block show nothing is lost by switching to it. `chosenWordEvaluation_tendsto_action`: if the approximate action  $p_n$  is asymptotically multiplicative with  $p_n(1) = 1$ , the words satisfy that mapping the letters through  $s$  and multiplying returns  $g$ , and each generator  $\sigma_n(i)$  converges in normalized Hamming distance to  $p_n(s i)$ , then the evaluation of  $g$  converges to  $p_n(g)$ ; the proof compares both to the intermediate product of the  $p_n(s i)$  along the word, using `list_permutation_hamming_tendsto` on one side and `approximate_action_list_prod_tendsto` on the other. Building on that, `chosenWordEvaluation_multiplicative_tendsto` proves the evaluation is itself asymptotically multiplicative: the distance between the evaluation of  $ag$  and the product of the evaluations of  $a$  and  $g$  vanishes, via a four-term triangle chain through  $p_n(ag)$  and  $p_n(a)p_n(g)$  combined with `normalizedHamming_mul_le`. Together these results let later constructions replace a sofic approximation's action by a strictly word-generated map that satisfies the generation axioms exactly while retaining every vanishing-error estimate.

*Remaining declarations in this section:*

|                                                   |     |                                                                                                                      |
|---------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------|
| <code>normalizedHamming_mul_le</code>             | THM | Normalized Hamming distance of products is bounded by the sum of the factorwise distances.                           |
| <code>list_permutation_hamming_tendsto</code>     | THM | Letterwise normalized Hamming convergence of permutation families implies convergence of their list products.        |
| <code>approximate_action_list_prod_tendsto</code> | THM | Asymptotically multiplicative approximate actions evaluate list products to products of evaluations, asymptotically. |
| <code>chosenWordEvaluation</code>                 | DEF | Evaluates the chosen word of $g$ as a product of generator permutations.                                             |
| <code>chosenWordEvaluation_tendsto_action</code>  | THM | The chosen word evaluation converges in normalized Hamming distance to the approximate action.                       |

## 7.18 Completing restricted generators (CompletedSourceChosenWordRestrictionTransfer)

Shows the completion step of the pruning construction is asymptotically canonical. `completedRestriction` closes the restriction of a permutation  $\sigma$  to a retained subset  $Z$  into a genuine permutation of  $Z$ . If a candidate permutation  $\tau$  of  $Z$  agrees with  $\sigma$  at every point whose image stays inside  $Z$ , their permutation distance is at most  $|\text{univ} \setminus Z|$ : disagreements can only occur on the exit set, whose size is bounded by the deleted set because  $\sigma$  is a bijection. Dividing by  $|Z|$  gives the normalized bound, and for sequences whose relative deleted density vanishes,  $\tau_n$  converges to the completed restriction in normalized Hamming distance. Later segments use this to identify independently constructed permutations on retained supports with completed restrictions of the ambient model.

**`completedGenerator_normalizedHamming_tendsto_zero`** thm

```

theorem completedGenerator_normalizedHamming_tendsto_zero
  (V : ℕ → Type*) [V n, Fintype (V n)]
  [V n, DecidableEq (V n)]
  (σ : (n : ℕ) → Equiv.Perm (V n))
  (Z : (n : ℕ) → Finset (V n))
  (τ : (n : ℕ) → Equiv.Perm {x : V n // x ∈ Z n})
  (hagrees : ∀ n, ∀ x : {x : V n // x ∈ Z n},
    σ n (x : V n) ∈ Z n →
      ((τ n x : {x : V n // x ∈ Z n}) : V n) =
        σ n (x : V n))
  (hdeleted : Tendsto
    (fun n => (((Finset.univ \ Z n).card : ℝ) / (Z n).card))
    atTop (nhds 0)) :
  Tendsto

```

```

(fun n =>
  SoficGroups.normalizedHamming
    (τ n) (completedRestriction (σ n) (Z n)))
  atTop (nhds 0) := by
-- proof: 9 lines, omitted

```

The convergence lemma for the completion step. `completedRestriction` closes the restriction of a permutation  $\sigma$  to a subset  $Z$  into a genuine permutation of the subtype. Suppose  $\tau_n$  is any permutation of  $Z_n$  that agrees with  $\sigma_n$  at every point whose image stays inside  $Z_n$ . The finitary bound `completedGenerator_permutationDistance_le_deleted` shows the permutation distance from  $\tau_n$  to the completed restriction is at most  $|\text{univ} \setminus Z_n|$ : disagreements are confined to the exit set (points of  $Z$  mapped outside  $Z$ ), formalized through `permutationDistance_le_subtypeBad` and `card_subtypeBad`, and `card_permutation_exit_le_deleted` bounds the exits by the deleted set using bijectivity of  $\sigma$ , since every point leaving  $Z$  occupies a slot in the complement. Division by  $|Z_n|$  gives `completedGenerator_normalizedHamming_le_deleted`, and this theorem concludes by squeezing: if the relative deleted density  $|\text{univ} \setminus Z_n|/|Z_n|$  tends to zero, then the normalized Hamming distance between  $\tau_n$  and the completed restriction tends to zero. The significance: after pruning bad points from a sofic model the restricted maps must be re-closed into permutations, and this shows any two reasonable closures agree asymptotically, so downstream estimates are insensitive to the particular completion chosen.

*Remaining declarations in this section:*

|                                                                |     |                                                                                             |
|----------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------|
| <code>completedGenerator_permutationDistance_le_deleted</code> | THM | Distance from $\tau$ to the completed restriction is at most the deleted set's cardinality. |
| <code>completedGenerator_normalizedHamming_le_deleted</code>   | THM | Normalized Hamming version: bounded by the deleted cardinality divided by $ Z $ .           |

## 7.19 Two-stage completion generator transfer (CompletedSourceFinalGeneratorTransfer)

Transfers generator convergence through the two-stage restriction used to build the final finite models: first restrict the ambient model to a support  $P_n$ , then delete a further bad set  $D_n$  inside it, completing to a permutation at each stage. Auxiliary limits show the surviving density tends to one and the deleted mass is negligible even relative to the survivors. The main theorem concludes that if  $\sigma_n(i) = p_n(s i)$ , the candidate generators  $\tau_n(i)$  agree with the once-completed restriction wherever the image survives, and  $|D_n|/|P_n| \rightarrow 0$ , then each  $\tau_n(i)$  converges in normalized Hamming distance to the twice-completed restriction of  $p_n(s i)$ . This certifies that the final generator system inherits all asymptotic closeness from the original sofic approximation.

**twiceCompleted\_sourceGenerator\_normalizedHamming\_tendsto\_zero thm**

```

theorem twiceCompleted_sourceGenerator_normalizedHamming_tendsto_zero
  (V : ℕ → Type*) [V n, Fintype (V n)]
  [V n, DecidableEq (V n)]
  {ι J : Type*}
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))
  (p : (n : ℕ) → J → Equiv.Perm (V n))
  (s : ι → J)
  (P : (n : ℕ) → Finset (V n))
  (D : (n : ℕ) → Finset {x : V n // x ∈ P n})
  (τ : (n : ℕ) → (i : ι) →
    Equiv.Perm
      {x : {x : V n // x ∈ P n} //
        x ∈ (Finset.univ \ D n :
          Finset {x : V n // x ∈ P n})})
  (hs : ∀ n i, σ n i = p n (s i))
  (hag : ∀ n i,
    ∀ x : {x : {x : V n // x ∈ P n} //
      x ∈ (Finset.univ \ D n :
        Finset {x : V n // x ∈ P n})},
    completedRestriction (σ n i) (P n) x.val ∈
      (Finset.univ \ D n :
        Finset {x : V n // x ∈ P n}) →

```

```

      (τ n i x).val =
        completedRestriction (σ n i) (P n) x.val)
(hZ : ∀ n,
  (Finset.univ \ D n :
    Finset {x : V n // x ∈ P n}).Nonempty)
(hdeleted : Tendsto
  (fun n => ((D n).card : ℝ) / (P n).card)
  atTop (nhds 0)) :
∀ i : ι,
  Tendsto
    (fun n =>
      SoficGroups.normalizedHamming
        (τ n i)
        (completedRestriction
          (completedRestriction (p n (s i)) (P n))
          (Finset.univ \ D n)))
    atTop (nhds 0) := by
-- proof: 40 lines, omitted

```

The models eventually used are produced in two stages: the ambient model on  $V_n$  is restricted to a retained support  $P_n$  (as a subtype), then a further bad set  $D_n$  inside that subtype is deleted, each stage completed to an honest permutation by `completedRestriction`. This theorem certifies the final generators still converge. Hypotheses:  $\sigma_n(i) = p_n(s i)$ ; the candidate generators  $\tau_n(i)$ , permutations of the survivor subtype, agree with the once-completed restriction wherever its image survives the deletion; the survivors are nonempty; and the deleted density  $|D_n|/|P_n|$  tends to zero. Conclusion: for each generator index  $i$ , the normalized Hamming distance from  $\tau_n(i)$  to the twice-completed restriction of  $p_n(s i)$  tends to zero. The proof first converts the absolute deleted density into density relative to the survivors, via `surviving_card_ratio_tendsto_one` (survivor density tends to one, an algebraic manipulation of  $1 - x$ ) and `deleted_density_relative_survivors` (a quotient of two limits); it then rewrites the complement of the survivor set as  $D_n$  and applies `completedGenerator_normalizedHamming_tendsto_zero` with  $Z$  the survivor set inside the subtype of  $P_n$ . This is the last transfer step showing that the doubly pruned and completed generator system inherits all asymptotic closeness to the original sofic approximation.

*Remaining declarations in this section:*

|                                                 |     |                                                                                 |
|-------------------------------------------------|-----|---------------------------------------------------------------------------------|
| <code>surviving_card_ratio_tendsto_one</code>   | THM | The density of surviving points tends to one when the deleted density vanishes. |
| <code>deleted_density_relative_survivors</code> | THM | The deleted set's cardinality relative to the survivors tends to zero.          |

## 7.20 Fixed radius multiplication bad sets (CompletedPrescribedSpectralRadiusSchedule)

Constructs the explicit vanishing bad sets witnessing local multiplicativity at a prescribed word radius. `mem_generator_pow_of_chosen_word_length` shows an element whose chosen word has length at most  $r$  lies in  $S^r$ , padding with the identity. `multiplicationBad` collects the points where  $\varphi(ag)$  and  $\varphi(a)\varphi(g)$  differ for one pair, and `fixedRadiusRootBad` unions these over  $S^r \times S^r$ . Outside this set the evaluation is exactly multiplicative for every pair whose combined word lengths stay below  $r$ , and for an asymptotically multiplicative family the set's density vanishes for each fixed  $r$ , being a finite union of sets whose densities are normalized Hamming distances. These bad sets are precisely the sets  $B$  required by the sparse-cut and uniform-radius improvement theorems, on a schedule of radii chosen later in the file.

**fixedRadiusRootBad\_rooted** thm

```

theorem fixedRadiusRootBad_rooted
  {G V : Type*} [Group G] [DecidableEq G]
  [Fintype V] [DecidableEq V]
  (S : Finset G) (hone : 1 ∈ S)
  (w : G → List ιS)
  (hw : ∀ a : G,

```

```

      ((w a).map fun i : ιS => (i : G)).prod = a)
    (φ : G → Equiv.Perm V) (r : ℕ)
    (a g : G)
    (hr : (w a).length + (w g).length +
      (w (a * g)).length ≤ r)
    (x : V) (hx : x ∉ fixedRadiusRootBad φ S r) :
    φ (a * g) x = (φ a * φ g) x := by
-- proof: 12 lines, omitted

```

multiplicationBad for  $\varphi, a, g$  is the set of points where  $\varphi(ag)$  and  $\varphi(a)\varphi(g)$  disagree, and fixedRadiusRootBad unions these failure sets over all pairs  $(a, g) \in S^r \times S^r$ . This theorem verifies the rootedness interface required by the Kun machinery: assuming  $1 \in S$  and that the chosen words evaluate correctly (mapping the letters of  $w(a)$  into the group and multiplying returns  $a$ ), then for any pair  $a, g$  whose combined word lengths, including that of  $w(ag)$ , total at most  $r$ , and any point  $x$  outside fixedRadiusRootBad, one has  $\varphi(ag)(x) = (\varphi(a)\varphi(g))(x)$ . The proof rests on mem\_generator\_pow\_of\_chosen\_word\_length: a list of letters of  $S$  multiplies into the power of  $S$  of its length, by induction on the list, and since  $1 \in S$  the monotonicity Finset.pow\_subset\_pow\_right pads shorter words up to  $S^r$ ; hence both  $a$  and  $g$  lie in  $S^r$ , so any failure point would belong to the union by construction. The statement has exactly the shape of the local-multiplicativity hypotheses of exists\_completed\_goodPermutationGraph\_sparse\_cut and exists\_uniform\_radius\_hasAlmostCentralizerImprovement, with the bad set  $B$  instantiated as fixedRadiusRootBad.

#### fixedRadiusRootBad\_density\_tendsto\_zero thm

```

theorem fixedRadiusRootBad_density_tendsto_zero
  {G : Type*} [Group G] [DecidableEq G]
  (V : ℕ → Type*)
  [V n, Fintype (V n)] [V n, DecidableEq (V n)]
  (φ : ∀ n, G → Equiv.Perm (V n))
  (S : Finset G)
  (hmul : ∀ a g : G,
    Tendsto
      (fun n => SoficGroups.normalizedHamming
        (φ n (a * g)) (φ n a * φ n g))
      atTop (ℕ 0))
  (r : ℕ) :
  Tendsto
    (fun n =>
      ((fixedRadiusRootBad (φ n) S r).card : ℝ) /
      Fintype.card (V n))
    atTop (ℕ 0) := by
-- proof: 18 lines, omitted

```

The companion density estimate: if the family  $\varphi_n$  is asymptotically multiplicative, meaning that for every fixed pair  $a, g$  the normalized Hamming distance between  $\varphi_n(ag)$  and  $\varphi_n(a)\varphi_n(g)$  tends to zero, then for each fixed radius  $r$  the density of fixedRadiusRootBad for  $\varphi_n, S, r$  inside  $V_n$  tends to zero. The proof observes that the indexing set  $S^r \times S^r$  is a fixed finite set independent of  $n$ ; for each pair, the density of multiplicationBad is literally the normalized Hamming distance in question, as both count the disagreement points of two permutations, and the general lemma finite\_union\_bad\_density\_tendsto\_zero shows a finite union of vanishing-density sets has vanishing density. Together with fixedRadiusRootBad\_rooted, this produces for every radius  $r$  a sequence of bad sets  $B_n$  of vanishing density outside which the model is exactly multiplicative for all radius- $r$  word pairs. That is exactly the input consumed, along a diverging schedule of radii, by the sparse-cut theorem and by exists\_uniform\_radius\_hasAlmostCentralizerImprovement when the whole machine is finally aimed at a hypothetical sofic approximation of the nine-prefix source group.

*Remaining declarations in this section:*

|                                         |     |                                                                                 |
|-----------------------------------------|-----|---------------------------------------------------------------------------------|
| mem_generator_pow_of_chosen_word_length | THM | Elements whose chosen word has length at most $r$ lie in the power $S^r$ .      |
| multiplicationBad                       | DEF | Points where $\varphi$ fails multiplicativity for the fixed pair $(a, g)$ .     |
| fixedRadiusRootBad                      | DEF | Union of the multiplication failure sets over all pairs from $S^r \times S^r$ . |



## 8 Poincaré inequalities, finite models, and the Thompson obstruction

The estimates converge: explicit Kazhdan generators for the nine-prefix group, an additive median Poincaré inequality, and vanishing midrank variance force the completed models to become expanding centralizer finite models, and `SourceTopLevelCompression` shows soficity plus such finite models implies LEF. Against this stands the short second pillar: Thompson-like two-relator identities collapse in any finite group, so the local prefix transposition group is not LEF.

### 8.1 Expander partitions from Kazhdan pairs (KunSourceUnconditionalFullDecomposition)

Converts a Kazhdan spectral gap into combinatorial expander decompositions of every finite model in a sofic approximation. `exists_source_sparse_cut_at_radius` turns the abstract rooted-word sparse-cut theorem into a concrete improvement step: any vertex set with  $\gamma$ -sparse boundary that avoids a radius- $r$  exceptional set can be replaced by a nearby set with  $\alpha$ -sparse boundary. `exists_source_full_finpartition_sequence_of_kazhdan` iterates this with tolerances  $\alpha_j \rightarrow 0$  and a diagonalization over radii to produce a constant  $\gamma > 0$  and full finpartitions of every model whose parts are  $\gamma$ -expanding while their total boundary density vanishes. `exists_source_subgroup_and_ambient_full_finpartition_sequences` runs the construction simultaneously for a subgroup acting through an injective homomorphism (via the pullback sofic approximation) and for the ambient group, on the same models — the paired partition sequences underlying the later transport and rank arguments.

**exists\_source\_sparse\_cut\_at\_radius** *thm*

```

theorem exists_source_sparse_cut_at_radius
  {G : Type} [Group G] [DecidableEq G]
  (P : SoficGroups.KazhdanPair.{0, 0} G)
  (S : Finset G) (hone : 1 ∈ S)
  (hcover : P.generators ⊆ S)
  (hsymmetric : ∀ g ∈ S, g-1 ∈ S)
  (hgenerates : Subgroup.closure (S : Set G) = T)
  (γ α : ℝ) (hγ : 0 < γ) (hα : 0 < α)
  (hsmall :
    216 * γ <
      (S.card : ℝ) *
        (1 - kazhdanMarkovContractionFactor P S) ^ 2) :
  ∃ r : ℕ,
    ∀ (A : SoficGroups.SoficApproximation G) (n : ℕ)
      (T : Finset (Fin (A.model n).size)),
      Disjoint T
        (chosenCayleyRadiusBad A S
          (symmetricGeneratorWord S hsymmetric hgenerates) n r) →
      (SoficGroups.boundary
        (fun i : ιS => (A.model n).action (i : G)) T : ℝ) <
        γ * (T.card : ℝ) →
      ∃ U : Finset (Fin (A.model n).size),
        3 * (U ∆ T).card < T.card ∧
        (SoficGroups.boundary
          (fun i : ιS => (A.model n).action (i : G)) U : ℝ) ≤
          α * (U.card : ℝ) := by
-- proof: 67 lines, omitted

```

Given a Kazhdan pair  $P$  for  $G$  and a finite symmetric generating set  $S$  that contains the identity and covers  $P.\text{generators}$ , together with tolerances  $\gamma, \alpha > 0$  satisfying the numerical gap condition  $216\gamma < |S|(1 - q)^2$ , where  $q$  is the Markov contraction factor `kazhdanMarkovContractionFactor`, the theorem produces a single radius  $r$  uniform over all sofic approximations and indices: whenever a vertex set  $T$  in the  $n$ -th model is disjoint from the radius- $r$  exceptional set `chosenCayleyRadiusBad` and its boundary under the  $S$ -action is below  $\gamma|T|$ , there is a replacement  $U$  with  $3|U \triangle T| < |T|$  and boundary at most  $\alpha|U|$ . The proof instantiates the abstract rooted-word sparse-cut theorem `exists_rooted_word_radius_sparse_cut_of_boundary` at the word map `symmetricGeneratorWord`, then checks that the concrete Markov model `sourceRootedIndicatorMarkovModel` attached to  $T$  is generated by the  $S$ -letters and rooted at radius  $r$  precisely because  $T$  avoids the exceptional set; a short case

analysis matches the model's indicator function with membership in  $T$ . This is the single-step repair lemma: sets that fail to be  $\alpha$ -sparse expanders but avoid a sparse exceptional set can be locally improved, which the greedy partition construction of the next theorem iterates.

**exists\_source\_full\_finpartition\_sequence\_of\_kazhdan** *thm*

```

theorem exists_source_full_finpartition_sequence_of_kazhdan
  {G : Type} [Group G] [DecidableEq G]
  (A : SoficGroups.SoficApproximation G)
  (P : SoficGroups.KazhdanPair.{0, 0} G)
  (S : Finset G) (hS : 1 ∈ S)
  (hcover : P.generators ⊆ S)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set G) = T) :
  ∃ (γ : ℝ)
    (Q : (n : ℕ) →
      Finpartition
        (Finset.univ : Finset (Fin (A.model n).size))),
  0 < γ ∧
  (∀ n, ∀ C ∈ (Q n).parts,
    ∀ E : Finset (Fin (A.model n).size),
      E ⊆ C →
        2 * E.card ≤ C.card →
        γ * (E.card : ℝ) ≤
          (SoficGroups.boundary
            (fun i : 1S =>
              (A.model n).action (i : G)) E : ℝ)) ∧
  Tendsto
    (fun n =>
      (∑ C ∈ (Q n).parts,
        (SoficGroups.boundary
          (fun i : 1S =>
            (A.model n).action (i : G)) C : ℝ)) /
        (A.model n).size)
    atTop (N 0) := by
-- proof: 107 lines, omitted

```

Produces a uniform expansion constant  $\gamma > 0$  together with, for every index  $n$ , a finpartition  $Q_n$  of the entire vertex set of the  $n$ -th sofic model such that every part  $C$  is a  $\gamma$ -expander for the  $S$ -action — any  $E \subseteq C$  with  $2|E| \leq |C|$  has boundary at least  $\gamma|E|$  — while the total boundary of all parts, divided by the model size, tends to zero. The constant is  $\gamma = |S|(1-q)^2/288$ , chosen so that the gap hypothesis  $216\gamma < |S|(1-q)^2$  of the sparse-cut lemma holds, with  $q < 1$  supplied by `kazhdanMarkovContractionFactor_lt_one`. The proof takes tolerances  $\alpha_j = 1/(j+1)$ , obtains a radius for each  $j$  from `exists_source_sparse_cut_at_radius`, and notes that the exceptional-set densities of `chosenCayleyRadiusBad` vanish at each fixed radius; the diagonalization lemma `exists_diverging_radius_with_vanishing_diagonal_error` selects a slowly diverging index  $m_n$  so that the tolerance and the exceptional density vanish simultaneously. Feeding the resulting improvement step into the engine `exists_expanding_full_finpartition_sequence` of `KunResidualExpanderDecomposition` yields the partitions. These partition sequences are the combinatorial skeleton of the transport argument: component sizes of  $Q_n$  later define the log-rank functions, and the vanishing boundary budget drives the density limits throughout the following namespaces.

*Remaining declarations in this section:*

|                                                                              |                                                                                                                                                    |
|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> exists_source_subgroup_and_ambient_ full_finpartition_sequences </pre> | <p><i>THM</i>    Runs the partition construction simultaneously for the pulled-back subgroup action and the ambient action on the same models.</p> |
|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|

## 8.2 Infinitude of the compressed source group (KunActualCompressedSourceGroupFoundations)

A single foundational fact for the compressed source group: the prefix elementary group of the alpha-zero prefix code is infinite. The proof transports the previously established infinitude of the elementary Leavitt

group `binaryLeavittElementaryGroup` at width 3 through the group equivalence `binaryPrefixElementaryGroupEquiv` specialized at `alphaZeroPrefixCode`. Infinitude of this group is a background nondegeneracy requirement for the source-group constructions that follow: quantitative statements about sofic approximations of the compressed source group are vacuous or degenerate for finite groups, so this lemma guarantees that the objects being analyzed are genuinely infinite, with no Kazhdan or property (T) input needed.

**alphaZeroPrefixElementaryGroup\_infinite** *thm*

```
theorem alphaZeroPrefixElementaryGroup_infinite :
  Infinite
  (SoficGroups.prefixElementaryGroup SoficGroups.alphaZeroPrefixCode) := by
  -- proof: 8 lines, omitted
```

States that `prefixElementaryGroup` at `alphaZeroPrefixCode` is an infinite group. The proof is a short transport: the file has already established `binaryLeavittEL3_infinite`, infinitude of `binaryLeavittElementaryGroup` at width 3, and it has constructed the group equivalence `binaryPrefixElementaryGroupEquiv` between that elementary group and the prefix elementary group of a suitable binary prefix code, here instantiated at the alpha-zero code. Applying `Infinite.of_injective` to the equivalence and its injectivity transfers infinitude. The statement requires no Kazhdan or property (T) input; it is purely a cardinality fact inherited through the Leavitt-algebra picture, where the elementary group at width three was shown infinite earlier in the file. In the surrounding architecture the alpha-zero prefix group serves as the compressed model of the source group, and several later quantitative arguments about its sofic approximations are meaningful only for an infinite group; this lemma discharges that nondegeneracy requirement unconditionally and is the only content of its namespace.

### 8.3 Kazhdan pairs on prescribed generators (KunExactKazhdanGeneratorChange)

Standard Kazhdan generator-change argument, needed because property (T) supplies a Kazhdan pair on some generating set while the combinatorial machinery requires the generator finset to be an arbitrarily prescribed symmetric one. `unitary_word_displacement_le` bounds the displacement of a vector under a product of unitaries by word length times the maximal letter displacement. `exists_word_of_symmetric_generating_finset` and its packaged versions `symmetricGeneratingWord` and `symmetricGeneratingWord_prod` express arbitrary elements as words over a symmetric generating finset via closure induction. `kazhdanPairOnSymmetricGeneratingFinset` divides the Kazhdan constant by one plus the total length of the words representing the old generators, producing a Kazhdan pair whose generator field is definitionally the new set; `exists_kazhdanPair_with_exact_symmetric_generators` states the resulting existence principle under a `HasPropertyT` instance.

**kazhdanPairOnSymmetricGeneratingFinset** *def*

```
def kazhdanPairOnSymmetricGeneratingFinset
  {G : Type u} [Group G]
  (P : SoficGroups.KazhdanPair.{u, v} G)
  (S : Finset G)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set G) = T) :
  SoficGroups.KazhdanPair.{u, v} G := by
  classical
  let w : G → List ιS :=
    symmetricGeneratingWord S hsymmetric hgenerates
  let L : ℕ := 1 + ∑ g ∈ P.generators, (w g).length
  have hL : 0 < L := by
    dsimp [L]
    omega
  have hLreal : 0 < (L : ℝ) := by
    exact_mod_cast hL
  refine
    { generators := S
      kazhdanConstant := P.kazhdanConstant / (L : ℝ)
      positive := div_pos P.positive hLreal
      invariant := ?_ }
  intro H _ _ π ξ hξ hS
```

```

apply P.invariant H  $\pi$   $\xi$  h $\xi$ 
intro g hg
have hlength :
  (w g).length < L := by
    have hsingle :
      (w g).length  $\leq$ 
-- 37 more lines, omitted

```

Given an arbitrary Kazhdan pair  $P$  for  $G$  and a symmetric generating finset  $S$ , this definition manufactures a new Kazhdan pair whose generator field is literally  $S$ . Every original generator  $g$  is written as a word  $w(g)$  over  $S$  using `symmetricGeneratingWord`, and  $L$  is one plus the sum of the lengths of all these words, so each individual word is strictly shorter than  $L$ . The new Kazhdan constant is the old one divided by  $L$ , still positive. For the invariance property, suppose a unitary representation  $\pi$  moves a vector  $\xi$  by less than the new constant on every element of  $S$ . By the displacement estimate `unitary_word_displacement_le`, the word  $w(g)$  then moves  $\xi$  by at most its length times the new constant, which is strictly less than the old constant  $P.kazhdanConstant$ ; since the word multiplies out to  $g$  by `symmetricGeneratingWord_prod`, the hypothesis of the original pair holds on  $P.generators$  and its invariance conclusion applies verbatim. The companion lemma `kazhdanPairOnSymmetricGeneratingFinset_generators` records that the generator field equals  $S$  by `rfl` — the exactness downstream constructions require when they pattern-match on the generator finset — and `exists_kazhdanPair_with_exact_symmetric_generators` turns this into a bare existence statement from any `HasPropertyT` instance.

*Remaining declarations in this section:*

|                                                                 |     |                                                                                                                     |
|-----------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------|
| <code>unitary_word_displacement_le</code>                       | THM | Displacement of a vector under a product of unitaries is at most word length times the maximal letter displacement. |
| <code>exists_word_of_symmetric_generating_finset</code>         | THM | Every element is a product of letters from a symmetric generating finset, by closure induction.                     |
| <code>symmetricGeneratingWord</code>                            | DEF | Chooses such a representing word for each group element via choice.                                                 |
| <code>symmetricGeneratingWord_prod</code>                       | THM | Specification: the chosen word's letter product equals the given element.                                           |
| <code>kazhdanPairOnSymmetricGeneratingFinset_generators</code>  | THM | The rescaled pair's generator finset is definitionally $S$ .                                                        |
| <code>exists_kazhdanPair_with_exact_symmetric_generators</code> | THM | Under property (T), some Kazhdan pair has generators equal to any prescribed symmetric generating finset.           |

## 8.4 Explicit ambient generators for nine-prefix group (KunExactActualSourceAmbientGenerators)

Builds the explicit generating data linking the alpha prefix elementary group (the subgroup side) to the nine-prefix elementary group (the ambient side). `sourceAlphaInclusion` is the injective inclusion homomorphism; `sourceCompressionU` and `sourceCompressionV` package the two compression units. Given any generating finset  $S_\Gamma$  of the subgroup, `sourcePositiveGenerators` adjoins the compressions to the included generators and `sourceAmbientSymmetricGenerators` adds the identity and inverses. The central theorem `sourceAmbientSymmetricGenerators_generate` shows these finitely many elements generate the whole ambient group, by passing to the units of the binary Leavitt algebra and invoking `sourceGeneratedGroup_eq_nine`; combined with the previous namespace this yields a Kazhdan pair on exactly this generating set. A second block reindexes the positive generators by the sum of  $S_\Gamma$ -indices and `Fin 2` (`sourcePositiveGeneratorMap`), computes its range, and shows the range generates — the indexed family over which the later energy sums run.

**`sourceAmbientSymmetricGenerators`** `def`

```

noncomputable def sourceAmbientSymmetricGenerators
  (S $\Gamma$  : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode)) :
  Finset

```

```

(SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode) := by
classical
exact insert 1
(sourcePositiveGenerators SΓ u
(sourcePositiveGenerators SΓ).image fun g => g-1)

```

The canonical ambient generating finset. Starting from any finite generating set  $S_\Gamma$  of the alpha prefix elementary group, `sourcePositiveGenerators` first forms the images of  $S_\Gamma$  under `sourceAlphaInclusion` and adjoins the two compression elements `sourceCompressionU` and `sourceCompressionV`, the elements of the nine-prefix elementary group carrying the compression units `compressionU` and `compressionV` of the binary Leavitt algebra. `sourceAmbientSymmetricGenerators` then inserts the identity and closes under inverses by taking the union with the image under  $g \mapsto g^{-1}$ . The normalization lemmas that follow verify precisely the hypotheses demanded elsewhere: `one_mem_sourceAmbientSymmetricGenerators` shows the identity is present, `sourceAmbientSymmetricGenerators_inv_mem` shows closure under inversion by case analysis on the three membership sources (identity, positive generator, inverse of a positive generator), and the generation theorem below completes the triple. This shape — a finite symmetric set containing the identity — is exactly what the Kazhdan generator-change construction and the expander partition machinery of `KunSourceUnconditionalFullDecomposition` consume, so this finset is the ambient generating set threaded through the remainder of the transport argument, with its positive half indexed separately for the energy sums of `KunPositiveWordMidrankEnergy`.

#### **sourceAmbientSymmetricGenerators\_generate thm**

```

theorem sourceAmbientSymmetricGenerators_generate
(SΓ : Finset
(SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
(hgeneratesΓ :
Subgroup.closure
(SΓ : Set
(SoficGroups.prefixElementaryGroup
SoficGroups.alphaPrefixCode)) = T) :
Subgroup.closure
(sourceAmbientSymmetricGenerators SΓ : Set
(SoficGroups.prefixElementaryGroup
SoficGroups.ninePrefixCode)) = T := by
-- proof: 75 lines, omitted

```

The generation theorem for the ambient group. Let  $K$  be the subgroup generated by the symmetrized generators. First, every positive generator lies in  $K$ ; consequently every included element does, because the generating hypothesis on  $S_\Gamma$  transfers along `Subgroup.comap` of the inclusion: the closure of  $S_\Gamma$  is the whole alpha group and lands inside the preimage of  $K$ . Next,  $K$  is pushed forward along the subtype embedding into the units of the binary Leavitt algebra, giving a subgroup  $U$  that contains all corner elements coming from the alpha prefix group and both compression units — that is, all generators of `sourceGeneratedGroup` from the `SourceGeneration` development — so the source generated group is contained in  $U$ . The previously proven identification `sourceGeneratedGroup_eq_nine` says this generated group is exactly the nine-prefix elementary group, so every element of the ambient group, viewed as a Leavitt unit, is the image of some element of  $K$ ; injectivity of the subtype map then gives  $K = T$ . This is the bridge from the fixed two-compression presentation of the ambient group to an arbitrarily chosen generating set of the subgroup: it makes possible a Kazhdan pair supported on exactly these generators (`exists_kazhdanPair_on_exact_sourceAmbientSymmetricGenerators`) and the simultaneous subgroup-and-ambient partition sequences built earlier.

*Remaining declarations in this section:*

|                                             |     |                                                                                                        |
|---------------------------------------------|-----|--------------------------------------------------------------------------------------------------------|
| <code>sourceAlphaInclusion</code>           | DEF | Monoid homomorphism including the alpha prefix elementary group into the nine-prefix elementary group. |
| <code>sourceAlphaInclusion_injective</code> | THM | The inclusion is injective, since it preserves the underlying Leavitt unit.                            |
| <code>sourceCompressionU</code>             | DEF | The compression element <code>compressionU</code> as an element of the nine-prefix elementary group.   |

|                                                                           |     |                                                                                                           |
|---------------------------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------|
| <code>sourceCompressionV</code>                                           | DEF | The compression element <code>compressionV</code> as an element of the nine-prefix elementary group.      |
| <code>sourcePositiveGenerators</code>                                     | DEF | Positive generators: the image of $S_\Gamma$ under the inclusion together with both compression elements. |
| <code>one_mem_sourceAmbientSymmetricGenerators</code>                     | THM | The identity belongs to the symmetrized ambient generator finset.                                         |
| <code>sourceAmbientSymmetricGenerators_inv_mem</code>                     | THM | The symmetrized ambient generator finset is closed under inverses.                                        |
| <code>exists_kazhdanPair_on_exact_sourceAmbientSymmetricGenerators</code> | THM | Under property (T), yields a Kazhdan pair with generators exactly the symmetrized ambient generators.     |
| <code>sourceCompressionTable</code>                                       | DEF | Two-entry table listing <code>sourceCompressionU</code> and <code>sourceCompressionV</code> .             |
| <code>sourcePositiveGeneratorMap</code>                                   | DEF | Indexed positive generator map on the sum of $S_\Gamma$ -indices and <code>Fin 2</code> .                 |
| <code>range_sourcePositiveGeneratorMap</code>                             | THM | The map's range is exactly the positive generator finset.                                                 |
| <code>sourcePositiveGenerators_generate</code>                            | THM | The positive generators alone already generate the nine-prefix elementary group.                          |
| <code>sourcePositiveGeneratorMap_range_generate</code>                    | THM | Hence the range of the indexed positive generator map generates the whole group.                          |

## 8.5 Unconditional generator and Kazhdan data (KunUnconditionalActualSourceGeneratorData)

Removes all remaining hypotheses from the generator preparation. `exists_symmetric_generating_finset_of_finitelyGenerated` shows any finitely generated group admits a finite generating set containing the identity and closed under inverses, by symmetrizing a given finite generating set. Specializing to the finitely generated alpha prefix elementary group gives `exists_actual_source_alpha_symmetric_generators`. The main theorem `exists_unconditional_actual_source_generator_data` then combines this with the unconditional property (T) instances for the alpha and nine-prefix elementary groups to produce, with no assumptions, a generating finset  $S_\Gamma$ , a Kazhdan pair on the subgroup with exactly those generators, and a Kazhdan pair on the ambient group with exactly the ambient symmetric generators, together with injectivity of the inclusion and generation by the indexed positive generator map. Later sections destructure this one statement to launch the partition and transport constructions.

**`exists_unconditional_actual_source_generator_data`** thm

```

theorem exists_unconditional_actual_source_generator_data :
  ∃ (SΓ : Finset
    (SoficGroups.prefixElementaryGroup
      SoficGroups.alphaPrefixCode))
  (PΓ : SoficGroups.KazhdanPair.{0, 0}
    (SoficGroups.prefixElementaryGroup
      SoficGroups.alphaPrefixCode))
  (PG : SoficGroups.KazhdanPair.{0, 0}
    (SoficGroups.prefixElementaryGroup
      SoficGroups.ninePrefixCode)),
  1 ∈ SΓ ∧
  (∀ g ∈ SΓ, g⁻¹ ∈ SΓ) ∧
  Subgroup.closure
    (SΓ : Set
      (SoficGroups.prefixElementaryGroup
        SoficGroups.alphaPrefixCode)) = T ∧
  PΓ.generators = SΓ ∧
  1 ∈ sourceAmbientSymmetricGenerators SΓ ∧
  (∀ g ∈ sourceAmbientSymmetricGenerators SΓ,
    g⁻¹ ∈ sourceAmbientSymmetricGenerators SΓ) ∧
  Subgroup.closure
    (sourceAmbientSymmetricGenerators SΓ : Set

```

```

      (SoficGroups.prefixElementaryGroup
        SoficGroups.ninePrefixCode)) = T ∧
PG.generators = sourceAmbientSymmetricGenerators SΓ ∧
Function.Injective sourceAlphaInclusion ∧
Subgroup.closure
  (Set.range (sourcePositiveGeneratorMap SΓ)) = T := by
-- proof: 23 lines, omitted

```

The culminating existence package of the generator preparation: there exist a finset  $S_\Gamma$  of the alpha prefix elementary group and Kazhdan pairs  $P_\Gamma$  (for the alpha group) and  $P_G$  (for the nine-prefix group) such that  $S_\Gamma$  contains the identity, is symmetric, and generates;  $P_\Gamma$  has generators exactly  $S_\Gamma$ ; the ambient set `sourceAmbientSymmetricGenerators` of  $S_\Gamma$  likewise contains the identity, is symmetric, and generates the nine-prefix group;  $P_G$  has generators exactly that ambient set; `sourceAlphaInclusion` is injective; and the indexed positive generator map has generating range. The proof wires together the previous two namespaces and instantiates property (T) with the unconditional instances `alphaPrefixElementaryGroup_hasPropertyT_unconditional` and `ninePrefixElementaryGroup_hasPropertyT_unconditional`, so the statement carries no hypotheses at all. Downstream, this theorem is destructured to feed `exists_source_subgroup_and_ambient_full_finpartition_sequences`: the two Kazhdan pairs give the expanding partition sequences for the subgroup and ambient actions on any hypothetical sofic approximation of the nine-prefix group, and the exactness of the generator fields eliminates any mismatch between the analytic Kazhdan data and the combinatorial partition data. It is the point where the argument becomes fully unconditional on the group-theoretic side.

*Remaining declarations in this section:*

|                                                                      |     |                                                                                             |
|----------------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------|
| <code>exists_symmetric_generating_finset_of_finitelyGenerated</code> | THM | Any finitely generated group has a finite symmetric generating set containing the identity. |
| <code>exists_actual_source_alpha_symmetric_generators</code>         | THM | Produces such a symmetric generating finset for the alpha prefix elementary group.          |

## 8.6 Component size comparison bad sets (SourceCommonComponentComparisonBad)

Controls the set of vertices at which a permutation shrinks the size of their partition component by more than a factor  $(1 - \eta)$  (`comparisonBad`; family version `familyComparisonBad`). Two regimes are treated. For permutations coming from the subgroup action, a comparison-bad vertex must cross parts of the partition, so its density is dominated by the vanishing crossing density (`subgroup_comparisonBad_density_tendsto_zero`). For transported permutations, a matching argument through the transported partition, its insufficient-overlap components, and the maximum-overlap parts confines bad vertices to a set of size at most the insufficient-overlap mass plus the overlap losses (`comparisonBad_card_le_overlap_and_loss`), both of which vanish in density by hypothesis (`transported_comparisonBad_density_tendsto_zero`). `combined_positive_comparisonBad_density_tendsto_zero` joins the two families by `Sum.elim` and sums the limits, certifying that component sizes are asymptotically almost preserved by the entire combined family.

**`comparisonBad_subset_matchedWordPreimageBad` thm**

```

theorem comparisonBad_subset_matchedWordPreimageBad
  {V : Type*} [Fintype V] [DecidableEq V]
  (Q : Finpartition (Finset.univ : Finset V))
  (T : Equiv.Perm V) (eta : ℝ) :
  comparisonBad Q T eta ≤
    SoficGroups.matchedWordPreimageBad
      (Finset.univ : Finset V)
      (overlapRetainedComponents Q T eta)
      (SoficGroups.maximumOverlapPart Q) T := by
-- proof: 46 lines, omitted

```

Handles the transported permutation  $T$ , which need not respect the partition  $Q$  at all. The transported partition `transportedUnivFinpartition` has as parts the  $T$ -images of the parts of  $Q$ . Discarding the parts with insufficient overlap against  $Q$  (`insufficientOverlapComponents`) leaves `overlapRetainedComponents`. For a



vertex  $x$ , its  $Q$ -part maps under  $T$  to the transported part containing  $T(x)$ ; if moreover  $T(x)$  lies in the matched core — the region where a retained transported part meets its maximum-overlap part `maximumOverlapPart` of  $Q$  — then the retention condition yields the quantitative bound that the component size of  $T(x)$  is at least  $(1 - \eta)$  times that of  $x$ , via `partitionComponentSize_transport_lower_of_retained`, so  $x$  is not comparison-bad. Contrapositively, every comparison-bad vertex lies in `matchedWordPreimageBad`. Chaining with `matchedWordPreimageBad_card_le` and the counting lemma `matchedCore_missing_card_le_overlap_and_loss` bounds the number of bad vertices by the insufficient-overlap mass plus the total loss  $|C| - |C \cap D|$ , where  $D$  is the maximum-overlap part, summed over transported parts — precisely the two quantities that the transport hypotheses drive to zero in density. This lemma is where the geometric matching between a partition and its permuted image becomes a size-comparison statement usable by the rank quantization machinery.

**combined\_positive\_comparisonBad\_density\_tendsto\_zero** *thm*

```

theorem combined_positive_comparisonBad_density_tendsto_zero
  (V : ℕ → Type*) [V n, Fintype (V n)]
  [V n, DecidableEq (V n)]
  (ι κ : Type*) [Fintype ι] [Fintype κ]
  (Q : (n : ℕ) → Finpartition (Finset.univ : Finset (V n)))
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))
  (T : (n : ℕ) → κ → Equiv.Perm (V n))
  (eta : ℕ → ℝ) (heta : ∀ n, 0 ≤ eta n)
  (hcross : ∀ i : ι, Tendsto
    (fun n =>
      ((SoficGroups.partitionWordCrossing
        (Q n) (σ n i)).card : ℝ) /
        Fintype.card (V n))
    atTop (nhds 0))
  (hoverlap : ∀ j : κ, Tendsto
    (fun n =>
      (∑ C ∈ SoficGroups.insufficientOverlapComponents
        (SoficGroups.transportedUnivFinpartition
          (Q n) (T n j)) (Q n) (eta n), (C.card : ℝ)) /
        Fintype.card (V n))
    atTop (nhds 0))
  (hloss : ∀ j : κ, Tendsto
    (fun n =>
      (∑ C ∈
        (SoficGroups.transportedUnivFinpartition
          (Q n) (T n j)).parts,
        ((C.card : ℝ) -
          ((C n SoficGroups.maximumOverlapPart (Q n) C).card : ℝ))) /
        Fintype.card (V n))
    atTop (nhds 0)) :
  Tendsto
    (fun n =>
      ((familyComparisonBad (Q n)
        (Sum.elim (σ n) (T n)) (eta n)).card : ℝ) /
        Fintype.card (V n))
    atTop (nhds 0)) := by
  -- proof: 35 lines, omitted

```

Merges the two control mechanisms into a single statement about the joint family `Sum.elim` of  $\sigma_n$  and  $T_n$  indexed by  $\iota \oplus \kappa$ : the density of `familyComparisonBad` — index-vertex pairs where the component size shrinks by more than a  $(1 - \eta_n)$  factor, divided by the vertex count — tends to zero. For a subgroup index the bound comes from the vanishing partition-crossing density via `subgroup_comparisonBad_density_tendsto_zero`: a bad vertex must change parts, and crossings are asymptotically rare. For a transported index it comes from the vanishing insufficient-overlap and overlap-loss densities via `transported_comparisonBad_density_tendsto_zero`. The proof is then a finite-sum limit argument: `familyComparisonBad_card` decomposes the joint count as the sum over indices of individual bad counts, and `tendsto_finetSum` adds up the individual vanishing limits after rewriting the density as a sum of densities. This theorem supplies exactly the bad-density hypothesis required by the offset-selection theorem of the next namespace; it certifies that along the hypothetical sofic sequence, almost every vertex sees its  $Q$ -component size essentially preserved by every permutation in the combined

subgroup-plus-transport family, which is what makes a common almost-invariant rank function possible.

*Remaining declarations in this section:*

|                                                             |     |                                                                                                                   |
|-------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------|
| <code>comparisonBad</code>                                  | DEF | Vertices whose partition component size shrinks below $(1 - \eta)$ times its original size under the permutation. |
| <code>familyComparisonBad</code>                            | DEF | Index-vertex pairs that are comparison-bad for the corresponding member of a permutation family.                  |
| <code>familyComparisonBad_card</code>                       | THM | The family bad set's cardinality is the sum of the individual bad sets' cardinalities.                            |
| <code>comparisonBad_subset_partitionWord Crossing</code>    | THM | For $\eta \geq 0$ , every comparison-bad vertex crosses partition parts.                                          |
| <code>overlapRetainedComponents</code>                      | DEF | Transported partition parts retaining sufficient overlap with the reference partition.                            |
| <code>matchedCore_missing_card_le_overlap_and_loss</code>   | THM | Bounds vertices outside the matched core by insufficient-overlap mass plus maximum-overlap losses.                |
| <code>comparisonBad_card_le_overlap_and_loss</code>         | THM | The comparison-bad count is at most insufficient-overlap mass plus maximum-overlap losses.                        |
| <code>cast_card_sdiff_eq_sub_card_inter</code>              | THM | Real-cast counting identity $ C \setminus D  =  C  -  C \cap D $ .                                                |
| <code>subgroup_comparisonBad_density_tendsto_zero</code>    | THM | Bad density vanishes for permutations with vanishing partition crossing density.                                  |
| <code>transported_comparisonBad_density_tendsto_zero</code> | THM | Bad density vanishes for transports with vanishing overlap-deficiency and overlap-loss densities.                 |

## 8.7 Random-offset log-rank quantization (SourceCommonOffsetMidrankEnergy)

Quantizes component sizes into an integer rank almost invariant under the permutation family. `componentLogRank` applies the offset floor rank construction to the logarithm of `realComponentSize`, with bucket width  $H$  and offset  $r$ . Since the previous segment makes the comparison-bad density vanish, a rank drop at a good vertex forces the logarithm of the component size to cross a bucket boundary while moving only by roughly  $\eta_n$ ; averaging over the offset makes this rare when  $\eta_n/H_n \rightarrow 0$ . `exists_common_component_log_rank_with_vanishing_drops` extracts deterministic offsets  $r_n \in [0, H_n)$  with vanishing rank-drop density via the exceptional-rank-offset machinery, and `exists_common_component_log_rank_with_vanishing_midrank_energy` augments this with a vanishing midrank permutation energy computed against a second partition with vanishing crossing density.

**`exists_common_component_log_rank_with_vanishing_drops`** thm

```
theorem exists_common_component_log_rank_with_vanishing_drops
  (V : ℕ → Type*)
  [V n, Fintype (V n)] [V n, Nonempty (V n)]
  [V n, DecidableEq (V n)]
  (ι : Type*) [Fintype ι]
  (Q : (n : ℕ) → Finpartition (Finset.univ : Finset (V n)))
  (p : (n : ℕ) → ι → Equiv.Perm (V n))
  (eta H : ℕ → ℝ)
  (heta0 : ∀ n, 0 ≤ eta n)
  (heta1 : ∀ n, eta n < 1)
  (hH : ∀ n, 0 < H n)
  (heta : Tendsto eta atTop (nhds 0))
  (hratio : Tendsto (fun n => eta n / H n) atTop (nhds 0))
  (hbad : Tendsto
    (fun n =>
      ((componentSizeComparisonBad (Q n) (p n) (eta n)).card : ℝ) /
      Fintype.card (V n))
    atTop (nhds 0)) :
  ∃ r : ℕ → ℝ,
    (∀ n, r n ∈ Set.Ico 0 (H n)) ∧
```

```

Tendsto
  (fun n =>
    (∑ i : ι,
      ((SoficGroups.MidrankPermutationEnergy.rankDecreasingVertices
        Finset.univ
        (componentLogRank (Q n) (H n) (r n))
        (p n i)).card : ℝ)) /
      Fintype.card (V n))
    atTop (nhds 0) := by
-- proof: 59 lines, omitted

```

The random-offset argument. Fix bucket widths  $H_n > 0$  and tolerances  $\eta_n \rightarrow 0$  with  $\eta_n/H_n \rightarrow 0$ . Ranks are given by `componentLogRank`: the floor quantization of the logarithm of the component size into buckets of width  $H_n$  shifted by an offset  $r$ . A permutation can decrease the rank of a vertex only if either the corresponding pair lies in `componentSizeComparisonBad` (whose density vanishes by hypothesis), or the component size shrinks by at most the factor  $(1 - \eta_n)$  — so its logarithm moves very little — yet still crosses a bucket boundary, an event whose measure, averaged over a uniformly distributed offset  $r \in [0, H_n)$ , is of order  $\eta_n/H_n$  per pair. The averaging is carried out abstractly by `exists_common_log_rank_offsets_tendsto_zero_except` from the `ExceptionalRankOffset` development, applied to the index set of pairs with the before and after value functions built from `realComponentSize`; positivity of component sizes (parts of a finpartition are nonempty) keeps the logarithms defined. The conclusion is a deterministic sequence of offsets  $r_n \in [0, H_n)$  for which the density of rank-decreasing vertices, summed over the whole family, vanishes; `rankDropCount_eq_sum_rankDecreasingVertices` rewrites the abstract drop count into the concrete per-permutation form. Vanishing rank drops make the quantized rank an almost-invariant integer label, the structure the midrank energy bound exploits next.

**exists\_common\_component\_log\_rank\_with\_vanishing\_midrank\_energy** *thm*

```

theorem exists_common_component_log_rank_with_vanishing_midrank_energy
  (V : ℕ → Type*)
  [∀ n, Fintype (V n)] [∀ n, Nonempty (V n)]
  [∀ n, DecidableEq (V n)]
  (ι : Type*) [Fintype ι]
  (Q A : (n : ℕ) → Finpartition (Finset.univ : Finset (V n)))
  (p : (n : ℕ) → ι → Equiv.Perm (V n))
  (eta H : ℕ → ℝ)
  (heta0 : ∀ n, 0 ≤ eta n)
  (heta1 : ∀ n, eta n < 1)
  (hH : ∀ n, 0 < H n)
  (heta : Tendsto eta atTop (nhds 0))
  (hratio : Tendsto (fun n => eta n / H n) atTop (nhds 0))
  (hbad : Tendsto
    (fun n =>
      ((componentSizeComparisonBad (Q n) (p n) (eta n)).card : ℝ) /
      Fintype.card (V n))
    atTop (nhds 0))
  (hcross : Tendsto
    (fun n =>
      (∑ i : ι,
        ((SoficGroups.partitionWordCrossing (A n) (p n i)).card : ℝ)) /
        Fintype.card (V n))
    atTop (nhds 0)) :
  ∃ r : ℕ → ℝ,
  (∀ n, r n ∈ Set.Ico 0 (H n)) ∧
  Tendsto
    (fun n =>
      (∑ i : ι,
        ((SoficGroups.MidrankPermutationEnergy.rankDecreasingVertices
          Finset.univ
          (componentLogRank (Q n) (H n) (r n))
          (p n i)).card : ℝ)) /
          Fintype.card (V n))
    atTop (nhds 0) ∧
  Tendsto

```

```

(fun n =>
  (∑ i : ι, ∑ x : V n,
    (SoficGroups.MidrankPermutationEnergy.partitionVertexMidrank
      (A n) (componentLogRank (Q n) (H n) (r n)) (p n i x) -
    -- statement truncated; proof: 13 lines, omitted

```

Extends the offset selection with an energy conclusion: the same offsets  $r_n$  additionally make the midrank permutation energy vanish, namely the normalized sum, over the family and over all vertices, of the squared difference of `partitionVertexMidrank` values before and after the permutation, where the midrank of a vertex is computed from the second partition  $A_n$  and the rank function `componentLogRank`. The proof first obtains the offsets from `exists_common_component_log_rank_with_vanishing_drops`, then applies the engine `partitionVertexMidrank_permutation_energy_tendsto_zero` of `MidrankPermutationEnergy`, whose two hypotheses are exactly the vanishing crossing density of  $A_n$  under the family and the vanishing rank-drop density just established. The intuition: the midrank of a vertex changes only if the permutation moves it across a part of  $A_n$  (rare by the crossing hypothesis) or changes its integer rank (rare by the offset choice), and the engine converts these rarity statements into an  $\ell^2$  energy estimate. The resulting package — offsets in  $[0, H_n)$ , vanishing drops, vanishing midrank energy — is precisely the shape consumed by the next namespace, which instantiates it at the combined subgroup-plus-transport permutation family.

*Remaining declarations in this section:*

|                                                 |     |                                                                                |
|-------------------------------------------------|-----|--------------------------------------------------------------------------------|
| <code>realComponentSize</code>                  | DEF | The cardinality of a vertex's partition component, cast to the reals.          |
| <code>componentLogRank</code>                   | DEF | Integer vertex rank: offset floor rank of the logarithm of the component size. |
| <code>componentSizeComparisonBad</code>         | DEF | Pairs where the real component size shrinks by more than factor $(1 - \eta)$ . |
| <code>componentSizeComparison_of_not_mem</code> | THM | Pairs outside the bad set satisfy the component size comparison inequality.    |

## 8.8 Combined family rank offsets (SourceCommonComponentRankNoBad)

A one-theorem junction namespace. `exists_common_positive_component_log_rank_with_vanishing_midrank_energy` instantiates the offset-selection result at the combined family, joined by `Sum.elim`, of subgroup permutations  $\sigma$  and transported permutations  $T$ : from vanishing crossing density of the subgroup partition under  $\sigma$ , vanishing insufficient-overlap and overlap-loss densities of the transports, and vanishing crossing density of the ambient partition under the joint family, it produces offsets  $r_n \in [0, H_n)$  for which both the rank-drop density and the midrank permutation energy of the joint family vanish. Its proof identifies the two definitions of the comparison-bad set used in the two supporting namespaces and chains their main theorems. This is the packaged almost-invariant rank structure that a hypothetical sofic approximation would have to carry.

**`exists_common_positive_component_log_rank_with_vanishing_midrank_energy` thm**

```

theorem exists_common_positive_component_log_rank_with_vanishing_midrank_energy
  (V : ℕ → Type*)
  [∀ n, Fintype (V n)] [∀ n, Nonempty (V n)]
  [∀ n, DecidableEq (V n)]
  (ι κ : Type*) [Fintype ι] [Fintype κ]
  (Q A : (n : ℕ) → Finpartition (Finset.univ : Finset (V n)))
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))
  (T : (n : ℕ) → κ → Equiv.Perm (V n))
  (eta H : ℕ → ℝ)
  (heta0 : ∀ n, 0 ≤ eta n)
  (heta1 : ∀ n, eta n < 1)
  (hH : ∀ n, 0 < H n)
  (heta : Tendsto eta atTop (nhds 0))
  (hratio : Tendsto (fun n => eta n / H n) atTop (nhds 0))
  (hcrossQ : ∀ i : ι, Tendsto

```

```

(fun n =>
  ((SoficGroups.partitionWordCrossing
    (Q n) (σ n i)).card : ℝ) /
  Fintype.card (V n))
atTop (nhds 0))
(hoverlap : ∀ j : κ, Tendsto
  (fun n =>
    (∑ C ∈ SoficGroups.insufficientOverlapComponents
      (SoficGroups.transportedUnivFinpartition
        (Q n) (T n j)) (Q n) (eta n), (C.card : ℝ)) /
      Fintype.card (V n))
    atTop (nhds 0))
  (hloss : ∀ j : κ, Tendsto
    (fun n =>
      (∑ C ∈
        (SoficGroups.transportedUnivFinpartition
          (Q n) (T n j)).parts,
          ((C.card : ℝ) -
            ((C n SoficGroups.maximumOverlapPart (Q n) C).card : ℝ))) /
          Fintype.card (V n))
        atTop (nhds 0))
    (hcrossA : Tendsto
      (fun n =>
        (∑ i : ι ⊕ κ,
          ((SoficGroups.partitionWordCrossing
            -- statement truncated; proof: 46 lines, omitted

```

An assembly theorem at an important junction: it specializes the offset machinery to the concrete combined family. Inputs: the subgroup-side partition sequence  $Q_n$  (providing component sizes) and the ambient-side sequence  $A_n$  (providing midranks); subgroup permutations  $\sigma$  with vanishing  $Q$ -crossing density; transported permutations  $T$  with vanishing insufficient-overlap and overlap-loss densities; and vanishing crossing density of  $A_n$  under the joined family. The proof first invokes `combined_positive_comparisonBad_density_tendsto_zero` to conclude that the family comparison-bad density for the `Sum.elim` family vanishes; a short extensionality argument identifies the two syntactically different bad sets used by the supporting namespaces (`componentSizeComparisonBad` versus `familyComparisonBad` — both filter the same inequality between real component sizes). It then feeds this into `exists_common_component_log_rank_with_vanishing_midrank_energy` at index type  $\iota \oplus \kappa$ . Output: offsets  $r_n \in [0, H_n)$  such that both the rank-drop density and the midrank permutation energy of the combined family vanish. In the global argument this says that a hypothetical sofic approximation of the nine-prefix group would carry an integer log-rank labelling that is almost invariant under both the subgroup action and the compression transports — the structure that the spectral-gap inequalities will ultimately show cannot exist.

## 8.9 Squared energy under sofic actions (KunPositiveWordMidrankEnergy)

Develops the squared  $\ell^2$  energy `squaredPermutationEnergy` of a permutation acting on a test function and shows that, along a sofic approximation, vanishing normalized energy is a property closed under the group operations. Energy is symmetric under inverses, almost subadditive under products (`squaredPermutationEnergy_mul_le`), and, for  $[0, 1]$ -valued functions, stable under replacing a permutation by a Hamming-close one (`squaredPermutationEnergy_le_add_hammingDist`); the approximate multiplicativity defining a sofic approximation then transfers these facts to the action maps (`sofic_action_squared_energy_mul`, `sofic_action_squared_energy_inv`). The main theorem `action_energy_tendsto_zero_of_positive_generator_sum` performs closure induction: if the summed energies over an indexed generating family vanish, the energy of every element's action vanishes, and hence so does the sum over any finite subset. Energy decay thus needs checking only on the positive generators built earlier.

**squaredPermutationEnergy\_le\_add\_hammingDist** *thm*

```

theorem squaredPermutationEnergy_le_add_hammingDist
  {V : Type*} [Fintype V] [DecidableEq V]
  (f : V → ℝ) (hf0 : ∀ x, 0 ≤ f x) (hf1 : ∀ x, f x ≤ 1)
  (p q : Equiv.Perm V) :

```

```

squaredPermutationEnergy f p ≤
  squaredPermutationEnergy f q +
    (hammingDist (fun x => p x) (fun x => q x) : ℝ) := by
-- proof: 37 lines, omitted

```

The perturbation lemma that lets energy estimates survive replacing an exact composite permutation by the sofic model's approximate one. For  $f$  with values in  $[0, 1]$  and permutations  $p, q$ , it bounds the energy of  $p$  by the energy of  $q$  plus the unnormalized Hamming distance between  $p$  and  $q$  as functions. The pointwise claim: at a vertex where  $px = qx$  the summands agree; where they differ,  $(f(px) - f(x))^2 \leq 1$  is absorbed by the indicator 1, the `nlarith` certificate being  $(1 - (f(px) - f(x)))(1 + (f(px) - f(x))) \geq 0$  — exactly the statement that the difference lies in  $[-1, 1]$ . Summing over vertices and converting the indicator sum into `hammingDist` via `Finset.sum_boole` finishes. The sequential corollary `squaredPermutationEnergy_tendsto_of_normalizedHamming` divides by the vertex count: vanishing normalized energy for  $q_n$  together with vanishing `normalizedHamming` distance between  $p_n$  and  $q_n$  gives vanishing normalized energy for  $p_n$ . Since the defining property of a sofic approximation is precisely that the action of  $gh$  is normalized-Hamming-close to the composite of the actions of  $g$  and  $h$ , this lemma is the interface between the exact algebraic closure argument and the merely approximate multiplicativity of the finite models.

#### **action\_energy\_tendsto\_zero\_of\_positive\_generator\_sum thm**

```

theorem action_energy_tendsto_zero_of_positive_generator_sum
  {G ι : Type*} [Group G] [Fintype ι]
  (A : SoficGroups.SoficApproximation G)
  (s : ι → G)
  (hgenerate : Subgroup.closure (Set.range s) = ⊤)
  (f : (n : ℕ) → Fin (A.model n).size → ℝ)
  (hf0 : ∀ n x, 0 ≤ f n x)
  (hf1 : ∀ n x, f n x ≤ 1)
  (hpositive : Tendsto
    (fun n =>
      (∑ i : ι,
        squaredPermutationEnergy (f n)
          ((A.model n).action (s i))) /
        (A.model n).size) atTop (ℕ 0)) :
  ∀ g : G,
  Tendsto
    (fun n =>
      squaredPermutationEnergy (f n) ((A.model n).action g) /
        (A.model n).size) atTop (ℕ 0) := by
-- proof: 40 lines, omitted

```

Upgrades generator-level energy decay to all group elements. Hypotheses: an indexed family  $s : \iota \rightarrow G$  whose range generates  $G$ , test functions  $f_n$  with values in  $[0, 1]$ , and the single limit that the sum over  $i$  of the normalized energies of the actions of  $s(i)$  tends to zero. Step one extracts vanishing of each individual generator energy, since each nonnegative term is dominated by the vanishing sum. Step two is closure induction over the fact that `Subgroup.closure` of the range is  $\top$ : the base case is a generator; the identity has zero energy because the action of 1 is the identity permutation; products are handled by `sofic_action_squared_energy_mul`, which chains the doubling inequality `squaredPermutationEnergy_mul_le` with the Hamming perturbation lemma and the approximate multiplicativity of the sofic approximation; inverses by `sofic_action_squared_energy_inv`, which combines invariance of energy under permutation inverse (`squaredPermutationEnergy_inv`) with the asymptotic closeness of the inverse permutation and the action of the inverse element (`sofic_action_inverse_normalizedHamming_tendsto_zero`). The corollary `sum_action_energy_tendsto_zero_of_positive_generator_sum` then sums over any finite subset of  $G$ . In context the  $f_n$  are midrank-derived test functions: energy decay needs verification only on the finitely many positive generators — the included subgroup generators and the two compressions — but is then available for arbitrary elements appearing in the spectral-gap argument.

*Remaining declarations in this section:*

|                                       |                  |                                                                                     |
|---------------------------------------|------------------|-------------------------------------------------------------------------------------|
| <code>squaredPermutationEnergy</code> | <code>DEF</code> | The squared energy $\sum_x (f(px) - f(x))^2$ of a permutation acting on a function. |
|---------------------------------------|------------------|-------------------------------------------------------------------------------------|

|                                                          |     |                                                                                                               |
|----------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------|
| squaredPermutationEnergy_nonneg                          | THM | The squared permutation energy is nonnegative.                                                                |
| squaredPermutationEnergy_inv                             | THM | Energy is invariant under inverting the permutation, by reindexing the sum.                                   |
| squaredPermutationEnergy_mul_le                          | THM | Energy of a product is at most twice the sum of the factors' energies.                                        |
| squaredPermutationEnergy_tendsto_of_normalizedHamming    | THM | Vanishing normalized energy transfers between permutation sequences at vanishing normalized Hamming distance. |
| squaredPermutationEnergy_mul_tendsto_zero                | THM | Vanishing normalized energies for two sequences imply vanishing for their pointwise products.                 |
| sofic_action_inverse_normalizedHamming_tendsto_zero      | THM | The inverse of an action permutation is asymptotically Hamming-close to the action of the inverse.            |
| sofic_action_squared_energy_mul                          | THM | Vanishing normalized action energy is preserved under group multiplication in a sofic approximation.          |
| sofic_action_squared_energy_inv                          | THM | Vanishing normalized action energy is preserved under group inversion in a sofic approximation.               |
| sum_action_energy_tendsto_zero_of_positive_generator_sum | THM | Hence summed action energy over any finite set of group elements vanishes.                                    |

## 8.10 Additive median Poincare inequality (KunAdditiveMedianPoincare)

Proves a Poincare inequality for permutation families satisfying an approximate Cheeger expansion hypothesis  $\gamma \min(|A|, N - |A|) \leq \partial A + \varepsilon$ . The  $\ell^1$  Dirichlet form is `permutationRealVariation`. A layer-cake identity (`permutationRealVariation_subtract_layer`) shows that subtracting the bottom value layer of a nonnegative function removes exactly twice that value times the boundary of its support, and strong induction on support size yields the coarea bound `additive_permutation_small_support_coarea` for functions supported on at most half the vertices. Splitting an arbitrary  $[0, 1]$ -valued function at a finite median with small level sets into positive and negative parts whose variations add exactly gives the median absolute deviation bound, which is then converted, via  $(f - m)^2 \leq |f - m|$  and the mean-minimization property, into the variance inequality `additive_permutation_finiteVariance`:  $2\gamma N \text{Var}(f) \leq \text{variation} + 4\varepsilon$ .

**additive\_permutation\_small\_support\_coarea** *thm*

```

theorem additive_permutation_small_support_coarea
  {V ι : Type*} [Fintype V]
  [Fintype ι] [DecidableEq V]
  (σ : ι → Equiv.Perm V) (γ ε : ℝ)
  (hε : 0 ≤ ε)
  (hexp : ∀ A : Finset V,
    γ * min (A.card : ℝ)
      ((Fintype.card V : ℝ) - A.card) ≤
      (SoficGroups.boundary σ A : ℝ) + ε)
  (f : V → ℝ) (M : ℝ) (hM : 0 ≤ M)
  (hf : ∀ x, 0 ≤ f x)
  (hfM : ∀ x, f x ≤ M)
  (hhalf :
    2 * (SoficGroups.CheegerPoincare.positiveSupport f).card ≤
    Fintype.card V) :
  2 * γ * (∑ x : V, f x) ≤
  permutationRealVariation σ f + 2 * ε * M := by
-- proof: 138 lines, omitted

```

The discrete coarea, or Cheeger, inequality in  $\ell^1$  form. Setting: a family  $\sigma$  of permutations satisfying the approximate expansion hypothesis  $\gamma \min(|A|, N - |A|) \leq \partial A + \varepsilon$  for every vertex set  $A$ , and a function  $f$  with  $0 \leq f \leq M$  whose positive support occupies at most half the vertices. Conclusion:  $2\gamma \sum f \leq \text{variation} + 2\varepsilon M$ . The proof is strong induction on the size of the positive support  $A$ , peeling one value layer at a time. With  $m$  the minimum positive value of  $f$ , the identity `permutationRealVariation_subtract_layer` shows the variation of  $f$  equals the variation of the truncated function  $g = f - m$  on  $A$  (zero outside) plus exactly



$2m \cdot \partial A$ ; its proof matches entering and exiting crossings of  $A$  via `card_entering_eq_card_exiting` from `KunThomFiberCoarea`. The truncation  $g$  has strictly smaller support — a vertex attaining the minimum drops out — values in  $[0, M - m]$ , and the same half-support bound, so the inductive hypothesis applies. The expansion hypothesis at  $A$ , where the minimum in the hypothesis is attained at  $|A|$  by the half-support assumption, contributes  $\gamma m|A| \leq m(\partial A + \varepsilon)$ , and the mass identity  $\sum f = \sum g + m|A|$  lets `nlinarith` close the bound. The error term  $2\varepsilon M$  accumulates the  $\varepsilon$ -slack across all layers, totaling twice  $\varepsilon$  times the maximal value rather than the number of layers.

**additive\_permutation\_finiteVariance** *thm*

```

theorem additive_permutation_finiteVariance
  {V 1 : Type*} [Fintype V] [Nonempty V]
  [Fintype 1] [DecidableEq V]
  (σ : 1 → Equiv.Perm V) (γ ε : ℝ)
  (hγ : 0 ≤ γ)
  (hε : 0 ≤ ε)
  (hexp : ∀ A : Finset V,
    γ * min (A.card : ℝ)
      ((Fintype.card V : ℝ) - A.card) ≤
      (SoficGroups.boundary σ A : ℝ) + ε)
  (f : V → ℝ)
  (hf : ∀ x, 0 ≤ f x)
  (hf_one : ∀ x, f x ≤ 1) :
  2 * γ * (Fintype.card V : ℝ) *
    SoficGroups.CheegerPoincare.finiteVariance f ≤
    permutationRealVariation σ f + 4 * ε := by
-- proof: 6 lines, omitted

```

The segment’s final Poincare inequality and the exported form: for a permutation family satisfying the approximate Cheeger condition and any  $f$  with values in  $[0, 1]$ ,  $2\gamma N \cdot \text{Var}(f) \leq \text{variation} + 4\varepsilon$ , where  $N$  is the vertex count and  $\text{Var}$  is `finiteVariance` from the `CheegerPoincare` development. It is derived from the median bound in three steps. `additive_permutation_median_absolute_deviation` splits  $f$  at a median  $m \in [0, 1]$ , produced by `exists_bounded_finite_real_median` so that both strict level sets occupy at most half the vertices, into the parts  $\max(f - m, 0)$  and  $\max(m - f, 0)$ ; each satisfies the half-support hypothesis of the coarea bound with  $M = 1$ , their variations add exactly to that of  $f$  (`permutationRealVariation_positive_negative`, from the pointwise identity `positive_negative_abs_eq`), and their masses add to  $\sum |f - m|$ , giving  $2\gamma \sum_x |f(x) - m| \leq \text{variation} + 4\varepsilon$ . Then the pointwise inequality  $(f(x) - m)^2 \leq |f(x) - m|$ , valid because differences lie in  $[-1, 1]$ , and the mean-minimization fact `sum_sq_sub_finiteMean_le` replace absolute deviation about the median by squared deviation about the mean; `card_mul_finiteVariance` rewrites the sum in variance form. In the global scheme this inequality is the spectral-gap side of the eventual contradiction: expansion forces  $N \cdot \text{Var}(f)$  to be controlled by the  $\ell^1$  variation, which the sofic energy estimates make vanish for the midrank test functions.

*Remaining declarations in this section:*

|                                                             |     |                                                                                                        |
|-------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------|
| <code>permutationRealVariation</code>                       | DEF | The $\ell^1$ variation $\sum_i \sum_x  f(\sigma_i x) - f(x) $ of a permutation family.                 |
| <code>sum_indicator_real_eq_mul_card</code>                 | THM | Summing a constant over an indicator equals the constant times the filtered cardinality.               |
| <code>permutationRealVariation_subtract_layer</code>        | THM | Layer-cake identity: subtracting the bottom layer $m$ on $A$ removes exactly $2m$ times the boundary.  |
| <code>positive_negative_abs_eq</code>                       | THM | Positive-part and negative-part differences sum to the full absolute difference.                       |
| <code>permutationRealVariation_positive_negative</code>     | THM | The variation of $f$ splits exactly into the variations of its positive and negative parts about $m$ . |
| <code>exists_bounded_finite_real_median</code>              | THM | A median $m \in [0, 1]$ exists whose strict lower and upper level sets each cover at most half.        |
| <code>additive_permutation_median_absolute_deviation</code> | THM | Median bound $2\gamma \sum_x  f(x) - m  \leq \text{variation} + 4\varepsilon$ .                        |

additive\_permutation\_median\_variance THM Mean-square bound:  $2\gamma$  times summed squared deviation from the mean is at most variation  $+4\epsilon$ .

## 8.11 Variation calculus for completed component permutations (KunCompletedComponentMidrankVariation)

This segment develops a total-variation calculus for real test functions under a finite family of permutations. The quantity `permutationRealVariation` sums  $|f(\sigma_i x) - f x|$  over all generators and vertices; a Cauchy-Schwarz estimate bounds its square by the Dirichlet energy, so vanishing normalized energy forces vanishing normalized variation. The section then compares the global variation with the variations of completed per-component permutations: restricting  $\sigma$  to a part  $C$  of a finpartition and completing it to an honest permutation of  $C$  changes the variation by at most the boundary of  $C$  (for functions valued in  $[0, 1]$ ), and summing over all parts costs only the total boundary. These bounds let the expander and transport argument localize the midrank test function to components of an almost-invariant partition at asymptotically negligible cost.

`completed_component_real_variation_le` thm

```
theorem completed_component_real_variation_le
  {V 1 : Type*} [DecidableEq V] [Fintype 1]
  (σ : 1 → Equiv.Perm V)
  (C : Finset V)
  (τ : 1 → Equiv.Perm {x : V // x ∈ C})
  (ht : ∀ i (x : V) (hx : x ∈ C) (_hy : σ i x ∈ C),
    ((τ i (x, hx) : {x : V // x ∈ C}) : V) = σ i x)
  (f : V → ℝ)
  (hf0 : ∀ x, 0 ≤ f x)
  (hf1 : ∀ x, f x ≤ 1) :
  permutationRealVariation τ (fun x => f (x : V)) ≤
    (Σ i : 1, Σ x ∈ C, |f (σ i x) - f x|) +
    (SoficGroups.boundary σ C : ℝ) := by
-- proof: 58 lines, omitted
```

`completed_component_real_variation_le` bounds the variation of a completed permutation family on a single part. The setting:  $\sigma$  is a family of permutations of the ambient vertex set,  $C$  a finite subset, and  $\tau$  a family of permutations of the subtype of  $C$  that agrees with  $\sigma$  at every point  $x \in C$  whose image  $\sigma_i x$  also lies in  $C$ ; the test function  $f$  takes values in  $[0, 1]$ . The conclusion is that the variation of the restriction of  $f$  under  $\tau$  is at most  $\sum_i \sum_{x \in C} |f(\sigma_i x) - f x|$  plus the boundary count `SoficGroups.boundary` of  $C$  under  $\sigma$ . The proof is pointwise: if  $\sigma_i x$  stays in  $C$ , the completed and original increments coincide by the agreement hypothesis; if it exits, the increment is at most one (since  $0 \leq f \leq 1$ ) and the point is exactly one of those counted by the boundary filter. Summing and converting subtype sums to sums over  $C$  gives the bound. This is the key cost estimate that lets the argument replace the internal, non-bijective restriction of a sofic model to a partition part by an honest permutation of that part, paying only boundary terms which later vanish in density.

*Remaining declarations in this section:*

|                                                                         |     |                                                                                                                               |
|-------------------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------|
| <code>permutationRealVariation</code>                                   | DEF | Total variation of $f$ under a permutation family: $\sum_i \sum_x  f(\sigma_i x) - f x $ .                                    |
| <code>permutationRealVariation_sq_le_card_mul_energy</code>             | THM | Cauchy-Schwarz bound: the squared variation is at most index cardinality times vertex cardinality times the Dirichlet energy. |
| <code>normalized_permutationRealVariation_tendsto_zero_of_energy</code> | THM | If the normalized Dirichlet energy vanishes, the normalized variation vanishes, via the square-root bound and squeezing.      |
| <code>sum_partition_parts_eq</code>                                     | THM | Summing a function part by part over a finpartition equals summing over all vertices.                                         |
| <code>sum_completed_component_real_variation_le</code>                  | THM | Summed over parts: completed component variations total at most the global variation plus the sum of part boundaries.         |

## 8.12 Expansion forces vanishing weighted midrank variance (KunGlobalActualAdditiveMidrankVariance)

This segment combines the variation calculus with the additive median Poincare inequality. Under the hypothesis that every small subset of every partition part expands (boundary at least  $\gamma|E|$  whenever  $2|E| \leq |C|$ ), the finite-level theorem bounds  $2\gamma$  times the weighted sum of component `midrankVariance` values by the variation of the global partition midrank function plus five boundary terms. Two asymptotic corollaries follow by squeezing: if the normalized total part boundary and the normalized midrank Dirichlet energy both vanish, then the normalized weighted component midrank variance vanishes; and the required completions of the internal permutations can be chosen canonically via `exists_completion_of_internal_permutation`, removing the explicit completion hypothesis. The output is the quantitative bridge from spectral expansion of the hypothetical sofic model to near-constancy of ranks on components, feeding the rank invariance section.

**weighted\_component\_midrankVariance\_additive\_bound** *thm*

```
theorem weighted_component_midrankVariance_additive_bound
  {V 1 : Type*} [Fintype V] [DecidableEq V] [Fintype 1]
  (P : Finpartition (Finset.univ : Finset V))
  (σ : 1 → Equiv.Perm V)
  (τ : (C : Finset V) → 1 → Equiv.Perm {x : V // x ∈ C})
  (γ : ℝ) (hγ : 0 ≤ γ)
  (hτ : ∀ C ∈ P.parts, ∀ i (x : V)
    (hx : x ∈ C) (hy : σ i x ∈ C),
    ((τ C i (x, hx) : {x : V // x ∈ C}) : V) = σ i x)
  (hexpand : ∀ C ∈ P.parts, ∀ E : Finset V, E ⊆ C →
    2 * E.card ≤ C.card →
    γ * (E.card : ℝ) ≤ (SoficGroups.boundary σ E : ℝ))
  (b : V → ℤ) :
  2 * γ *
    (∑ C ∈ P.parts,
      (C.card : ℝ) *
        SoficGroups.midrankVariance
          (SoficGroups.componentRankMassList C b)) ≤
    SoficGroups.KunCompletedComponentMidrankVariation.permutationRealVariation
      σ (SoficGroups.MidrankPermutationEnergy.partitionVertexMidrank P b) +
    5 * (∑ C ∈ P.parts, (SoficGroups.boundary σ C : ℝ)) := by
  -- proof: 106 lines, omitted
```

`weighted_component_midrankVariance_additive_bound` is the finite-level spectral estimate at the heart of this segment. Hypotheses:  $P$  is a finpartition of the vertex set,  $\tau$  completes  $\sigma$  on each part, and every part is an additive expander for  $\sigma$ : any subset  $E \subseteq C$  with  $2|E| \leq |C|$  has boundary at least  $\gamma|E|$ . The conclusion bounds  $2\gamma \sum_C |C|$  times the `midrankVariance` of the component rank mass lists by the variation of the global function `partitionVertexMidrank` plus five times the total part boundary. The proof works one part at a time: `completed_component_additive_expansion` converts the expansion hypothesis into a cut inequality for the completed permutations with an additive boundary error, then the median Poincare inequality `additive_permutation_finiteVariance` bounds the finite variance of the component midrank function by the component variation plus four boundary terms; `componentVertexMidrank_finiteVariance_eq` identifies this finite variance with  $|C|$  times the midrank variance. Summing over parts and applying `sum_completed_component_real_variation_le` to remove the completions costs one further boundary term, giving the factor five. The two subsequent corollaries take normalized limits, showing that vanishing boundary density plus vanishing midrank Dirichlet energy force the weighted midrank variance density to zero.

*Remaining declarations in this section:*

|                                                                                       |            |                                                                                                                                                                   |
|---------------------------------------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>weighted_component_midrankVariance_tendsto_zero_of_additive_expansion</code>    | <i>THM</i> | If boundary density and midrank energy vanish, the normalized weighted component midrank variance tends to zero.                                                  |
| <code>weighted_component_midrankVariance_tendsto_zero_of_source_half_expansion</code> | <i>THM</i> | Same vanishing conclusion using default completions from <code>exists_completion_of_internal_permutation</code> , eliminating the explicit completion hypothesis. |

### 8.13 Common rank selection and arc invariance (KunCommonRankArcInvariance)

This segment converts vanishing midrank variance into a common rank selection on components. For each part one chooses a rank value  $j_n(C)$  maximizing `componentRankMass`; the density of vertices whose rank differs from the selected one vanishes, being bounded by twelve times the weighted variance density via the dominant-mass estimate. Consequently, for every word permutation whose partition-crossing density vanishes, the density of its rank-changing arc vanishes as well, since each rank change is charged either to a crossing point or to an omitted point. Everything is packaged in `exists_common_rank_invariance_of_midrank_variance`. A final independent lemma records that in any sofic approximation the action of  $g^{-1}$  agrees with the inverse of the action of  $g$  up to vanishing normalized Hamming distance, a compatibility fact used when words involving inverses are evaluated in the model.

**exists\_common\_rank\_invariance\_of\_midrank\_variance** thm

```
theorem exists_common_rank_invariance_of_midrank_variance
  {V : N → Type*}
  [V n, Fintype (V n)] [V n, Nonempty (V n)]
  [V n, DecidableEq (V n)]
  {κ : Type*}
  (P : (n : N) → Finpartition (Finset.univ : Finset (V n)))
  (b : (n : N) → V n → Z)
  (w : (n : N) → κ → Equiv.Perm (V n))
  (hvariance : Tendsto
    (fun n =>
      (Σ C ∈ (P n).parts,
        (C.card : ℝ) *
          SoficGroups.midrankVariance
            (SoficGroups.componentRankMassList C (b n))) /
          Fintype.card (V n))
    atTop (nhds 0))
  (hcross : ∀ i : κ, Tendsto
    (fun n =>
      ((SoficGroups.partitionWordCrossing
        (P n) (w n i)).card : ℝ) /
        Fintype.card (V n))
    atTop (nhds 0)) :
  ∃ j : (n : N) → Finset (V n) → Z,
  (∀ n C, C ∈ (P n).parts →
    j n C ∈ C.image (b n) ∧
    ∀ k ∈ C.image (b n),
      SoficGroups.componentRankMass C (b n) k ≤
      SoficGroups.componentRankMass C (b n) (j n C)) ∧
  Tendsto
    (fun n =>
      (((Finset.univ : Finset (V n)) \
        SoficGroups.RankArcCharging.selectedRankSupport
          (P n) (b n) (j n)).card : ℝ) /
        Fintype.card (V n))
    atTop (nhds 0) ∧
  ∀ i : κ, Tendsto
    (fun n =>
      ((SoficGroups.RankArcCharging.rankChangingArc
        (Finset.univ : Finset (V n))
        (b n) (w n i)).card : ℝ) /
        Fintype.card (V n))
    atTop (nhds 0))
  -- statement truncated; proof: 8 lines, omitted
```

`exists_common_rank_invariance_of_midrank_variance` packages the rank-selection pipeline of this section. Given a partition sequence  $P_n$ , integer rank labels  $b_n$ , and word permutations  $w_n$ , assume the weighted component midrank variance density vanishes and every word has vanishing partition-crossing density. The theorem produces a selection  $j_n(C)$  of a rank value lying in the image of  $b_n$  on each part and maximizing `componentRankMass`, such that the density of vertices outside the selected rank support (points whose rank is not the chosen one) tends to zero, and for every word index the density of the rank-changing arc of  $w_n$  tends to zero. The proof strings together the three preceding lemmas: `exists_maximizing_partition_ranks` chooses maximizers by classical choice; `maximizing_partition_rank_omitted_density_tendsto_zero` bounds the omitted density

by twelve times the variance density, using `actual_weighted_midrank_dominant_mass_le` together with `card_sdiff_selectedRankSupport`; and `rankChangingArc_density_tendsto_zero_of_selected_support` charges each rank-changing arc either to a partition-crossing point or to an omitted point. The upshot: on the hypothetical sofic model, after discarding a vanishing fraction of points, each component carries one common rank preserved by all tracked words. This invariance is what ultimately collides with the behavior the sofic approximation would have to exhibit.

*Remaining declarations in this section:*

|                                                                       |     |                                                                                                                                                            |
|-----------------------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>exists_maximizing_partition_ranks</code>                        | THM | Selects, for each part, a rank value maximizing <code>componentRankMass</code> , using <code>exists_maximal_componentRankMass</code> and classical choice. |
| <code>maximizing_partition_rank_omitted_density_tendsto_zero</code>   | THM | Density of vertices missed by the maximizing rank support vanishes, bounded by twelve times the weighted variance density.                                 |
| <code>rankChangingArc_density_tendsto_zero_of_selected_support</code> | THM | If the omitted density and all partition crossing densities vanish, each word's rank-changing arc density vanishes.                                        |
| <code>sofic_action_inverse_normalized_Hamming_tendsto_zero</code>     | THM | Normalized Hamming distance between the inverse of the action of $g$ and the action of $g^{-1}$ vanishes.                                                  |

## 8.14 First-factor Cayley balls in product approximations (ChosenCayleyMatchedComponent Realization)

A short specialization of the chosen Cayley radius machinery to product groups  $K \times J$ . The sofic approximation of the product is pulled back along the injective first-factor inclusion `MonoidHom.inl`, yielding `sourceFirstFactorApproximation`, a sofic approximation of  $K$  whose action of  $g$  is the product action of  $(g, 1)$ . The generic construction `chosenCayleyRadiusBad`, instantiated with the word scheme `symmetricGeneratorWord` for a finite symmetric generating set  $S$ , gives the bad set `sourceFirstFactorCayleyRadiusBad`, whose density vanishes, and outside of which the orbit map on the ball  $S^k$  is injective. This supplies the largeness input for the matched component realization of the next segment.

**`sourceFirstFactor_injective_ball`** thm

```
theorem sourceFirstFactor_injective_ball
  {K J : Type*} [Group K] [Group J] [DecidableEq K]
  (A : SoficGroups.SoficApproximation (K × J))
  (S : Finset K)
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set K) = ⊤)
  (n k : ℕ)
  {x : Fin (A.model n).size}
  (hx : x ∉ sourceFirstFactorCayleyRadiusBad
    A S hsymmetric hgenerates n k) :
  Set.InjOn
    (fun g : K => (A.model n).action (g, 1) x)
    (↑(S ^ k) : Set K) := by
-- proof: 5 lines, omitted
```

`sourceFirstFactor_injective_ball` is the payoff of this short section. Working with a sofic approximation  $A$  of a product group  $K \times J$ , the section first pulls  $A$  back along the injective homomorphism `MonoidHom.inl` to obtain `sourceFirstFactorApproximation`, a sofic approximation of  $K$  alone whose action of  $g$  is the action of  $(g, 1)$ . The generic Cayley radius machinery, `chosenCayleyRadiusBad` applied with the word scheme `symmetricGeneratorWord` attached to a finite symmetric generating set  $S$  of  $K$ , then supplies a bad set `sourceFirstFactorCayleyRadiusBad` of vanishing density. The theorem states that for any point  $x$  outside this bad set, the orbit map sending  $g$  to the action of  $(g, 1)$  at  $x$  is injective on the ball  $S^k$ . In other words, away from a negligible set, every point of the finite model carries a faithfully embedded copy of a large Cayley ball of the first factor. This is precisely the input the matched component realization needs: combined with the capture lemmas of

the next section it shows that components of the almost-invariant partition contain half-radius Cayley balls and are therefore large, which eventually collides with the spectral estimates.

*Remaining declarations in this section:*

|                                                                    |     |                                                                                                                                    |
|--------------------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------|
| <code>sourceFirstFactorApproximation</code>                        | DEF | Pulls back a sofic approximation of $K \times J$ to $K$ along the injective inclusion <code>MonoidHom.inl</code> .                 |
| <code>sourceFirstFactorCayleyRadiusBad</code>                      | DEF | Bad set where the first-factor Cayley radius test fails, via <code>chosenCayleyRadiusBad</code> for the pulled-back approximation. |
| <code>sourceFirstFactorCayleyRadiusBad_density_tendsto_zero</code> | THM | The first-factor Cayley radius bad set has vanishing normalized density.                                                           |

## 8.15 Canonical bad set captures completion failures (CanonicalProductRadiusBadMatchedCapture)

This segment builds the canonical bad set for the product-radius test and proves the capture lemmas. The finite label set `sourceProductRadiusLabels` collects first-factor ball elements  $(a, 1)$  with  $a \in S^k$ , tracked second-factor table elements  $(1, j)$ , and mixed products  $(a, j)$ ; four membership lemmas record the inclusions. `canonicalProductRadiusBad` unions the first-factor Cayley bad set with `finiteRootBad` on these labels and has vanishing density by a union bound. Two capture results follow: every failure of the source word test (commutation, multiplicativity, or separation) forces membership in the finite root bad set, and each part's `sourceCompletionBad` is contained in the intersection of the part with the canonical `matchedRadiusBad`. The main realization theorem then embeds the half-radius Cayley ball injectively into any part containing a good point.

**`sourceCompletionBad_subset_canonical_matchedRadiusBad` thm**

```
theorem sourceCompletionBad_subset_canonical_matchedRadiusBad
  {K J : Type*} [Group K] [Group J]
  [DecidableEq K] [DecidableEq J]
  (A : SoficGroups.SoficApproximation (K × J))
  (S : Finset K)
  (hsymmetric : ∀ a ∈ S, a⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set K) = T)
  (F : Finset J) (n k : ℕ)
  {U : Finset (Fin (A.model n).size)}
  (P : Finpartition U)
  (C : Finset (Fin (A.model n).size))
  (hC : C ∈ P.parts) :
  sourceCompletionBad
    (fun i : iS => (A.model n).action ((i : K), 1))
    (fun j : J => (A.model n).action (1, j))
  F C ⊆
  C n SoficGroups.matchedRadiusBad
  P (sourceProductRadiusLabels S F k)
  ((A.model n).action)
  (canonicalProductRadiusBad
    A S hsymmetric hgenerates F n k) := by
-- proof: 74 lines, omitted
```

`sourceCompletionBad_subset_canonical_matchedRadiusBad` is the main capture lemma. Fix a part  $C$  of a finpartition  $P$  of the finite model. The set `sourceCompletionBad` collects the points of  $C$  at which completing the second-factor permutations relative to  $C$  can fail: a generator or tracked element pushes the point out of  $C$ , a composite label exits, or one of the word tests (commutation, multiplicativity, separation) fails. The theorem shows this entire set is contained in the intersection of  $C$  with `matchedRadiusBad`, the bad set of the canonical product radius test built from `canonicalProductRadiusBad` and the crossing sets of the labels in `sourceProductRadiusLabels`. The proof is a case analysis over the covering lemma `sourceCompletionBad_subset_exit_union_wordBad`: each exit event is converted into membership in a `partitionWordCrossing` set for an appropriate label, using `component_exit_mem_partitionWordCrossing`, and for mixed labels the multiplicativity of good root points rewrites the composite action as a two-step action; word-test failures are absorbed into `finiteRootBad` via `sourceWordTestBad_subset_finiteProductRootBad`. Its significance: the union bound on `canonicalProduct`

RadiusBad together with the crossing densities makes the matched bad set negligible, so almost every point of almost every component is simultaneously good for completion, commutation, and separation, exactly the hypotheses required to build the centralizer finite model.

**canonical\_source\_matched\_component\_cayley\_ball\_realization** *thm*

```

theorem canonical_source_matched_component_cayley_ball_realization
  {K J : Type*} [Group K] [Group J]
  [DecidableEq K] [DecidableEq J]
  (A : SoficGroups.SoficApproximation (K × J))
  (S : Finset K) (hones : 1 ∈ S)
  (hsymmetric : ∀ a ∈ S, a-1 ∈ S)
  (hgenerates : Subgroup.closure (S : Set K) = T)
  (F : Finset J) (n k : ℕ)
  {U : Finset (Fin (A.model n).size)}
  (P : Finpartition U)
  (C : Finset (Fin (A.model n).size))
  (hC : C ∈ P.parts)
  (x : Fin (A.model n).size) (hx : x ∈ C)
  (hgood : x ∉
    SoficGroups.matchedRadiusBad
      P (sourceProductRadiusLabels S F k)
      ((A.model n).action)
      (canonicalProductRadiusBad
        A S hsymmetric hgenerates F n k)) :
  ∃ f : K → Fin (A.model n).size,
    Set.MapsTo f
      (↑(S ^ (k / 2)) : Set K)
      (↑C : Set (Fin (A.model n).size)) ∧
    Set.InjOn f (↑(S ^ (k / 2)) : Set K) := by
-- proof: 36 lines, omitted

```

`canonical_source_matched_component_cayley_ball_realization` converts goodness of a single point into geometric largeness of its component. Suppose  $x$  lies in a part  $C$  and avoids the matched radius bad set built from `canonicalProductRadiusBad`. The theorem produces a map  $f$  from  $K$  to model vertices, namely sending  $a$  to the action of  $(a, 1)$  at  $x$ , that maps the half-radius ball  $S^{k/2}$  into  $C$  and is injective there. Injectivity comes from `sourceFirstFactor_injective_ball`: the point  $x$  is outside the first-factor Cayley radius bad set, which is one component of the canonical bad set. Containment in  $C$  is the matched part: for  $a \in S^{k/2} \subseteq S^k$  (using that  $1 \in S$  and `Finset.pow_subset_pow_right`), the label  $(a, 1)$  belongs to `sourceProductRadiusLabels`, so if the action of  $(a, 1)$  exited  $C$ , then  $x$  would lie in the corresponding `partitionWordCrossing` set and hence in the matched bad set, a contradiction. Consequently every good component contains at least  $|S^{k/2}|$  points. Since balls in the relevant Cayley graph grow without bound, components of the almost-invariant partition are eventually large, which is essential for the counting and spectral arguments downstream.

*Remaining declarations in this section:*

|                                                             |     |                                                                                                                         |
|-------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------|
| <code>sourceProductRadiusLabels</code>                      | DEF | Finite label set in $K \times J$ : ball elements paired with one, tracked table elements, and generator-table products. |
| <code>firstFactor_mem_sourceProductRadiusLabels</code>      | THM | Ball elements $(a, 1)$ with $a \in S^k$ lie in the label set.                                                           |
| <code>secondFactor_mem_sourceProductRadiusLabels</code>     | THM | Tracked table elements $(1, j)$ lie in the label set.                                                                   |
| <code>composite_mem_sourceProductRadiusLabels</code>        | THM | Mixed products $(a, j)$ of a generator and a tracked table element lie in the label set.                                |
| <code>generator_mem_sourceProductRadiusLabels</code>        | THM | Generators $(a, 1)$ lie in the label set, since one belongs to the tracked table.                                       |
| <code>canonicalProductRadiusBad</code>                      | DEF | Bad set combining the first-factor Cayley radius bad set with <code>finiteRootBad</code> for the product radius labels. |
| <code>canonicalProductRadiusBad_density_tendsto_zero</code> | THM | The canonical product radius bad set has vanishing density, by a union bound on the two constituent densities.          |



|                                                   |     |                                                                                                                                |
|---------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------|
| sourceWordTestBad_subset_finite<br>ProductRootBad | THM | Every commutation, multiplication, or separation failure of the source word test forces membership in the finite root bad set. |
| component_exit_mem_partitionWord<br>Crossing      | THM | A point of part $C$ sent outside $C$ lies in the partition word crossing set.                                                  |

## 8.16 Eventual table properties of completed restrictions (CompletedSelectedCentralizer PairwiseSource)

Two eventual statements derived from vanishing bad-set densities. First, if the density of `sourceCompletionBad` inside the selected sets  $Z_n$  tends to zero, then eventually five times its cardinality is at most  $|Z_n|$ . Second, under the same hypothesis and with a tolerance sequence  $t_n$  dominating twice the index cardinality times the bad count, the completed restrictions of the second-factor permutations to  $Z_n$  eventually form a table with three quantitative properties: approximate multiplicativity on the finite set  $F$  (five times the permutation distance between the completed product and the product of completions is at most the subtype cardinality), pairwise separation of distinct elements of  $F$ , and commutation defect at most  $t_n$  against the completed first-factor family for every tracked table element. These are the exact eventual hypotheses consumed by the finite model construction.

### **eventually\_completed\_sourceCentralizer\_table\_of\_bad\_density** *thm*

```

theorem eventually_completed_sourceCentralizer_table_of_bad_density
  (V : ℕ → Type*) [∀ n, Fintype (V n)]
  [∀ n, DecidableEq (V n)]
  {ι J : Type*} [Fintype ι] [Group J]
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))
  (p : (n : ℕ) → J → Equiv.Perm (V n))
  (F : Finset J)
  (Z : (n : ℕ) → Finset (V n))
  (hZ : ∀ n, (Z n).Nonempty)
  (hbad : Tendsto
    (fun n =>
      ((sourceCompletionBad
        (σ n) (p n) F (Z n)).card : ℝ) /
        (Z n).card)
    atTop (nhds 0))
  (σZ : (n : ℕ) → ι →
    Equiv.Perm {x : V n // x ∈ Z n})
  (hσZ : ∀ n i (x : V n) (hx : x ∈ Z n)
    (_hi : σ n i x ∈ Z n),
    ((σZ n i (x, hx) :
      {x : V n // x ∈ Z n} : V n) = σ n i x)
  (t : ℕ → ℕ)
  (ht : ∀ n,
    2 * Fintype.card ι *
      (sourceCompletionBad
        (σ n) (p n) F (Z n)).card ≤ t n) :
  ∀ n in atTop,
  (∀ a ∈ F, ∀ b ∈ F,
    5 * SoficGroups.permutationDistance
      (completedRestriction (p n) (a * b)) (Z n))
    (completedRestriction (p n) a) (Z n) *
      completedRestriction (p n) b) (Z n)) ≤
    Fintype.card {x : V n // x ∈ Z n} ∧
  (∀ a ∈ F, ∀ b ∈ F, a ≠ b →
    Fintype.card {x : V n // x ∈ Z n} <
      5 * SoficGroups.permutationDistance
        (completedRestriction (p n) a) (Z n))
        (completedRestriction (p n) b) (Z n))) ∧
  (∀ a ∈ SoficGroups.CompressionCriterion.productTrackedTable F,
    SoficGroups.permutationCommutationDefect
-- statement truncated; proof: 30 lines, omitted

```

eventually\_completed\_sourceCentralizer\_table\_of\_bad\_density upgrades density hypotheses to an eventual, fully quantitative table of properties for the completed permutations. The data: a sequence of finite vertex types, first-factor permutations  $\sigma_n$  completed on selected sets  $Z_n$  by permutations of the subtype agreeing with  $\sigma_n$  where possible, second-factor permutations  $p_n$ , a finite set  $F$ , and a tolerance sequence  $t_n$  dominating twice the index cardinality times the completion bad count. Assuming the density of sourceCompletionBad on  $Z_n$  vanishes, eventually three things hold simultaneously. First, for all  $a, b \in F$ , five times the permutation distance between the completed restriction of  $p_n(ab)$  and the product of the completed restrictions of  $p_n(a)$  and  $p_n(b)$  is at most the cardinality of the subtype (approximate multiplicativity). Second, distinct elements of  $F$  have completed restrictions at distance strictly more than a fifth of the cardinality (separation). Third, for every element of the tracked table productTrackedTable the commutation defect against the completed first-factor family is at most  $t_n$ . Each item follows from a finite transfer lemma (completedRestriction\_mul\_distance\_le\_sourceCompletionBad, completedRestriction\_separated\_of\_sourceCompletionBad, completedRestriction\_commutationDefect\_le\_sourceCompletionBad) once five times the bad count is below the component size, which the first theorem of the section provides eventually.

*Remaining declarations in this section:*

|                                      |     |                                                               |
|--------------------------------------|-----|---------------------------------------------------------------|
| eventually_five_mul_sourceCompletion | THM | From vanishing density, eventually five times the source com- |
| Bad_le_of_density                    |     | pletion bad count is at most the selected set size.           |

## 8.17 Building the expanding centralizer finite model (KunActualSelectedCentralizerFiniteModel)

This segment assembles the expanding centralizer finite model at a single good index. trackedCompletedAlmostCentralizer packages each permutation  $p(j)$  with  $j$  in the tracked table productTrackedTable as an AlmostCentralizerElement of the family  $\sigma$  with defect at most  $t$ , and sends untracked elements to the identity. Four lemmas verify that on tracked elements it carries exactly  $p(j)$ , that it maps the group identity to the identity, and that approximate multiplicativity and pairwise separation on  $F$  transfer from  $p$  to the packaged assignment, since  $F$ , products from  $F$ , and the identity all lie in the tracked table. The main theorem then combines six eventual hypotheses at a single index and applies expandingCentralizerFiniteModelOfImprovement to produce a nonempty ExpandingCentralizerFiniteModel, the object driving the LEF reduction.

**nonempty\_expandingCentralizerFiniteModel\_of\_selected\_completed\_sequence** thm

```
theorem nonempty_expandingCentralizerFiniteModel_of_selected_completed_sequence
  {J : Type} [Group J] (F : Finset J)
  (V : ℕ → Type)
  [V n, Fintype (V n)] [V n, DecidableEq (V n)]
  (ι : Type) [Fintype ι]
  (σ : (n : ℕ) → ι → Equiv.Perm (V n))
  (p : (n : ℕ) → J → Equiv.Perm (V n))
  (hp : ∀ n, p n 1 = 1)
  (t : ℕ → ℕ)
  (h : ℝ) (hpositive : 0 < h)
  (hdefect : ∀ n in atTop,
    ∀ j ∈ SoficGroups.CompressionCriterion.productTrackedTable F,
    SoficGroups.permutationCommutationDefect
      (σ n) (p n j) ≤ t n)
  (hexp : ∀ n in atTop, ∀ E : Finset (V n),
    h * min (E.card : ℝ)
      ((Fintype.card (V n) : ℝ) - E.card) ≤
      (SoficGroups.boundary (σ n) E : ℝ))
  (hsmall : ∀ n in atTop,
    (10 : ℝ) * t n < h * Fintype.card (V n))
  (himprove : ∀ n in atTop,
    SoficGroups.HasAlmostCentralizerImprovement (σ n) (t n))
  (hmul : ∀ n in atTop, ∀ x ∈ F, ∀ y ∈ F,
    5 * SoficGroups.permutationDistance
      (p n (x * y)) (p n x * p n y) ≤ Fintype.card (V n))
  (hsep : ∀ n in atTop, ∀ x ∈ F, ∀ y ∈ F, x ≠ y →
    Fintype.card (V n) <
```

```

5 * SoficGroups.permutationDistance (p n x) (p n y)) :
Nonempty (SoficGroups.ExpandingCentralizerFiniteModel J F) := by
-- proof: 17 lines, omitted

```

`nonempty_expandingCentralizerFiniteModel_of_selected_completed_sequence` is the culminating existence theorem of the centralizer construction. It assumes six eventual properties of a sequence of finite models: commutation defects of the  $p_n(j)$  against  $\sigma_n$  bounded by  $t_n$  for all tracked  $j$ ; a uniform Cheeger-type expansion for  $\sigma_n$  with constant  $h > 0$ ; smallness  $10 t_n < h \cdot |V_n|$ ; the improvement property `HasAlmostCentralizerImprovement`; approximate multiplicativity of  $p_n$  on  $F$ ; and pairwise separation on  $F$ . Since all six are eventual filters, some single index  $n$  satisfies them simultaneously. At that index, `trackedCompletedAlmostCentralizer` packages each  $p_n(j)$  with tracked  $j$  as an `AlmostCentralizerElement` of defect at most  $t_n$ , with the identity as fallback off the tracked table, and the three accompanying lemmas convert the unit, multiplicativity, and separation hypotheses into the corresponding properties of the packaged assignment. Feeding these into `expandingCentralizerFiniteModelOfImprovement` yields an inhabitant of `ExpandingCentralizerFiniteModel` for  $J$  and  $F$ . This structure, a finite model of the almost-centralizer of an expanding permutation family separating the finite set  $F$ , is exactly what `lef_of_expanding_centralizer_models` consumes, so this theorem is the last quantitative step before the top-level compression.

*Remaining declarations in this section:*

|                                                                   |     |                                                                                                                 |
|-------------------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------|
| <code>trackedCompletedAlmostCentralizer</code>                    | DEF | Packages permutations $p(j)$ as almost-centralizer elements for tracked $j$ , and the identity otherwise.       |
| <code>trackedCompletedAlmostCentralizer_permutation_of_mem</code> | THM | For tracked $j$ , the packaged almost-centralizer element carries exactly the permutation $p(j)$ .              |
| <code>trackedCompletedAlmostCentralizer_map_one</code>            | THM | The packaged assignment sends the group identity to the identity permutation, given $p(1) = 1$ .                |
| <code>trackedCompletedAlmostCentralizer_multiplicative</code>     | THM | The packaged assignment inherits approximate multiplicativity on $F$ from the corresponding hypothesis on $p$ . |
| <code>trackedCompletedAlmostCentralizer_separated</code>          | THM | The packaged assignment inherits pairwise separation on $F$ from the corresponding hypothesis on $p$ .          |

## 8.18 Sofic plus finite models implies LEF (SourceTopLevelCompression)

A single conditional theorem forming the skeleton of item 2 in the global chain. It assumes that for every sofic approximation of `prefixElementaryGroup ninePrefixCode` whose model sizes diverge, and for every finite subset  $F$  of the local prefix transposition group at `sourceLocalWord`, there is an expanding centralizer finite model for  $F$ . Under this hypothesis, soficity of the nine-prefix elementary group implies that the local prefix transposition group is LEF. The proof extracts a size-diverging sofic approximation from soficity and countability, applies the hypothesis to obtain models for every  $F$ , and invokes the formal reduction `lef_of_expanding_centralizer_models`. The theorem cleanly separates the analytic work of constructing the models from the formal derivation of LEF.

**`sourceLocalPrefixTranspositionGroup_lef_of_source_finite_models` thm**

```

theorem sourceLocalPrefixTranspositionGroup_lef_of_source_finite_models
(hmodels :
  ∀ A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode),
  Tendsto (fun n => (A.model n).size) atTop atTop →
  ∀ F : Finset
    (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
     SoficGroups.ThompsonPrefixLocal.sourceLocalWord),
  Nonempty (SoficGroups.ExpandingCentralizerFiniteModel
    (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
     SoficGroups.ThompsonPrefixLocal.sourceLocalWord) F))
(hsource : SoficGroups.Sofic
  (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode)) :

```

```

SoficGroups.LEF
(SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
 SoficGroups.ThompsonPrefixLocal.sourceLocalWord) := by
-- proof: 10 lines, omitted

```

`sourceLocalPrefixTranspositionGroup_lef_of_source_finite_models` is the conditional form of item 2 in the global chain. Its hypothesis `hmodels` says: for every sofic approximation  $A$  of `prefixElementaryGroup nine PrefixCode` whose model sizes tend to infinity, and every finite subset  $F$  of `localPrefixTranspositionGroup` at `sourceLocalWord`, there is an expanding centralizer finite model for  $F$ . Under this hypothesis, soficity of the nine-prefix elementary group implies the local prefix transposition group is LEF. The proof is short: soficity plus countability of the nine-prefix group (`ninePrefixElementaryGroup_countable`) yields, through `exists_soficApproximation_size_tendsto`, a sofic approximation with diverging sizes; applying `hmodels` to it gives finite models for every  $F$ , and `lef_of_expanding_centralizer_models` converts a family of expanding centralizer finite models into local embeddability into finite groups. The theorem thus isolates precisely what the long Kun-style analytic development must deliver, namely the finite models, from the purely formal derivation of LEF. Combined with the unconditional non-LEF theorem proved a few sections later, it will show that the nine-prefix elementary group cannot be sofic once the models are constructed unconditionally from a hypothetical sofic approximation.

## 8.19 Thompson relations collapse in finite groups (ThompsonFFiniteQuotient)

Pure group theory with no sofic content. With `conjugateTerm` defined as  $a^{-n}ba^n$ , the two Thompson-F-style relations, namely that  $ab^{-1}$  commutes with the first and with the second conjugate term, convert into conjugation identities advancing the index by one. A shift lemma and a two-step induction (`Nat.twoStep Induction`) propagate these identities from the base cases at indices one and two to every index. In a finite group the element  $a$  has finite order, so the conjugate terms are periodic; evaluating the propagated relation one step before the period forces  $b = a^{-1}ba$ , that is,  $a$  and  $b$  commute. This finite quotient collapse is the algebraic engine that later defeats local embeddability into finite groups for the concrete witness.

**finite\_group\_commute\_of\_thompsonF\_two\_relations** *thm*

```

theorem finite_group_commute_of_thompsonF_two_relations
  {G : Type*} [Group G] [Finite G] (a b : G)
  (hfirst : Commute (a * b⁻¹) (a⁻¹ * b * a))
  (hsecond : Commute (a * b⁻¹) ((a ^ 2)⁻¹ * b * a ^ 2)) :
  Commute a b := by
-- proof: 28 lines, omitted

```

`finite_group_commute_of_thompsonF_two_relations` shows that Thompson-F-style relations collapse in finite groups. Write  $c_n = a^{-n}ba^n$ , which is `conjugateTerm`. The hypotheses say  $ab^{-1}$  commutes with  $c_1$  and with  $c_2$ . By `conjugacy_relation_of_commute`, each commutation converts into a conjugation identity  $b^{-1}c_nb = c_{n+1}$  for  $n = 1, 2$ . The lemma `conjugacy_relation_all` then propagates these two base cases to every index by a two-step induction (`Nat.twoStepInduction`), where the induction step `conjugacy_relation_step` rewrites conjugation by  $b$  as conjugation by  $b^{-1}c_1b$  and applies the shift lemma `conjugacy_relation_shift` twice. Finiteness enters only now:  $a$  has finite order, so  $c_{\text{ord}(a)} = b$  by `pow_orderOf_eq_one`, and  $c_{\text{ord}(a)+1} = a^{-1}ba$ . Evaluating the propagated relation at index  $\text{ord}(a) - 1$  gives  $b = a^{-1}ba$ , so  $a$  and  $b$  commute. This is the group-theoretic engine of the non-LEF proof: any finite image of the two-relator witness, in particular any image in a finite symmetric group, must be abelian on its generators, while the actual witness constructed later is not.

*Remaining declarations in this section:*

|                                 |     |                                                                                                   |
|---------------------------------|-----|---------------------------------------------------------------------------------------------------|
| <code>conjugateTerm</code>      | DEF | Defines <code>conjugateTerm</code> as $a^{-n}ba^n$ .                                              |
| <code>conjugateTerm_succ</code> | THM | Recursion: the $(n + 1)$ -st conjugate term is the $n$ -th term conjugated by $a$ .               |
| <code>conjugation_shift</code>  | THM | Conjugating a conjugacy identity by $a$ preserves it; a pure computation via <code>group</code> . |

|                                            |     |                                                                                                      |
|--------------------------------------------|-----|------------------------------------------------------------------------------------------------------|
| <code>conjugacy_relation_of_commute</code> | THM | If $ab^{-1}$ commutes with the $n$ -th conjugate term, conjugation by $b$ advances the index.        |
| <code>conjugacy_relation_shift</code>      | THM | Shifts a conjugacy relation one step: conjugating the conjugator by $a$ advances both indices.       |
| <code>conjugacy_relation_step</code>       | THM | Two-step induction step: relations at indices one, $n$ , and $n + 1$ yield the relation at $n + 2$ . |
| <code>conjugacy_relation_all</code>        | THM | By two-step induction, the base relations at indices one and two propagate to all indices.           |

## 8.20 Abstract two-relator non-LEF criterion (ThompsonFTwoRelatorLEF)

This segment defines the relators in the free group on two letters and proves the abstract non-LEF criterion. `thompsonFRelator  $n$`  is the commutator of  $ab^{-1}$  with  $a^{-n}ba^n$ , and `thompsonFGeneratorCommutator` is the commutator of the generators. The main theorem: if a group has elements  $a, b$  satisfying the relators at  $n = 1, 2$  but not commuting, and if in every finite symmetric group those relations force commutation, then the group is not LEF. The proof handles the subtlety that a locally multiplicative map need not compute long words correctly: `exists_local_word_control` chooses finite control sets for each tracked word, and an LEF approximation on a support containing all controls transports the relators into the finite permutation group, forces the images to commute, and injectivity pulls back the contradiction with noncommutativity.

**not\_lef\_of\_thompsonF\_two\_relations** thm

```

theorem not_lef_of_thompsonF_two_relations
  {G : Type*} [Group G] (a b : G)
  (h1 : Commute (a * b⁻¹) (a⁻¹ * b * a))
  (h2 : Commute (a * b⁻¹) ((a ^ 2)⁻¹ * b * a ^ 2))
  (hne : ¬ Commute a b)
  (hfinite : ∀ (n : ℕ) (x y : Equiv.Perm (Fin n)),
    Commute (x * y⁻¹) (x⁻¹ * y * x) →
    Commute (x * y⁻¹) ((x ^ 2)⁻¹ * y * x ^ 2) →
    Commute x y) :
  ¬ SoficGroups.LEF G := by
-- proof: 64 lines, omitted

```

`not_lef_of_thompsonF_two_relations` is the abstract non-LEF criterion. Suppose a group  $G$  contains  $a, b$  satisfying the two relators, so the lift  $\varphi$  from the free group on two letters sends both relators to 1, yet  $a$  and  $b$  do not commute; and suppose that in every finite symmetric group the two relations force commutation. If  $G$  were LEF, one could locally embed a finite support into some `Equiv.Perm` on a finite type. The subtlety is that evaluating a free-group word at the composite of a locally multiplicative map with  $\varphi$  requires multiplicativity on all intermediate products; this is handled by `exists_local_word_control`, which for each tracked word (the two relators and the generator commutator  $w$ ) chooses a finite control set such that any map multiplicative on the controls computes that word correctly. Taking the support to contain 1, the image of  $w$ , and all controls, the induced free-group homomorphism  $\psi$  into the finite permutation group kills both relators, so by the finite collapse hypothesis the images of  $a$  and  $b$  commute, hence  $\psi w = 1$ . But then the approximation sends both the image of  $w$  and 1 to the same permutation, and injectivity on the support forces the image of  $w$  to be 1, contradicting noncommutativity. The criterion reduces non-LEF to exhibiting a concrete two-relator witness, done in the next section.

*Remaining declarations in this section:*

|                                           |     |                                                                                            |
|-------------------------------------------|-----|--------------------------------------------------------------------------------------------|
| <code>thompsonFRelator</code>             | DEF | The $n$ -th Thompson relator: commutator of $ab^{-1}$ with $a^{-n}ba^n$ in the free group. |
| <code>thompsonFGeneratorCommutator</code> | DEF | The commutator of the two free generators, used as the noncommutativity witness word.      |

## 8.21 Concrete Thompson witness in the Leavitt units (ThompsonFLocalWitness)

This long segment constructs the concrete witness inside the unit group of the binary Leavitt algebra. `rootRotation`, a product of three cylinder swaps, performs the elementary rotation of the binary tree; `generatorA` and `generatorB` are the inverses of the root rotation and of its insertion below prefix `[1]`. Prefix-action calculus (inversion, transport along `prefixInsertionHom`, appending) identifies the conjugates  $a^{-1}ba$  and  $a^{-2}ba^2$  as insertions of  $a$  at `[1, 1]` and `[1, 1, 1]`, shows  $ab^{-1}$  fixes the cylinder `[1, 1]`, and derives both Thompson relators from the cylinder-fixing commutation criterion. An explicit action computation on `[1, 1, 0]` proves the generators do not commute. Both generators lie in `binaryPrefixTranspositionGroup`, and everything transports through `prefixInsertionHom` at `sourceLocalWord` into the local prefix transposition group, giving `sourceGeneratorA`, `sourceGeneratorB` with the same relations and noncommutativity.

**rootRotation** `def`

```
def rootRotation : BinaryLeavitt* :=
  cylinderSwap [0] [1, 0] (by decide) (by decide) *
  cylinderSwap [0] [1, 1] (by decide) (by decide) *
  cylinderSwap [0] [1] (by decide) (by decide)
```

`rootRotation` is the concrete seed of the Thompson F witness. It is defined in the unit group of the binary Leavitt algebra as the product of three cylinder swaps: `cylinderSwap` of `[0]` with `[1, 0]`, then of `[0]` with `[1, 1]`, then of `[0]` with `[1]`, each pair verified incomparable by `decide`. The three accompanying action lemmas compute its effect on basic cylinders through the `PrefixWordAction` relation: it carries the cylinder at `[0, 0]` to `[0]`, the cylinder at `[0, 1]` to `[1, 0]`, and the cylinder at `[1]` to `[1, 1]`. This is exactly the elementary rotation of the infinite binary tree that generates Thompson's group F: it coarsens on the left branch and refines along the right. `generatorA` is its inverse, and `generatorB` is the inverse of `rightRotation`, the same rotation inserted below the prefix `[1]` via `prefixInsertionHom`, mirroring the classical presentation of F in which the second generator is the first acting one level down the right spine. Because each cylinder swap visibly lies in `binaryPrefixTranspositionGroup`, so do both generators, which is what later allows the witness to be transported into the local prefix transposition group at `sourceLocalWord`.

**commute\_prefixInsertion\_of\_prefixWordAction\_fixed** `thm`

```
theorem commute_prefixInsertion_of_prefixWordAction_fixed
  (g : BinaryLeavitt*) (w : List (Fin 2))
  (h : PrefixWordAction g w w) (u : BinaryLeavitt*) :
  Commute g (prefixInsertionHom w u) := by
  -- proof: 8 lines, omitted
```

`commute_prefixInsertion_of_prefixWordAction_fixed` is the mechanism behind both relator verifications. It says: if a unit  $g$  satisfies `PrefixWordAction` from  $w$  to  $w$ , meaning  $g$  carries the cylinder at the word  $w$  to itself acting as the identity on it, then  $g$  commutes with `prefixInsertionHom` of any unit  $u$  at  $w$ , that is, with anything supported below the prefix  $w$ . The proof conjugates: `prefixInsertionHom_conjugate_of_prefixWordAction` shows that conjugating the insertion of  $u$  by  $g$  returns the same insertion when  $g$  fixes  $w$ , and commutation follows by rewriting. The two relators are then immediate: `generator_two_eq_prefixInsertion` identifies  $a^{-1}ba$  with `generatorA` inserted at `[1, 1]`, and `generator_difference_action_one_one` shows  $ab^{-1}$  fixes the cylinder `[1, 1]`, giving `relator_one`. Similarly `generator_three_eq_prefixInsertion` exhibits  $a^{-2}ba^2$  as an insertion at `[1, 1, 1]`, and appending a letter preserves the fixed-cylinder property, giving `relator_two`. This reproduces the standard reason the Thompson relations hold: the element  $ab^{-1}$  is supported away from the deep right spine where the conjugated generators live.

**generators\_not\_commute** `thm`

```
theorem generators_not_commute : ¬ Commute generatorA generatorB := by
  -- proof: 54 lines, omitted
```

`generators_not_commute` certifies that the witness pair is genuinely nonabelian, by an explicit cylinder computation in the Leavitt algebra. The proof first records actions of the generators obtained by inverting

the root rotation lemmas:  $a$  carries  $[1, 0]$  to  $[0, 1]$  and  $[1, 1]$  to  $[1]$ , while  $b$  carries  $[1, 1, 0]$  to  $[1, 0, 1]$  and  $[1, 0]$  to  $[1, 0, 0]$ . Composing, the product  $ab$  carries the cylinder  $[1, 1, 0]$  to  $[0, 1, 1]$ , whereas  $ba$  carries  $[1, 1, 0]$  to  $[1, 0, 0]$ . If the generators commuted, the prefixing identities of `PrefixWordAction` would force `leavittWordS` of  $[0, 1, 1]$  to equal `leavittWordS` of  $[1, 0, 0]$ . But then 1, which equals `leavittWordT` of  $[0, 1, 1]$  times `leavittWordS` of  $[0, 1, 1]$  by `leavittWordT_mul_wordS_self`, would equal the mixed product, which vanishes by `leavittWordT_mul_wordS_of_incomparable` since the two words are incomparable, contradicting  $1 \neq 0$  in the Leavitt algebra. The subsequent definitions `sourceGeneratorA` and `sourceGeneratorB` push the pair into the local group at `sourceLocalWord`; the relations transfer along the insertion homomorphism and noncommutativity transfers back along its injectivity, completing the local witness.

*Remaining declarations in this section:*

|                                                              |     |                                                                                                                                 |
|--------------------------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------|
| <code>prefixWordAction_inv</code>                            | THM | Inverting a unit reverses its prefix word action: $g^{-1}$ carries $b$ back to $a$ .                                            |
| <code>prefixWordAction_prefixInsertion</code>                | THM | Prefix word actions transport along <code>prefixInsertionHom</code> : the inserted unit acts on words with prefix $l$ attached. |
| <code>rightRotation</code>                                   | DEF | Defines <code>rightRotation</code> as the root rotation inserted below prefix $[1]$ .                                           |
| <code>generatorA</code>                                      | DEF | Defines <code>generatorA</code> as the inverse of the root rotation.                                                            |
| <code>generatorB</code>                                      | DEF | Defines <code>generatorB</code> as the inverse of the right rotation.                                                           |
| <code>rootRotation_action_zero_zero</code>                   | THM | The root rotation carries the cylinder at $[0, 0]$ to $[0]$ .                                                                   |
| <code>rootRotation_action_zero_one</code>                    | THM | The root rotation carries the cylinder at $[0, 1]$ to $[1, 0]$ .                                                                |
| <code>rootRotation_action_one</code>                         | THM | The root rotation carries the cylinder at $[1]$ to $[1, 1]$ .                                                                   |
| <code>generatorB_eq_prefixInsertion</code>                   | THM | <code>generatorB</code> equals <code>generatorA</code> inserted below prefix $[1]$ .                                            |
| <code>generator_two_eq_prefixInsertion</code>                | THM | The conjugate $a^{-1}ba$ equals <code>generatorA</code> inserted below prefix $[1, 1]$ .                                        |
| <code>generator_three_eq_prefixInsertion</code>              | THM | The conjugate $a^{-2}ba^2$ equals <code>generatorA</code> inserted below prefix $[1, 1, 1]$ .                                   |
| <code>generator_difference_action_one_one</code>             | THM | The difference $ab^{-1}$ fixes the cylinder at prefix $[1, 1]$ .                                                                |
| <code>relator_one</code>                                     | THM | First Thompson relation: $ab^{-1}$ commutes with $a^{-1}ba$ .                                                                   |
| <code>relator_two</code>                                     | THM | Second Thompson relation: $ab^{-1}$ commutes with $a^{-2}ba^2$ .                                                                |
| <code>cylinderSwap_mem_binaryPrefixTranspositionGroup</code> | THM | Cylinder swaps belong to the binary prefix transposition group, being among its closure generators.                             |
| <code>rootRotation_mem_binaryPrefixTranspositionGroup</code> | THM | The root rotation lies in the binary prefix transposition group, as a product of three cylinder swaps.                          |
| <code>generatorA_mem_binaryPrefixTranspositionGroup</code>   | THM | <code>generatorA</code> lies in the binary prefix transposition group, by closure under inverses.                               |
| <code>generatorB_mem_binaryPrefixTranspositionGroup</code>   | THM | <code>generatorB</code> lies in the binary prefix transposition group, via the prefix insertion membership lemma.               |
| <code>sourceGeneratorA</code>                                | DEF | Transports <code>generatorA</code> into the local prefix transposition group by insertion at the source local word.             |
| <code>sourceGeneratorB</code>                                | DEF | Transports <code>generatorB</code> into the local prefix transposition group by insertion at the source local word.             |
| <code>source_relator_one</code>                              | THM | The first Thompson relation transfers to the source generators along the insertion homomorphism.                                |
| <code>source_relator_two</code>                              | THM | The second Thompson relation transfers to the source generators along the insertion homomorphism.                               |
| <code>source_generators_not_commute</code>                   | THM | The source generators do not commute, by injectivity of the prefix insertion homomorphism.                                      |



## 8.22 The source transposition group is not LEF

A single unconditional top-level theorem, item 3 of the global chain: the local prefix transposition group at `ThompsonPrefixLocal.sourceLocalWord` is not LEF. The proof instantiates the abstract two-relator criterion `not_lef_of_thompsonF_two_relations` with the concrete source generators and their transported relations and noncommutativity from `ThompsonFLocalWitness`, and discharges the finite-collapse hypothesis by applying `finite_group_commute_of_thompsonF_two_relations` to the finite groups `Equiv.Perm` on `Fin n`. Together with the conditional LEF statement derived from soficity, this theorem produces the contradiction that makes the nine-prefix elementary group non-sofic at the end of the file.

**sourceLocalPrefixTranspositionGroup\_notLEF** thm

```
theorem sourceLocalPrefixTranspositionGroup_notLEF :
  ¬ LEF
    (ThompsonPrefixInsertion.localPrefixTranspositionGroup
     ThompsonPrefixLocal.sourceLocalWord) := by
  -- proof: 10 lines, omitted
```

`sourceLocalPrefixTranspositionGroup_notLEF` is item 3 of the global chain and the unconditional half of the final contradiction. It asserts that `localPrefixTranspositionGroup` at `ThompsonPrefixLocal.sourceLocalWord` is not LEF. The proof instantiates the abstract criterion `not_lef_of_thompsonF_two_relations` with the concrete data assembled in `ThompsonFLocalWitness`: the source generators `sourceGeneratorA` and `sourceGeneratorB`, the transported relations `source_relator_one` and `source_relator_two`, and the transported noncommutativity `source_generators_not_commute`. The remaining hypothesis of the criterion, that in every finite permutation group the two relations force commutation, is discharged by `finite_group_commute_of_thompsonF_two_relations` applied to `Equiv.Perm` on `Fin n`, which is a finite group. Conceptually, the local prefix transposition group contains a Thompson-F-like two-generator system whose defining relations survive in finite quotients only by abelianizing, while the group itself is visibly nonabelian on those generators; hence no injective locally multiplicative map into a finite symmetric group can exist. Together with `sourceLocalPrefixTranspositionGroup_lef_of_sofic`, the conditional LEF statement derived from soficity of the nine-prefix elementary group, this yields the non-soficity result at the top of the file's final chain of theorems.

## 8.23 Unconditional dual expander decomposition data (KunUnconditionalActualSourceDecomposition)

A single packaging theorem: for an arbitrary sofic approximation of the nine-prefix elementary group, it produces the complete dual decomposition data needed by the Kun-style analysis, with no unproved side conditions. The data comprise a finite symmetric generating set of the alpha-prefix-code elementary group containing the identity, the corresponding ambient symmetric generating set of the nine-prefix group, injectivity of `sourceAlphaInclusion`, generation by the positive generator map, and two finpartition sequences of the model vertex sets with positive expansion constants: inside every part, every subset occupying at most half the part has boundary at least a constant multiple of its size, for both the subgroup and the ambient generator actions. Finally, the normalized total boundary of the parts vanishes for both partitions. It merges the generator data theorem with the full finpartition existence theorem.

**exists\_unconditional\_actual\_source\_dual\_finpartition\_sequences** thm

```
theorem exists_unconditional_actual_source_dual_finpartition_sequences
  (A : SoficGroups.SoficApproximation
   (SoficGroups.prefixElementaryGroup
    SoficGroups.ninePrefixCode)) :
  ∃ (SΓ : Finset
    (SoficGroups.prefixElementaryGroup
     SoficGroups.alphaPrefixCode))
    (SG : Finset
     (SoficGroups.prefixElementaryGroup
      SoficGroups.ninePrefixCode))
    (QΓ QG : (n : ℕ) →
     Finpartition
```

```

    (Finset.univ : Finset (Fin (A.model n).size)))
  (γΓ γG : ℝ),
  SG = sourceAmbientSymmetricGenerators SΓ ∧
  1 ∈ SΓ ∧
  (∀ g ∈ SΓ, g⁻¹ ∈ SΓ) ∧
  Subgroup.closure
    (SΓ : Set
      (SoficGroups.prefixElementaryGroup
        SoficGroups.alphaPrefixCode)) = T ∧
  1 ∈ SG ∧
  (∀ g ∈ SG, g⁻¹ ∈ SG) ∧
  Subgroup.closure
    (SG : Set
      (SoficGroups.prefixElementaryGroup
        SoficGroups.ninePrefixCode)) = T ∧
  Function.Injective sourceAlphaInclusion ∧
  Subgroup.closure
    (Set.range (sourcePositiveGeneratorMap SΓ)) = T ∧
  0 < γΓ ∧
  0 < γG ∧
  (∀ n, ∀ C ∈ (QΓ n).parts,
    ∀ E : Finset (Fin (A.model n).size),
      E ⊆ C →
      2 * E.card ≤ C.card →
      γΓ * (E.card : ℝ) ≤
      (SoficGroups.boundary
        (fun i : ιSΓ =>
          (A.model n).action
            -- statement truncated; proof: 23 lines, omitted

```

exists\_unconditional\_actual\_source\_dual\_finpartition\_sequences packages, for an arbitrary sofic approximation  $A$  of `prefixElementaryGroup ninePrefixCode`, the entire dual decomposition data needed by the Kun-style analysis. It produces: a finite symmetric generating set  $S\Gamma$  of the alpha-prefix-code elementary group containing 1 and generating the whole group; the ambient set  $SG$ , equal to `sourceAmbientSymmetricGenerators` of  $S\Gamma$ , with the same three properties for the nine-prefix group; injectivity of `sourceAlphaInclusion` together with the fact that the range of the positive generator map has full closure; and two finpartition sequences  $Q\Gamma_n$  and  $QG_n$  of the model vertex sets with positive constants  $\gamma\Gamma$  and  $\gamma G$  such that inside every part each subset  $E$  with  $2|E| \leq |C|$  expands: its boundary under the corresponding generator action is at least  $\gamma|E|$ . Finally, the total boundary of the parts, normalized by model size, tends to zero for both partitions. The proof combines `exists_unconditional_actual_source_generator_data`, which supplies the generators, presentations, and faithfulness, with `exists_source_subgroup_and_ambient_full_finpartition_sequences`, which builds the expander-decomposed partitions simultaneously for the subgroup and the ambient actions. This theorem is the entry point through which every hypothetical sofic approximation is decomposed into almost-invariant expander components, the setting where the midrank variance and rank invariance machinery developed above operates.

## 8.24 Diagonal selection of radii and tolerances (KunCombinedPrescribedRootSourceTolerance)

Numerical bookkeeping for the final construction. A helper lemma shows that when the normalized tolerance vanishes, any fixed positive multiple of the tolerance is eventually below any fixed positive multiple of the model size. The main theorem is a long diagonal-selection result: given radius-indexed root bad sets and a source bad set of vanishing density, a vanishing positive tolerance schedule  $\delta$ , a radius schedule  $R$ , a degree  $d$ , an expansion constant  $\ell$ , and a parameter  $q < 1$ , it constructs a diverging index sequence  $m$ , a rounding term `rootError` equal to the ceiling of  $\delta(m_n)$  times the cardinality, and a tolerance  $t_n$ , verifying thirteen properties: exact defining equations, budget inequalities dominating both bad sets and the rounding term, vanishing normalized tolerance, a comparison with  $\ell\delta$ , and two eventual spectral smallness inequalities used downstream.

**exists\_prescribed\_radius\_union\_source\_completed\_tolerance** *thm*

```

theorem exists_prescribed_radius_union_source_completed_tolerance
  (V : ℕ → Type*)
  [V n, Fintype (V n)] [V n, DecidableEq (V n)]
  (hVpositive : ∀ n, 0 < Fintype.card (V n))
  (hV : Tendsto (fun n => Fintype.card (V n)) atTop atTop)
  (root : (n : ℕ) → ℕ → Finset (V n))
  (source : (n : ℕ) → Finset (V n))
  (hroot : ∀ r : ℕ,
    Tendsto
      (fun n => ((root n r).card : ℝ) / Fintype.card (V n))
      atTop (ℳ 0))
  (hsource : Tendsto
    (fun n => ((source n).card : ℝ) / Fintype.card (V n))
    atTop (ℳ 0))
  (δ : ℕ → ℝ)
  (hδpositive : ∀ j, 0 < δ j)
  (hδ : Tendsto δ atTop (ℳ 0))
  (R : ℕ → ℕ) (hR : ∀ j, j ≤ R j)
  (d : ℕ) (hd : 0 < d)
  (ell q : ℝ) (hell : 0 < ell) (hq : q < 1) :
  ∃ m rootError t : ℕ → ℕ,
    Tendsto m atTop atTop ∧
    Tendsto (fun n => R (m n)) atTop atTop ∧
    Tendsto (fun n => δ (m n)) atTop (ℳ 0) ∧
    Tendsto
      (fun n =>
        (((root n (R (m n)) ∪ source n).card : ℝ) /
          (δ (m n) * (Fintype.card (V n) : ℝ))))
      atTop (ℳ 0) ∧
  (∀ n,
    rootError n =
      Nat.ceil (δ (m n) * (Fintype.card (V n) : ℝ))) ∧
  (∀ n,
    δ (m n) ≤
      (rootError n : ℝ) / (Fintype.card (V n) : ℝ)) ∧
  Tendsto
    (fun n =>
      (rootError n : ℝ) / (Fintype.card (V n) : ℝ))
    atTop (ℳ 0) ∧
  (∀ n,
    -- statement truncated; proof: 209 lines, omitted

```

exists\_prescribed\_radius\_union\_source\_completed\_tolerance is a long but elementary diagonal-selection theorem that fixes all numerical parameters for the final construction. Inputs: a finite model sequence with diverging cardinalities  $N_n$ , radius-indexed bad sets and a source bad set, each of vanishing density; positive tolerances  $\delta_j \rightarrow 0$ ; a radius schedule  $R$  with  $j \leq R_j$ ; a degree  $d$ , an expansion constant  $\ell$ , and a parameter  $q < 1$ . Setting  $B_n^j$  to be the union of the root bad set at radius  $R_j$  with the source bad set, the diagonal lemma exists\_diverging\_radius\_with\_vanishing\_diagonal\_error produces an index sequence  $m_n \rightarrow \infty$  along which even the rescaled density  $|B_n^{m_n}|/(\delta(m_n)N_n)$  vanishes. The theorem then defines rootError as  $\lceil \delta(m_n)N_n \rceil$  and the tolerance  $t_n = 2d|B_n^{m_n}| + \text{rootError}_n + 1$ , and verifies thirteen properties: the exact defining equations, positivity, the budget inequalities showing  $2d$  times each bad-set cardinality is at most  $t_n$ , domination of the rounding term, vanishing of  $t_n/N_n$ , a lower bound comparing  $\ell\delta(m_n)$  with the normalized tolerance, and two eventual spectral inequalities,  $10t_n < \ell N_n$  and  $5(4 + \ell(216/(d(1-q)^2) + 1/d))t_n \leq \ell N_n$ , both via eventually\_scaled\_tolerance\_lt. These are precisely the smallness conditions later matched against the spectral gap constants of the property (T) machinery.

*Remaining declarations in this section:*

|                                |     |                                                                                               |
|--------------------------------|-----|-----------------------------------------------------------------------------------------------|
| eventually_scaled_tolerance_lt | THM | If $t_n/N_n$ vanishes, then eventually $c \cdot t_n < \ell \cdot N_n$ for positive constants. |
|--------------------------------|-----|-----------------------------------------------------------------------------------------------|

## 9 Final assembly: selected components and the main theorem

The last stretch discharges every hypothesis accumulated along the way: expander components are selected and pruned into exact expanders, guards are stripped from matched densities, approximate multiplicativity of the twice-completed restrictions is verified, and the finite-model bridge composes into `ninePrefixElementaryGroup_not_sofic`, `binaryLeavittElementaryGroup_not_sofic`, and finally `exists_finitelyPresented_nonsofic_group`.

### 9.1 Conjugacy disagreement bad sets (KunLiteralSourceSelectedComponents)

A short interlude quantifying how far the finite models of a sofic approximation are from genuine homomorphisms on conjugates. `sourceConjugacyDisagreementBad` collects the model points at level  $n$  where the permutation assigned to  $ugu^{-1}$  disagrees with the conjugate of the assigned permutations of  $u$  and  $g$ , and the companion theorem shows this set has vanishing normalized density. These bad sets are the price paid whenever the argument replaces honestly acted group conjugates by conjugates of permutations; they are later unioned over a generating family (idx 1389) and folded into the guarded bad sets that protect the component selection.

**sourceConjugacyDisagreementBad\_density\_tendsto\_zero** *thm*

```
theorem sourceConjugacyDisagreementBad_density_tendsto_zero
  {G : Type*} [Group G]
  (A : SoficGroups.SoficApproximation G)
  (u g : G) :
  Tendsto
    (fun n =>
      ((sourceConjugacyDisagreementBad A u g n).card : ℝ) /
      (A.model n).size)
    atTop (nhds 0) := by
  -- proof: 6 lines, omitted
```

`sourceConjugacyDisagreementBad_density_tendsto_zero` states that the fraction of points of the level- $n$  model on which the permutation assigned to the conjugate  $ugu^{-1}$  differs from the conjugate  $\pi(u)\pi(g)\pi(u)^{-1}$  of the assigned permutations tends to zero. The proof is a direct translation: the filtered set defining `sourceConjugacyDisagreementBad` is exactly the disagreement set of the two permutations, so its normalized cardinality is their normalized Hamming distance, and the earlier lemma `SourceCompressionTransportCrossing.tendsto_action_conjugate` (a consequence of the asymptotic multiplicativity built into `SoficApproximation`) gives the limit; a simple unfolding `normalizedHamming` and `hammingDist` closes the goal. The statement matters because the transport strategy repeatedly replaces honestly acted conjugates, group elements of the form  $ugu^{-1}$ , which word-crossing and Cayley-ball lemmas understand, by conjugates of permutations, which the transported-partition lemmas understand. Every such replacement is only valid outside this vanishing bad set. The sets for all generators are later unioned into `sourceGeneratorFamilyConjugacyBad` (idx 1389) and folded into `sourceSelectedRadiusBad` (idx 1391), so that all subsequent component selections take place away from them, allowing reference and transported generator families to be interchanged with vanishing error.

*Remaining declarations in this section:*

|                                             |            |                                                                                                                     |
|---------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------|
| <code>sourceConjugacyDisagreementBad</code> | <b>DEF</b> | Finset of model points where the acted conjugate $ugu^{-1}$ disagrees with the conjugate of the acted permutations. |
|---------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------|

### 9.2 Common slow overlap scales for transports (KunActualBothTransportedOverlapScales)

Transports the expander structure of the source partitions along the two compression-table permutations. The first theorem shows that vanishing summed partition boundaries for the acted alpha generator family persist when every generator is conjugated by an acted compression-table element: each conjugate is rewritten as the action of a group word using `sourceCompressionTable_conjugates_alpha`, and word-crossing and conjugated-crossing estimates finish. Building on this, `exists_source_both_common_slow_overlap_scales` feeds the gamma-expansion hypothesis and both boundary limits into the abstract `KunTransportedAmbientOverlap`

machinery to obtain positive antitone null sequences  $\eta$  and  $H$  with  $\eta/H \rightarrow 0$  such that, for both table indices, the transported partitions have vanishing insufficient-overlap density and vanishing dominant-overlap loss, plus a tail on which  $\eta < 1$ . These scales are the quantitative backbone of the retained-component matching.

**exists\_source\_both\_common\_slow\_overlap\_scales** *thm*

```

theorem exists_source_both_common_slow_overlap_scales
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (S : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure
    (S : Set
      (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode)) =
      T)
  (Q : ∀ n, Finpartition
    (Finset.univ : Finset (Fin (A.model n).size)))
  (gamma : ℝ) (hgamma : 0 < gamma)
  (hexpand : ∀ n C, C ∈ (Q n).parts →
    ∀ E : Finset (Fin (A.model n).size), E ⊆ C →
      2 * E.card ≤ C.card →
        gamma * (E.card : ℝ) ≤
          (SoficGroups.boundary
            (fun i : ιS =>
              (A.model n).action
                (SoficGroups.SourceBothCompressionNormalization.sourceAlphaElement
                  (i : SoficGroups.prefixElementaryGroup
                    SoficGroups.alphaPrefixCode))) E : ℝ))
  (hsource : Tendsto
    (fun n =>
      (∑ C ∈ (Q n).parts,
        (SoficGroups.boundary
          (fun i : ιS =>
            (A.model n).action
              (SoficGroups.SourceBothCompressionNormalization.sourceAlphaElement
                (i : SoficGroups.prefixElementaryGroup
                  SoficGroups.alphaPrefixCode))) C : ℝ)) /
        (A.model n).size)
      atTop (nhds 0)) :
    ∃ eta H : ℕ → ℝ,
      (∀ n, 0 < eta n) ∧
      Antitone eta ∧
      Tendsto eta atTop (nhds 0) ∧
      (∀ n, 0 < H n) ∧
      Antitone H ∧
    -- statement truncated; proof: 81 lines, omitted

```

`exists_source_both_common_slow_overlap_scales` takes a sofic approximation  $A$  of the nine-prefix group, a symmetric generating finset  $S$  of the alpha prefix group, model partitions  $Q_n$ , a gamma-expansion hypothesis for the acted generator family on small subsets of every part, and vanishing summed part boundaries. It returns positive antitone null sequences  $\eta$  and  $H$  with  $\eta/H \rightarrow 0$  such that, for each table index  $j$ , the partition transported along the acted `sourceCompressionTable` permutation satisfies two limits: the total density of components whose overlap with their best reference part is insufficient at tolerance  $\eta_n$  vanishes, and the total loss  $\sum_C (|C| - |C \cap \text{overlap}(C)|)$  over transported parts vanishes; additionally there is a tail index past which  $\eta < 1$ . The proof instantiates `KunTransportedAmbientOverlap.exists_common_slow_overlap_scales_for_transported_partitions` with  $\sigma$  the acted alpha generators and  $T$  the acted tables, feeding it the transported boundary vanishing of the preceding theorem; the loss conclusion comes from `dominant_component_loss_density_tendsto_zero` combined with `transportedUnivFinpartition_half_expansion`, which transfers half-expansion to transported partitions. These  $\eta, H$  are exactly the scales at which retained components are defined and at which the smallness condition  $2(e^H - 1 + 2\eta) < 1$  is later imposed.

*Remaining declarations in this section:*

|                                                                                               |                  |                                                                                                                                               |
|-----------------------------------------------------------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>source_both_transported_generator_</code><br><code>boundary_density_tendsto_zero</code> | <code>THM</code> | Vanishing summed partition boundaries for the alpha generator family persist after conjugation by either acted compression-table permutation. |
|-----------------------------------------------------------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|

### 9.3 Compression isomorphism and product approximation (SourceProductThroughAlpha)

Builds the algebraic bridge from the alpha prefix elementary group to the compressed alphaZero group and onward to the product group used in the endgame. Conjugation by `compressionU` is packaged as a homomorphism and upgraded to an isomorphism; generating finsets transport across it (identity, inverses, closure), yielding a Kazhdan pair for the alphaZero group with exactly the compressed generators via its unconditional property (T). The product of the alphaZero group with the local prefix transposition group maps into the alpha group, and this map factors the product embedding into the nine-prefix group. Consequently the sofic approximation pulls back to the product group with identical model sizes, and crossing densities for embedded product elements, finite families, and canonical radius label sets all vanish, providing the crossing inputs for the guarded selection.

**sourceCompressionUAlphaEquiv** `def`

```

noncomputable def sourceCompressionUAlphaEquiv :
  SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode ==
  SoficGroups.prefixElementaryGroup SoficGroups.alphaZeroPrefixCode := by
  apply MulEquiv.ofBijjective sourceCompressionUAlphaHom
  constructor
  · intro g g' heq
    apply Subtype.ext
    apply (MulAut.conj SoficGroups.compressionU).injective
    exact congrArg
      (fun k : SoficGroups.prefixElementaryGroup
        SoficGroups.alphaZeroPrefixCode =>
        (k : SoficGroups.BinaryLeavitt*)) heq
  · intro k
    have hk :
      (k : SoficGroups.BinaryLeavitt*) ∈
      (SoficGroups.prefixElementaryGroup
        SoficGroups.alphaPrefixCode).map
        (MulAut.conj SoficGroups.compressionU).toMonoidHom := by
      rw [SoficGroups.compressionU_map_alphaPrefixElementaryGroup]
      exact k.property
    obtain (g, hg, heq) := hk
    refine ({g, hg}, ?_)
    apply Subtype.ext
    exact heq

```

`sourceCompressionUAlphaEquiv` upgrades `sourceCompressionUAlphaHom` to a multiplicative equivalence between `prefixElementaryGroup alphaPrefixCode` and `prefixElementaryGroup alphaZeroPrefixCode`. The underlying homomorphism is conjugation `MulAut.conj` by the unit `compressionU` of the binary Leavitt algebra, restricted to the alpha subgroup and corestricted using `compressionU_map_alphaPrefixElementaryGroup`, the earlier computation that conjugation by `compressionU` carries the alpha prefix elementary group exactly onto the alphaZero one. Injectivity descends from injectivity of conjugation in the unit group via `Subtype.ext`; surjectivity is read off from the same image computation by pulling any element of the alphaZero group back through the mapped subgroup. Why it matters here: the alphaZero group carries unconditional property (T) (consumed at idx 1355), while all the sofic-approximation estimates are phrased for the alpha group sitting inside the nine-prefix group. This isomorphism lets generating finsets, Kazhdan pairs, and Cayley balls be moved between the two: a generating finset  $S$  becomes `sourceCompressedGeneratingFinset S` with identity, inverse-closure, and generation preserved (idx 1352 to 1354). Moreover, at idx 1383 applying the equivalence and re-embedding into the nine-prefix group is shown to coincide with conjugating by `sourceCompressionUElement`, which ties this algebraic compression directly to the permutation transport happening in the finite models.

**exists\_sourceCompressedKazhdanPair\_with\_same\_generators** thm

```

theorem exists_sourceCompressedKazhdanPair_with_same_generators
  (SΓ : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (hsymmetric : ∀ g ∈ SΓ, g⁻¹ ∈ SΓ)
  (hgenerates : Subgroup.closure
    (SΓ : Set
      (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode)) =
    T) :
  ∃ P : SoficGroups.KazhdanPair.{0, v}
    (SoficGroups.prefixElementaryGroup
      SoficGroups.alphaZeroPrefixCode),
    P.generators = sourceCompressedGeneratingFinset SΓ := by
  -- proof: 10 lines, omitted

```

**exists\_sourceCompressedKazhdanPair\_with\_same\_generators**: given a symmetric generating finset of the alpha prefix elementary group, there exists a **KazhdanPair** for the **alphaZero** prefix elementary group whose generator finset is exactly the compressed image **sourceCompressedGeneratingFinset SΓ**. The proof installs the instance **HasPropertyT** via **alphaZeroPrefixElementaryGroup\_hasPropertyT\_unconditional** (the unconditional property (T) result proved much earlier in the file through the **Kazhdan-pair** and **Ershov-Jaikin** machinery for elementary groups over noncommutative rings) and then applies **KunExactKazhdanGeneratorChange.exists\_kazhdanPair\_with\_exact\_symmetric\_generators**, which converts property (T) into a **Kazhdan** pair with any prescribed symmetric generating finset; the symmetry and closure obligations are discharged by idx 1353 and 1354. The significance is that the spectral-gap and expansion machinery of the **Kun** namespaces consumes **Kazhdan** pairs whose generator finsets are explicitly known: the **Kazhdan** constant of the pair is what ultimately supplies the gamma-expansion of partition parts against the corresponding acted generator families. Being able to prescribe the generators as the compressed image of an arbitrary generating finset of the alpha group means the entire compressed product construction can run with generators that match, after the conjugation identity of idx 1383, the transported generator families appearing in the finite models.

**sourceCompressedLocalProductApproximation** def

```

def sourceCompressedLocalProductApproximation
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode)) :
  SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup
      SoficGroups.alphaZeroPrefixCode ×
      SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
      SoficGroups.ThompsonPrefixLocal.sourceLocalWord) :=
  SoficGroups.pullbackSoficApproximation
    SoficGroups.ThompsonPrefixInsertion.sourceCompressedLocalProductEmbedding
    SoficGroups.ThompsonPrefixInsertion.sourceCompressedLocalProductEmbedding_injective
  A

```

**sourceCompressedLocalProductApproximation** pulls the given sofic approximation *A* of the nine-prefix group back along **sourceCompressedLocalProductEmbedding**, the injective homomorphism from the product of the **alphaZero** prefix elementary group with the local prefix transposition group at **sourceLocalWord**, yielding a **SoficApproximation** of the product with literally the same finite models; idx 1361 records the size equality by **rfl**. The construction is **pullbackSoficApproximation**, which precomposes each model action with the embedding; injectivity guarantees that the separation requirement of sofic approximations survives the pullback. Its role: the endgame studies how Cayley balls of the product group sit inside components of the transported partitions. Via the factorization at idx 1357, the product embedding equals **sourceAlphaInclusion** composed with **sourceCompressedLocalProductToAlpha**, so every crossing estimate proved for words in the alpha generators transfers verbatim to acted product elements: idx 1358 handles a single element, idx 1359 sums over a finite family, and idx 1362 specializes to the canonical label sets **sourceProductRadiusLabels** of the compressed generating finset at radius *k*. These vanishing crossing sums are precisely the inputs that **canonicalProductRadiusBad** and the guarded selection theorem (idx 1395) require in order to realize injective Cayley-ball images inside selected components.



Remaining declarations in this section:

|                                                                                 |     |                                                                                                                                                                                        |
|---------------------------------------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sourceCompressionUAlphaHom</code>                                         | DEF | Monoid homomorphism from the <code>alpha</code> to the <code>alphaZero</code> prefix elementary group, given by conjugation with <code>compressionU</code> .                           |
| <code>sourceCompressedGeneratingFinset</code>                                   | DEF | Image of an <code>alpha</code> -group generating finset under <code>sourceCompressionUAlphaEquiv</code> .                                                                              |
| <code>sourceCompressedGeneratingFinset_one_mem</code>                           | THM | The compressed generating finset contains the identity whenever the original finset does.                                                                                              |
| <code>sourceCompressedGeneratingFinset_inv_mem</code>                           | THM | The compressed generating finset is closed under inverses whenever the original finset is.                                                                                             |
| <code>sourceCompressedGeneratingFinset_closure</code>                           | THM | The compressed finset generates the <code>alphaZero</code> prefix elementary group, by surjectivity of the isomorphism.                                                                |
| <code>sourceCompressedLocalProductToAlpha</code>                                | DEF | Homomorphism from the product of the <code>alphaZero</code> group and the local prefix transposition group into the <code>alpha</code> group.                                          |
| <code>sourceCompressedLocalProductEmbedding_factors_through_alpha</code>        | THM | Composing <code>sourceAlphaInclusion</code> with <code>sourceCompressedLocalProductToAlpha</code> recovers <code>sourceCompressedLocalProductEmbedding</code> ; proved definitionally. |
| <code>source_compressed_local_product_word_crossing_density_tendsto_zero</code> | THM | Partition crossing density of the acted image of any embedded product element tends to zero.                                                                                           |
| <code>source_compressed_local_product_ball_crossing_density_tendsto_zero</code> | THM | Summed crossing densities over any finite family of embedded product elements tend to zero.                                                                                            |
| <code>sourceCompressedLocalProductApproximation_model_size</code>               | THM | The pulled-back product model has the same size as the original model; definitional simp lemma.                                                                                        |
| <code>source_canonical_product_radius_crossing_density_tendsto_zero</code>      | THM | Crossing sums over the canonical radius- $k$ product label set in the pulled-back approximation have vanishing density.                                                                |

## 9.4 Retained transported component matching (SourceRetainedActualMatching)

Defines the logarithmic component rank, the offset floor at scale  $H$  and offset  $r$  of the logarithm of a point's component size, and the retained components of a transported partition, then proves the two matching statements at the heart of the transport argument. Pointwise, every retained component  $C$  overlaps a unique part of the reference partition in proportion at least  $1 - \eta$ , its symmetric difference with that part is below  $(e^H - 1 + 2\eta)|C|$ , and under the smallness condition the matched part is dominated by the intersection. In sequence form, assuming vanishing ambient weighted midrank variance, vanishing inverse crossings, and vanishing insufficient-overlap densities, the retained components match reference parts with vanishing discarded support, vanishing total symmetric difference, and an injective matching.

**retained\_transport\_component\_matching\_bounds** thm

```

theorem retained_transport_component_matching_bounds
  {V : Type*} [Fintype V] [DecidableEq V]
  (Q : Finpartition (Finset.univ : Finset V))
  (T : Equiv.Perm V)
  (H r eta : ℝ) (hH : 0 < H)
  (C : Finset V)
  (hC : C ∈ retainedTransportedComponents Q T H r eta) :
  SoficGroups.maximumOverlapPart Q C ∈ Q.parts ∧
  (1 - eta) * (C.card : ℝ) ≤
  ((C n SoficGroups.maximumOverlapPart Q C).card : ℝ) ∧
  ((C Δ SoficGroups.maximumOverlapPart Q C).card : ℝ) <
  (Real.exp H - 1 + 2 * eta) * (C.card : ℝ) ∧
  (2 * (Real.exp H - 1 + 2 * eta) < 1 →
  (SoficGroups.maximumOverlapPart Q C).card <
  2 * (C n SoficGroups.maximumOverlapPart Q C).card) := by
-- proof: 63 lines, omitted

```

`retained_transport_component_matching_bounds` is the pointwise geometric heart of the matching. For a component  $C$  in `retainedTransportedComponents Q T H r eta` it derives four facts: `maximumOverlapPart Q C` is a genuine part of  $Q$ ; the intersection satisfies  $(1 - \eta)|C| \leq |C \cap \text{overlap}|$ ; the symmetric difference satisfies  $|C \triangle \text{overlap}| < (e^H - 1 + 2\eta)|C|$ ; and when  $2(e^H - 1 + 2\eta) < 1$  the matched part is strictly smaller than twice the intersection, so  $C$  contains a strict majority of its matched part. The proof unpacks `SourceCompressionRetainedDiscard.retainedComponents_spec`, which supplies the part membership, the overlap bound, and a witness point  $y \in C \cap \text{overlap}$  whose logarithmic rank is invariant under  $T$ . Writing  $C$  as the image under  $T$  of a reference part and pulling the witness back through  $T$ , the lemma `SourceCompressionMatching.transported_maximumOverlapPart_card_lt_exp_mul` bounds the matched part size by  $e^H|C|$ : the floor rank pins the logarithms of the two component sizes to a common window of width  $H$ . Combining this with the overlap bound yields the symmetric-difference estimate (`symmDiff_card_lt_of_overlap_and_exp_card`) and the majority estimate (`target_majority_of_overlap_and_exp_card`). The majority bound is what makes the matching injective and measure-preserving up to vanishing error in `idx 1366`.

**`retained_source_transport_matching_of_ambient_midrank_variance` `thm`**

```
theorem retained_source_transport_matching_of_ambient_midrank_variance
  (V : ℕ → Type*)
  [V n, Fintype (V n)] [V n, Nonempty (V n)]
  [V n, DecidableEq (V n)]
  (κ : Type*)
  (Q A : (n : ℕ) → Finpartition
    (Finset.univ : Finset (V n)))
  (T : (n : ℕ) → κ → Equiv.Perm (V n))
  (H r eta : ℕ → ℝ)
  (hHpos : ∀ n, 0 < H n)
  (hHzero : Tendsto H atTop (nhds 0))
  (heta0 : ∀ n, 0 ≤ eta n)
  (hetazero : Tendsto eta atTop (nhds 0))
  (hvariance : Tendsto
    (fun n =>
      (∑ C ∈ (A n).parts,
        (C.card : ℝ) *
          SoficGroups.midrankVariance
            (SoficGroups.componentRankMassList C
              (logarithmicComponentRank (Q n) (H n) (r n)))) /
          Fintype.card (V n))
    atTop (nhds 0))
  (hcross : ∀ i : κ, Tendsto
    (fun n =>
      ((SoficGroups.partitionWordCrossing
        (A n) (T n i).symm).card : ℝ) /
        Fintype.card (V n))
    atTop (nhds 0))
  (hoverlap : ∀ i : κ, Tendsto
    (fun n =>
      ((∑ C ∈ SoficGroups.insufficientOverlapComponents
        (SoficGroups.transportedUnivFinpartition (Q n) (T n i))
        (Q n) (eta n), C.card : ℕ) : ℝ) /
        Fintype.card (V n))
    atTop (nhds 0)) :
  ∀ i : κ,
    (∀ n, retainedTransportedComponents
      (Q n) (T n i) (H n) (r n) (eta n) ≤
        (SoficGroups.transportedUnivFinpartition
          (Q n) (T n i)).parts) ∧
  -- statement truncated; proof: 68 lines, omitted
```

`retained_source_transport_matching_of_ambient_midrank_variance` is the sequence-level matching theorem. Its inputs are partition sequences  $Q_n$  and ambient partitions  $A_n$  on finite vertex types, permutation families  $T_n(i)$ , positive scales  $H \rightarrow 0$ , nonnegative  $\eta \rightarrow 0$ , a vanishing weighted midrank variance of `logarithmicComponentRank` measured over the ambient parts, vanishing crossings of the inverse permutations with respect to  $A_n$ , and vanishing insufficient-overlap densities. For each index  $i$  it concludes: retained components are

parts of the transported partition; their maximum-overlap parts are genuine parts of  $Q_n$ ; the majority bound holds under the smallness condition; the density of vertices missed by the matched retained support tends to zero; the summed symmetric differences have vanishing density; and `maximumOverlapPart` is injective on retained components once  $2(e^H - 1 + 2\eta) < 1$ . Structurally, `KunCommonRankArcInvariance.exists_common_rank_invariance_of_midrank_variance` turns the variance and crossing hypotheses into rank invariance along the permutations; `SourceCompressionRetainedDiscard.retained_missing_density_tendsto_zero` controls the discarded support; `SourceCompressionMatching.symmDiff_density_tendsto_zero_of_log_rank_bounds` integrates the pointwise bound from idx 1365 into a density statement; and `finpartition_dominant_matching_injOn` extracts injectivity from the majority bound. It is applied at idx 1396 with  $T$  the acted compression tables, evaluated at index zero, that is, the acted `sourceCompressionUElement`.

*Remaining declarations in this section:*

|                                            |     |                                                                                                                           |
|--------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------|
| <code>logarithmicComponentRank</code>      | DEF | Integer rank: offset floor at scale $H$ of the logarithm of a point's partition component size.                           |
| <code>retainedTransportedComponents</code> | DEF | Components of the transported partition retained by the discard procedure for the logarithmic rank and tolerance $\eta$ . |

## 9.5 Capped overlap scales (KunActualBothTransportedOverlapScales)

A normalization step for the overlap scales produced at idx 1348. Since  $\eta \rightarrow 0$ , replacing  $\eta$  by  $\min(\eta, 1/2)$  preserves positivity, antitonicity, both limits, the ratio  $\eta/H \rightarrow 0$ , and the vanishing insufficient-overlap conclusion, transferred by eventual equality of the two sequences, while the loss conclusion does not mention  $\eta$  at all. The gain is that  $\eta_n < 1$  now holds at every index, which is the pointwise form required by the common-log-rank selection (idx 1371) and the final unconditional package (idx 1396), avoiding tail bookkeeping in every later existential statement.

### `exists_source_both_capped_overlap_scales` thm

```

theorem exists_source_both_capped_overlap_scales
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (S : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (hsymmetric : ∀ g ∈ S, g-1 ∈ S)
  (hgenerates : Subgroup.closure
    (S : Set
      (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode)) =
      T)
  (Q : ∀ n, Finpartition
    (Finset.univ : Finset (Fin (A.model n).size)))
  (gamma : ℝ) (hgamma : 0 < gamma)
  (hexpand : ∀ n C, C ∈ (Q n).parts →
    ∀ E : Finset (Fin (A.model n).size), E ⊆ C →
      2 * E.card ≤ C.card →
        gamma * (E.card : ℝ) ≤
          (SoficGroups.boundary
            (fun i : ιS =>
              (A.model n).action
                (SoficGroups.SourceBothCompressionNormalization.sourceAlphaElement
                  (i : SoficGroups.prefixElementaryGroup
                    SoficGroups.alphaPrefixCode))) E : ℝ))
  (hsource : Tendsto
    (fun n =>
      (∑ C ∈ (Q n).parts,
        (SoficGroups.boundary
          (fun i : ιS =>
            (A.model n).action
              (SoficGroups.SourceBothCompressionNormalization.sourceAlphaElement
                (i : SoficGroups.prefixElementaryGroup
                  SoficGroups.alphaPrefixCode))) C : ℝ)) /

```

```

      (A.model n).size)
    atTop (nhds 0)) :
  ∃ eta H : ℕ → ℝ,
    (∀ n, 0 < eta n) ∧
    (∀ n, eta n < 1) ∧
    Antitone eta ∧
    Tendsto eta atTop (nhds 0) ∧
    (∀ n, 0 < H n) ∧
  -- statement truncated; proof: 54 lines, omitted

```

`exists_source_both_capped_overlap_scales` has the same hypotheses as `idx 1348` and almost the same conclusion, but strengthens the eventual bound to  $\eta_n < 1$  for every  $n$ , and drops the explicit tail witness. The construction is elementary: obtain  $\eta, H$  from `exists_source_both_common_slow_overlap_scales` and define the capped sequence  $\min(\eta_n, 1/2)$ . Positivity follows from `lt_min`, the strict bound below one from  $\min \leq 1/2$ , and antitonicity from monotonicity of  $\min$ . Because  $\eta \rightarrow 0$ , the capped sequence agrees with  $\eta$  eventually (the filter-theoretic `EventuallyEq` obtained from `gt_mem_nhds`), so the limit, the ratio  $\eta/H \rightarrow 0$ , and the insufficient-overlap conclusion, which depends on  $\eta$  only through its eventual values, all transfer by `Tendsto.congr'`. The dominant-overlap loss conclusion never mentions  $\eta$ , so it passes through unchanged. Though routine, this step is load-bearing: the downstream rank-selection theorem (`idx 1371`) and the matching theorem (`idx 1366`) require  $0 \leq \eta_n$  and  $\eta_n < 1$  at every index, since their internal constructions divide by quantities like  $1 - \eta_n$ ; phrasing those theorems with merely eventual bounds would force tail-shift bookkeeping through every subsequent existential, including the final package at `idx 1396`.

## 9.6 Ambient crossings and common log rank (KunLiteralNineSourceCompletedCentralizerModels)

Crossing estimates in the ambient nine-prefix group and the selection of a common rank offset. From vanishing summed boundaries for an ambient generating finset, the crossing density of any fixed acted element, of finite families, and of inverse permutations is shown to vanish, the last using the almost-inverse property of sofic actions. The main theorem then selects offsets  $r_n \in [0, H_n)$  so that the weighted midrank variance of the logarithmic rank of the source partition, measured over the ambient partition, vanishes, and so do the crossings of the inverse compression tables. The proof pushes vanishing midrank energy from the positive generator family to the full ambient generating set using the positivity hypothesis, then applies the half-expansion of the ambient partition.

**`exists_source_common_log_rank_with_ambient_midrank_variance` thm**

```

theorem exists_source_common_log_rank_with_ambient_midrank_variance
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (SΓ : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (SG : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (QG QG : (n : ℕ) →
    Finpartition (Finset.univ : Finset (Fin (A.model n).size)))
  (yG : ℝ)
  (hsymmetricG : ∀ g ∈ SG, g⁻¹ ∈ SG)
  (hgeneratesG : Subgroup.closure
    (SG : Set
      (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode)) =
      T)
  (hpositive : Subgroup.closure
    (Set.range
      (SoficGroups.KunExactActualSourceAmbientGenerators.sourcePositiveGeneratorMap
        SΓ)) = T)
  (hyG : 0 < yG)
  (hexpandG : ∀ n C, C ∈ (QG n).parts →
    ∀ E : Finset (Fin (A.model n).size), E ⊆ C →
      2 * E.card ≤ C.card →
        yG * (E.card : ℝ) ≤
          (SoficGroups.boundary
            (fun i : ιSG =>

```

```

      (A.model n).action
      (i : SoficGroups.prefixElementaryGroup
       SoficGroups.ninePrefixCode)) E : ℝ))
(hboundaryΓ : Tendsto
 (fun n =>
  (∑ C ∈ (QΓ n).parts,
   (SoficGroups.boundary
    (fun i : ιSΓ =>
     (A.model n).action
      (SoficGroups.KunExactActualSourceAmbientGenerators.sourceAlphaInclusion
       (i : SoficGroups.prefixElementaryGroup
        SoficGroups.alphaPrefixCode))) C : ℝ)) /
    (A.model n).size)
  atTop (N 0))
-- statement truncated; proof: 155 lines, omitted

```

`exists_source_common_log_rank_with_ambient_midrank_variance` is the analytic pivot of this chunk. Given both partition sequences (source and ambient), generating finsets of the alpha and nine-prefix groups, the ambient expansion constant  $\gamma_G$ , positivity of the closure of `sourcePositiveGeneratorMap` (alpha generators together with the two compression tables), vanishing boundary sums for both partitions, and capped scales  $\eta, H$  with the transported overlap and loss conclusions, it produces offsets  $r_n \in [0, H_n)$  such that: the ambient-partition-weighted midrank variance of `componentLogRank` of the source partition at scale  $H_n$ , offset  $r_n$ , tends to zero; and the crossings of the inverse compression-table permutations with respect to the ambient partition vanish. The proof first invokes `SourceCommonComponentRankNoBad.exists_common_positive_component_log_rank_with_vanishing_midrank_energy` to find  $r$  with vanishing midrank energy for the positive generator family. It then defines the potential  $f_n$  as `MidrankPermutationEnergy.partitionVertexMidrank` of the ambient partition with the chosen rank, checks  $0 \leq f_n \leq 1$ , rewrites the energy as `squaredPermutationEnergy`, and applies `KunPositiveWordMidrankEnergy.sum_action_energy_tendsto_zero_of_positive_generator_sum`, using that the positive generators generate the whole group, to spread the vanishing energy to the full ambient generating set. Finally `KunGlobalActualAdditiveMidrankVariance.weighted_component_midrankVariance_tendsto_zero_of_source_half_expansion` converts vanishing generator energy plus the  $\gamma_G$  half-expansion into the variance conclusion; the inverse crossings are `idx 1370`.

*Remaining declarations in this section:*

|                                                   |                  |                                                                                                                    |
|---------------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>sourceAmbientGeneratedWord_crossing_</code> | <code>THM</code> | Crossing density of any fixed acted nine-prefix element vanishes, given vanishing ambient generator boundary sums. |
| <code>density_tendsto_zero</code>                 |                  |                                                                                                                    |
| <code>sourceAmbientFiniteFamily_crossing_</code>  | <code>THM</code> | Summed crossing densities over a finite family of acted nine-prefix elements vanish.                               |
| <code>density_tendsto_zero</code>                 |                  |                                                                                                                    |
| <code>sourceAmbientActualInverse_crossing_</code> | <code>THM</code> | Crossing density of the inverse of any acted element vanishes, via the almost-inverse property of sofic actions.   |
| <code>density_tendsto_zero</code>                 |                  |                                                                                                                    |

## 9.7 Expansion transfer past generator disagreement (KunActualFirstStageReferenceExpansion)

Develops a purely combinatorial transfer principle: if a reference permutation family gamma-expands small subsets of a component  $C$ , then any actual family disagreeing with it on few points, completed to permutations of  $C$ , satisfies a two-sided min-form expansion inequality with an additive defect controlled by the reference boundary of  $C$  plus twice the disagreement count. The defect is normalized by the component size, shown to vanish under vanishing boundary and disagreement densities, and repackaged with an explicit vanishing sequence. Finally the disagreement count is bounded by the generator count times the intersection with a designated bad set, giving a variant driven by a selected bad set of vanishing relative density, ready to apply to retained components carrying guarded bad sets.

**completed\_actual\_component\_additive\_expansion\_of\_reference** `thm`

```

theorem completed_actual_component_additive_expansion_of_reference
  {V ι : Type*} [Fintype ι] [DecidableEq V]
  (σref oact : ι → Equiv.Perm V) (C : Finset V)

```

```

(τ : ι → Equiv.Perm {x : V // x ∈ C})
(hτ : ∀ i (x : V) (hx : x ∈ C) (_hy : σact i x ∈ C),
  ((τ i (x, hx) : {x : V // x ∈ C}) : V) = σact i x)
(γ : ℝ)
(hexpand : ∀ E : Finset V, E ⊆ C →
  2 * E.card ≤ C.card →
  γ * (E.card : ℝ) ≤ (SoficGroups.boundary σref E : ℝ)) :
∀ E : Finset {x : V // x ∈ C},
  γ * min (E.card : ℝ) ((C.card : ℝ) - E.card) -
  ((SoficGroups.boundary σref C : ℝ) +
    2 * (componentGeneratorDisagreement C σref σact : ℝ)) ≤
  (SoficGroups.boundary τ E : ℝ) := by
-- proof: 80 lines, omitted

```

completed\_actual\_component\_additive\_expansion\_of\_reference: fix a component  $C$ , a reference family that  $\gamma$ -expands subsets  $E \subseteq C$  with  $2|E| \leq |C|$ , an actual family, and permutations  $\tau_i$  of the subtype of  $C$  that agree with the actual family wherever it stays inside  $C$ . The conclusion: for every subset  $E$  of the subtype,  $\gamma \cdot \min(|E|, |C| - |E|)$  minus the defect, the reference boundary of  $C$  plus twice `componentGeneratorDisagreement`, is a lower bound for the  $\tau$ -boundary of  $E$ . For the small half  $2|E| \leq |C|$ , the proof pushes  $E$  down to a subset  $D \subseteq C$ , applies the reference expansion to  $D$ , and chains three comparison inequalities: reference-to-actual boundary via the disagreement count (idx 1374); actual-to-completed boundary via `KunResidualExpanderDecomposition.original_boundary_le_completed_add_component_boundary`, since points that exit  $C$  are counted in the boundary of  $C$  itself; and the actual boundary of  $C$  back to the reference boundary via disagreement again; `linarith` finishes. For the large half the complement trick applies: `boundary_complement` shows the  $\tau$ -boundary of  $E$  and of its complement in the subtype coincide, and the min collapses to  $|C| - |E|$ . This converts ambient reference expansion into intrinsic expansion of the completed permutations on each retained component, exactly the shape needed by the spectral-gap contradiction; the additive defect is normalized and eliminated in idx 1377 to 1382.

**exists\_vanishing\_completedRestriction\_additive\_expansion\_of\_selected\_bad** thm

```

theorem exists_vanishing_completedRestriction_additive_expansion_of_selected_bad
  (V : ℕ → Type*) [V n, Fintype (V n)]
  [V n, DecidableEq (V n)]
  (ι : Type*) [Fintype ι]
  (C B : (n : ℕ) → Finset (V n))
  (hC : ∀ n, (C n).Nonempty)
  (σref σact : (n : ℕ) → ι → Equiv.Perm (V n))
  (γ : ℝ)
  (hexpand : ∀ n, ∀ E : Finset (V n), E ⊆ C n →
    2 * E.card ≤ (C n).card →
    γ * (E.card : ℝ) ≤
      (SoficGroups.boundary (σref n) E : ℝ))
  (hbad : ∀ n i (x : V n), x ∈ C n →
    σref n i x ≠ σact n i x → x ∈ B n)
  (hboundary : Tendsto
    (fun n =>
      (SoficGroups.boundary (σref n) (C n) : ℝ) /
      (C n).card)
    atTop (nhds 0))
  (hbad_density : Tendsto
    (fun n =>
      (((C n n B n).card : ℝ) / (C n).card))
    atTop (nhds 0)) :
  ∃ a : ℕ → ℝ,
    (∀ n, 0 ≤ a n) ∧
    Tendsto a atTop (nhds 0) ∧
    ∀ n, ∀ E : Finset {x : V n // x ∈ C n},
      γ * min (E.card : ℝ)
        ((Fintype.card {x : V n // x ∈ C n} : ℝ) - E.card) -
        a n * Fintype.card {x : V n // x ∈ C n} ≤
        (SoficGroups.boundary
          (fun i =>
            SoficGroups.MatchedComponentCompletion.completedRestriction

```

```

      (oact n i) (C n)) E : ℝ) := by
-- proof: 5 lines, omitted

```

`exists_vanishing_completedRestriction_additive_expansion_of_selected_bad` packages the transfer principle in the exact form consumed downstream. Hypotheses: sequences of nonempty components  $C_n$  and bad sets  $B_n$ ; reference and actual permutation families whose disagreement points on  $C_n$  all lie in  $B_n$ ;  $\gamma$ -expansion of the reference family on small subsets of  $C_n$ ; vanishing of the reference boundary of  $C_n$  relative to  $|C_n|$ ; and vanishing relative density of  $C_n \cap B_n$ . Conclusion: there is a nonnegative null sequence  $a_n$  such that for every  $n$  and every subset  $E$  of the subtype of  $C_n$ ,  $\gamma \min(|E|, |C_n| - |E|) - a_n |C_n|$  bounds from below the boundary of the family `MatchedComponentCompletion.completedRestriction` of the actual permutations on  $C_n$ . The proof composes `idx 1380`, which bounds the disagreement count by the number of generators times  $|C_n \cap B_n|$ , with `idx 1381` to get vanishing disagreement density, then applies `idx 1379`, where  $a_n$  is the `normalizedReferenceCompletionError` of `idx 1377` and the expansion inequality is `idx 1376`. The point of the selected-bad formulation: in the application, disagreements between reference generators (honest acted conjugates in the group) and transported generators (conjugates of permutations) are certified to lie in the conjugacy bad sets of `idx 1389`, so retained components chosen to avoid the guarded bad sets inherit genuine expander behavior with vanishing defect.

*Remaining declarations in this section:*

|                                                                                    |     |                                                                                                                                               |
|------------------------------------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>componentGeneratorDisagreement</code>                                        | DEF | Counts, summed over generators, the points of $C$ where the reference and actual permutation families disagree.                               |
| <code>componentGeneratorDisagreement_comm</code>                                   | THM | The disagreement count is symmetric in the reference and actual families.                                                                     |
| <code>boundary_le_boundary_add_componentGeneratorDisagreement</code>               | THM | The reference boundary of $E \subseteq C$ is at most the actual boundary plus the disagreement count over $C$ .                               |
| <code>completedRestriction_additive_expansion_of_reference</code>                  | THM | Specializes the additive expansion bound to the canonical <code>completedRestriction</code> permutations of the actual family on a component. |
| <code>normalizedReferenceCompletionError</code>                                    | DEF | Normalized defect: reference boundary of $C$ plus twice the disagreement count, divided by the component size.                                |
| <code>normalizedReferenceCompletionError_tendsto_zero</code>                       | THM | The normalized completion error tends to zero when boundary and disagreement densities both vanish.                                           |
| <code>exists_vanishing_completedRestriction_additive_expansion_of_reference</code> | THM | Packages the additive expansion with an explicit nonnegative vanishing defect sequence $a_n$ .                                                |
| <code>componentGeneratorDisagreement_le_card_mul_component_bad</code>              | THM | Bounds the disagreement count by the number of generators times the bad-set intersection cardinality.                                         |
| <code>componentGeneratorDisagreement_density_tendsto_zero_of_component_bad</code>  | THM | Disagreement density vanishes whenever the relative density of the bad set inside $C_n$ vanishes.                                             |

## 9.8 Compression element conjugation bridges (SourceProductThroughAlpha)

Three bridges tying the algebraic compression to the model-level transport. The identity `source_u_conjugated_product_generator` shows that embedding the pair of a compressed generator and the identity through the product embedding is literally the conjugate of the included alpha element by `sourceCompressionUElement`; thus transported reference generators are embedded product elements. The remaining two theorems transport the gamma-expansion hypothesis and the vanishing boundary sums from the source partitions to the partitions pushed forward along the acted compression element, with the conjugated generator family, by direct application of the `KunTransportedAmbientOverlap` half-expansion and boundary-transport lemmas.

`source_u_conjugated_product_generator` thm



```

theorem source_u_conjugated_product_generator
  (g : SoficGroups.prefixElementaryGroup
    SoficGroups.alphaPrefixCode) :
  SoficGroups.ThompsonPrefixInsertion.sourceCompressedLocalProductEmbedding
    (sourceCompressionUAlphaEquiv g, 1) =
  SoficGroups.SourceBothCompressionNormalization.sourceCompressionUElement *
    SoficGroups.SourceGeneratedWordCrossing.sourceAlphaInclusion g *
    (SoficGroups.SourceBothCompressionNormalization.sourceCompressionUElement)-1 := by
  -- proof: 15 lines, omitted

```

`source_u_conjugated_product_generator` is a pure algebra identity in the nine-prefix elementary group: for  $g$  in the alpha prefix group, embedding the pair  $(\text{equiv}(g), 1)$  via `sourceCompressedLocalProductEmbedding`, where the first coordinate is `sourceCompressionUAlphaEquiv` applied to  $g$ , equals the product `sourceCompressionUElement` times `sourceAlphaInclusion` of  $g$  times the inverse of `sourceCompressionUElement`. The proof is by `Subtype.ext`: on underlying units of the binary Leavitt algebra the left side is  $(\text{equiv}(g)) \cdot 1$ , and since the equivalence was defined as conjugation `MulAut.conj` by `compressionU`, both sides reduce to `compressionU` times  $g$  times its inverse, closed by `MulAut.conj_apply`. Its significance is that it identifies two a priori different objects handled by different toolkits: transported reference generators, conjugates by the compression element, about which the transported-partition expansion and boundary lemmas of idx 1384 and 1385 speak; and embedded product elements of the form  $(k, 1)$ , about which the canonical product-radius machinery, the Cayley-ball realization, and the pulled-back approximation of idx 1360 speak. Thanks to this identity, avoiding the canonical product bad set simultaneously controls the transported generator family, up to the conjugacy disagreement sets of idx 1345, which is exactly how `sourceSelectedRadiusBad` (idx 1391) and the guarded selection (idx 1395) are wired together.

*Remaining declarations in this section:*

|                                                   |     |                                                                                                                                                |
|---------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>source_u_transported_reference_half_</code> | THM | Gamma-expansion of source partition parts transfers to the partition transported by the acted compression element, with conjugated generators. |
| <code>expansion</code>                            |     |                                                                                                                                                |
| <code>source_u_transported_reference_</code>      | THM | Vanishing boundary sums transfer to the transported partition for the compression-conjugated generator family.                                 |
| <code>boundary_density_tendsto_zero</code>        |     |                                                                                                                                                |

## 9.9 Transported generators and guard sets (KunLiteralSourceSelectedComponents)

Concrete instantiations at the model level. `sourceCompressionPermutation` is the acted compression element; conjugating each acted alpha generator by it gives `sourceTransportedReferenceGenerators`, and pushing the source partition along it gives `sourceTransportedPartition`. The conjugacy disagreement bad sets from idx 1345 are unioned over a generating finset into `sourceGeneratorFamilyConjugacyBad`, whose density vanishes as a finite union of vanishing sets; outside it, the transported reference generators agree pointwise with the acted honest conjugates. Finally `sourceSelectedRadiusBad` unions the canonical product radius bad set of the compressed product approximation with this conjugacy bad family, forming the concrete guard set used in the selection argument.

**sourceSelectedRadiusBad** `def`

```

noncomputable def sourceSelectedRadiusBad
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (S : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (hsymmetric : ∀ g ∈ S, g-1 ∈ S)
  (hgenerates : Subgroup.closure
    (S : Set
      (SoficGroups.prefixElementaryGroup
        SoficGroups.alphaPrefixCode)) = T)
  (F : Finset
    (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
      SoficGroups.ThompsonPrefixLocal.sourceLocalWord))

```

```

(n k : N) : Finset (Fin (A.model n).size) := by
classical
let SK :=
  SoficGroups.SourceProductThroughAlpha.sourceCompressedGeneratingFinset S
let AP :=
  SoficGroups.SourceProductThroughAlpha.sourceCompressedLocalProductApproximation A
exact
  SoficGroups.CanonicalProductRadiusBadMatchedCapture.canonicalProductRadiusBad
    AP SK
    (SoficGroups.SourceProductThroughAlpha.sourceCompressedGeneratingFinset_inv_mem
      S hsymmetric)
    (SoficGroups.SourceProductThroughAlpha.sourceCompressedGeneratingFinset_closure
      S hgenerates)
  F n k u sourceGeneratorFamilyConjugacyBad A S n

```

`sourceSelectedRadiusBad` assembles, at level  $n$  and radius parameter  $k$ , the full guard set for component selection: the union of `canonicalProductRadiusBad`, computed in the pulled-back product approximation `sourceCompressedLocalProductApproximation` with generating finset `sourceCompressedGeneratingFinset` of  $S$  (whose inverse-closure and closure proofs are threaded in as arguments) and auxiliary finset  $F$ , with `sourceGeneratorFamilyConjugacyBad`. A point outside this union is good in two independent senses. Outside the canonical bad set, the Cayley ball of radius about  $k/2$  in the compressed product generators embeds injectively into the point's matched component; this is the realization property invoked at idx 1395. Outside the conjugacy bad family, all transported reference generators, which are conjugates of permutations, agree at the point with the acted honest group conjugates, so word estimates carried out in the group itself apply. The definition uses classical choice for decidability and local abbreviations for the compressed generating finset and the pulled-back approximation. No density statement is proved in this declaration, but for each fixed  $k$  it follows from `canonicalProductRadiusBad_density_tendsto_zero` together with idx 1390 via a union bound; this is exactly the pattern abstracted by `guardedCanonicalProductRadiusBad` (idx 1394), of which `sourceSelectedRadiusBad` is the intended concrete instance with the conjugacy family in the guard slot.

*Remaining declarations in this section:*

|                                                                     |     |                                                                                                                      |
|---------------------------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------|
| <code>sourceCompressionPermutation</code>                           | DEF | The level- $n$ model permutation realizing <code>sourceCompression</code> <code>UElement</code> .                    |
| <code>sourceTransportedReferenceGenerators</code>                   | DEF | Generator family obtained by conjugating each acted alpha generator with <code>sourceCompressionPermutation</code> . |
| <code>sourceTransportedPartition</code>                             | DEF | The universal partition transported along <code>sourceCompressionPermutation</code> .                                |
| <code>sourceGeneratorFamilyConjugacyBad</code>                      | DEF | Union over the generating finset of the conjugacy disagreement bad sets for the compression element.                 |
| <code>sourceGeneratorFamilyConjugacyBad_density_tendsto_zero</code> | THM | Density of the generator-family conjugacy bad set vanishes, as a finite union of vanishing bad sets.                 |

## 9.10 Guarded canonical component selection (SourceGuardedCanonicalSelection)

Sets up and executes the guarded selection. After abbreviating the two product factors as `SourceK` and `SourceJ`, `guardedCanonicalProductRadiusBad` augments the canonical product-radius bad set of the compressed approximation by an arbitrary guard family  $E$  of vanishing density. The main theorem converts retained-matching data, retained components inside the transported partition with matched parts, majority bounds, and vanishing discard and symmetric-difference densities, into an actual selection: a tail shift  $N \geq N_0$  past which the scale conditions hold, radii  $r_n \rightarrow \infty$ , and components  $C_n$  of unbounded size whose normalized boundary vanishes and which meet the guarded matched-radius bad set in vanishing proportion. The key input is a Cayley-ball realization: good points of retained components inject balls of the compressed generating set into the component.

`exists_source_guarded_canonical_selected_components_of_retained_matching` *thm*

```

theorem exists_source_guarded_canonical_selected_components_of_retained_matching
  [DecidableEq SourceK] [DecidableEq SourceJ]
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (SΓ : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (hone : 1 ∈ SΓ)
  (hsymmetric : ∀ g ∈ SΓ, g-1 ∈ SΓ)
  (hgenerates : Subgroup.closure
    (SΓ : Set
      (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode)) =
    T)
  (P Q : (n : ℕ) → Finpartition
    (Finset.univ : Finset (Fin (A.model n).size)))
  {κ : Type*} [Fintype κ]
  (σ : (n : ℕ) → κ → Equiv.Perm (Fin (A.model n).size))
  (R : (n : ℕ) → Finset (Finset (Fin (A.model n).size)))
  (hR : ∀ n, R n ⊆ (P n).parts)
  (D : (n : ℕ) → Finset (Fin (A.model n).size) →
    Finset (Fin (A.model n).size))
  (hD : ∀ n C, C ∈ R n → D n C ∈ (Q n).parts)
  (H eta : ℕ → ℝ)
  (hH : Tendsto H atTop (nhds 0))
  (heta : Tendsto eta atTop (nhds 0))
  (hmajor : ∀ n,
    eta n ≤ (1 : ℝ) / 2 →
    2 * (Real.exp (H n) - 1 + 2 * eta n) < 1 →
    ∀ C, C ∈ R n →
      (D n C).card < 2 * (C n D n C).card)
  (hdiscard : Tendsto
    (fun n =>
      (((Finset.univ : Finset (Fin (A.model n).size)) \
        SoficGroups.matchedRetainedSupport (R n)).card : ℝ) /
      (A.model n).size))
    atTop (nhds 0))
  (hsymm : Tendsto
    (fun n =>
      ((∑ C ∈ R n, (C Δ D n C).card : ℕ) : ℝ) /
      (A.model n).size))
    atTop (nhds 0))
-- statement truncated; proof: 155 lines, omitted

```

`exists_source_guarded_canonical_selected_components_of_retained_matching` is the selection engine of the endgame. Inputs: a generating finset of the alpha group containing the identity; partitions  $P$  and  $Q$  of the models; a finite permutation family  $\sigma$  with vanishing summed boundary over  $P$ ; retained components  $R_n \subseteq P_n$  parts with a matched-part map  $D$  landing in  $Q_n$  parts; the majority bound under the scale condition; vanishing discard and symmetric-difference densities; guard sets  $E$  with vanishing density for each radius; vanishing source boundary sums over  $Q$ ; and a threshold  $N_0$ . Output: a shift  $N \geq N_0$ , radii  $r_n \rightarrow \infty$ , and components  $C_n \in R_{n+N}$  such that  $\eta \leq 1/2$  and  $2(e^H - 1 + 2\eta) < 1$  hold on the whole tail, the  $\sigma$ -boundary of  $C_n$  normalized by  $|C_n|$  vanishes,  $|C_n| \rightarrow \infty$ , and the fraction of  $C_n$  lying in `matchedRadiusBad` for the label family `sourceProductRadiusLabels` at radius  $r_n$  with the guarded bad set tends to zero. The proof translates everything into the pulled-back product approximation: crossing sums over canonical labels vanish (idx 1362); the guarded bad set has vanishing density by a union estimate combining `canonicalProductRadiusBad_density_tendsto_zero` with the guard hypothesis; and `canonical_source_matched_component_cayley_ball_realization` shows every good point of a retained component injects the product Cayley ball of radius  $k/2$  into that component, using that `SourceK` is infinite. The abstract tail-selection theorem `exists_matched_slow_diagonal_large_components_on_source_scale_tail` then produces the data.

*Remaining declarations in this section:*

|                      |      |                                                                                                 |
|----------------------|------|-------------------------------------------------------------------------------------------------|
| <code>SourceK</code> | ABBR | Abbreviation for the alphaZero prefix elementary group, the property (T) factor of the product. |
| <code>SourceJ</code> | ABBR | Abbreviation for the local prefix transposition group at the source local word.                 |

guardedCanonicalProductRadiusBad DEF Canonical product radius bad set of the compressed product approximation, united with an arbitrary extra guard family  $E$ .

## 9.11 Unconditional retained matching package (KunActualSourceURetainedNoPremise)

The capstone of this stretch: with no hypotheses beyond a sofic approximation  $A$  of the nine-prefix group, produces the full retained-matching state for the compression element. It chains the unconditional dual finpartition sequences, the capped common slow overlap scales (idx 1367), the common log rank with ambient midrank variance (idx 1371), and the sequence matching theorem (idx 1366) specialized to table index zero, whose permutation is the acted sourceCompressionUElement. The output bundles a generating finset with its expansion constant, the source partitions, the scale sequences  $H, \eta, r$ , and the retained component families together with all matching, majority, discard, symmetric-difference, and injectivity properties, in exactly the form consumed by the guarded canonical selection.

**exists\_actual\_source\_u\_retained\_matching** thm

```

theorem exists_actual_source_u_retained_matching
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode)) :
  ∃ (SΓ : Finset
    (SoficGroups.prefixElementaryGroup
      SoficGroups.alphaPrefixCode))
  (QΓ : (n : ℕ) →
    Finpartition (Finset.univ : Finset (Fin (A.model n).size)))
  (γΓ : ℝ)
  (H eta r : ℕ → ℝ)
  (R : (n : ℕ) → Finset (Finset (Fin (A.model n).size))),
  1 ∈ SΓ ∧
  (∀ g ∈ SΓ, g-1 ∈ SΓ) ∧
  Subgroup.closure
    (SΓ : Set
      (SoficGroups.prefixElementaryGroup
        SoficGroups.alphaPrefixCode)) = T ∧
  0 < γΓ ∧
  (∀ n, ∀ C ∈ (QΓ n).parts,
    ∀ E : Finset (Fin (A.model n).size),
      E ⊆ C →
        2 * E.card ≤ C.card →
          γΓ * (E.card : ℝ) ≤
            (SoficGroups.boundary
              (fun i : ιSΓ =>
                (A.model n).action
                  (SoficGroups.SourceBothCompressionNormalization.sourceAlphaElement
                    (i : SoficGroups.prefixElementaryGroup
                      SoficGroups.alphaPrefixCode))) E : ℝ)) ∧

  Tendsto
    (fun n =>
      (∑ C ∈ (QΓ n).parts,
        (SoficGroups.boundary
          (fun i : ιSΓ =>
            (A.model n).action
              (SoficGroups.SourceBothCompressionNormalization.sourceAlphaElement
                (i : SoficGroups.prefixElementaryGroup
                  SoficGroups.alphaPrefixCode))) C : ℝ)) /
        (A.model n).size)
      atTop (ℕ 0) ∧
  -- statement truncated; proof: 156 lines, omitted

```

`exists_actual_source_u_retained_matching` removes every hypothesis: from a bare sofic approximation  $A$  of `prefixElementaryGroup ninePrefixCode` it manufactures the entire retained-matching state for the compression element. Step one: `KunUnconditionalActualSourceDecomposition.exists_unconditional_actual_source_dual_finpartition_sequences` yields generating finsets of the alpha and nine-prefix groups, dual partition sequences,

expansion constants  $\gamma_T$  and  $\gamma_G$ , positivity of the positive-generator closure, and both vanishing boundary sums; this is where the property (T) expansion enters. Step two: the capped overlap scales (idx 1367) give  $\eta, H$  with  $\eta_n < 1$  pointwise and the transported overlap and loss conclusions for both compression tables. Step three: idx 1371 selects  $r_n \in [0, H_n)$  with vanishing ambient midrank variance and vanishing inverse-table crossings; a `simpa` aligns the two definitionally equal notions of logarithmic rank and the two spellings of the compression tables. Step four: the sequence matching theorem (idx 1366), instantiated with index type `Fin 2` and  $T$  the acted compression tables, is evaluated at index zero, whose permutation is definitionally the acted `sourceCompressionUElement`, producing the retained families with parts containment, matched-part membership, the majority bound, vanishing discarded support and symmetric-difference densities, and injectivity of `maximumOverlapPart`. The statement fixes the retained families definitionally as `retainedTransportedComponents`, so later theorems can unfold them; this package is what the guarded canonical selection consumes on the way to contradicting soficity.

## 9.12 Selected expander components for the source model (KunLiteralSourceSelectedComponents)

Opens the literal nine-prefix stage of the argument. `sourceSelectedProductRadiusLabels` forms the finite set of product-group labels obtained by pairing the compressed generating finset of  $S$  with  $F$  at word radius  $k$ . The centerpiece, `exists_source_selected_components`, distills the entire earlier selection machinery: from any sofic approximation of the nine-prefix elementary group it extracts a symmetric generating set  $S$  containing the identity, partitions with vanishing boundary density, an expansion constant  $\gamma > 0$ , radii tending to infinity, and a sequence of nonempty partition components that expand sub-half subsets, have vanishing relative boundary and vanishing matched-radius bad density, and grow without bound. This package is the sole input that the finite-model construction of this final stretch consumes.

**exists\_source\_selected\_components** *thm*

```
theorem exists_source_selected_components
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (F : Finset
    (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
      SoficGroups.ThompsonPrefixLocal.sourceLocalWord)) :
  ∃ (S : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
    (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
    (hgenerates : Subgroup.closure
      (S : Set
        (SoficGroups.prefixElementaryGroup
          SoficGroups.alphaPrefixCode)) = T)
    (Q : (n : ℕ) →
      Finpartition (Finset.univ : Finset (Fin (A.model n).size)))
    (gamma : ℝ) (N : ℕ) (r : ℕ → ℕ)
    (C : (n : ℕ) → Finset (Fin (A.model (n + N)).size)),
    1 ∈ S ∧
    0 < gamma ∧
    Tendsto
      (fun n =>
        (∑ D ∈ (Q n).parts,
          (SoficGroups.boundary
            (fun i : ιS =>
              (A.model n).action
                (SoficGroups.SourceGeneratedWordCrossing.sourceAlphaInclusion
                  i)) D : ℝ)) /
          (A.model n).size)
        atTop (ℕ 0) ∧
      Tendsto r atTop atTop ∧
    (∀ n, C n ∈ (sourceTransportedPartition A Q (n + N)).parts) ∧
    (∀ n, (C n).Nonempty) ∧
    (∀ n, ∀ D : Finset (Fin (A.model (n + N)).size),
      D ⊆ C n → 2 * D.card ≤ (C n).card →
        gamma * (D.card : ℝ) ≤
```

```

      (SoficGroups.boundary
        (sourceTransportedReferenceGenerators A S (n + N)) D : ℝ)) ∧
Tendsto
  (fun n =>
    (SoficGroups.boundary
      -- statement truncated; proof: 110 lines, omitted

```

`exists_source_selected_components` is the data-extraction theorem for the literal nine-prefix setting. Given a hypothetical sofic approximation  $A$  of `prefixElementaryGroup` at `ninePrefixCode` and a finite set  $F$  of local prefix transpositions, it returns a finite symmetric generating set  $S$  of the alpha prefix group containing the identity, partitions  $Q_n$  of the finite models, a constant  $\gamma > 0$ , an offset  $N$ , radii  $r_n \rightarrow \infty$ , and components  $C_n$  of the transported partition, such that: the total boundary density of the alpha generators over the parts of  $Q_n$  tends to zero; every  $C_n$  is nonempty and  $\gamma$ -expands each subset of at most half its size against the transported reference generators; the relative boundary of  $C_n$  itself vanishes; the fraction of  $C_n$  lying in the matched-radius bad set built from `sourceSelectedProductRadiusLabels` and `sourceSelectedRadiusBad` at radius  $r_n$  vanishes; and  $|C_n| \rightarrow \infty$ . The proof pulls the retained-matching package from `exists_actual_source_u_retained_matching`, transports partitions and generators by conjugation with the compression element via `sourceTransportedPartition` and `sourceTransportedReferenceGenerators`, invokes `exists_source_guarded_canonical_selected_components_of_retained_matching`, and converts the raw expansion hypothesis through `source_u_transport_reference_half_expansion`. It is exactly the package consumed by `source_ambient_nonempty_expandingCentralizerFiniteModel`.

*Remaining declarations in this section:*

```

sourceSelectedProductRadiusLabels  DEF  Finite label set pairing compressed alpha-zero generators of
                                         S with F at word radius k.

```

### 9.13 Reindexing generators through the compression equivalence (SourceProductThroughAlpha)

A brief technical bridge inside `SourceProductThroughAlpha`. The compression equivalence `sourceCompressionUAlphaEquiv` carries alpha-prefix elements to alpha-zero-prefix elements; `sourceCompressedGeneratorSubtypeEquiv` restricts it to a bijection between a generating finset and its image `sourceCompressedGeneratingFinset`. The companion lemma shows boundary counts of any vertex set are invariant under reindexing a permutation family along this bijection, by rewriting the boundary sum with `Equiv.sum_comp`. Later segments use this to translate expansion statements indexed by the alpha generators into statements indexed by the compressed generating finset expected by the product approximation machinery.

#### **boundary\_sourceCompressedGeneratorSubtypeEquiv** *thm*

```

theorem boundary_sourceCompressedGeneratorSubtypeEquiv
  {V : Type*} [DecidableEq V]
  (SΓ : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (σ : ι(sourceCompressedGeneratingFinset SΓ) → Equiv.Perm V)
  (E : Finset V) :
  SoficGroups.boundary
    (fun i : ιSΓ =>
      σ (sourceCompressedGeneratorSubtypeEquiv SΓ i)) E =
    SoficGroups.boundary σ E := by
  -- proof: 6 lines, omitted

```

`boundary_sourceCompressedGeneratorSubtypeEquiv` states that for any permutation family  $\sigma$  indexed by the subtype of the compressed generating finset and any finset  $E$  of vertices, the boundary of  $E$  computed against the reindexed family  $i \mapsto \sigma(ei)$ , where  $e$  is the subtype equivalence and the index now runs over the original generating finset, equals the boundary computed against  $\sigma$  itself. Since `boundary` is defined as a sum over the index type of the number of points of  $E$  sent outside  $E$ , the proof is a single application of `Equiv.sum_comp` with `sourceCompressedGeneratorSubtypeEquiv`, which is constructed by restricting `sourceCompressionUAlphaEquiv` to the

finset and its image, with both inverse laws checked by `Subtype.ext`. This lemma is the hinge that lets `exists_vanishing_completed_first_factor_additive_expansion_of_selected_source` restate expansion bounds proved for the alpha-indexed generator family in the compressed indexing demanded by the product-approximation machinery, without any loss in the constants.

*Remaining declarations in this section:*

|                                               |     |                                                                                                                                 |
|-----------------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------|
| <code>sourceCompressedGeneratorSubtype</code> | DEF | Bijection between a generating finset of the alpha prefix group and its image under <code>sourceCompressionUAlphaEquiv</code> . |
| <code>Equiv</code>                            |     |                                                                                                                                 |

## 9.14 Stripping the guard from matched densities (KunGuardedCanonicalMatchedDensity)

Two small monotonicity facts about matched-radius bad sets. `matchedRadiusBad_mono_bad` shows the matched bad set grows with its bad-set argument, since both branches of the union defining it are preserved under enlargement. `canonical_matched_density_of_guarded` deduces that if the density of components inside the matched bad set for a union  $B \cup E$  tends to zero, the same holds for  $B$  alone, by a cardinality comparison and `squeeze_zero`. This is used to strip the auxiliary conjugacy-disagreement guard from the selected-components conclusion when instantiating the canonical composition machinery in the nine-prefix setting.

**canonical\_matched\_density\_of\_guarded** thm

```
theorem canonical_matched_density_of_guarded
  (V : ℕ → Type*) [V n, DecidableEq (V n)]
  (U : (n : ℕ) → Finset (V n))
  (P : (n : ℕ) → Finpartition (U n))
  {ι : Type*} (I : ℕ → Finset ι)
  (w : (n : ℕ) → ι → Equiv.Perm (V n))
  (B E C : (n : ℕ) → Finset (V n))
  (hguarded : Tendsto
    (fun n =>
      (((C n n SoficGroups.matchedRadiusBad
        (P n) (I n) (w n) (B n ∪ E n)).card : ℝ) /
        (C n).card))
    atTop (nhds 0)) :
  Tendsto
    (fun n =>
      (((C n n SoficGroups.matchedRadiusBad
        (P n) (I n) (w n) (B n)).card : ℝ) /
        (C n).card))
    atTop (nhds 0)) := by
  -- proof: 21 lines, omitted
```

`canonical_matched_density_of_guarded` removes a guard set from a matched-density hypothesis. The selection pipeline naturally controls the density of  $C_n$  inside the matched-radius bad set for the union  $B_n \cup E_n$ , where  $E_n$  is an auxiliary conjugacy-disagreement guard; the downstream composition theorem, however, requires the density for  $B_n$  alone. Because `matchedRadiusBad_mono_bad` shows the matched-radius bad set is monotone in its bad-set argument (membership comes from a union of a bad-component branch and a crossing branch, and both are preserved when the bad set is enlarged), the intersection of  $C_n$  with the matched bad set for  $B_n$  is contained in the intersection for  $B_n \cup E_n$ . Hence its normalized cardinality is dominated termwise by the guarded density, and `squeeze_zero` together with nonnegativity (via positivity) concludes the limit is zero. This is a routine but structurally necessary step: it is applied in `source_ambient_nonempty_expandingCentralizerFiniteModel` to pass from the selected bad set, which is definitionally a union of the canonical bad set with `sourceGeneratorFamilyConjugacyBad`, to the canonical bad set expected by `nonempty_expandingCentralizerFiniteModel_of_canonical_selected_additive`.

*Remaining declarations in this section:*

|                                        |     |                                                                                                                       |
|----------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------|
| <code>matchedRadiusBad_mono_bad</code> | THM | <code>matchedRadiusBad</code> is monotone in its bad-set argument: enlarging $B$ enlarges the matched-radius bad set. |
|----------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------|



## 9.15 Pruning components into exact expanders (KunActualCanonicalSelectedPruning)

One long theorem that upgrades approximate expansion to exact expansion by pruning. Given selected components of a product sofic approximation whose completed first-factor generator restrictions satisfy a  $\gamma$ -expansion inequality with additive error  $a_n \rightarrow 0$ , it deletes a set  $D_n$  of vanishing relative density from each component and builds honest permutations  $\tau_n$  of the survivors that agree with the completed restrictions wherever both a point and its image survive, and that expand every subset with the clean constant  $\gamma/2$  and no error term. The stagewise construction is delegated to `exists_completed_pruned_expander`; the offset  $N$  makes the additive error small enough for that lemma to apply, and survivor cardinalities are shown to tend to infinity.

**exists\_actual\_uniformly\_expanding\_pruned\_selected\_components** *thm*

```
theorem exists_actual_uniformly_expanding_pruned_selected_components
  {K J : Type*} [Group K] [Group J]
  (A : SoficGroups.SoficApproximation (K × J))
  (S : Finset K)
  (v : ℕ → ℕ)
  (Q : (n : ℕ) → Finpartition
    (Finset.univ : Finset (Fin (A.model (v n)).size)))
  (C : (n : ℕ) → Finset (Fin (A.model (v n)).size))
  (hC : ∀ n, C n ∈ (Q n).parts)
  (hlarge : Tendsto (fun n => (C n).card) atTop atTop)
  (γ : ℝ) (hγ : 0 < γ)
  (a : ℕ → ℝ)
  (ha : Tendsto a atTop (nhds 0))
  (hadd : ∀ n,
    ∀ E : Finset {x : Fin (A.model (v n)).size // x ∈ C n},
    γ * min (E.card : ℝ)
      ((Fintype.card
        {x : Fin (A.model (v n)).size // x ∈ C n} : ℝ) -
        E.card) -
    a n * Fintype.card
      {x : Fin (A.model (v n)).size // x ∈ C n} ≤
    (SoficGroups.boundary
      (fun i : ιS =>
        completedRestriction
          ((A.model (v n)).action ((i : K), (1 : J)))
          (C n)) E : ℝ)) :
  ∃ (N : ℕ)
  (D : (n : ℕ) → Finset
    {x : Fin (A.model (v (n + N))).size // x ∈ C (n + N)})
  (τ : (n : ℕ) → ιS → Equiv.Perm
    {x : {y : Fin (A.model (v (n + N))).size //
      y ∈ C (n + N)} //
    x ∈ (Finset.univ \ D n :
      Finset {y : Fin (A.model (v (n + N))).size //
        y ∈ C (n + N)}}}),
  (∀ n,
    (Finset.univ \ D n :
      Finset {x : Fin (A.model (v (n + N))).size //
        x ∈ C (n + N)}).Nonempty) ∧
  Tendsto
-- statement truncated; proof: 149 lines, omitted
```

`exists_actual_uniformly_expanding_pruned_selected_components` converts approximate expansion into exact expansion at the price of deleting a vanishing fraction of each component. The hypothesis is additive: for the family of completed restrictions of the first-factor generator actions to  $C_{v(n)}$ , every subset  $E$  satisfies  $\gamma \min(|E|, |C| - |E|) - a_n |C| \leq \partial E$ , where  $\partial E$  is the boundary count and  $a_n \rightarrow 0$ . The conclusion produces an offset  $N$ , deletion sets  $D_n$  with  $|D_n|/|C_{n+N}| \rightarrow 0$ , and permutations  $\tau_n$  of the surviving subtype  $\text{univ} \setminus D_n$  that agree with the completed restrictions at every surviving point whose image also survives, and expand every subset with the two-sided bound at constant  $\gamma/2$  and no additive error. The proof sets  $\ell = \gamma/2$ , computes the threshold  $(\gamma - \ell)^2/c$  with  $c = 2(2(\gamma - \ell) + |S|)$ , and picks  $N$  so that  $a_{n+N}$  stays below it; each

stage is handled by `exists_completed_pruned_expander`, which yields  $D$  with  $2|D| \leq |C|$  and  $|D| \leq a|C|/(\gamma - \ell)$ . Nonemptiness of the survivors follows from the half bound, the deleted density vanishes by comparison with  $a_{n+N}/(\gamma - \ell)$ , and survivor cardinalities tend to infinity by `pruned_component_card_tendsto_atTop`. This supplies the exact expanders required by the Kazhdan-pair bridge.

## 9.16 Completion bad density on the survivors (KunActualCanonicalPrunedSourceBadDensity)

A single transfer theorem. Assuming vanishing density of the components inside the canonical matched-radius bad set and vanishing relative density of the deleted sets, it concludes that `sourceCompletionBad`, the completion-failure set of the first-factor and second-factor completed permutation families over  $F$ , has vanishing density relative to the surviving set. The proof instantiates the general lemma `pruned_sourceCompletionBad_density_tendsto_zero_of_matchedRadius` with the containment `sourceCompletionBad_subset_canonical_matchedRadiusBad`. It provides the bad-density hypothesis consumed by the finite-model bridge of the following segments.

**pruned\_sourceCompletionBad\_density\_tendsto\_zero\_of\_canonical\_matched** thm

```

theorem pruned_sourceCompletionBad_density_tendsto_zero_of_canonical_matched
  {K J : Type*} [Group K] [Group J]
  [DecidableEq K] [DecidableEq J]
  (A : SoficGroups.SoficApproximation (K × J))
  (S : Finset K)
  (hsymmetric : ∀ a ∈ S, a⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set K) = T)
  (F : Finset J) (v : ℕ → ℕ)
  (Q : (n : ℕ) → Finpartition
    (Finset.univ : Finset (Fin (A.model (v n)).size)))
  (C : (n : ℕ) → Finset (Fin (A.model (v n)).size))
  (hC : ∀ n, C n ∈ (Q n).parts)
  (r : ℕ → ℕ)
  (hmatched : Tendsto
    (fun n =>
      (((C n n SoficGroups.matchedRadiusBad
        (Q n) (sourceProductRadiusLabels S F (r n))
        ((A.model (v n)).action)
        (canonicalProductRadiusBad
          A S hsymmetric hgenerates F (v n) (r n))).card : ℝ) /
        (C n).card)))
    atTop (ℕ 0))
  (D : (n : ℕ) → Finset
    {x : Fin (A.model (v n)).size // x ∈ C n})
  (hZ : ∀ n,
    (Finset.univ \ D n :
      Finset {x : Fin (A.model (v n)).size // x ∈ C n}).Nonempty)
  (hdeleted : Tendsto
    (fun n => ((D n).card : ℝ) / (C n).card)
    atTop (ℕ 0)) :
  Tendsto
    (fun n =>
      ((sourceCompletionBad
        (fun i : 1S =>
          completedRestriction
            ((A.model (v n)).action ((i : K), (1 : J))) (C n))
        (fun j : J =>
          completedRestriction
            ((A.model (v n)).action ((1 : K), j)) (C n))
        F (Finset.univ \ D n)).card : ℝ) /
      (C n).card))
    atTop (ℕ 0))

```

-- statement truncated; proof: 21 lines, omitted

`pruned_sourceCompletionBad_density_tendsto_zero_of_canonical_matched` supplies the bad-density input of the finite-model bridge. Here `sourceCompletionBad` collects the surviving points at which the completed first-factor generator family  $i \mapsto \text{completedRestriction}((A.\text{model}(\nu n)).\text{action}(i, 1))(C_n)$  and the completed second-factor family over  $F$  fail the bookkeeping needed later, namely the commutation and product agreement that the centralizer construction depends on. The theorem assumes that the density of  $C_n$  inside the canonical

matched-radius bad set, with labels `sourceProductRadiusLabels` at radius  $r_n$  and bad set `canonicalProductRadiusBad`, tends to zero, and that the deleted sets  $D_n$  have vanishing relative density; it concludes that the density of `sourceCompletionBad` relative to the surviving set  $\text{univ} \setminus D_n$  also tends to zero. The proof is a direct application of the previously established general transfer lemma `pruned_sourceCompletionBad_density_tendsto_zero_of_matchedRadius`, whose key containment hypothesis is discharged by `sourceCompletionBad_subset_canonical_matchedRadiusBad`: every completion-bad point of a component is already matched-radius bad for the canonical bad set. The remaining arguments are bookkeeping of indices through the reindexing function  $\nu$ .

## 9.17 Approximate multiplicativity of twice-completed restrictions (KunActualTwice PrunedFirstFactorRootBad)

Quantitative theory of how completed restrictions compose, culminating in asymptotic multiplicativity of the twice-completed first-factor family. `restrictionMultiplicationBad` isolates the points of  $Z$  where composition can fail; off it, the completed restriction of  $r$  equals the product of those of  $p$  and  $q$ . Consequently the permutation distance is bounded by the pointwise failure count plus twice the deleted complement. Specializing to model actions of the first-factor elements  $(a, 1)$ ,  $(g, 1)$ ,  $(ag, 1)$  with  $a, g, ag \in S^k$ , every failure is charged to the canonical matched-radius bad set. A small limit lemma converts absolute densities into survivor-relative densities, and the final theorem concludes that the normalized Hamming distance between  $\rho_n(ag)$  and  $\rho_n(a)\rho_n(g)$  tends to zero.

**firstFactor\_completed\_mul\_distance\_le\_canonical\_matched thm**

```
theorem firstFactor_completed_mul_distance_le_canonical_matched
  {K J : Type*} [Group K] [Group J]
  [DecidableEq K] [DecidableEq J]
  (A : SoficGroups.SoficApproximation (K × J))
  (S : Finset K)
  (hsymmetric : ∀ a ∈ S, a⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set K) = T)
  (F : Finset J) (n k : ℕ)
  {U : Finset (Fin (A.model n).size)}
  (Q : Finpartition U)
  (C : Finset (Fin (A.model n).size))
  (hC : C ∈ Q.parts)
  (a g : K)
  (ha : a ∈ S ^ k) (hg : g ∈ S ^ k)
  (hag : a * g ∈ S ^ k) :
  SoficGroups.permutationDistance
    (completedRestriction
      ((A.model n).action (a * g, (1 : J))) C)
    (completedRestriction
      ((A.model n).action (a, (1 : J))) C *
      completedRestriction
      ((A.model n).action (g, (1 : J))) C) ≤
  (C n SoficGroups.matchedRadiusBad
    Q (sourceProductRadiusLabels S F k)
    ((A.model n).action)
    (canonicalProductRadiusBad A S hsymmetric hgenerates F n k)).card := by
  -- proof: 63 lines, omitted
```

`firstFactor_completed_mul_distance_le_canonical_matched` is the combinatorial heart of approximate multiplicativity. Fix  $a, g \in S^k$  with  $ag \in S^k$ , and consider the completed restrictions to a component  $C$  of the model actions of  $(a, 1)$ ,  $(g, 1)$ , and  $(ag, 1)$ . The theorem bounds the permutation distance between the completed restriction of  $ag$  and the product of the other two by the number of points of  $C$  lying in the canonical matched-radius bad set. The proof routes through `permutationDistance_le_subtypeBad`: it suffices that composition holds off `restrictionMultiplicationBad`, and a failure point  $x$  must fail in one of three ways. If the action of  $g$  or of  $ag$  sends  $x$  out of  $C$ , then  $x$  belongs to `partitionWordCrossing` for a label of `sourceProductRadiusLabels` (the first-factor labels contain all elements of  $S^k$  paired with the identity, by `firstFactor_mem_sourceProductRadiusLabels`), hence  $x$  is matched-radius bad. Otherwise the product identity itself fails at  $x$ ; but if  $x$  avoided `finiteRootBad`, then `finiteRootBad_multiplicative` would force the actions of  $a$ ,  $g$ , and  $ag$  to

compose at  $x$ , a contradiction, so  $x$  is root-bad and again captured. Thus every multiplication defect of the completed family is charged to the single matched bad set whose density the selection theorem controls.

**twiceCompleted\_firstFactor\_mul\_normalizedHamming\_tendsto\_zero\_of\_canonical\_matched** thm

```

theorem twiceCompleted_firstFactor_mul_normalizedHamming_tendsto_zero_of_canonical_matched
  {K J : Type*} [Group K] [Group J]
  [DecidableEq K] [DecidableEq J]
  (A : SoficGroups.SoficApproximation (K × J))
  (S : Finset K) (hS : 1 ∈ S)
  (hsymmetric : ∀ a ∈ S, a⁻¹ ∈ S)
  (hgenerates : Subgroup.closure (S : Set K) = T)
  (F : Finset J) (v : ℕ → ℕ)
  (Q : (n : ℕ) → Finpartition
    (Finset.univ : Finset (Fin (A.model (v n)).size)))
  (C : (n : ℕ) → Finset (Fin (A.model (v n)).size))
  (hC : ∀ n, C n ∈ (Q n).parts)
  (r : ℕ → ℕ) (hr : Tendsto r atTop atTop)
  (hmatched : Tendsto
    (fun n =>
      (((C n n SoficGroups.matchedRadiusBad
        (Q n) (sourceProductRadiusLabels S F (r n))
        ((A.model (v n)).action)
        (canonicalProductRadiusBad
          A S hsymmetric hgenerates F (v n) (r n))).card : ℝ) /
        (C n).card))
    atTop (nhds 0))
  (D : (n : ℕ) → Finset
    {x : Fin (A.model (v n)).size // x ∈ C n})
  (hZ : ∀ n,
    (Finset.univ \ D n :
      Finset {x : Fin (A.model (v n)).size // x ∈ C n}).Nonempty)
  (hdeleted : Tendsto
    (fun n => ((D n).card : ℝ) / (C n).card)
    atTop (nhds 0))
  (a g : K) :
  Tendsto
    (fun n =>
      SoficGroups.normalizedHamming
        (completedRestriction
          (completedRestriction
            ((A.model (v n)).action (a * g, (1 : J))) (C n))
            (Finset.univ \ D n))
          (completedRestriction
            (completedRestriction

```

-- statement truncated; proof: 108 lines, omitted

`twiceCompleted_firstFactor_mul_normalizedHamming_tendsto_zero_of_canonical_matched` establishes that the twice-completed first-factor family  $\rho_n(g)$ , the model action of  $(g, 1)$  completed first to the component  $C_n$  and then to the survivors  $\text{univ} \setminus D_n$ , is an asymptotic homomorphism: for every pair  $a, g$ , the normalized Hamming distance between  $\rho_n(ag)$  and  $\rho_n(a)\rho_n(g)$  tends to zero. The proof chooses generator words for  $a, g$ , and  $ag$  via `symmetricGeneratorWord`; since  $r_n \rightarrow \infty$ , eventually all three word lengths are below  $r_n$ , so  $a, g, ag \in S^{r_n}$  by `mem_generator_pow_of_chosen_word_length`, and `firstFactor_completed_mul_distance_le_canonical_matched` bounds the first-stage multiplication defect by the matched bad count  $B_n$ . The second completion is handled by `completedRestriction_mul_distance_le_failure_add_deleted`, which shows completing to  $Z_n$  adds at most  $2|D_n|$  to the defect. Dividing by the survivor cardinality, `density_relative_survivors` and `deleted_density_relative_survivors` convert the hypotheses, which are stated as densities relative to  $|C_n|$ , into vanishing bounds relative to  $|Z_n|$ , and `squeeze_zero'` closes the limit. This is precisely the asymptotic-multiplicativity hypothesis fed to the finite-model bridge in the composition theorem.

*Remaining declarations in this section:*

|                                           |     |                                                                           |
|-------------------------------------------|-----|---------------------------------------------------------------------------|
| <code>restrictionMultiplicationBad</code> | DEF | The points of $Z$ where $q$ or $r$ exits $Z$ , or where $rx \neq p(qx)$ . |
|-------------------------------------------|-----|---------------------------------------------------------------------------|

|                                      |     |                                                                                                                                      |
|--------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------|
| completedRestriction_mul_of_not_mem_ | THM | Off restrictionMultiplicationBad, the completed restriction of $r$ equals the product of the completed restrictions of $p$ and $q$ . |
| restrictionMultiplicationBad         |     |                                                                                                                                      |
| completedRestriction_mul_distance_   | THM | Bounds the distance between completedRestriction of $r$ and the product by the failure count plus twice the deleted complement.      |
| le_failure_add_deleted               |     |                                                                                                                                      |
| density_relative_survivors           | THM | If $b_n$ and the deleted set have vanishing density in $V_n$ , then $b_n$ over survivor cardinality vanishes.                        |

## 9.18 Bridge to expanding centralizer finite models (KunActualSelectedPrunedSourceFiniteModel Bridge)

The bridge from pruned combinatorial data to the finite models. A preparatory lemma shows that agreement with an ambient family forces  $\tau$  to send identity generators to the identity permutation. The main theorem assumes a Kazhdan pair for  $G$ , exact expansion of  $\tau_n$  on growing survivor sets, an asymptotic homomorphism  $\rho_n$  close to  $\tau_n$  on generators, and vanishing sourceCompletionBad density, and produces Nonempty of Expanding CentralizerFiniteModel. The proof evaluates generator words, controls root-defect densities, invokes the Kazhdan-Markov contraction and uniform-radius improvement results together with a tolerance scheduler to certify HasAlmostCentralizerImprovement, and finishes with the terminal selected-completed-sequence lemma. Property (T) enters the endgame precisely here.

**nonempty\_expandingCentralizerFiniteModel\_of\_actual\_selected\_pruned\_source** thm

```

theorem nonempty_expandingCentralizerFiniteModel_of_actual_selected_pruned_source
  {G J : Type} [Group G] [Group J]
  [DecidableEq G]
  (P : SoficGroups.KazhdanPair.{0, 0} G)
  (S : Finset G) (honeS : 1 ∈ S)
  (hcover : P.generators ⊆ S)
  (hsymmetric : ∀ g ∈ S, g-1 ∈ S)
  (hgenerates : Subgroup.closure (S : Set G) = T)
  (F : Finset J)
  (W : ℕ → Type)
  [∀ n, Fintype (W n)] [∀ n, DecidableEq (W n)]
  (σ : (n : ℕ) → ιS → Equiv.Perm (W n))
  (hoone : ∀ n (i : ιS), (i : G) = 1 → σ n i = 1)
  (p : (n : ℕ) → J → Equiv.Perm (W n))
  (hp : ∀ n, p n 1 = 1)
  (Z : (n : ℕ) → Finset (W n))
  (hZ : ∀ n, (Z n).Nonempty)
  (hlarge : Tendsto (fun n => (Z n).card) atTop atTop)
  (τ : (n : ℕ) → ιS →
    Equiv.Perm {x : W n // x ∈ Z n})
  (ht : ∀ n i (x : W n) (hx : x ∈ Z n)
    (_hi : σ n i x ∈ Z n),
    ((τ n i (x, hx) :
      {x : W n // x ∈ Z n}) : W n) = σ n i x)
  (p : (n : ℕ) → G →
    Equiv.Perm {x : W n // x ∈ Z n})
  (hpone : ∀ n, p n 1 = 1)
  (hpmul : ∀ a g : G,
    Tendsto
      (fun n => SoficGroups.normalizedHamming
        (p n (a * g)) (p n a * p n g))
        atTop (ℵ 0))
  (hpgenerator : ∀ i : ιS,
    Tendsto
      (fun n => SoficGroups.normalizedHamming
        (τ n i) (p n (i : G)))
        atTop (ℵ 0))
  (ell : ℝ) (hell : 0 < ell)

```

```

(hexp : ∀ n, ∀ E : Finset {x : W n // x ∈ Z n},
  ell * min (E.card : ℝ)
-- statement truncated; proof: 177 lines, omitted

```

`nonempty_expandingCentralizerFiniteModel_of_actual_selected_pruned_source` is the analytic bridge from pruned combinatorial data to the finite models whose existence ultimately contradicts non-LEF-ness. Its inputs: a Kazhdan pair  $P$  for  $G$  whose generators lie in the symmetric generating finset  $S$ ; a family  $\tau_n$  of permutations of survivor sets  $Z_n$  (with  $|Z_n| \rightarrow \infty$ ) expanding every subset with constant  $\ell > 0$ ; an asymptotic homomorphism  $\rho_n$  with  $\tau_n(i)$  asymptotically close to  $\rho_n(i)$  on generators; agreement of  $\tau$  with the ambient family  $\sigma$ ; second-factor permutations  $p_n$ ; and vanishing density of `sourceCompletionBad`. The conclusion is `Nonempty` of `ExpandingCentralizerFiniteModel` for  $J$  and  $F$ . The proof evaluates chosen generator words to define  $\varphi_n(g)$  as products of  $\tau_n$  along `symmetricGeneratorWord`, so  $\varphi_n$  is asymptotically multiplicative by `chosenWordEvaluation_multiplicative_tendsto` and agrees with  $\tau_n$  on generators. Root-defect sets `fixedRadiusRootBad` have vanishing density. The Kazhdan side enters through `kazhdanMarkovContractionFactor`, a contraction factor  $q < 1$ , and `exists_uniform_radius_hasAlmostCentralizerImprovement`, which for each tolerance produces a uniform radius; the scheduler `exists_prescribed_radius_union_source_completed_tolerance` then selects tolerances  $t_n$  compatible with the root, source, and deletion budgets, establishing `HasAlmostCentralizerImprovement` for  $\tau_n$  eventually. Together with the commutation-defect, multiplicativity, and separation tables for the completed second-factor family from `eventually_completed_sourceCentralizer_table_of_bad_density`, the terminal lemma `nonempty_expandingCentralizerFiniteModel_of_selected_completed_sequence` produces the model.

*Remaining declarations in this section:*

```

completed_generator_one_of_internal_ THM  If  $\tau$  agrees with  $\sigma$  inside  $Z$  and  $\sigma$  kills identity generators, so
agreement                                does  $\tau$ .

```

## 9.19 Composing pruning, multiplicativity, and the bridge (KunActualCanonicalSelectedFiniteModelComposition)

A pure composition theorem with no new mathematics: it takes the canonical selected-components package (vanishing matched-radius bad density, growing components, additive expansion with vanishing error, and a Kazhdan pair covering the generators) and derives the expanding centralizer finite model. It calls the pruning theorem, shifts all sequences by the resulting offset  $N$ , defines the once-completed and twice-completed permutation families, verifies their identity normalization, asymptotic multiplicativity, generator agreement, and pruned bad-density hypotheses using the three preceding namespaces, and hands everything to the bridge with expansion constant  $\gamma/2$ . Its interface is exactly what the literal nine-prefix instantiation can satisfy directly.

`nonempty_expandingCentralizerFiniteModel_of_canonical_selected_additive` thm

```

theorem nonempty_expandingCentralizerFiniteModel_of_canonical_selected_additive
  {K J : Type} [Group K] [Group J]
  [DecidableEq K] [DecidableEq J]
  (A : SoficGroups.SoficApproximation (K × J))
  (P : SoficGroups.KazhdanPair.{0, 0} K)
  (S : Finset K) (hone : 1 ∈ S)
  (hcover : P.generators ⊆ S)
  (hsymmetric : ∀ g ∈ S, g-1 ∈ S)
  (hgenerates : Subgroup.closure (S : Set K) = T)
  (F : Finset J) (v : ℕ → ℕ)
  (Q : (n : ℕ) → Finpartition
    (Finset.univ : Finset (Fin (A.model (v n)).size)))
  (C : (n : ℕ) → Finset (Fin (A.model (v n)).size))
  (hC : ∀ n, C n ∈ (Q n).parts)
  (r : ℕ → ℕ) (hr : Tendsto r atTop atTop)
  (hmatched : Tendsto
    (fun n =>
      (((C n n SoficGroups.matchedRadiusBad
        (Q n) (sourceProductRadiusLabels S F (r n))

```

```

      ((A.model (v n)).action)
      (canonicalProductRadiusBad
        A S hsymmetric hgenerates F (v n) (r n)).card : ℝ) /
      (C n).card))
  atTop (N 0))
(hlarge : Tendsto (fun n => (C n).card) atTop atTop)
(γ : ℝ) (hγ : 0 < γ)
(a : ℕ → ℝ) (ha : Tendsto a atTop (N 0))
(hadd : ∀ n,
  ∀ E : Finset {x : Fin (A.model (v n)).size // x ∈ C n},
  γ * min (E.card : ℝ)
    ((Fintype.card
      {x : Fin (A.model (v n)).size // x ∈ C n} : ℝ) -
      E.card) -
  a n * Fintype.card
    {x : Fin (A.model (v n)).size // x ∈ C n} ≤
  (SoficGroups.boundary
    (fun i : ιS =>
      completedRestriction
        ((A.model (v n)).action ((i : K), (1 : J)))
        (C n)) E : ℝ)) :
-- statement truncated; proof: 108 lines, omitted

```

`nonempty_expandingCentralizerFiniteModel_of_canonical_selected_additive` composes the three preceding namespaces into one clean interface. Its hypotheses are exactly what the literal source construction can furnish: canonical selected components  $C_n$  with growing cardinality and vanishing matched-radius bad density, radii  $r_n \rightarrow \infty$ , a Kazhdan pair whose generators lie in the symmetric generating finset  $S$ , and the additive expansion estimate for the completed first-factor generator restrictions with vanishing error  $a_n$ . The proof first applies `exists_actual_uniformly_expanding_pruned_selected_components` to obtain the offset  $N$ , deletion sets  $D_n$ , and exact  $\gamma/2$ -expanders  $\tau_n$ ; it then shifts every sequence by  $N$ , defines  $\sigma$  and  $p$  as the completed restrictions of the first-factor and second-factor actions to  $C_{n+N}$ , and  $\rho$  as their second completion to the survivors  $\text{univ} \setminus D_n$ . Identity normalization is checked by unfolding `completedRestriction_one` through the model's `map_one`; asymptotic multiplicativity of  $\rho$  comes from `twiceCompleted_firstFactor_mul_normalizedHamming_tendsto_zero_of_canonical_matched`; generator closeness from `twiceCompleted_sourceGenerator_normalizedHamming_tendsto_zero`; and the pruned completion-bad density from `pruned_sourceCompletionBad_density_tendsto_zero_of_canonical_matched`. All hypotheses of `nonempty_expandingCentralizerFiniteModel_of_actual_selected_pruned_source` are then met, yielding the finite model.

## 9.20 Disagreement points land in the conjugacy bad set (KunLiteralSourceSelectedComponents)

A one-lemma return to the literal source namespace. `sourceTransportedReferenceGenerator_disagreement_mem` shows that any point where the transported reference generator, that is the  $u$ -conjugated alpha generator action, disagrees with the compressed product approximation's action of the corresponding compressed generator, lies in `sourceGeneratorFamilyConjugacyBad`. Since the two group elements coincide by `source_u_conjugated_product_generator`, disagreement of the induced permutations is exactly a conjugacy disagreement, and unfolding the definition places the point in the bad set. It licenses the family swap performed in the next segment's additive-expansion theorem.

**`sourceTransportedReferenceGenerator_disagreement_mem` *thm***

```

theorem sourceTransportedReferenceGenerator_disagreement_mem
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (SΓ : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (n : ℕ) (i : ιSΓ) (x : Fin (A.model n).size)
  (h : sourceTransportedReferenceGenerators A SΓ n i x ≠
    ((SoficGroups.SourceProductThroughAlpha.sourceCompressedLocalProductApproximation A).model n).action
      (SoficGroups.SourceProductThroughAlpha.sourceCompressionUAlphaEquiv
        (i : SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode),
        1) x) :

```



```

x ∈ sourceGeneratorFamilyConjugacyBad A SΓ n := by
-- proof: 19 lines, omitted

```

`sourceTransportedReferenceGenerator_disagreement_mem` is the pointwise justification for swapping generator families. The transported reference generator at index  $i$  acts as the model action of the  $u$ -conjugated alpha inclusion of  $i$ , where  $u$  is `sourceCompressionUElement`, while the compressed product approximation acts through the image of  $i$  under `sourceCompressionUAlphaEquiv`. By `source_u_conjugated_product_generator` these two prescriptions agree as permutations arising from equal group elements, so any point  $x$  at which the two permutations disagree witnesses a conjugacy disagreement. The proof unfolds `sourceGeneratorFamilyConjugacyBad` as a `Finset.biUnion` over the generators of the filters `sourceConjugacyDisagreementBad`, exhibits the generator underlying  $i$  as the witness, and rewrites the disagreement with `source_u_conjugated_product_generator` to match the filter's predicate. The lemma matters because the expansion estimates of the selection pipeline are stated for the transported reference generators, while the composition theorem needs them for the compressed product generators: `exists_vanishing_completed_first_factor_additive_expansion_of_selected_source` uses this lemma to charge every disagreement between the two families to a bad set already known to have vanishing density inside the selected components.

## 9.21 Additive expansion for completed compressed generators (KunActualSourceSelected CompletedGeneratorExpansion)

Builds the additive expansion input for the composition theorem in the literal setting. Given subset expansion of the transported reference generators on the selected components with constant  $\gamma$ , vanishing relative boundary of the components, and the guarded matched-density hypothesis, the theorem outputs a nonnegative vanishing error sequence  $a_n$  such that the completed restrictions to  $C_n$  of the compressed product first-factor generator actions expand every subset of the component subtype with defect at most  $a_n$  times the component size. The disagreement between the two generator families is absorbed into `sourceGeneratorFamilyConjugacyBad`, and the final boundary is reindexed to the compressed generating finset via the boundary-invariance lemma.

**exists\_vanishing\_completed\_first\_factor\_additive\_expansion\_of\_selected\_source** *thm*

```

theorem exists_vanishing_completed_first_factor_additive_expansion_of_selected_source
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (S : Finset
    (SoficGroups.prefixElementaryGroup SoficGroups.alphaPrefixCode))
  (hsymmetric : ∀ g ∈ S, g⁻¹ ∈ S)
  (hgenerates : Subgroup.closure
    (S : Set
      (SoficGroups.prefixElementaryGroup
        SoficGroups.alphaPrefixCode)) = T)
  (Q : (n : ℕ) →
    Finpartition (Finset.univ : Finset (Fin (A.model n).size)))
  (F : Finset
    (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
      SoficGroups.ThompsonPrefixLocal.sourceLocalWord))
  (N : ℕ) (r : ℕ → ℕ)
  (C : (n : ℕ) → Finset (Fin (A.model (n + N)).size))
  (gamma : ℝ)
  (hC : ∀ n, (C n).Nonempty)
  (hexpand : ∀ n, ∀ D : Finset (Fin (A.model (n + N)).size),
    D ⊆ C n → 2 * D.card ≤ (C n).card →
      gamma * (D.card : ℝ) ≤
        (SoficGroups.boundary
          (SoficGroups.KunLiteralSourceSelectedComponents.sourceTransportedReferenceGenerators
            A S (n + N)) D : ℝ))
  (hboundary : Tendsto
    (fun n =>
      (SoficGroups.boundary
        (SoficGroups.KunLiteralSourceSelectedComponents.sourceTransportedReferenceGenerators
          A S (n + N)) (C n) : ℝ) / (C n).card)

```

```

    atTop (nhds 0))
(hguarded : Tendsto
(fun n =>
  (((C n n SoficGroups.matchedRadiusBad
    (SoficGroups.KunLiteralSourceSelectedComponents.sourceTransportedPartition
      A Q (n + N))
    (SoficGroups.KunLiteralSourceSelectedComponents.sourceSelectedProductRadiusLabels
      S F (r n))
    (((SoficGroups.SourceProductThroughAlpha.sourceCompressedLocalProductApproximation
      A).model (n + N)).action)
  -- statement truncated; proof: 88 lines, omitted

```

`exists_vanishing_completed_first_factor_additive_expansion_of_selected_source` manufactures the additive expansion hypothesis for the composition theorem. Starting from subset expansion of the transported reference generators on the selected components  $C_n$  (constant  $\gamma$ , for subsets of at most half the component), together with vanishing relative boundary of  $C_n$  and the guarded matched-density hypothesis, it produces a nonnegative error sequence  $a_n \rightarrow 0$  such that the completed restrictions to  $C_n$  of the compressed product generator actions expand every subtype subset  $E$  with defect at most  $a_n$  times the component size. The first step shows that the density of  $C_n$  inside `sourceGeneratorFamilyConjugacyBad` vanishes: that set is one branch of the union defining `sourceSelectedRadiusBad`, hence contained in the guarded matched bad set, and `squeeze_zero` applies. The analytic work is done by the general lemma `exists_vanishing_completedRestriction_additive_expansion_of_selected_bad`, instantiated with the transported reference family as the expanding family and the compressed product family as the target, using `sourceTransportedReferenceGenerator_disagreement_mem` to charge every family disagreement to the bad set. Finally, `boundary_sourceCompressedGeneratorSubtypeEquiv` reindexes the boundary from the alpha generating finset to the compressed generating finset, matching the exact statement demanded by `nonempty_expandingCentralizerFiniteModel_of_canonical_selected_additive`.

## 9.22 Finite models from a nine-prefix sofic approximation (KunLiteralNineSourceFinite Models)

The concrete instantiation that closes the quantitative argument: from any sofic approximation of the nine-prefix elementary group and any finite set  $F$  of local prefix transpositions, an expanding centralizer finite model for the local prefix transposition group exists. The proof assembles the selected-components package, names the ambient product data (the alpha-zero prefix group, the local transposition group, the compressed local product approximation, and the compressed generating finset), obtains a Kazhdan pair with exactly those generators, strips the conjugacy guard from the matched density, invokes the additive-expansion theorem, and applies the composition theorem. Every step after this theorem is purely logical transfer.

**`source_ambient_nonempty_expandingCentralizerFiniteModel`** *thm*

```

theorem source_ambient_nonempty_expandingCentralizerFiniteModel
  (A : SoficGroups.SoficApproximation
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode))
  (F : Finset
    (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
      SoficGroups.ThompsonPrefixLocal.sourceLocalWord)) :
  Nonempty
    (SoficGroups.ExpandingCentralizerFiniteModel
      (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
        SoficGroups.ThompsonPrefixLocal.sourceLocalWord) F) := by
  -- proof: 102 lines, omitted

```

`source_ambient_nonempty_expandingCentralizerFiniteModel` instantiates the entire abstract pipeline at the concrete nine-prefix code: from any sofic approximation  $A$  of `prefixElementaryGroup` at `ninePrefixCode` and any finite set  $F$  of local prefix transpositions, it constructs an expanding centralizer finite model for the local prefix transposition group over  $F$ . The proof first runs `exists_source_selected_components` to obtain the generating set  $S$ , partitions  $Q$ , constant  $\gamma$ , offset  $N$ , radii  $r$ , and components  $C$ . It then names the ambient product data:  $K$  the alpha-zero prefix group,  $J$  the local transposition group,  $AP$  the compressed local product approximation of  $A$ , and  $SK$  the compressed generating finset, obtaining a Kazhdan pair  $PK$  whose generators are exactly

$SK$  from `exists_sourceCompressedKazhdanPair_with_same_generators`. Two definitional identities, verified by `rfl`, identify the selected bad set with the union of the canonical bad set and the conjugacy guard and identify the label sets; `canonical_matched_density_of_guarded` then strips the guard to give the canonical matched density. The additive expansion is supplied by `exists_vanishing_completed_first_factor_additive_expansion_of_selected_source`, and `nonempty_expandingCentralizerFiniteModel_of_canonical_selected_additive` concludes. This theorem is the last quantitative step of the whole development.

### 9.23 Final chain to a nonsofic finitely presented group (SourceTopLevelCompressionFinal)

The final chain of the file. Soficity of the nine-prefix elementary group would yield, through the finite models just constructed and the top-level compression argument, that the source local prefix transposition group is LEF; but `sourceLocalPrefixTranspositionGroup_notLEF`, proved earlier, says it is not, so `prefixElementaryGroup` at nine `PrefixCode` is not sofic. Non-soficity is then transferred along the isomorphism `ninePrefixElementaryGroupEquiv` to `binaryLeavittElementaryGroup` at parameter 9, since soficity is inherited through injective homomorphisms. Finally, `exists_finitelyPresented_not_sofic_of_not_sofic` converts this into the headline existential statement: there is a type carrying a group structure that is finitely presented and not sofic, completing the formalization.

**sourceLocalPrefixTranspositionGroup\_lef\_of\_sofic** *thm*

```
theorem sourceLocalPrefixTranspositionGroup_lef_of_sofic :
  SoficGroups.Sofic
    (SoficGroups.prefixElementaryGroup SoficGroups.ninePrefixCode) →
  SoficGroups.LEF
    (SoficGroups.ThompsonPrefixInsertion.localPrefixTranspositionGroup
     SoficGroups.ThompsonPrefixLocal.sourceLocalWord) := by
  -- proof: 6 lines, omitted
```

`sourceLocalPrefixTranspositionGroup_lef_of_sofic` is the conditional heart of the contradiction. It assumes sofic for the nine-prefix elementary group and derives LEF for the source local prefix transposition group. The proof is only two lines because the work is packaged elsewhere: `sourceLocalPrefixTranspositionGroup_lef_of_source_finite_models`, from the `SourceTopLevelCompression` namespace, states that LEF follows provided every sofic approximation of the nine-prefix group and every finite subset  $F$  admits an expanding centralizer finite model, and `source_ambient_nonempty_expandingCentralizerFiniteModel` provides precisely those models. Conceptually: a sofic approximation of the ambient elementary group would give finite permutation models that approximate the local transposition group well enough, with expansion and centralizer control descending from the Kazhdan-pair machinery, to embed arbitrarily large finite portions of that group into finite symmetric groups, which is the definition of local embeddability into finite groups. Since `sourceLocalPrefixTranspositionGroup_notLEF` shows the group is in fact not LEF, this conditional is exactly the lever that refutes soficity of the nine-prefix elementary group in the next theorem.

**exists\_finitelyPresented\_nonsofic\_group** *thm*

```
theorem exists_finitelyPresented_nonsofic_group :
  ∃ (G : Type) ( _ : Group G),
  Group.IsFinitelyPresented G ∧ ¬ SoficGroups.Sofic G := by
  -- proof: 3 lines, omitted
```

`exists_finitelyPresented_nonsofic_group` is the headline result of the file: there exists a type  $G$  with a group structure that is finitely presented and not sofic. The proof composes the final chain assembled in this segment. `ninePrefixElementaryGroup_not_sofic` refutes soficity of the nine-prefix elementary group by pitting `sourceLocalPrefixTranspositionGroup_lef_of_sofic` against `sourceLocalPrefixTranspositionGroup_notLEF`: soficity would imply LEF, which is refuted. `binaryLeavittElementaryGroup_not_sofic` then transfers this along the group isomorphism `ninePrefixElementaryGroupEquiv`: since soficity passes through injective homomorphisms by `sofic_of_injective`, applied here to the inverse of the equivalence, soficity of `binaryLeavittElementaryGroup` at parameter 9 would force soficity of the nine-prefix elementary group. Finally, `exists_finitelyPresented_not_sofic_of_not_sofic`, established earlier in the file, converts non-soficity of the binary Leavitt elementary group into the existential conclusion  $\exists G, G$  finitely presented and not sofic, packaging the finite-presentability side of the argument.

This four-step block is the formal counterpart of the informal statement that not every finitely presented group is sofic, and every quantitative ingredient feeding it was closed off in the preceding segments.

*Remaining declarations in this section:*

|                                                     |     |                                                                                                                                                                            |
|-----------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ninePrefixElementaryGroup_not_sofic</code>    | THM | The nine-prefix elementary group is not sofic, since the induced LEF property contradicts <code>sourceLocalPrefixTranspositionGroup_notLEF</code> .                        |
| <code>binaryLeavittElementaryGroup_not_sofic</code> | THM | Transfers non-soficity to <code>binaryLeavittElementaryGroup</code> at parameter 9 along <code>ninePrefixElementaryGroupEquiv</code> via <code>sofic_of_injective</code> . |

## 10 Provenance and verification notes

- **Source.** `NonSoficGroup.lean`, repository [github.com/openai/ten-proofs](https://github.com/openai/ten-proofs), commit `94bc0feb6a9f` (August 2, 2026), Apache License 2.0. The file is reproduced verbatim as `sophic.lean` next to this document. The repository accompanies OpenAI’s report *Ten advances in mathematics and theoretical computer science* (<https://cdn.openai.com/pdf/ten-proofs-oai.pdf>).
- **Checking.** The file contains no sorry and declares no axioms of its own; it imports Mathlib and builds with Lean 4.32.0 via `lake build NonSoficGroup`. The repository also ships a statement-only challenge file (`ComparatorChallenges/D_NonSoficGroup.lean`) whose theorem statement matches `exists_finitelyPresented_nonsofic_group` as displayed in Section 1, for independent proof checking.
- **About this digest.** The explanations in this document were generated by Claude (Anthropic) by reading the Lean source; they aim to be faithful but are not themselves machine-checked. The Lean file is the authority. Kind counts and section structure were extracted mechanically, and every declaration of the file appears in this document exactly once, either as a deep dive or as a table row.
- **Background.** Sofic groups originate with Gromov’s work on symbolic dynamics and were named by Weiss; standard background includes M. Gromov, *Endomorphisms of symbolic algebraic varieties*, J. Eur. Math. Soc. 1 (1999), and B. Weiss, *Sofic groups and dynamical systems*, Sankhyā Ser. A 62 (2000). Property (T), Leavitt algebras, and Thompson’s group  $F$  are classical; the file develops all it needs from scratch on top of Mathlib.